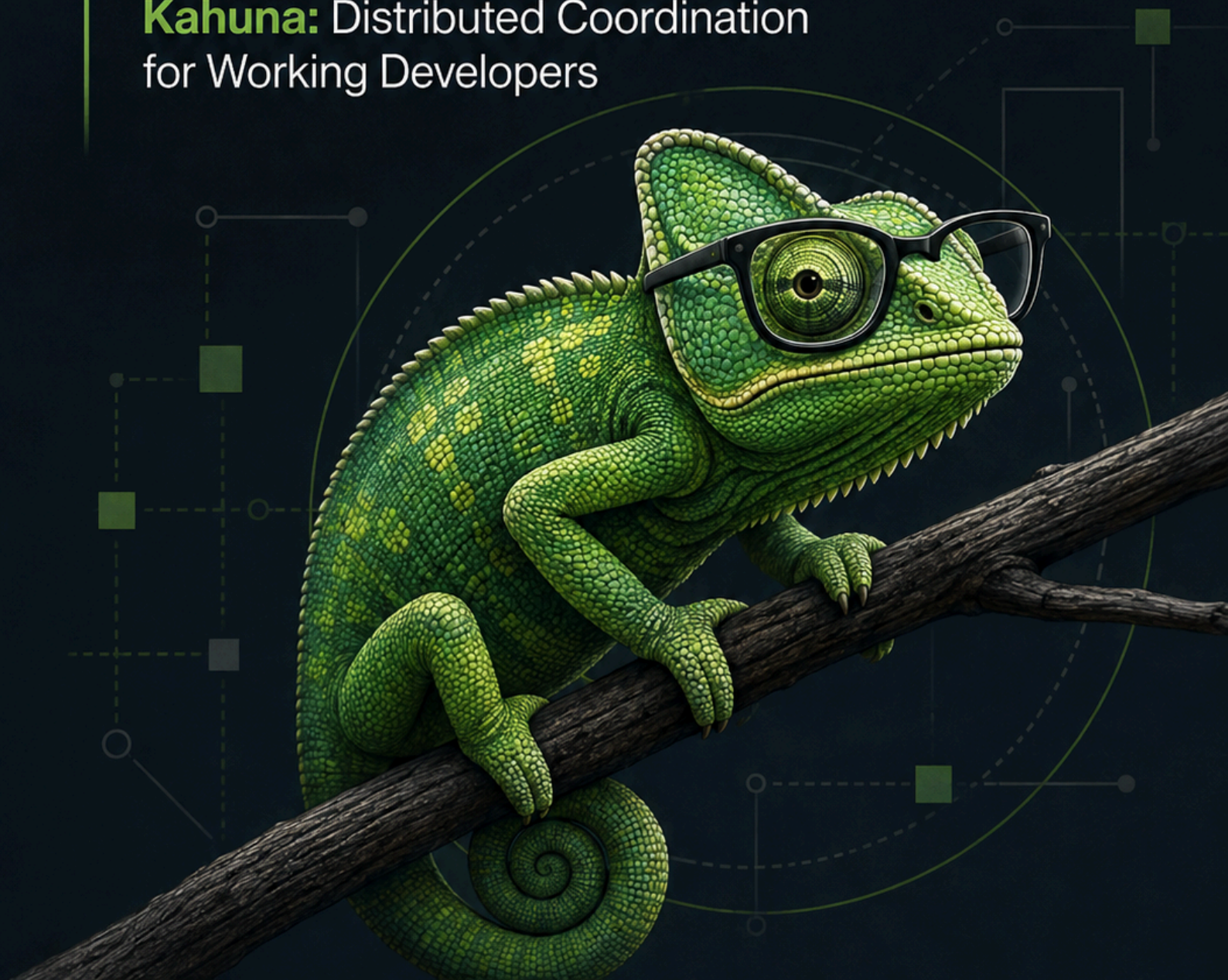
The Book of Kahuna

Kahuna: Distributed Coordination for Working Developers



Kahuna

Distributed Coordination for Working Developers

Andres Gutierrez

2025

Contents

1 Preface 13
1.1 Who This Book Is For 13
1.2 How This Book Is Organized 13
1.3 Conventions 14
1.4 Acknowledgments 14
2 What Is Kahuna? 15
2.1 The Coordination Problem 15
2.2 What Kahuna Is 16
2.3 The Three Primitives 16
2.4 When to Use Kahuna 18
2.5 When Not to Use Kahuna 19
2.6 How Kahuna Compares 19
2.7 Architecture at a Glance 21
2.8 A First Look at the Client SDK 22
2.9 What's Ahead 23
3 Getting Started 25
3.1 Prerequisites 25
3.2 Installing the Server 25
3.3 Installing the Command-Line Client 26
3.4 First Operations with the CLI 26
3.5 Running a Three-Node Cluster 27
3.6 Connecting from C# 29
3.7 A Complete Example 31
3.8 Your First Failure Experiment 33
3.9 Summary 34
4 Key-Value Operations 35
4.1 Keys and Values 35
4.2 Revisions 35
4.3 Conditional Writes 36
4.4 Time-to-Live (TTL) 37
4.5 Durability Levels 38
4.6 Deleting Keys 39
4.7 Checking Existence 39
4.8 No-Revision Writes 39
4.9 Batch Operations 39
4.10 Prefix Queries 41
4.11 Range Scans 41
4.12 Snapshot Reads 42
4.13 The KahunaKeyValue Object 44
4.14 CLI Reference 45
4.15 Putting It Together 45
4.16 Summary 46
5 Distributed Locking 47
5.1 Why Locks Need Leases 47

5.2 Acquiring a Lock 47
5.3 Fencing Tokens 48
5.4 Waiting for a Lock 50
5.5 Extending a Lease 51
5.6 What Happens When a Lock Holder Crashes 52
5.7 Unlocking 52
5.8 Querying Lock State 53
5.9 Ephemeral Locks 53
5.10 CLI Lock Operations 54
5.11 Common Patterns 54
5.12 Summary 55
6 The Distributed Sequencer 56
6.1 Creating a Sequence 56
6.2 Getting the Next Value 56
6.3 Reserving a Range 56
6.4 How Block Allocation Works 57
6.5 Increment 58
6.6 Maximum Value 58
6.7 Idempotent Allocation 59
6.8 Querying a Sequence 60
6.9 Updating a Sequence 60
6.10 Deleting a Sequence 61
6.11 CLI Commands 62
6.12 Server Configuration 62
6.13 Use Cases 63
6.14 Guarantees and Non-Guarantees 64
6.15 Summary 64
7 Transactions 65
7.1 Script Transactions 65
7.2 Interactive Transactions 68
7.3 Locking Modes 70
7.4 Read Validation and Write-Skew Detection 71
7.5 Conflict Policy 72
7.6 RetryableTransaction 72
7.7 Transaction Options Reference 74
7.8 Common Patterns 75
7.9 Summary 77
8 The Script Language 78
8.1 Language Overview 78
8.2 Data Types 78
8.3 Variables 78
8.4 Key-Value Commands 79
8.5 Placeholders 81
8.6 Expressions 82
8.7 Control Flow 83
8.8 Transactions in Scripts 84

8.9 RETURN 86
8.10 SLEEP 87
8.11 THROW 87
8.12 Built-in Functions 87
8.13 Complete Example: Balance Transfer 90
8.14 Complete Example: Batch Initialization 91
8.15 Complete Example: Snapshot Audit 91
8.16 Script Caching 91
8.17 Reserved Words 91
8.18 Operator Precedence 92
8.19 User-Defined Functions 92
8.20 Summary 93
9 Leader Election and Job Coordination 94
9.1 The Simplest Leader Election 94
9.2 Heartbeat Renewal 95
9.3 The Leader Election Loop 96
9.4 Why Fencing Tokens Matter 98
9.5 Publishing Leader Identity 100
9.6 Distributed Task Queue 101
9.7 Failure Scenarios 104
9.8 Tuning Leader Election 104
9.9 Combining Leader Election with Task Queues 105
9.10 Summary 106
10 Idempotency and Duplicate Prevention 107
10.1 The Retry Problem 107
10.2 Delivery Semantics 107
10.3 Naturally Idempotent Operations 107
10.4 Non-Idempotent Operations 108
10.5 Revision-Based Idempotency (CRAS) 108
10.6 Value-Based Idempotency (CVAS) 109
10.7 SetIfNotExists as an Idempotency Guard 109
10.8 Fencing-Based Exactly-Once Writes 111
10.9 Sequence-Based Deduplication 112
10.10 Transaction Operation IDs 113
10.11 Building an Idempotent Pipeline 113
10.12 Choosing the Right Mechanism 116
10.13 Common Mistakes 116
10.14 Summary 117
11 Globally Unique Identifiers 118
11.1 The ID Generation Problem 118
11.2 Common ID Schemes 118
11.3 Comparison Table 120
11.4 Patterns for ID Generation 120
11.5 Tuning Block Size 123
11.6 Failure Scenarios 125
11.7 When Not to Use Kahuna Sequences 126

11.8 A Complete Example 126
11.9 Summary 127
12 Coordinating Services 129
12.1 Distributed Configuration 129
12.2 Service Presence Detection 131
12.3 Session Management 134
12.4 Game Lobby Coordination 136
12.5 Failure Scenarios 139
12.6 Choosing Between Persistent and Ephemeral 140
12.7 Distributed Rate Limiting 140
12.8 Summary 142
13 Transactional Workflows 144
13.1 The Problem with Separate Writes 144
13.2 Single-Transaction Workflows 144
13.3 Choosing a Locking Mode 146
13.4 Multi-Transaction Workflows 147
13.5 Transaction Boundaries 149
13.6 Handling Conflicts 151
13.7 Script Transactions for Complex Logic 151
13.8 Failure Scenarios 153
13.9 Workflow Design Checklist 153
13.10 Summary 154
14 Handling Failures and Retries 155
14.1 Error Classification 155
14.2 The Error Decision Table 156
14.3 Retry Strategies 156
14.4 Timeout Configuration 158
14.5 Failure Scenarios 159
14.6 Application-Level Circuit Breaker 161
14.7 Error Handling Checklist 162
14.8 Summary 163
15 Architecture Overview 164
15.1 Project Layout 164
15.2 External Libraries 164
15.3 The IKahuna Facade 165
15.4 KahunaManager 166
15.5 The Shared Pattern: Manager, Locator, Actor 166
15.6 Inter-Node Communication 168
15.7 Persistence Backends 168
15.8 The Meta Partition 168
15.9 Client Communication 169
15.10 Summary 169
16 The Request Lifecycle 170
16.1 The Write Path: Setting a Key 170
16.2 The Read Path: Getting a Key 174
16.3 Read Path vs Write Path 175

16.4 Synchronous vs Asynchronous Boundaries 176
16.5 Where Latency Comes From 176
16.6 Failure Scenarios 177
16.7 Summary 177
17 Partitioning and Data Distribution 178
17.1 Two Routing Modes 178
17.2 The Range Map 180
17.3 The Meta Partition 181
17.4 Range Splits 181
17.5 Range Merges 183
17.6 The Generation Fence 184
17.7 State Transfer During Splits 185
17.8 Prefix Operations and Splits 185
17.9 Server Configuration 185
17.10 Failure Scenarios 186
17.11 Summary 187
18 Consensus and Replication 188
18.1 The Consensus Problem 188
18.2 Kommander: The Raft Library 188
18.3 Per-Partition Raft Groups 188
18.4 Leader Election 189
18.5 Log Entry Types 191
18.6 The Replication Pipeline 193
18.7 Linearizable Reads 194
18.8 Hybrid Logical Clocks 195
18.9 Voter and Learner Roles 196
18.10 Replication Factor 197
18.11 State Transfer 197
18.12 WAL Compaction 197
18.13 Failure Scenarios 197
18.14 Summary 199
19 The Actor Model and Concurrency 200
19.1 The Actor Model 200
19.2 Nixie: The Actor Framework 200
19.3 Consistent-Hash Routing 202
19.4 Kahuna's Actor Hierarchy 202
19.5 KeyValueActor 203
19.6 Ephemeral vs Persistent Actors 205
19.7 LockActor 205
19.8 SequenceActor 206
19.9 PartitionWriteAggregatorActor 206
19.10 BackgroundWriterActor 206
19.11 The Single-Threaded Invariant 206
19.12 Failure Scenarios 207
19.13 Summary 208
20 MVCC and Snapshot Isolation 209

20.1 The Version Chain ... 209
20.2 Write Intents ... 211
20.3 MVCC Entries: Read-Your-Writes ... 212
20.4 Snapshot Reads ... 212
20.5 The Snapshot Floor ... 213
20.6 Revision Pruning ... 214
20.7 Safe Timestamp ... 215
20.8 Prepared Intents and MVCC ... 215
20.9 Failure Scenarios ... 216
20.10 Summary ... 216
21 Transaction Internals ... 218
21.1 Transaction Sessions ... 218
21.2 Admission Control ... 218
21.3 The Two-Phase Commit Protocol ... 219
21.4 Durable Transaction Finalization ... 220
21.5 Deferred Settlement ... 222
21.6 Conflict Detection ... 224
21.7 Script Transactions ... 225
21.8 Transaction Record Store ... 225
21.9 Completion Receipts ... 225
21.10 Session Reaping ... 226
21.11 Transaction Recovery ... 226
21.12 Outcome Contract ... 227
21.13 Failure Scenarios ... 228
21.14 Summary ... 229
22 Lock and Sequencer Internals ... 230
22.1 Lock Entry ... 230
22.2 The Lock Pipeline ... 230
22.3 Fencing Token Monotonicity ... 232
22.4 Lock Replication ... 233
22.5 Lock Restoration ... 233
22.6 The Unflushed Writes Overlay ... 234
22.7 Lock Collection ... 234
22.8 Sequence Internals ... 234
22.9 Failure Scenarios ... 238
22.10 Summary ... 239
23 Persistence and Storage Engines ... 241
23.1 The Persistence Interface ... 241
23.2 The Memory Backend ... 242
23.3 The SQLite Backend ... 242
23.4 The RocksDB Backend ... 243
23.5 The Background Writer ... 245
23.6 The Unflushed Overlay ... 247
23.7 Flush Notification ... 248
23.8 IO Scheduling and Backpressure ... 248
23.9 Cache Eviction ... 249

23.10 Failure Scenarios 250
24 Recovery and Fault Tolerance 252
24.1 The Startup Sequence 252
24.2 Raft Log Replay 252
24.3 The KeyValueRestorer 253
24.4 The Lock Restorer 253
24.5 Leader Change Recovery 254
24.6 Transaction Recovery 254
24.7 Whole-Partition State Transfer 255
24.8 Snapshot Holds and Revision Safety 256
24.9 PITR Backup and Restore 256
24.10 Durability Tracking 257
24.11 Failure Scenarios 257
25 Deployment and Cluster Sizing 259
25.1 Standalone Mode 259
25.2 Multi-Node Cluster 260
25.3 Docker Compose Deployment 261
25.4 Kubernetes Deployment 263
25.5 HTTPS Configuration 263
25.6 Cluster Sizing 265
25.7 Replication Factor 266
25.8 Storage Path Configuration 267
25.9 Failure Scenarios 268
25.10 Production Checklist 268
26 Cluster Membership and Scaling 270
26.1 The Committed Roster 270
26.2 Member Roles 270
26.3 Adding a Node 271
26.4 Learner Promotion 271
26.5 Removing a Node 272
26.6 Rolling Restarts 273
26.7 The SWIM Failure Detector 274
26.8 Auto-Rejoin 275
26.9 REST Endpoints 275
26.10 Configuration Flags 276
26.11 Failure Scenarios 276
27 Range Management and Rebalancing 278
27.1 Key-Range Registration 278
27.2 Monitoring Ranges 279
27.3 Automatic Splitting 279
27.4 Automatic Merging 280
27.5 Manual Split and Merge 281
27.6 Leader Balancing 282
27.7 Replica Placement 284
27.8 Per-Partition Replication Factor 286
27.9 The Kahuna-Side Placement Coordinator 286

27.10 Configuration Reference 287
27.11 Failure Scenarios 290
28 Backup and Point-in-Time Recovery 292
28.1 Backup Types 292
28.2 Backup Storage 292
28.3 Taking a Full Backup 293
28.4 Taking an Incremental Backup 294
28.5 Coordinated Cluster Snapshots 295
28.6 The Backup Manifest 296
28.7 Manifest Authentication 296
28.8 Backup Chains 297
28.9 Listing Backups 297
28.10 Restoring to a Point in Time 297
28.11 PITR Bootstrap 299
28.12 Retention and Garbage Collection 299
28.13 REST Endpoints 300
28.14 Backup Outcomes 301
28.15 Backup Storage Abstraction 302
28.16 Configuration Reference 303
28.17 Failure Scenarios 303
29 Observability and Performance Tuning 305
29.1 Metrics Architecture 305
29.2 Status Dashboard 305
29.3 Key Metrics 306
29.4 The Benchmark Tool 316
29.5 IO Scheduler Tuning 319
29.6 Write Coalescing Tuning 319
29.7 Cache Configuration 320
29.8 RocksDB Shared Memory 321
29.9 Transaction Admission Control 322
29.10 Tuning Recipes 323
29.11 Failure Scenarios 324
30 Testing Distributed Systems 325
30.1 Embedded Cluster Tests 325
30.2 Jepsen Testing 326
30.3 What Jepsen Found 329
30.4 Interpreting Jepsen Results 333
30.5 Why Conventional Tests Are Not Enough 333
30.6 Configuration Reference 334
30.7 Failure Scenarios 335
31 Appendix A: Script Language Reference 336
31.1 Data Types 336
31.2 Variables 336
31.3 Key-Value Commands 337
31.4 SET Flags 338
31.5 Operators 338

31.6 Control Flow 339
31.7 Transaction Blocks 340
31.8 Flow Control Statements 341
31.9 Built-in Functions 341
31.10 Reserved Words 343
31.11 Operator Precedence 343
31.12 Placeholders 343
31.13 Complete Example 344
32 Appendix B: Configuration Reference 345
32.1 Server and Networking 345
32.2 Storage and WAL 345
32.3 RocksDB 346
32.4 Cluster and Node Identity 347
32.5 Actor Workers 347
32.6 IO Scheduler 348
32.7 Raft Executor 348
32.8 Raft Consensus Timers 349
32.9 Raft WAL and Compaction 351
32.10 Raft gRPC Transport 352
32.11 Raft WAL Shard Tuning 354
32.12 Raft Snapshot Transfer 355
32.13 Raft Backfill 357
32.14 Membership: Learner Promotion, Gossip, and SWIM 357
32.15 Quiescence 358
32.16 Leader Balancer 359
32.17 Slow Node Avoidance 361
32.18 Replica Placement 361
32.19 Cache 363
32.20 Write Batching 363
32.21 Transactions and Admission Control 365
32.22 Sequences 368
32.23 Persistence 369
32.24 Backup and Point-in-Time Recovery 371
32.25 Key-Range Split and Merge 373
32.26 Internal-Only Parameters 374
33 Appendix C: CLI Command Reference 375
33.1 Global Options 375
33.2 Key-Value Commands 375
33.3 Lock Commands 376
33.4 Sequence Commands 377
33.5 Script Commands 378
33.6 Cluster Commands 379
33.7 Range Commands 379
33.8 Backup Commands 380
33.9 Interactive Shell Commands 381
34 Appendix D: Error Codes and Response Types 382

34.1 Key-Value Responses 382
34.2 Lock Responses 383
34.3 Sequence Responses 384
34.4 Transaction Status 385
34.5 Backup Outcomes 386
34.6 Cluster Leave Outcomes 388
34.7 Split Range Status 388
34.8 Merge Ranges Status 389
34.9 Register Key Range Status 390
34.10 Remove Key Range Status 390
34.11 Common Patterns 391
35 Appendix E: Comparison with etcd, ZooKeeper, Consul, and Redis 393
35.1 Feature Matrix 394
35.2 Architecture Comparison 395
35.3 Distributed Locks 396
35.4 Transactions 397
35.5 Sharding and Scalability 397
35.6 Use-Case Fit 398
35.7 Summary 398
36 Appendix F: Glossary 399

To Gaby and Dani, for every late night and every early morning you gave me so I could design, architect, build, and test this system. This book exists because you believed in it before anyone else did.

1 Preface

Kahuna started as a question: what would it take to build a coordination system that a working developer could pick up in an afternoon and run in production the next week? Not a research prototype. Not a system that requires a PhD to operate. A system with locks, key-value storage, transactions, and sequences, all backed by consensus, all accessible through a simple API.

That question turned into code, and the code turned into something that needed a book.

1.1 Who This Book Is For

This book is for developers and operators who build and run distributed systems. You do not need prior experience with consensus protocols, MVCC, or actor models. The book starts with practical usage (how to store a value, acquire a lock, run a transaction) and progresses to internal mechanisms (how Raft replicates entries, how the two-phase commit protocol works, how the WAL is structured).

If you are evaluating Kahuna for a project, Part I (Chapters 1 through 7) gives you everything you need to make that decision. If you are already running Kahuna and need to understand its behavior under failure, Part III (Chapters 14 through 23) explains the internals. If you are operating a production cluster, Part IV (Chapters 24 through 29) covers deployment, scaling, backup, observability, and testing.

1.2 How This Book Is Organized

The book is divided into four parts:

Part I: Using Kahuna (Chapters 1 through 13). These chapters teach you how to use Kahuna's three primitives (key-value store, distributed locks, distributed sequences), how to write transactions and scripts, and how to apply these tools to common patterns such as leader election, idempotent operations, and transactional workflows.

Part II: Architecture (Chapters 14 and 15). These two chapters provide the map. Chapter 14 introduces the component hierarchy. Chapter 15 traces a single request through every layer from client to disk and back. Every subsequent chapter references these two.

Part III: Mechanisms (Chapters 16 through 23). Each chapter explains one internal subsystem: partitioning, consensus, the actor model, MVCC, transactions, locks and sequences, persistence, and recovery. These chapters are independent of each other. Read them in any order after Part II.

Part IV: Operations (Chapters 24 through 29). These chapters cover deployment, cluster membership, range management, backup and point-in-time recovery, observability and tuning, and testing. They combine the external knowledge from Part I with the internal knowledge from Part III.

The appendices provide reference material: the script language syntax, the configuration reference, the CLI command reference, error codes, a comparison with other systems, and a glossary.

1.3 Conventions

Code examples use C# for client code and Kahuna Script for server-side scripts. Command-line examples show the `kahuna-cli` interactive shell or the `kahuna-server` flags. Configuration flags appear with their default values in tables throughout the text.

Each internal chapter ends with a "Failure Scenarios" section that describes what goes wrong when specific conditions occur. These sections are based on real behavior observed through the Jepsen test suite.

1.4 Acknowledgments

Building a distributed system is not a solo effort, even when it feels like one at 3 AM. The Jepsen test suite found real bugs that made Kahuna better. The open-source community around .NET provided the runtime and tooling that made the implementation possible.

And thank you for picking up this book. I hope it helps you build something reliable.

Andres Gutierrez

2 What Is Kahuna?

Imagine you are running three copies of a payment service behind a load balancer. A customer submits an order. The request lands on node A, which checks the customer's balance, deducts the amount, and records the charge. But what happens if the same request, retried after a timeout, lands on node B? Node B checks the balance, sees the old value (the deduction hasn't propagated yet), and charges the customer a second time.

This is not a hypothetical scenario. It is one of the most common bugs in distributed systems: two processes operating on shared state without coordination. The fix is not more careful coding. The fix is a coordination primitive (a lock, a conditional write, or an atomic transaction) that both processes agree to respect.

Building those coordination primitives yourself is hard. Getting them right under network partitions, process crashes, and clock drift is harder still. This is the problem that Kahuna solves.

2.1 The Coordination Problem

When an application runs on a single machine, coordination is straightforward. A thread acquires a mutex, does its work, and releases the mutex. The operating system guarantees that only one thread holds the mutex at a time. The application doesn't need to think about what happens if the CPU loses power mid-operation, because the OS handles that, too.

Distributed systems don't have a single machine. They have multiple processes running on multiple nodes, connected by a network that can delay, reorder, or drop messages. There is no shared memory. There is no global clock. There is no single entity that can decide "this process goes first."

And yet, distributed applications need all of the things that a single-machine application takes for granted:

- **Mutual exclusion.** Only one worker should process a given job at a time.
- **Consistent state.** If two clients update the same key, every subsequent read should see the same value, regardless of which node handles the read.
- **Atomic updates.** When a transaction writes to three keys, either all three writes take effect or none of them do.
- **Ordered identifiers.** Order numbers, event IDs, and log sequence numbers need to be unique and monotonically increasing across the entire cluster.

Each of these requirements is a coordination problem. Solving any one of them requires agreement among multiple nodes. Solving them correctly, under failures, requires a consensus protocol: a formal mechanism by which a group of nodes agrees on a single value or a single order of operations, even when some nodes are unreachable.

Consensus protocols are well-studied. Raft, Paxos, and Zab have been implemented in production systems for over a decade. But implementing a consensus protocol is not the same as having a coordination system. A coordination system wraps the protocol in a usable API: locks you can acquire, keys you can read and write, transactions you can commit, and sequences you can allocate from.

That is what Kahuna provides.

2.2 What Kahuna Is

Kahuna is an open-source distributed coordination system built in C# on .NET. It provides three core primitives:

1. **A distributed key-value store.** Store and retrieve data across a cluster of nodes, with strong consistency, multi-version concurrency control (MVCC), and support for transactions.
2. **Distributed locks.** Acquire time-bounded locks (leases) on named resources, with monotonically increasing fencing tokens that prevent stale lock holders from corrupting data.
3. **A distributed sequencer.** Generate globally unique, monotonically increasing numbers without a single-node bottleneck.

These three primitives share a single cluster, a single consensus layer (Raft), and a single operational surface. You deploy one system instead of three.

Kahuna organizes data into partitions. Each partition is an independent unit with its own Raft group: its own leader, its own replicas, and its own log. This design allows Kahuna to scale horizontally: more partitions means more throughput, because different partitions can be led by different nodes.

Every write in Kahuna goes through Raft consensus. This means that once a write is acknowledged, it is durable on a majority of replicas. If a node fails, another node can take over as leader for that partition without losing committed data.

> *Kahuna* is a Hawaiian word that refers to an expert in any field. Historically, it has been used to refer to doctors, surgeons, priests, and sorcerers.

2.3 The Three Primitives

2.3.1 Key-Value Storage

At its core, Kahuna is a distributed key-value store. You write a value under a string key, and you can read it back from any node in the cluster. Kahuna routes the key to the correct partition, forwards the request to the partition's leader, and replicates the write through Raft before acknowledging it.

But Kahuna's key-value store goes well beyond simple get and set. It provides:

- **Revisions.** Every write to a key increments a revision number. You can read the current revision, and you can use it for compare-and-swap operations: "set this key to X, but only if its current revision is 7." If another writer changed the key in the meantime, the operation fails instead of silently overwriting.
- **Conditional writes.** Beyond compare-and-swap on revisions, you can set a key only if it doesn't exist yet, only if it already exists, or only if its current value matches a specific value.
- **Time-to-live (TTL).** Keys can expire after a specified duration. This is useful for session tokens, ephemeral state, and presence detection.
- **Durability levels.** Keys can be *persistent* (survive restarts, replicated via Raft) or *ephemeral* (fast, in-memory only, lost on restart). You choose per key.

- **Range queries.** You can scan keys by prefix or by range, with pagination support for large result sets.
- **Transactions.** You can read and write multiple keys atomically, with either optimistic or pessimistic concurrency control.

Here is what a simple key-value operation looks like in C#:

```
var client = new KahunaClient("http://localhost:2070");

// Set a key
await client.SetKeyValue("user:1001:name", "Alice");

// Get a key
KahunaKeyValue result = await client.GetKeyValue("user:1001:name");
Console.WriteLine(result.ValueAsString()); // "Alice"
Console.WriteLine(result.Revision);        // 1
```

The key `user:1001:name` is routed to a partition based on its prefix. The write goes through Raft consensus. The read goes to the partition's leader, which confirms it is still the leader before responding.

2.3.2 Distributed Locks

A distributed lock is a lease: a time-bounded claim on a named resource. While a process holds the lock, no other process can acquire it. When the lease expires (because the holder crashed, or because it simply didn't renew in time), the lock becomes available to other processes.

Leases alone are not enough, though. Consider this scenario:

1. Process A acquires a lock with a 10-second lease.
2. Process A starts a long operation.
3. The lease expires because the operation takes longer than expected.
4. Process B acquires the lock.
5. Process A finishes the operation and writes its result, but it no longer holds the lock.

Process A just wrote to a shared resource without holding the lock. This is the *stale lock holder* problem, and it can corrupt data even when the lock implementation itself is correct.

Kahuna prevents this with **fencing tokens**. Every time a lock is acquired, Kahuna assigns it a monotonically increasing integer. Process A might get fencing token 5. When the lease expires and process B acquires the lock, process B gets fencing token 6. If the downstream system checks the fencing token before accepting a write, it can reject process A's stale write (token 5) because it has already seen a higher token (6).

```
// Acquire a lock with a 10-second lease
await using KahunaLock lockHandle =
    await client.GetOrCreateLock(
        "resource:payment:1001",
        expiresMs: 10000);

if (lockHandle.IsAcquired)
{
```

```
        Console.WriteLine($"Lock acquired, fencing token:
{lockHandle.FencingToken}");
    // Do the protected work
}
// Lock is released automatically when lockHandle is disposed
```

The `await using` pattern ensures the lock is released even if the code throws an exception. If the process crashes instead of throwing, the lease expires and the lock becomes available.

2.3.3 Distributed Sequencer

Many applications need unique identifiers: order numbers, event IDs, invoice numbers, log sequence numbers. These identifiers must be:

- **Unique** across the entire cluster.
- **Monotonically increasing.** A higher number always means a later event.
- **Available without a single-node bottleneck.** The system should not funnel all ID generation through one machine.

Database auto-increment columns satisfy the first two requirements, but not the third. UUIDs satisfy the first and third, but not the second (they are not sortable by generation time in their standard form). Snowflake IDs satisfy all three, but require careful clock management and machine ID assignment.

Kahuna's sequencer takes a different approach. It uses **block-based allocation**: a node reserves a block of sequence values (for example, 1000 at a time) with a single Raft commit. It then serves individual values from that block without any further consensus operations. When the block is exhausted, it reserves another one.

This design means that one Raft commit can produce hundreds or thousands of unique values. The trade-off is **gaps**: if a node crashes with 300 values remaining in its block, those 300 values are never used. The sequence jumps from wherever it was to the next block boundary. Values are guaranteed to be unique, but they are not guaranteed to be gap-free.

```
// Create a sequence
await client.CreateSequence("order-ids");

// Allocate the next value
KahunaSequence seq = await client.NextSequenceValue("order-ids");
Console.WriteLine($"Order ID: {seq.CurrentValue}"); // 1

// Allocate a batch of values
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids", 100);
Console.WriteLine($"Range: {range.Start} to {range.End}"); // e.g. 2 to 101
```

For most applications, gaps are acceptable. If you need gap-free sequences (for example, invoice numbers in jurisdictions that require them), you can set the block size to 1, which forces a Raft commit per value. This eliminates gaps but reduces throughput.

2.4 When to Use Kahuna

Kahuna is designed for **coordination workloads**: operations where multiple processes need to agree on shared state. Here are the scenarios where it fits well:

Distributed locking. Protecting shared resources across services: job processing queues, rate limiters, payment flows, leader election, and any workflow where "only one at a time" is a correctness requirement.

Coordination metadata. Storing configuration that multiple services read and update: feature flags, service registrations, routing tables, cluster membership records.

Transactional updates to small key sets. Atomically updating a handful of related keys: account balances, inventory counts, state machines. Kahuna's transactions are designed for small working sets (tens of keys), not bulk data operations.

Unique ID generation. Producing globally unique, monotonically increasing identifiers without a centralized bottleneck.

Session and presence management. Tracking which services or users are currently active, using ephemeral keys with TTL.

2.5 When Not to Use Kahuna

Kahuna is not a general-purpose database, and using it as one will lead to poor results.

Bulk data storage. If you need to store millions of rows with complex queries, use a database (PostgreSQL, CockroachDB, CamusDB). Kahuna stores coordination metadata and small working sets, not application data.

Message queuing. If you need publish-subscribe, message routing, or stream processing, use a message broker (Kafka, RabbitMQ, NATS). Kahuna can coordinate producers and consumers, but it is not a message transport.

High-throughput caching. For large-scale cache workloads with millions of reads per second, use Redis or Memcached. Kahuna can serve as a cache for low-to-medium workloads (it supports TTL, ephemeral durability, and no-revision writes), but its strong consistency and Raft consensus path add overhead that dedicated cache systems avoid.

Analytics and reporting. If you need aggregations, joins, or full-text search, use a database or search engine. Kahuna's query model is key-based, not relational.

The simplest rule: if the data is *coordination state* (who holds the lock, what is the current sequence number, what is the latest configuration version), Kahuna is a good fit. If the data is *application state* (user profiles, product catalogs, order histories), use a database.

2.6 How Kahuna Compares

Several established systems provide overlapping capabilities. Understanding the differences helps you choose the right tool.

2.6.1 etcd

etcd is a distributed key-value store built on Raft. It is the coordination backbone of Kubernetes. Like Kahuna, it provides linearizable reads and writes, leases, and watches.

Differences: etcd does not provide a built-in sequencer, a transaction scripting language, or MVCC-based snapshot isolation for multi-key transactions. etcd's lock API (in the `concurrency` package) does not return a fencing token directly. A developer can read the key's `CreateRevision` and use it as a fencing value, but the application must implement that pattern

manually. Kahuna returns a fencing token as an explicit field on every lock acquisition. etcd uses a single Raft group, so every node holds every key. This limits etcd to small clusters (typically 3 or 5 nodes) and a total data size of a few gigabytes. Kahuna uses multiple Raft groups (one per partition) and a configurable replication factor. Each partition is replicated to a subset of nodes, not all of them. This means a Kahuna cluster can grow well beyond 5 or 9 nodes: keys and locks are distributed across many partitions, and each partition only consumes resources on its replicas. etcd is written in Go; Kahuna is written in C#.

2.6.2 Apache ZooKeeper

ZooKeeper is the original distributed coordination service. It provides a hierarchical namespace (similar to a filesystem), ephemeral nodes, watches, and sequential nodes for ordering.

Differences: ZooKeeper's data model is tree-structured (znodes), while Kahuna's is flat (key-value). ZooKeeper does not provide multi-key transactions, a scripting language, or a built-in sequencer with block allocation. ZooKeeper's lock recipe uses sequential ephemeral znodes. The sequence number on the znode (or the znode's `czxid`) is monotonically increasing and can serve as a fencing token, but the application must read and propagate it manually. Like etcd, ZooKeeper uses a single consensus group (Zab), so every node holds every znode. This limits cluster size and total data capacity. Kahuna distributes data across partitions with per-partition replication, which allows it to scale to larger clusters. ZooKeeper is written in Java; Kahuna is written in C#.

2.6.3 HashiCorp Consul

Consul provides service discovery, health checking, and a key-value store. Its KV store supports CAS operations and distributed locks via sessions.

Differences: Consul is primarily a service mesh and discovery tool; its KV store is a secondary feature. Consul does not provide multi-key transactions, MVCC, snapshot isolation, or a sequencer. Consul's KV entries carry a `ModifyIndex` that increases monotonically, so a lock holder can use it as a fencing value, but the application must implement that pattern. Consul is written in Go; Kahuna is written in C#.

2.6.4 Redis

Redis is an in-memory data structure store. With Redlock (a distributed lock algorithm), it provides distributed locking. Redis Cluster provides horizontal scaling with hash-slot partitioning.

Differences: Redis prioritizes throughput and low latency over strong consistency. Redlock has been the subject of academic debate regarding its safety guarantees under certain failure modes. Kahuna's locks are built on Raft consensus with linearizable operations and fencing tokens. Redis does not provide multi-key transactions with snapshot isolation across partitions. Redis is written in C; Kahuna is written in C#.

2.6.5 Summary

Feature	Kahuna	etcd	ZooKeeper	Consul	Redis
Distributed KV store	Yes	Yes	Yes (tree)	Yes	Yes
Strong consistency	Yes (Raft)	Yes (Raft)	Yes (Zab)	Yes (Raft)	No (async replication)
Distributed locks	Yes	Yes (leases)	Yes (ephemeral nodes)	Yes (sessions)	Yes (Redlock)
Fencing tokens	Yes (built-in)	Manual (via revision)	Manual (via czxid)	Manual (via ModifyIndex)	No
Multi-key transactions	Yes (2PC + MVCC)	Yes (mini-transactions)	No	No	No
Snapshot isolation	Yes	No	No	No	No
Distributed sequencer	Yes	No	Yes (sequential nodes)	No	No
Transaction scripting	Yes	No	No	No	Yes (Lua)
Partitioned (multi-Raft-group)	Yes	No (single group)	No (single group)	No (single group)	Yes (hash slots)
Per-partition replication factor	Yes	No (all nodes)	No (all nodes)	No (all nodes)	Yes (per shard)
Horizontal data scaling	Yes	Limited	Limited	Limited	Yes
Primary language	C#	Go	Java	Go	C

This comparison is factual, not promotional. Each system makes different trade-offs, and the right choice depends on your workload, your team's expertise, and your operational requirements.

2.7 Architecture at a Glance

Before diving into the details in later chapters, here is a brief sketch of how Kahuna is organized internally. You don't need to understand all of this now. Part III of the book covers each component in depth.

A Kahuna cluster consists of multiple **nodes**. Each node runs the same server binary and can serve any request by routing it to the appropriate partition leader.

Data is organized into **partitions**. Each partition is a range of keys managed by its own independent **Raft group**. A Raft group has a **leader** (which handles writes and linearizable reads) and **followers** (which replicate the leader's log). If the leader fails, the remaining replicas elect a new leader.

When a client sends a request, the receiving node determines which partition owns the key (based on hashing or range mapping), finds the leader for that partition, and forwards the request. If the receiving node *is* the leader, it handles the request locally. The leader proposes the write to Raft, waits for a majority of replicas to acknowledge, and then responds to the client.

```
Client
  → Node (any node can receive)
  → Partition resolution (which partition owns this key?)
  → Leader (forward to leader if this node isn't it)
  → Raft consensus (replicate to majority)
  → Acknowledge to client
```

Kahuna uses **Hybrid Logical Clocks (HLC)** for ordering events across nodes. HLC combines a physical timestamp with a logical counter to produce globally unique, causally ordered timestamps without requiring synchronized clocks. Every committed write gets an HLC timestamp that establishes its position in the total order of operations.

The key-value store uses **Multi-Version Concurrency Control (MVCC)**: each key retains multiple versions, indexed by their HLC timestamp. This allows transactions to read a consistent snapshot of the data at a specific point in time, even while other transactions are writing to the same keys.

2.8 A First Look at the Client SDK

To close this chapter, here is a complete C# program that connects to a Kahuna cluster, uses all three primitives, and handles basic errors. Don't worry about understanding every detail. The following chapters will explain each operation thoroughly.

```
using Kahuna.Client;

// Connect to a Kahuna cluster
var client = new KahunaClient("http://localhost:2070");

// --- Key-Value ---
// Store a value
await client.SetKeyValue("config:feature:dark-mode", "enabled");

// Read it back
var kv = await client.GetKeyValue("config:feature:dark-mode");
Console.WriteLine($"Key: {kv.Key}, Value: {kv.ValueAsString()}, Revision:
{kv.Revision}");
```

```
// Conditional write: update only if the revision hasn't changed
var updated = await client.TryCompareRevisionAndSetKeyValue(
    "config:feature:dark-mode",
    "disabled",
    kv.Revision);
Console.WriteLine($"Update succeeded: {updated.Success}");

// --- Distributed Lock ---
// Acquire a lock with a 5-second lease
await using var lockHandle = await client.GetOrCreateLock(
    "job:nightly-report",
    expiresMs: 5000);

if (lockHandle.IsAcquired)
{
            Console.WriteLine($"Acquired   lock   with   fencing   token:
{lockHandle.FencingToken}");
    // Protected work goes here
}

// --- Sequencer ---
// Create a sequence and allocate values
await client.CreateSequence("invoice-numbers");
var next = await client.NextSequenceValue("invoice-numbers");
Console.WriteLine($"Next invoice number: {next.CurrentValue}");
```

This program doesn't handle retries, leader changes, or transaction conflicts yet. Chapter 13 covers all of those. For now, the point is to see the shape of the API: connect, operate, and let the cluster handle replication and consistency.

2.9 What's Ahead

The rest of this book is organized into four parts:

Part I, Using Kahuna (Chapters 2 to 7), walks through each primitive in detail. You'll start a cluster, learn the full key-value API, master distributed locks and fencing tokens, use the sequencer, write transactions, and learn the Kahuna scripting language.

Part II, Building Distributed Applications (Chapters 8 to 13), applies these primitives to real engineering problems: leader election, idempotent workflows, service coordination, transactional pipelines, and failure handling.

Part III, Understanding Kahuna Internals (Chapters 14 to 23), opens the hood. You'll trace a request from the client through routing, consensus, the actor model, MVCC, the two-phase commit protocol, persistence, and recovery. Every chapter connects to actual source code.

Part IV, Operating and Extending Kahuna (Chapters 24 to 29), covers production operations: deployment, cluster membership, range management, backup and point-in-time recovery, observability, performance tuning, and Jepsen testing.

You don't need to read the book linearly. Parts I and II are designed for developers who want to use Kahuna. Parts III and IV are for those who want to understand or operate it. But the concepts build on each other, and the book is designed to be read in order if you have the time.

Let's begin. The next chapter gets a cluster running on your machine.

3 Getting Started

This chapter takes you from zero to a working Kahuna environment. By the end, you will have a running cluster, a command-line client, and a C# application that reads, writes, locks, and generates sequence numbers. You will also crash a node and see the cluster recover on its own.

3.1 Prerequisites

Kahuna runs on .NET. You need:

- **.NET 10 SDK** (or later). Download it from dotnet.microsoft.com.
- **Docker** (optional, for multi-node clusters). Any recent version of Docker Desktop or Docker Engine works.

Verify your .NET installation:

```
dotnet --version
```

If this prints a version number of 10.0 or higher, you are ready.

3.2 Installing the Server

Kahuna is distributed as a .NET global tool. Install it with a single command:

```
dotnet tool install -g Kahuna.Server
```

This places the `kahuna-server` command on your PATH. Run it:

```
kahuna-server
```

With no arguments, the server starts a standalone node. It listens for HTTP connections on port 2070. The output shows the storage paths and the ports the node is using.

By default, the node stores its key-value data and Raft write-ahead log under your user data directory:

- **Linux/macOS:** `~/.local/share/kahuna`
- **Windows:** `%LOCALAPPDATA%\kahuna`

Both paths are printed at startup. To store data in a different location, set the `KAHUNA_HOME` environment variable, or pass `--storage-path` and `--wal-path` explicitly.

The standalone node elects itself as the leader for every partition. It does not need any peer configuration. This makes it the fastest way to start experimenting.

3.2.1 Storage Backends

Kahuna supports three storage backends:

Backend	Flag	Persistence	Use case
RocksDB	`--storage rocksdb`	Durable	Production (default)
SQLite	`--storage sqlite`	Durable	Alternative persistent backend
Memory	`--storage memory`	Ephemeral	Testing and prototyping

The default is RocksDB. For quick experiments where you do not need data to survive a restart, use the memory backend:

```
kahuna-server --storage memory --wal-storage memory
```

3.2.2 HTTPS

The standalone node serves HTTP only. To enable HTTPS, supply a certificate:

```
kahuna-server \
  --https-ports 2071 \
  --https-certificate /path/to/certificate.pfx
```

For local development, a self-signed certificate is sufficient. The client SDK has an option to skip certificate validation (covered later in this chapter).

3.3 Installing the Command-Line Client

The CLI is a separate .NET global tool:

```
dotnet tool install -g Kahuna.Control
```

This installs the `kahuna-cli` command. By default, it connects to `http://localhost:2070`, which matches the standalone server's default port.

3.4 First Operations with the CLI

Start a standalone server in one terminal window. Open a second terminal and try the following commands.

3.4.1 Setting and Getting a Key

Write a value:

```
kahuna-cli --set "users/alice" --value '{"name":"Alice","role":"admin"}'
```

Read it back:

```
kahuna-cli --get "users/alice"
```

The output shows the value and metadata, including the revision number. Every write increments the revision.

3.4.2 Setting a Key with an Expiry

Keys can have a time-to-live (TTL). The `--expires` flag takes a value in milliseconds:

```
kahuna-cli --set "session/abc123" --value "active" --expires 30000
```

This key expires after 30 seconds. After that, a `--get` on this key returns nothing.

3.4.3 Acquiring and Releasing a Lock

Acquire a lock named `jobs/send-email`. The `--expires` flag specifies the lease duration in milliseconds:

```
kahuna-cli --lock "jobs/send-email" --owner "worker-1" --expires 10000
```

The output confirms the lock is acquired. It also shows the fencing token, a monotonically increasing number. Each time the lock changes hands, the fencing token increases.

Release the lock:

```
kahuna-cli --unlock "jobs/send-email" --owner "worker-1"
```

Only the owner that acquired the lock can release it.

3.4.4 Creating and Using a Sequence

Create a sequence named `order-ids`:

```
kahuna-cli --create-sequence "order-ids"
```

Retrieve the next value:

```
kahuna-cli --next-sequence "order-ids"
```

Each call returns a new, unique, monotonically increasing number. You can also reserve a range of values at once:

```
kahuna-cli --reserve-sequence "order-ids" --count 10
```

This reserves 10 values and returns the start and end of the range.

3.4.5 Output Formats

By default, the CLI prints results in a human-readable console format. For scripting, use JSON output:

```
kahuna-cli --get "users/alice" --format json
```

3.5 Running a Three-Node Cluster

A single node is fine for development. For testing replication and fault tolerance, you need multiple nodes. Kahuna provides two ways to run a local cluster: Docker Compose and shell scripts.

3.5.1 Option 1: Docker Compose

The Kahuna repository includes a Docker Compose file at `docker/local.yml`. This file defines a three-node cluster on a Docker bridge network.

Clone the repository and start the cluster:

```
git clone https://github.com/kahunakv/kahuna.git
cd kahuna
docker compose -f docker/local.yml up -d
```

The three nodes expose these ports on your host:

Node	HTTP Port	HTTPS/Raft Port
kahuna1	8081	8082
kahuna2	8083	8084
kahuna3	8085	8086

Each node has its own storage volume. Each node's `--initial-cluster` flag lists the other two nodes' Raft endpoints. This is how nodes discover each other at startup.

Verify the cluster is running:

```
docker compose -f docker/local.yml ps
```

All three containers should show a healthy status.

Connect the CLI to any node in the cluster. For the Docker cluster, the HTTPS ports use a self-signed certificate, so add `--insecure` to skip validation:

```
kahuna-cli -c "https://localhost:8082" --insecure --set "test/key" --value
"hello"
kahuna-cli -c "https://localhost:8084" --insecure --get "test/key"
```

Notice that you write to node 1 and read from node 2. Kahuna routes the request to the correct partition leader internally, and the value is replicated through Raft. The read returns the same value regardless of which node you connect to.

3.5.2 Option 2: Local Shell Script

If you prefer to run the cluster outside Docker, the repository includes a script that starts three nodes as local processes:

```
cd kahuna
./scripts/run-cluster.sh
```

This script builds the server from source, then starts three nodes on localhost. The port layout is the same as the Docker cluster:

Node	HTTP	HTTPS/Raft
kahuna1	8081	8082
kahuna2	8083	8084
kahuna3	8085	8086

Press Ctrl+C to stop all three nodes.

For a standalone node from source, use:

```
./scripts/run-standalone.sh
```

Both scripts accept environment variables for customization:

Variable	Default	Description
`KAHUNA_STORAGE`	`rocksdb`	Storage backend (`rocksdb` or `memory`)
`KAHUNA_PARTITIONS`	`3`	Initial partition count
`KAHUNA_DATA_DIR`	`/tmp/kahuna-cluster`	Data directory (rocksdb only)

For example, to run a fully ephemeral cluster:

```
KAHUNA_STORAGE=memory ./scripts/run-cluster.sh
```

3.5.3 How Cluster Discovery Works

When a node starts, it needs to know about its peers. Kahuna uses static discovery: each node's `--initial-cluster` flag lists the Raft endpoints of the other nodes. The node contacts those peers and forms a Raft group for each partition.

The key configuration flags for clustering are:

Flag	Purpose
`--raft-nodename`	Unique name for this node
`--raft-nodeid`	Unique integer ID for this node
`--raft-host`	Host address for Raft communication
`--raft-port`	Port for Raft communication
`--initial-cluster`	Raft endpoints of the other nodes
`--initial-cluster-partitions`	Number of partitions (default: 3)

For example, to manually start node 1 of a three-node cluster:

```
kahuna-server \
  --raft-nodename kahuna1 \
  --raft-nodeid 1 \
  --raft-host 127.0.0.1 \
  --raft-port 8082 \
  --http-ports 8081 \
  --https-ports 8082 \
  --https-certificate certificate.pfx \
  --initial-cluster 127.0.0.1:8084 127.0.0.1:8086 \
  --initial-cluster-partitions 3
```

Nodes 2 and 3 use the same pattern, with their own node name, ID, ports, and peer list. Each node lists only the other nodes in `--initial-cluster`, not itself.

3.6 Connecting from C#

The Kahuna .NET client library communicates with the server over gRPC. Install it from NuGet:

```
dotnet add package Kahuna.Client
```

3.6.1 Creating a Client

For a single endpoint:

```
using Kahuna.Client;

var client = new KahunaClient("http://localhost:2070");
```

For a cluster with multiple endpoints, pass an array of URLs:

```
var client = new KahunaClient(new[]
{
    "https://localhost:8082",
    "https://localhost:8084",
    "https://localhost:8086"
});
```

The client distributes requests across the endpoints using round-robin. Kahuna routes each request to the correct partition leader internally, so any endpoint can handle any request.

3.6.2 Skipping Certificate Validation

When connecting to a cluster that uses self-signed certificates (such as the local Docker cluster), configure the client to skip TLS validation:

```
var client = new KahunaClient(
     new[] { "https://localhost:8082", "https://localhost:8084", "https://
localhost:8086" },
    options: new KahunaOptions { AllowInsecureCertificateValidation = true }
);
```

Do not use this setting in production.

3.6.3 Key-Value Operations

Write and read a value:

```
KahunaKeyValue setResult = await client.SetKeyValue("users/alice", "admin");
Console.WriteLine($"Revision: {setResult.Revision}");

KahunaKeyValue getResult = await client.GetKeyValue("users/alice");
Console.WriteLine($"Value: {getResult.ValueAsString()}");
Console.WriteLine($"Revision: {getResult.Revision}");
```

Every successful write returns a `KahunaKeyValue` object. The `Success` property tells you whether the write went through. The `Revision` property is a monotonically increasing number that changes with every update to that key.

Write with a TTL (in milliseconds):

```
await client.SetKeyValue("session/token-xyz", "active", expiryTime: 60000);
```

This key expires after 60 seconds.

3.6.4 Distributed Locks

Acquire a lock, do some work, and release it:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "jobs/send-email",
    expiry: TimeSpan.FromSeconds(10),
    durability: LockDurability.Persistent
);

if (lockHandle.IsAcquired)
{
                    Console.WriteLine($"Lock    acquired.    Fencing    token:
{lockHandle.FencingToken}");
    // Do the protected work here.
}
else
{
    Console.WriteLine("Could not acquire the lock. Another process holds it.");
}
// The lock is released automatically when the using block ends.
```

The `GetOrCreateLock` method returns a `KahunaLock` object. If the lock is already held by another owner, `IsAcquired` is false. The lock implements `IAsyncDisposable`, so it is released automatically at the end of the `await using` block.

The `FencingToken` is a monotonically increasing number. Each time a lock changes hands, the fencing token increases. You can pass this token to downstream services to ensure that a stale lock holder cannot overwrite newer data. Chapter 4 covers fencing in detail.

3.6.5 Sequences

Create a sequence and generate values:

```
KahunaSequence seq = await client.CreateSequence("order-ids");
Console.WriteLine($"Sequence created. Current value: {seq.CurrentValue}");

long nextId = await client.NextSequenceValue("order-ids");
Console.WriteLine($"Next value: {nextId}");

KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",
count: 10);
Console.WriteLine($"Reserved range: {range.Start} to {range.End}");
```

`NextSequenceValue` returns a single value. `ReserveSequenceRange` reserves a batch of values and returns the start and end of the range. The values in a reserved range are guaranteed to be unique and monotonically increasing across all clients.

3.7 A Complete Example

The following console application connects to a standalone Kahuna node and exercises all three primitives:

```
using Kahuna.Client;
using Kahuna.Shared.Locks;
```

```
var client = new KahunaClient("http://localhost:2070");

// --- Key-Value ---
KahunaKeyValue kv = await client.SetKeyValue("demo/greeting", "Hello, Kahuna!");
Console.WriteLine($"Set key 'demo/greeting' at revision {kv.Revision}");

KahunaKeyValue read = await client.GetKeyValue("demo/greeting");
Console.WriteLine($"Read: {read.ValueAsString()} (revision {read.Revision})");

// Update the key.
KahunaKeyValue updated = await client.SetKeyValue("demo/greeting", "Hello
again!");
Console.WriteLine($"Updated to revision {updated.Revision}");

// --- Distributed Lock ---
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "demo/my-lock",
    expiry: TimeSpan.FromSeconds(30),
    durability: LockDurability.Persistent
);

if (lockHandle.IsAcquired)
{
                    Console.WriteLine($"Lock acquired. Fencing token:
{lockHandle.FencingToken}");
}

// --- Sequence ---
await client.CreateSequence("demo/counter");
for (int i = 0; i < 5; i++)
{
    long id = await client.NextSequenceValue("demo/counter");
    Console.WriteLine($"Sequence value: {id}");
}

Console.WriteLine("Done.");
```

To run this example, create a new console project:

```
mkdir kahuna-demo && cd kahuna-demo
dotnet new console
dotnet add package Kahuna.Client
```

Replace the contents of `Program.cs` with the code above. Start a standalone Kahuna server in another terminal (`kahuna-server`), then run the application:

```
dotnet run
```

You should see output similar to:

```
Set key 'demo/greeting' at revision 1
Read: Hello, Kahuna! (revision 1)
Updated to revision 2
Lock acquired. Fencing token: 1
Sequence value: 0
Sequence value: 1
Sequence value: 2
Sequence value: 3
Sequence value: 4
Done.
```

3.8 Your First Failure Experiment

Distributed systems earn their value when something goes wrong. This experiment demonstrates that Kahuna keeps serving requests after a node failure.

3.8.1 Setup

Start the three-node Docker cluster:

```
docker compose -f docker/local.yml up -d
```

Write a key through node 1:

```
kahuna-cli -c "https://localhost:8082" --insecure \
  --set "experiment/counter" --value "42"
```

Verify the key is readable from all three nodes:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"
kahuna-cli -c "https://localhost:8084" --insecure --get "experiment/counter"
kahuna-cli -c "https://localhost:8086" --insecure --get "experiment/counter"
```

All three should return `42`.

3.8.2 Kill a Node

Stop the second container:

```
docker stop kahuna2
```

Now try reading and writing through the remaining nodes:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"
kahuna-cli -c "https://localhost:8086" --insecure \
  --set "experiment/counter" --value "43"
kahuna-cli -c "https://localhost:8082" --insecure --get "experiment/counter"
```

The reads and writes still work. Two out of three nodes form a majority (a quorum), so Raft can still reach consensus on new writes.

3.8.3 Bring the Node Back

```
docker start kahuna2
```

After a few seconds, node 2 catches up with the other two. Read the key through node 2:

```
kahuna-cli -c "https://localhost:8084" --insecure --get "experiment/counter"
```

It returns `43`, the value that was written while it was down. Raft replicated the missed writes when node 2 rejoined the cluster.

3.8.4 What Happens If Two Nodes Go Down?

Stop two nodes:

```
docker stop kahuna2 kahuna3
```

Now try to write:

```
kahuna-cli -c "https://localhost:8082" --insecure \
  --set "experiment/counter" --value "44"
```

This request fails or times out. With only one node remaining out of three, Raft cannot form a quorum. The cluster refuses to accept writes because it cannot guarantee that the write will be durable on a majority of replicas.

This is intentional. Kahuna chooses consistency over availability: it will not accept a write that it cannot safely replicate.

Bring the nodes back:

```
docker start kahuna2 kahuna3
```

After a few seconds, the cluster resumes normal operation.

3.9 Summary

In this chapter you installed Kahuna's server and CLI tools, ran a standalone node, operated a three-node cluster, and connected a C# application. You used all three primitives (key-value storage, distributed locks, and sequences) from both the CLI and the client SDK. You also ran a failure experiment that demonstrated Raft's ability to maintain availability with a majority of nodes and to replicate missed writes when a failed node returns.

The next chapter explores key-value operations in depth: conditional writes, revisions, TTL, range queries, and durability levels.

4 Key-Value Operations

Chapter 2 showed how to set and get a key. This chapter covers the full key-value API: conditional writes, revisions, time-to-live, prefix queries, range scans, batch operations, durability levels, and snapshot reads. By the end, you will know every tool Kahuna provides for working with key-value data.

4.1 Keys and Values

A Kahuna key is a string. A value is a byte array (or a UTF-8 string, which the client converts to bytes). There is no schema, no type enforcement, and no size hierarchy. Every key is flat.

Kahuna distinguishes a **null value** from an **empty value**. A null value means the key exists but has no payload (the value field is unset in the protobuf). An empty value means the key exists with a zero-length byte array. Both gRPC and REST transports preserve this distinction. A `SetKeyValue` call with `null` as the value stores a null payload. A call with `new byte[0]` stores an empty payload. A `GetKeyValue` call returns `null` or an empty array accordingly.

Keys often follow a convention of prefixed namespaces separated by `/`:

```
users/alice
users/bob
config/feature-flags/dark-mode
orders/2024/00042
```

This convention is not just cosmetic. Kahuna uses the key's prefix (everything up to and including the last `/`) to determine which partition stores the key. Keys that share a prefix route to the same partition. This matters for prefix queries (covered later in this chapter) and for transactions (covered in Chapter 6).

4.2 Revisions

Every key in Kahuna has a revision number. The revision starts at 0 on the first write and increases by one with every subsequent write to the same key.

```
var client = new KahunaClient("http://localhost:2070");

KahunaKeyValue r1 = await client.SetKeyValue("counter/a", "first");
Console.WriteLine(r1.Revision);  // 0

KahunaKeyValue r2 = await client.SetKeyValue("counter/a", "second");
Console.WriteLine(r2.Revision);  // 1

KahunaKeyValue r3 = await client.SetKeyValue("counter/a", "third");
Console.WriteLine(r3.Revision);  // 2
```

Revisions are per-key. Writing to `counter/a` does not affect the revision of `counter/b`.

The revision number serves two purposes:

1. **Conflict detection.** Conditional writes can check the revision before updating. If another client changed the key since you last read it, the revision will not match and the write will fail. This is optimistic concurrency control.

2. **History.** Kahuna stores previous revisions. You can read the value of a key at any past revision.

4.2.1 Reading a Specific Revision

```
KahunaKeyValue atRev1 = await client.GetKeyValueRevision("counter/a", revision:
1);
Console.WriteLine(atRev1.ValueAsString());  // "second"
```

If the requested revision does not exist, `Success` is false and `Value` is null.

4.3 Conditional Writes

An unconditional `SetKeyValue` always overwrites the current value. Conditional writes add a check: the write only succeeds if a condition is met. If the condition fails, `Success` is false and the key is unchanged.

Kahuna provides four conditional write modes, controlled by the `KeyValueFlags` parameter.

4.3.1 Set If Not Exists

Write only if the key does not already exist:

```
KahunaKeyValue result = await client.SetKeyValue(
    "config/db-connection",
    "Server=primary;Database=app",
    flags: KeyValueFlags.SetIfNotExists
);

if (result.Success)
    Console.WriteLine("Configuration key created.");
else
    Console.WriteLine("Key already exists. Not overwritten.");
```

This is useful for one-time initialization. Multiple processes can race to set the key. Exactly one will succeed.

4.3.2 Set If Exists

Write only if the key already exists:

```
KahunaKeyValue result = await client.SetKeyValue(
    "config/db-connection",
    "Server=secondary;Database=app",
    flags: KeyValueFlags.SetIfExists
);

if (result.Success)
    Console.WriteLine("Configuration updated.");
else
    Console.WriteLine("Key does not exist. Nothing to update.");
```

4.3.3 Compare-Revision-And-Set (CRAS)

Write only if the key's current revision matches a specified value. This is the primary mechanism for optimistic concurrency control.

```
// Read the current value and its revision.
KahunaKeyValue current = await client.GetKeyValue("inventory/widget-stock");
Console.WriteLine($"Current stock: {current.ValueAsString()}, revision:
{current.Revision}");

// Update the value, but only if nobody else changed it.
KahunaKeyValue updated = await client.TryCompareRevisionAndSetKeyValue(
    "inventory/widget-stock",
    "95",
    compareRevision: current.Revision
);

if (updated.Success)
    Console.WriteLine($"Updated to revision {updated.Revision}.");
else
     Console.WriteLine("Conflict: another client modified the key. Read and
retry.");
```

The pattern is read, modify, write-if-unchanged. If another client wrote to the key between your read and your write, the revision will differ and the write will fail. Your code can then re-read and retry.

4.3.4 Compare-Value-And-Set (CVAS)

Write only if the key's current value matches a specified byte sequence:

```
KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
    "state-machine/order-123",
    value: "shipped",
    compareValue: "paid"
);

if (result.Success)
    Console.WriteLine("State transitioned from 'paid' to 'shipped'.");
else
    Console.WriteLine("Current value is not 'paid'. Transition rejected.");
```

This is useful for state machines. The transition from "paid" to "shipped" only succeeds if the current state is "paid." If another process already moved the state to "cancelled," the write fails.

4.4 Time-to-Live (TTL)

Keys can have an expiration time. After the TTL elapses, the key is no longer readable. The TTL is specified in milliseconds.

```
// This key expires after 60 seconds.
await client.SetKeyValue("session/token-abc", "active", expiryTime: 60000);
```

You can also use a `TimeSpan`:

```
await client.SetKeyValue("session/token-abc",
"active", TimeSpan.FromMinutes(5));
```

An expiry of 0 (the default) means the key does not expire.

4.4.1 Extending a TTL

If a key is about to expire but you need it to live longer, extend its TTL without changing its value:

```
await client.ExtendKeyValue("session/token-abc", expiresMs: 60000);
```

Or using a `TimeSpan`:

```
await client.ExtendKeyValue("session/token-abc", TimeSpan.FromMinutes(5));
```

The extension resets the expiration clock from the current time. If the key does not exist, `Success` is false.

4.4.2 TTL and Conditional Writes

You can combine TTL with conditional write flags:

```
// Create a session key only if it doesn't already exist, with a 30-minute TTL.
KahunaKeyValue session = await client.SetKeyValue(
    "session/user-42",
    "session-data-here",
    expiryTime: 1800000,
    flags: KeyValueFlags.SetIfNotExists
);
```

4.5 Durability Levels

Every key-value operation accepts a `durability` parameter with two options:

Level	Behavior
`Persistent`	The write goes through Raft consensus and is replicated to a majority of nodes. This is the default.
`Ephemeral`	The write is stored in memory on the receiving node only. It is not replicated and does not survive a node restart.

```
// Persistent (default): safe, replicated.
await client.SetKeyValue("config/important", "value", durability:
KeyValueDurability.Persistent);

// Ephemeral: fast, local, not replicated.
await client.SetKeyValue("cache/temp-result", "value", durability:
KeyValueDurability.Ephemeral);
```

Use ephemeral durability for data you can afford to lose: caches, temporary counters, rate-limit windows. Use persistent durability for anything that must survive failures.

A read with a specific durability level only sees keys written at that same level. A key written as `Ephemeral` is not visible to a `Persistent` read, and vice versa.

4.6 Deleting Keys

Delete a key:

```
KahunaKeyValue deleted = await client.DeleteKeyValue("users/alice");
if (deleted.Success)
    Console.WriteLine("Key deleted.");
else
    Console.WriteLine("Key did not exist.");
```

A delete on a non-existent key sets `Success` to false.

4.7 Checking Existence

If you only need to know whether a key exists (without fetching its value), use `ExistsKeyValue`:

```
KahunaKeyValue exists = await client.ExistsKeyValue("users/alice");
if (exists.Success)
    Console.WriteLine($"Key exists at revision {exists.Revision}.");
else
    Console.WriteLine("Key does not exist.");
```

This is lighter than `GetKeyValue` because the server does not transfer the value bytes.

4.8 No-Revision Writes

By default, every write archives a revision entry so you can read the key's history. For high-throughput keys where you never need history (caches, counters, ephemeral state), you can skip revision archiving:

```
await client.SetKeyValueNoRevision("metrics/request-count", "4217");
```

The revision counter still increments and conditional writes still work, but the previous value is not stored for historical reads. This reduces storage overhead for keys that change frequently.

4.9 Batch Operations

When you need to read, write, or delete many keys at once, batch operations reduce the number of round trips.

4.9.1 Batch Set

```
var items = new List<KahunaSetKeyValueRequestItem>
{
    new() { Key = "users/alice", Value = "admin"u8.ToArray() },
    new() { Key = "users/bob", Value = "editor"u8.ToArray() },
    new() { Key = "users/carol", Value = "viewer"u8.ToArray() }
};

List<KahunaKeyValue> results = await client.SetManyKeyValues(items);
```

```
foreach (KahunaKeyValue kv in results)
     Console.WriteLine($"{kv.Key}: revision {kv.Revision}, success:
{kv.Success}");
```

Each item in the batch can have its own flags, expiry, and durability level.

4.9.2 Batch Get

```
var requests = new List<KahunaGetManyKeyValuesRequestItem>
{
    new() { Key = "users/alice" },
    new() { Key = "users/bob" },
    new() { Key = "users/carol" }
};

List<KahunaKeyValue> results = await client.GetManyKeyValues(requests);

foreach (KahunaKeyValue kv in results)
{
    if (kv.Success)
        Console.WriteLine($"{kv.Key} = {kv.ValueAsString()}");
    else
        Console.WriteLine($"{kv.Key}: not found");
}
```

4.9.3 Batch Delete

```
// Simple form: pass a list of keys.
List<KahunaKeyValue> deleted = await client.DeleteManyKeyValues(
    new[] { "users/alice", "users/bob", "users/carol" }
);

foreach (KahunaKeyValue kv in deleted)
    Console.WriteLine($"{kv.Key}: deleted = {kv.Success}");
```

A second overload accepts `KahunaDeleteKeyValueRequestItem` objects. This form lets you set the durability level for each key separately:

```
List<KahunaKeyValue> deleted = await client.DeleteManyKeyValues(
    new[]
    {
        new KahunaDeleteKeyValueRequestItem
            { Key = "cache/temp", Durability = KeyValueDurability.Ephemeral },
        new KahunaDeleteKeyValueRequestItem
            { Key = "config/old", Durability = KeyValueDurability.Persistent }
    }
);
```

4.9.4 Batch Exists

```
var requests = new List<KahunaGetManyKeyValuesRequestItem>
{
```

```
    new() { Key = "users/alice" },
    new() { Key = "users/bob" }
};

List<KahunaKeyValue> results = await client.ExistsManyKeyValues(requests);
```

All batch operations complete in a single round trip to the server.

4.10 Prefix Queries

Kahuna provides two ways to query keys by prefix. The difference between them is scope.

4.10.1 Single-Partition Prefix Query

`GetByBucket` returns all keys that share a prefix, within the partition that owns that prefix:

```
List<KahunaKeyValue> users = await client.GetByBucket(
    "users",
    KeyValueDurability.Persistent
);

foreach (KahunaKeyValue kv in users)
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString()}");
```

Because keys with the same prefix hash to the same partition, this query contacts only one partition. It is efficient, but it only finds keys that share the exact prefix.

The result set is capped at 4,096 entries. For larger sets, use range scans (next section).

4.10.2 All-Node Prefix Scan

`ScanAllByPrefix` searches all nodes for keys that match a prefix:

```
List<KahunaKeyValue> allConfigs = await client.ScanAllByPrefix(
    "config",
    KeyValueDurability.Persistent
);
```

This is a broader operation. It contacts every node in the cluster. Use it when you need to find keys across partitions. Like `GetByBucket`, the result is capped at 4,096 entries.

4.11 Range Scans

For large data sets or fine-grained control over which keys to return, use range scans.

4.11.1 Paginated Range Query

`GetByRange` returns keys within a prefix, optionally bounded by start and end keys:

```
List<KahunaKeyValue> page = await client.GetByRange(
    prefix: "orders/2024",
    startKey: "orders/2024/00100",
    startInclusive: true,
    endKey: "orders/2024/00200",
    endInclusive: false,
```

```
    limit: 50
);
```

This returns up to 50 keys in the range `[orders/2024/00100, orders/2024/00200)` within the `orders/2024` prefix. The `startInclusive` and `endInclusive` parameters control whether the boundary keys are included.

If you omit `startKey` and `endKey`, the query returns the first `limit` keys in the prefix.

4.11.2 Streaming Range Scan

`ScanByRange` returns an `IAsyncEnumerable<KahunaKeyValue>` that pages through results automatically:

```
await foreach (KahunaKeyValue kv in client.ScanByRange(
    prefix: "logs/2024-03",
    pageSize: 100))
{
    Console.WriteLine($"{kv.Key}: {kv.ValueAsString()}");
}
```

The client fetches pages of 100 entries from the server. Your code iterates over them as a continuous stream. This is the right choice when you do not know how many keys exist and want to process them one at a time without loading all results into memory.

You can also bound the scan with start and end keys:

```
await foreach (KahunaKeyValue kv in client.ScanByRange(
    prefix: "events",
    startKey: "events/2024-03-01",
    endKey: "events/2024-03-31",
    endInclusive: true,
    pageSize: 200))
{
    ProcessEvent(kv);
}
```

4.12 Snapshot Reads

By default, every read returns the latest committed value. Snapshot reads let you read the state of the key-value store at a specific point in time.

The `snapshotMs` parameter accepts a Unix epoch timestamp in milliseconds. When set, the read returns the value that was current at that moment.

```
// Write a value.
KahunaKeyValue v1 = await client.SetKeyValue("price/widget", "10.00");
long snapshotTime = v1.LastModified;

// Write a new value.
await client.SetKeyValue("price/widget", "12.00");

// Read the current value.
KahunaKeyValue current = await client.GetKeyValue("price/widget");
```

```
Console.WriteLine(current.ValueAsString());  // "12.00"

// Read the value at the snapshot time.
KahunaKeyValue snapshot = await client.GetKeyValue("price/widget", snapshotMs:
snapshotTime);
Console.WriteLine(snapshot.ValueAsString());  // "10.00"
```

The `LastModified` property on a `KahunaKeyValue` is a Unix epoch timestamp in milliseconds. You can use it as a snapshot anchor: read a key, save its `LastModified`, and later re-read at that exact point.

Snapshot reads also work with `GetByBucket`, `ScanAllByPrefix`, `GetByRange`, and `ScanByRange`. This gives you a consistent view across multiple keys at the same point in time.

```
// Read all user keys as they were at a specific moment.
List<KahunaKeyValue> usersAtSnapshot = await client.GetByBucket(
    "users",
    KeyValueDurability.Persistent,
    snapshotMs: snapshotTime
);
```

Snapshot reads depend on Kahuna's MVCC (multi-version concurrency control) layer. Chapter 19 covers the internals of how snapshots work.

4.12.1 Snapshot Holds

A snapshot read at a past timestamp works only if the MVCC layer still holds the revision. The revision pruning system can delete old revisions based on retention count and age. A snapshot hold prevents pruning from deleting revisions at or after a specific timestamp.

Use a snapshot hold when you need to read a consistent snapshot over multiple API calls (for example, to build a report across many keys):

```
// Acquire a hold at the current server time for 60 seconds.
(KeyValueResponseType result, string holdId, long leaseExpiry) =
    await client.AcquireSnapshotHold("report-builder-1", snapshotTime, leaseMs:
60000);

// Read many keys at that snapshot. The hold prevents revision pruning.
List<KahunaKeyValue> users = await client.GetByBucket(
    "users", KeyValueDurability.Persistent, snapshotMs: snapshotTime);
List<KahunaKeyValue> orders = await client.GetByBucket(
    "orders", KeyValueDurability.Persistent, snapshotMs: snapshotTime);

// Renew the hold if the report takes longer than expected.
await client.RenewSnapshotHold(holdId, leaseMs: 60000);

// Release the hold when done.
await client.ReleaseSnapshotHold(holdId);
```

`AcquireSnapshotHold` is idempotent: a repeated call with the same `holderId` and `timestamp` returns the same hold ID and renews the lease. `GetSnapshotFloor` returns the current effective floor timestamp and the number of live holds.

Without a hold, a long-running read may encounter pruned revisions and receive incomplete results. The hold keeps the effective snapshot floor at or below the held timestamp until the hold is released or its lease expires.

4.12.1.1 Restart Recovery

When a node restarts, snapshot holds that lapsed during the downtime are still registered in the durable state. The `SnapshotHoldStartupGraceWindow` (default 5 minutes) protects these holds. During the grace window, lapsed-but-registered holds continue to pin history. The revision pruner does not delete revisions at or after a held timestamp until the grace window expires. This gives clients time to renew or release their holds after a server restart.

Protection ends when the hold is removed through Raft replication, not when the lease expires locally. A lease expiry triggers a Raft proposal to remove the hold. Until that proposal commits, the hold remains active.

4.13 The KahunaKeyValue Object

Every key-value operation returns a `KahunaKeyValue` object. Here is a summary of its properties and methods:

Member	Type	Description
`Key`	`string`	The key that was operated on
`Success`	`bool`	Whether the operation succeeded
`Revision`	`long`	The key's revision after the operation
`Value`	`byte[]?`	The raw value bytes (null if not found or not fetched)
`Durability`	`KeyValueDurability`	The durability level used
`TimeElapsedMs`	`int`	Server-side time for the operation
`LastModified`	`long`	Commit timestamp (Unix epoch ms)
`ValueAsString()`	`string?`	Decode the value as UTF-8
`ValueAsLong()`	`long`	Parse the value as a long integer
`ValueAsBool()`	`bool`	Parse the value as a boolean
`Extend(TimeSpan)`	`Task<KahunaKeyValue>`	Extend the key's TTL
`Delete()`	`Task<KahunaKeyValue>`	Delete this key

The `Extend` and `Delete` methods operate on the same key and durability level that produced the result. They are convenience methods so you can chain operations without repeating the key name.

```
KahunaKeyValue session = await client.GetKeyValue("session/user-42");
if (session.Success)
```

```
{
    // Extend the session by 30 more minutes.
    await session.Extend(TimeSpan.FromMinutes(30));
}
```

4.14 CLI Reference

The command-line client supports all key-value operations. Here is a quick reference:

Operation	Command
Set a key	`kahuna-cli --set "key" --value "val"`
Set with TTL	`kahuna-cli --set "key" --value "val" --expires 60000`
Get a key	`kahuna-cli --get "key"`
Extend TTL	`kahuna-cli --extend "key" --expires 60000`
Prefix query (single partition)	`kahuna-cli --get-by-prefix "prefix"`
Prefix scan (all nodes)	`kahuna-cli --scan-by-prefix "prefix"`
JSON output	add `--format json` to any command

To connect to a specific node, use `-c`:

```
kahuna-cli -c "https://localhost:8082" --insecure --get "users/alice"
```

4.15 Putting It Together

The following example demonstrates several key-value patterns in a single program:

```
using Kahuna.Client;
using Kahuna.Shared.KeyValue;

var client = new KahunaClient("http://localhost:2070");

// 1. Create a key only if it doesn't exist (one-time initialization).
KahunaKeyValue init = await client.SetKeyValue(
    "app/version", "1.0.0", flags: KeyValueFlags.SetIfNotExists);
Console.WriteLine($"Init: success={init.Success}, revision={init.Revision}");

// 2. Read, modify, write with optimistic concurrency.
KahunaKeyValue config = await client.GetKeyValue("app/version");
KahunaKeyValue updated = await client.TryCompareRevisionAndSetKeyValue(
    "app/version", "1.1.0", compareRevision: config.Revision);

if (updated.Success)
    Console.WriteLine($"Updated to {updated.Revision}");
else
```

```
    Console.WriteLine("Conflict detected. Retry needed.");

// 3. State machine transition via compare-value-and-set.
await client.SetKeyValue("order/99/status", "pending");

KahunaKeyValue transition = await client.TryCompareValueAndSetKeyValue(
    "order/99/status", value: "confirmed", compareValue: "pending");
Console.WriteLine($"Transition: {transition.Success}");

// 4. Batch write with TTL.
var items = new List<KahunaSetKeyValueRequestItem>
{
   new() { Key = "cache/result-a", Value = "42"u8.ToArray(), ExpiresMs = 30000 },
   new() { Key = "cache/result-b", Value = "99"u8.ToArray(), ExpiresMs = 30000 }
};
await client.SetManyKeyValues(items);

// 5. Prefix query.
List<KahunaKeyValue> cacheEntries = await client.GetByBucket(
    "cache", KeyValueDurability.Persistent);
Console.WriteLine($"Cache entries: {cacheEntries.Count}");

// 6. Snapshot read.
KahunaKeyValue v1 = await client.SetKeyValue("price/gold", "2000");
long anchor = v1.LastModified;

await client.SetKeyValue("price/gold", "2050");

KahunaKeyValue latest = await client.GetKeyValue("price/gold");
KahunaKeyValue  past  =  await  client.GetKeyValue("price/gold",  snapshotMs:
anchor);

Console.WriteLine($"Latest: {latest.ValueAsString()}");   // 2050
Console.WriteLine($"At snapshot: {past.ValueAsString()}"); // 2000
```

4.16 Summary

Kahuna's key-value API goes beyond simple get and set. Conditional writes (SetIfNotExists, SetIfExists, CRAS, CVAS) give you optimistic concurrency control and state machine transitions without external locks. Revisions provide a per-key version history. TTL lets keys expire automatically. Prefix queries and range scans let you retrieve groups of related keys efficiently. Snapshot reads give you a consistent view of the store at a past point in time. Batch operations reduce round trips when you work with many keys at once.

The next chapter covers distributed locks: lease semantics, fencing tokens, lock contention, and what happens when a lock holder crashes.

5 Distributed Locking

On a single machine, a mutex is enough. A thread acquires the mutex, does its work, and releases it. The operating system guarantees that only one thread holds the mutex at a time.

In a distributed system, there is no shared memory and no single operating system. Two processes on different machines can both believe they hold "the lock" if the mechanism that coordinates them fails. This chapter explains how Kahuna's distributed locks work, why leases and fencing tokens are necessary, and how to use them correctly.

5.1 Why Locks Need Leases

A naive distributed lock works like this: write your name into a key. If the key is empty, you hold the lock. Delete the key when you are done.

The problem is "when you are done." What if the lock holder crashes before it deletes the key? The lock is held forever. No other process can acquire it.

The solution is a lease: a lock with an expiration time. If the holder does not release the lock before the lease expires, the lock becomes available to other clients. This guarantees progress. A crashed process cannot block the system indefinitely.

Kahuna's locks are lease-based. Every lock acquisition requires an expiry time. When the expiry elapses, the lock is automatically released.

5.2 Acquiring a Lock

The simplest lock acquisition takes a resource name and an expiry:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "jobs/send-invoice",
    expiry: TimeSpan.FromSeconds(30),
    durability: LockDurability.Persistent
);

if (lockHandle.IsAcquired)
{
            Console.WriteLine($"Lock   acquired.   Fencing   token:
{lockHandle.FencingToken}");
    // Do the protected work here.
}
else
{
    Console.WriteLine("Lock is held by another process.");
}
```

`GetOrCreateLock` returns a `KahunaLock` object. If the lock is already held by another owner, `IsAcquired` is false. The call does not block or wait. It tries once and returns immediately.

The lock implements `IAsyncDisposable`. When the `await using` block ends, the lock is released automatically. If the process crashes before the block ends, the lease expires and the lock becomes available.

5.2.1 Lock Parameters

Parameter	Type	Default	Description
`resource`	`string`	(required)	The name of the resource to lock
`expiry`	`TimeSpan` or `int` (ms)	30,000 ms	How long the lease lasts
`wait`	`TimeSpan` or `int` (ms)	0	How long to keep trying if the lock is held
`retry`	`TimeSpan` or `int` (ms)	0	How often to retry during the wait period
`durability`	`LockDurability`	`Persistent`	Replicated or in-memory

5.2.2 Lock Owner

Every lock acquisition generates a unique owner token (a GUID). This token identifies the holder. Only the owner that acquired the lock can release or extend it. You do not need to manage this token yourself; the `KahunaLock` object holds it internally.

You can read the owner for debugging:

```
if (lockHandle.IsAcquired)
    Console.WriteLine($"Owner: {lockHandle.OwnerAsString}");
```

5.3 Fencing Tokens

Leases solve the problem of a crashed lock holder. But they introduce a new problem: the stale lock holder.

5.3.1 The Stale Lock Holder Problem

Consider this scenario:

1. Process A acquires a lock with a 10-second lease.
2. Process A starts writing to a shared resource (a database, a file, an external API).
3. Process A experiences a long garbage collection pause, a network delay, or a slow disk. It does not crash, but it stops making progress for 15 seconds.
4. The lease expires after 10 seconds. Process A does not know this yet.
5. Process B acquires the lock. It starts writing to the same shared resource.
6. Process A resumes. It still believes it holds the lock. It writes to the shared resource, overwriting what process B just wrote.

Both processes believe they are the rightful lock holder. The data is now corrupted.

Extending the lease duration does not fix this. No matter how long the lease is, a sufficiently long pause can outlast it. The fundamental problem is that process A has no way to know its lease expired.

5.3.2 How Fencing Tokens Solve This

Kahuna assigns a fencing token to every lock acquisition. The fencing token is a monotonically increasing integer that is scoped to a specific resource. Each time the lock changes hands, the fencing token increases.

Here is the sequence with fencing tokens:

1. Process A acquires the lock. Fencing token: 5.
2. Process A pauses for 15 seconds. The lease expires.
3. Process B acquires the lock. Fencing token: 6.
4. Process A resumes and tries to write to the shared resource, passing fencing token 5.
5. The shared resource checks the fencing token. It has already seen token 6 from process B. It rejects token 5 because it is stale.

The downstream system (a database, an API, a message queue) must participate in fencing. It stores the highest fencing token it has seen and rejects any write with a lower token.

5.3.3 Fencing Token Behavior

- The first acquisition of a resource gets fencing token 0.
- Each new acquisition (by a different owner, or after expiry) increments the token by 1.
- Extending a lock does not change the fencing token.
- The same owner re-acquiring a lock before it expires gets the same fencing token back (no increment).
- Fencing tokens are per-resource. Locking `jobs/a` and `jobs/b` produce independent token sequences.

```
// First acquisition: token = 0
await using (KahunaLock first = await client.GetOrCreateLock("demo/fence",
TimeSpan.FromSeconds(5)))
{
    Console.WriteLine(first.FencingToken);  // 0
}

// Second acquisition (after release): token = 1
await using (KahunaLock second = await client.GetOrCreateLock("demo/fence",
TimeSpan.FromSeconds(5)))
{
    Console.WriteLine(second.FencingToken);  // 1
}
```

5.3.4 Using Fencing Tokens in Practice

Pass the fencing token to every downstream operation that the lock protects:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "orders/process-batch",
    TimeSpan.FromSeconds(30)
);

if (lockHandle.IsAcquired)
{
```

```
    long token = lockHandle.FencingToken;

    // Pass the token to the database.
    await db.ExecuteAsync(
        "UPDATE orders SET status = 'processed' WHERE batch_id = @batch AND
fence_token < @token",
        new { batch = batchId, token }
    );
}
```

The database query rejects the update if a higher fencing token has already written to this batch. This prevents a stale lock holder from overwriting newer data.

Not every system supports fencing natively. For systems that do not, you can store the fencing token as a column or field and check it in your application logic before writing.

5.4 Waiting for a Lock

By default, `GetOrCreateLock` tries once. If the lock is held, it returns immediately with `IsAcquired = false`. For workloads where you want to wait until the lock becomes available, pass `wait` and `retry` parameters:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    resource: "jobs/send-invoice",
    expiry: TimeSpan.FromSeconds(30),
    wait: TimeSpan.FromSeconds(10),
    retry: TimeSpan.FromMilliseconds(200)
);
```

This tries to acquire the lock. If it fails, it retries every 200 milliseconds (with a small random jitter of plus or minus 50 ms) until either the lock is acquired or 10 seconds elapse.

If the wait period expires without acquiring the lock, `IsAcquired` is false. The call does not throw an exception.

The `retry` parameter must be greater than zero when `wait` is greater than zero. Otherwise the client throws a `KahunaException`.

5.4.1 Contention with Multiple Clients

When multiple clients compete for the same lock using wait and retry, each client retries independently. Kahuna does not queue waiters or guarantee fairness. The first client to attempt acquisition after the lock is released wins. The jitter on the retry interval reduces the chance that all clients retry at exactly the same time.

```
// 10 workers competing for the same lock.
var tasks = Enumerable.Range(0, 10).Select(async i =>
{
    await using KahunaLock lk = await client.GetOrCreateLock(
        "shared/critical-section",
        expiry: TimeSpan.FromSeconds(5),
        wait: TimeSpan.FromSeconds(60),
        retry: TimeSpan.FromMilliseconds(100)
```

```
    );

    if (lk.IsAcquired)
    {
       Console.WriteLine($"Worker {i} acquired lock. Token: {lk.FencingToken}");
        await Task.Delay(500);  // Simulate work.
    }
});

await Task.WhenAll(tasks);
```

Each worker eventually acquires the lock because the lease is short (5 seconds) and the wait budget is long (60 seconds).

5.5 Extending a Lease

If your work takes longer than the original lease, extend it before it expires:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "jobs/long-running",
    TimeSpan.FromSeconds(10)
);

if (lockHandle.IsAcquired)
{
    // Start the work.
    await ProcessFirstBatch();

    // Extend the lease by another 10 seconds.
                    (bool     extended,     long     token)     =     await
lockHandle.TryExtend(TimeSpan.FromSeconds(10));

    if (extended)
    {
        Console.WriteLine($"Lease extended. Fencing token still: {token}");
        await ProcessSecondBatch();
    }
    else
    {
        Console.WriteLine("Extension failed. The lease may have expired.");
        // Stop work. Another process may now hold the lock.
    }
}
```

Key details about extension:

- `TryExtend` resets the expiration clock from the current time.
- Only the current owner can extend the lock. If the lease already expired and another process acquired it, `TryExtend` returns false.
- Extending does not change the fencing token. The token remains the same for the duration of a single ownership period.

- There is no automatic renewal. Your application must call `TryExtend` explicitly.

You can also extend a lock through the client directly, if you have the owner token:

```
(bool success, long token) = await client.TryExtendLock(
    "jobs/long-running",
    lockHandle.Owner,
    TimeSpan.FromSeconds(10),
    LockDurability.Persistent
);
```

5.6 What Happens When a Lock Holder Crashes

This is the scenario leases are designed for:

1. Process A acquires a lock with a 10-second lease.
2. Process A crashes (power failure, OOM kill, unhandled exception).
3. The `DisposeAsync` call never runs because the process is gone.
4. After 10 seconds, the lease expires.
5. Process B acquires the lock with a new, higher fencing token.

No manual intervention is required. The cluster does not need to detect the crash. The lease simply expires, and the lock becomes available.

This is why lease duration matters:

- **Too short:** The lock expires before the holder finishes its work. The holder must extend frequently, adding network round trips.
- **Too long:** After a crash, other processes wait for the lease to expire before they can make progress.

A good starting point is 10 to 30 seconds for most workloads. Adjust based on how long the protected work takes and how quickly you need recovery after a crash.

5.7 Unlocking

The `await using` pattern releases the lock automatically. You can also unlock manually:

```
KahunaLock lockHandle = await client.GetOrCreateLock(
    "jobs/send-email",
    TimeSpan.FromSeconds(30)
);

if (lockHandle.IsAcquired)
{
    await DoWork();
    bool released = await client.Unlock(
        "jobs/send-email",
        lockHandle.Owner,
        LockDurability.Persistent
    );
    Console.WriteLine($"Released: {released}");
}
```

Only the owner can unlock. If a different process tries to unlock with a different owner token, the call returns false.

If you do not unlock and do not dispose, the lock remains held until the lease expires. The `KahunaLock` finalizer logs a warning if the object is garbage-collected without being disposed.

5.8 Querying Lock State

You can inspect a lock's current state without acquiring it:

```
KahunaLockInfo? info = await client.GetLockInfo(
    "jobs/send-email",
    LockDurability.Persistent
);

if (info != null)
{
    Console.WriteLine($"Owner: {info.Owner}");
    Console.WriteLine($"Fencing token: {info.FencingToken}");
    Console.WriteLine($"Expires: {info.Expires}");
}
```

If no lock exists for the resource, `GetLockInfo` returns null.

You can also query through a `KahunaLock` object, even if you did not acquire the lock:

```
KahunaLock lockHandle = await client.GetOrCreateLock("jobs/send-email",
TimeSpan.FromSeconds(5));
KahunaLockInfo? info = await lockHandle.GetInfo();
```

Lock info is a diagnostic tool. Do not use it to make decisions about whether to proceed with work. Between the time you read the lock info and the time you act on it, the lock state can change. Use `GetOrCreateLock` for coordination, not `GetLockInfo`.

5.9 Ephemeral Locks

By default, locks are persistent: the acquisition goes through Raft consensus and is replicated to a majority of nodes. This provides strong guarantees but adds latency.

For workloads where speed matters more than durability (coordinating in-memory caches, local-only rate limiting), use ephemeral locks:

```
await using KahunaLock lockHandle = await client.GetOrCreateLock(
    "cache/rebuild",
    expiry: TimeSpan.FromSeconds(5),
    durability: LockDurability.Ephemeral
);
```

Ephemeral locks are stored in memory on the receiving node only. They are faster but do not survive a node restart. If the node crashes, the lock is gone immediately (no need to wait for lease expiry, but also no replication safety).

Use ephemeral locks only when losing the lock on a node failure is acceptable.

5.10 CLI Lock Operations

The command-line client supports lock operations:

Acquire a lock:

```
kahuna-cli --lock "jobs/send-email" --owner "worker-1" --expires 10000
```

Extend a lock:

```
kahuna-cli --extend-lock "jobs/send-email" --owner "worker-1" --expires 10000
```

Release a lock:

```
kahuna-cli --unlock "jobs/send-email" --owner "worker-1"
```

When using the CLI, you supply the owner name yourself. In the C# SDK, the owner is generated automatically as a GUID.

5.11 Common Patterns

5.11.1 Try-Once Pattern

Attempt the lock once. If it fails, skip the work or report the conflict:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "jobs/daily-report", TimeSpan.FromMinutes(5));

if (!lk.IsAcquired)
{
    Console.WriteLine("Another instance is already generating the report.");
    return;
}

await GenerateDailyReport();
```

This is appropriate when duplicate work is harmless (another instance is already doing the job) or when you have an external retry mechanism (a job scheduler that retries later).

5.11.2 Wait-and-Retry Pattern

Wait for the lock to become available:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "orders/checkout",
    expiry: TimeSpan.FromSeconds(15),
    wait: TimeSpan.FromSeconds(30),
    retry: TimeSpan.FromMilliseconds(100)
);

if (!lk.IsAcquired)
    throw new TimeoutException("Could not acquire checkout lock within 30
seconds.");

await ProcessCheckout();
```

5.11.3 Extend-While-Working Pattern

For work that may take longer than the initial lease:

```
await using KahunaLock lk = await client.GetOrCreateLock(
    "etl/load-customers", TimeSpan.FromSeconds(30));

if (!lk.IsAcquired) return;

foreach (var batch in customerBatches)
{
    await LoadBatch(batch);

    // Extend the lease before it expires.
    (bool ok, _) = await lk.TryExtend(TimeSpan.FromSeconds(30));
    if (!ok)
    {
        Console.WriteLine("Lost the lock. Stopping work.");
        return;
    }
}
```

Each iteration resets the lease clock. If the extension fails (because the lease expired between iterations), the code stops immediately to avoid conflicting with a new lock holder.

5.12 Summary

Kahuna's distributed locks combine leases and fencing tokens to solve two problems: a crashed lock holder (the lease expires and frees the lock) and a stale lock holder (the fencing token lets downstream systems reject outdated writes). Locks can wait and retry when contended, extend their lease for long-running work, and be queried for diagnostic purposes. Ephemeral locks trade durability for speed when replication is not needed.

The next chapter covers the distributed sequencer: generating globally unique, monotonically increasing numbers across the cluster.

6 The Distributed Sequencer

Many applications need unique, monotonically increasing numbers: order IDs, invoice numbers, event sequence numbers, log offsets. On a single machine, an auto-incrementing counter solves the problem. In a distributed system, there is no single machine to own the counter. Two nodes incrementing their own local counters will produce duplicates.

Kahuna's distributed sequencer generates globally unique, monotonically increasing numbers across the entire cluster. It does this without making every allocation wait for Raft consensus, thanks to a block-based allocation scheme that amortizes the cost of consensus over many values.

6.1 Creating a Sequence

A sequence is a named, persistent counter. Create one with a name:

```
KahunaSequence seq = await client.CreateSequence("order-ids");
```

The sequence starts at 0 with an increment of 1. You can customize the starting value, the increment, and an optional maximum:

```
KahunaSequence invoiceSeq = await client.CreateSequence(
    name: "invoice-numbers",
    initialValue: 1000,
    increment: 1,
    maxValue: 999999
);
```

This sequence starts allocating from 1001 (initialValue + increment) and stops at 999,999. If you try to allocate past the maximum, the client throws a `KahunaException` with a `MaxValueExceeded` response.

Creating a sequence that already exists throws a `KahunaException` with an `AlreadyExists` response. Use `GetSequence` to check whether a sequence exists before creating it.

6.2 Getting the Next Value

The simplest operation is requesting a single value:

```
long id = await client.NextSequenceValue("order-ids");
Console.WriteLine(id);  // 1
```

Each call returns the next value in the sequence. Values are unique across all clients and all nodes. Two clients calling `NextSequenceValue` concurrently on the same sequence will never receive the same number.

6.3 Reserving a Range

If you need many values at once, reserve a range:

```
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",
count: 100);
Console.WriteLine($"Start:   {range.Start},   End:   {range.End},   Count:
{range.Count}");
```

The returned range contains `count` consecutive values. Your application can use them locally without making further calls to Kahuna:

```
KahunaSequenceRange range = await client.ReserveSequenceRange("order-ids",
count: 50);

for (long id = range.Start; id <= range.End; id++)
{
    await InsertOrder(id);
}
```

Reserving a range is more efficient than calling `NextSequenceValue` in a loop because it requires only one round trip to the server.

6.4 How Block Allocation Works

Understanding block allocation explains why the sequencer is fast and what trade-offs it makes.

6.4.1 The Problem with Naive Allocation

A naive sequencer would store a counter in Raft and increment it on every request. Each request would require a Raft proposal, replication to a majority of nodes, and a durable write. At high throughput, this becomes a bottleneck: hundreds of Raft round trips per second for a single counter.

6.4.2 Block-Based Allocation

Kahuna uses a different approach. Instead of incrementing the counter by 1 on every request, the sequencer reserves a block of values in a single Raft operation.

Here is how it works:

1. The server stores a high-water mark for each sequence. The high-water mark is the highest value that has been reserved (not necessarily issued to clients).
2. When a client requests a value, the sequencer actor checks if it has any values left in its current block. If it does, it returns the next value from memory. No storage operation is needed.
3. When the current block is exhausted, the actor reserves a new block by bumping the high-water mark in the durable record. This is a single Raft compare-and-swap operation. The new block provides another batch of values to serve from memory.

The default block size is 1,000. This means one Raft commit is amortized over 1,000 values. The throughput of the sequencer is roughly 1,000 times higher than a naive per-value approach.

6.4.3 The Trade-Off: Gaps

Block allocation introduces the possibility of gaps in the sequence. If a node reserves a block of 1,000 values and only issues 50 before a restart or a leadership change, the remaining 950 values in that block are never issued. The next block starts from the new high-water mark.

For example:

1. The sequencer reserves block [1, 1000].

2. It issues values 1 through 50 to clients.
3. The node restarts.
4. The sequencer reserves a new block [1001, 2000].
5. Values 51 through 1000 are never issued.

This is the same trade-off that PostgreSQL makes with `CACHE` on sequences, or that SQL Server makes with sequence caching. The values are unique and monotonically increasing, but they are not contiguous.

6.4.4 When Gaps Are Not Acceptable

If your application requires gap-free numbering (some financial regulations require contiguous invoice numbers), set the block size to 1:

```
kahuna-server --sequencer-block-size 1
```

With a block size of 1, every allocation requires a Raft round trip. This is slower but produces no gaps. The right choice depends on your requirements.

6.4.5 Block Lease Revalidation

To bound stale-leader exposure, each block has a lease duration (default: 5 seconds). If a block sits in memory longer than the lease, the sequencer revalidates it against the durable record before serving more values. This prevents a stale leader from issuing values that conflict with values issued by a new leader.

6.5 Increment

The `increment` parameter controls the step size between values. With the default increment of 1, values go 1, 2, 3, 4. With an increment of 5:

```
KahunaSequence seq = await client.CreateSequence(
    "batch-ids", initialValue: 0, increment: 5);

long a = await client.NextSequenceValue("batch-ids");  // 5
long b = await client.NextSequenceValue("batch-ids");  // 10
long c = await client.NextSequenceValue("batch-ids");  // 15
```

The first value issued is `initialValue + increment`. Subsequent values increase by `increment`.

6.6 Maximum Value

Sequences can have an optional maximum. When the sequence reaches its maximum, further allocations fail:

```
KahunaSequence seq = await client.CreateSequence(
    "limited-ids", initialValue: 0, increment: 1, maxValue: 100);

// Allocate values until the sequence is exhausted.
for (int i = 0; i < 200; i++)
{
    try
    {
```

```
        long val = await client.NextSequenceValue("limited-ids");
        Console.WriteLine(val);
    }
    catch (KahunaException ex)
    {
        Console.WriteLine($"Sequence exhausted: {ex.Message}");
        break;
    }
}
```

When the maximum is reached, the client throws a `KahunaException`. The sequence does not wrap around.

If no maximum is set (the default), the sequence can grow until it reaches `long.MaxValue`.

6.7 Idempotent Allocation

In a distributed system, a client might send a request, experience a timeout, and retry. Without protection, the retry allocates new values. The client does not know whether the original request succeeded, so it may end up with two sets of values for the same logical operation.

Kahuna solves this with idempotency keys. Pass a key with your allocation request:

```
long id = await client.NextSequenceValue(
    "order-ids",
    idempotencyKey: "checkout-request-abc123"
);
```

If you retry with the same idempotency key, Kahuna returns the same value that was allocated on the first call. No new value is consumed. This is true even if a leadership change occurred between the first call and the retry, because the idempotent result is stored in the durable record.

Idempotent reserves work the same way:

```
KahunaSequenceRange range = await client.ReserveSequenceRange(
    "order-ids",
    count: 10,
    idempotencyKey: "batch-import-42"
);
```

Retrying with the same key and the same count returns the identical range. If you retry with the same key but a different count, Kahuna returns an `InvalidInput` error.

6.7.1 Retention Limits

Idempotency entries are retained for a limited time and number:

Setting	Default	Description
`--sequencer-idempotency-retention-max`	256	Maximum idempotency entries per sequence
`--sequencer-idempotency-retention-ttl`	600 seconds	Time window for idempotent replay

After an entry is evicted (by age or count), a retry with that key allocates fresh values instead of replaying the original allocation. Set these values based on how long your retry windows last.

6.8 Querying a Sequence

Read the metadata of an existing sequence:

```
KahunaSequence? seq = await client.GetSequence("order-ids");

if (seq != null)
{
    Console.WriteLine($"Name: {seq.Name}");
    Console.WriteLine($"Current value: {seq.CurrentValue}");
    Console.WriteLine($"Initial value: {seq.InitialValue}");
    Console.WriteLine($"Increment: {seq.Increment}");
    Console.WriteLine($"Max value: {seq.MaxValue}");
    Console.WriteLine($"Revision: {seq.Revision}");
}
```

The `CurrentValue` property is the high-water mark: the highest value that has been reserved (by block allocation), not the last value issued to a client. It may be higher than the last value any client received.

If the sequence does not exist, `GetSequence` returns null.

6.9 Updating a Sequence

`UpdateSequence` rewrites a sequence's parameters at runtime. Every field is optional. A field that is not set leaves the existing value unchanged.

```
await client.UpdateSequence(
    "order-ids",
    new SequenceUpdate
    {
        CurrentValue = 50000,
        Increment = 2,
        MaxValue = 999999
    }
);
```

The update starts a new **incarnation** of the sequence. The server waits out one block lease window before confirming the update. This ensures that no in-flight allocation from the old incarnation races with the new parameters.

The following fields are available on `SequenceUpdate`:

Field	Description
`CurrentValue`	New reserved high-water mark. The next issued value is this plus the increment.
`Increment`	New step between values. Must be positive.
`InitialValue`	New recorded starting value (descriptive only, does not move the counter).
`MaxValue`	New maximum.
`RemoveMaxValue`	Set to `true` to remove the maximum ceiling.
`BlockSize`	New per-sequence block size (overrides the server-wide `--sequencer-block-size`).
`RemoveBlockSize`	Set to `true` to revert the sequence to the server-wide block size.

6.9.1 Per-Sequence Block Size

By default, all sequences use the server-wide `--sequencer-block-size`. A per-sequence block size overrides this default for one sequence. Set it on create or update:

```
KahunaSequence seq = await client.CreateSequence(
    "invoice-numbers",
    initialValue: 0,
    increment: 1,
    blockSize: 1
);
```

A block size of 1 produces gap-free numbering for that sequence only. Other sequences continue to use the server-wide setting.

6.10 Deleting a Sequence

```
bool deleted = await client.DeleteSequence("order-ids");

if (deleted)
    Console.WriteLine("Sequence deleted.");
else
    Console.WriteLine("Sequence not found.");
```

Deleting a sequence removes its durable record. If you create a sequence with the same name afterward, it starts fresh from its initial value.

6.11 CLI Commands

Operation	Command
Create	`kahuna-cli --create-sequence "name"`
Create with options	`kahuna-cli --create-sequence "name" --initial-value 1000 --increment 5 --max-value 99999`
Get metadata	`kahuna-cli --get-sequence "name"`
Next value	`kahuna-cli --next-sequence "name"`
Next with idempotency	`kahuna-cli --next-sequence "name" --idempotency-key "req-123"`
Reserve range	`kahuna-cli --reserve-sequence "name" --count 100`
Update	`kahuna-cli --update-sequence "name" --current-value 50000`
Delete	`kahuna-cli --delete-sequence "name"`

6.12 Server Configuration

These server-side settings control sequencer behavior:

Setting	Default	Description
`--sequencer-block-size`	1000	Values reserved per Raft commit. 1 = gap-free (one commit per value).
`--sequencer-block-lease`	5 seconds	Time before a cached block is revalidated against the durable record.
`--sequencer-workers`	0 (auto)	Number of sequencer actor workers. 0 means auto-size: `max(8, CPU cores)`.
`--sequencer-max-sequences-per-actor`	10000	Maximum sequences one actor keeps in memory. Least recently used sequences are evicted.
`--sequencer-idempotency-retention-max`	256	Maximum idempotency entries per sequence record.
`--sequencer-idempotency-retention-ttl`	600 seconds	How long idempotent replays survive.

6.13 Use Cases

6.13.1 Order Numbers

```
await client.CreateSequence("orders", initialValue: 10000);

// In the order placement handler:
long orderId = await client.NextSequenceValue(
    "orders",
    idempotencyKey: $"place-order-{requestId}"
);
await SaveOrder(orderId, orderDetails);
```

The idempotency key ensures that a retried placement does not consume a second order number.

6.13.2 Event Offsets

```
await client.CreateSequence("event-log-offset");

// When appending an event:
KahunaSequenceRange batch = await client.ReserveSequenceRange("event-log-
offset", count: events.Count);

long offset = batch.Start;
foreach (var evt in events)
{
    evt.Offset = offset++;
    await AppendEvent(evt);
}
```

Reserving a range gives each event in the batch a unique, contiguous offset within that batch.

6.13.3 Sharded ID Generation

Multiple sequences can generate IDs for different shards:

```
// One sequence per shard, with non-overlapping ranges via increment and initial
value.
await client.CreateSequence("ids-shard-0", initialValue: 0, increment: 4);
await client.CreateSequence("ids-shard-1", initialValue: 1, increment: 4);
await client.CreateSequence("ids-shard-2", initialValue: 2, increment: 4);
await client.CreateSequence("ids-shard-3", initialValue: 3, increment: 4);

// Shard 0 produces: 4, 8, 12, 16, ...
// Shard 1 produces: 5, 9, 13, 17, ...
// Shard 2 produces: 6, 10, 14, 18, ...
// Shard 3 produces: 7, 11, 15, 19, ...
```

Each shard generates IDs independently. The increment of 4 and staggered initial values ensure the ranges never overlap.

6.14 Guarantees and Non-Guarantees

The sequencer provides these guarantees:

- **Uniqueness.** A value is issued at most once for the lifetime of the sequence, across all clients, nodes, and leadership changes.
- **Monotonic increase.** Values always climb. A later allocation returns a higher number than an earlier one (within the same ownership period).
- **Contiguous within a reserve.** A `ReserveSequenceRange(count: N)` call returns exactly N consecutive values.

The sequencer also provides **transparent forwarding.** When a client sends a sequence operation to a follower, the follower redirects the request to the partition leader within the hop budget. The same hop-budget mechanism that key-value operations use (described in Chapter 15) applies to sequences.

The sequencer does not guarantee:

- **No gaps.** Block allocation can leave gaps on restart, eviction, or leadership change. Set `--sequencer-block-size 1` for gap-free numbering at the cost of throughput.
- **Strict ordering across leadership changes.** During a leadership transition, the new leader starts from the next block boundary. This preserves uniqueness and monotonicity but may skip values.

6.15 Summary

Kahuna's distributed sequencer generates unique, monotonically increasing numbers across the cluster. Block-based allocation amortizes Raft consensus over many values, trading contiguity for throughput. Idempotency keys protect against duplicate allocation on retries. Sequences support custom increments, maximum values, and range reservations for batch workloads.

The next chapter covers transactions: how to read and write multiple keys atomically, with snapshot isolation and conflict detection.

7 Transactions

The previous chapters covered operations on individual keys, locks, and sequences. Each operation targets a single resource and completes in one round trip. Many real workloads need more than that. Transferring a balance between two accounts requires reading both keys, checking constraints, and writing both keys as a single atomic unit. If the process crashes after writing one key but before writing the other, the data is inconsistent.

Kahuna supports multi-key transactions with snapshot isolation. A transaction groups multiple reads and writes into a unit that either commits entirely or rolls back entirely. No partial results are visible to other clients.

Kahuna offers two transaction models:

- **Script transactions** send a self-contained script to the server. The server parses, plans, and executes the entire script as one atomic unit. This is the simpler model and the better choice when the logic can be expressed as a short sequence of key-value operations.
- **Interactive transactions** open a session between the client and the server. The client sends operations one at a time within the session, reads results, makes decisions in application code, and then commits or rolls back. This model is necessary when the transaction logic depends on intermediate results that only the application can evaluate.

Both models use two-phase commit (2PC) internally when the transaction touches keys on multiple partitions.

In both models, the server is the transaction coordinator. The client sends operations and a commit request, but the server drives the 2PC protocol and applies the writes. If the client crashes after it sends the commit request, the server still completes the transaction. The commit decision does not depend on the client remaining connected. This makes Kahuna transactions resilient to client failures at the critical moment.

7.1 Script Transactions

A script transaction is a string of key-value commands that the server executes atomically. If any command fails, the entire transaction is aborted and no changes are applied.

7.1.1 A Simple Script

```
string script = @"
    SET accounts/alice '950'
    SET accounts/bob '1050'
";

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script);

Console.WriteLine(result.Type);  // Set
```

This script writes two keys in a single atomic operation. Both writes succeed together or fail together. The `Type` property on the result indicates the outcome of the last command in the script.

7.1.2 Parameters

Hard-coding values into a script string is inconvenient and error-prone. Use parameters to pass values at runtime:

```
string script = "SET @key @value";

var parameters = new List<KeyValueParameter>
{
    new() { Key = "@key", Value = "accounts/alice" },
    new() { Key = "@value", Value = "950" }
};

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script, parameters:
parameters);
```

A parameter is a placeholder that starts with `@`. At execution time, Kahuna replaces each placeholder with the corresponding value from the parameter list.

7.1.3 The Result Object

`ExecuteKeyValueTransactionScript` returns a `KahunaKeyValueTransactionResult`. This object contains the outcome of the script:

Property	Type	Description
`Type`	`KeyValueResponseType`	The response type of the last command
`Values`	`List<...>`	List of values returned by GET commands
`FirstValue`	`byte[]?`	The value from the first entry in the list
`FirstValueAsString`	`string?`	The first value decoded as UTF-8
`FirstRevision`	`long`	The revision from the first entry
`TimeElapsedMs`	`int`	Server-side execution time

When a script contains one or more GET commands, the returned values appear in the `Values` list in order. Each entry carries the key, value, revision, expiration time, and last modification timestamp.

```
string script = @"
    SET users/1 'Alice'
    SET users/2 'Bob'
    GET users/1
";

KahunaKeyValueTransactionResult result =
    await client.ExecuteKeyValueTransactionScript(script);

Console.WriteLine(result.FirstValueAsString);  // Alice
```

7.1.4 Pre-Hashing for Reuse

Every time you call `ExecuteKeyValueTransactionScript` with a string script, the server must parse it. If you execute the same script many times (with different parameters), you can avoid repeated parsing by loading the script once:

```
KahunaTransactionScript transferScript = client.LoadTransactionScript(
    "SET @from @fromBalance SET @to @toBalance"
);
```

`LoadTransactionScript` computes a Blake3 hash of the script text. On the first execution, the server parses the script and caches the result keyed by this hash. On subsequent executions, the server finds the cached plan and skips parsing.

Execute the pre-hashed script with `Run`:

```
KahunaKeyValueTransactionResult result = await transferScript.Run(
    parameters: new()
    {
        new() { Key = "@from", Value = "accounts/alice" },
        new() { Key = "@fromBalance", Value = "900" },
        new() { Key = "@to", Value = "accounts/bob" },
        new() { Key = "@toBalance", Value = "1100" }
    }
);
```

7.1.5 Script Priority

By default, script transactions run at `Normal` priority. When the server is at its concurrency ceiling, you can influence which transactions start first:

```
KahunaKeyValueTransactionResult result = await transferScript.Run(
    priority: TransactionPriority.High,
    parameters: new()
    {
        new() { Key = "@key", Value = "config/critical" },
        new() { Key = "@value", Value = "updated" }
    }
);
```

Priority levels, from lowest to highest: `Background`, `Low`, `Normal`, `High`, `Critical`. Priority only affects admission order when the server is saturated. Below the concurrency ceiling, all transactions start immediately regardless of priority. A higher priority never preempts a transaction that is already running.

Each level has a different intended use:

- `Background`: bulk or deferrable work such as compaction sweeps or analytics. Yields to all other levels.
- `Low`: housekeeping tasks that can tolerate delay under load.
- `Normal`: the default for application traffic.
- `High`: latency-sensitive operations that should run ahead of normal traffic during saturation.

- `Critical`: reserved for maintenance transactions or work whose delay would harm availability. Do not tag ordinary traffic as `Critical`, because doing so defeats the ordering.

The next chapter covers the full script language in detail: variables, conditionals, loops, built-in functions, and multi-statement control flow.

7.2 Interactive Transactions

When the transaction logic requires reading a value from the server, making a decision in your application, and then writing based on that decision, use an interactive transaction session.

7.2.1 Starting a Session

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    }
);
```

`StartTransactionSession` opens a session with the server. The session receives a unique transaction ID and is pinned to a coordinator node for its lifetime. All operations within the session are part of the same transaction.

The session implements `IAsyncDisposable`. When the `await using` block ends:

- If the session is still in `Pending` status (no explicit commit or rollback), it rolls back automatically.
- If the session was committed or rolled back, disposal is a no-op.

7.2.2 Session Operations

Inside a session, you can perform the same key-value operations as the main `KahunaClient`, but they execute within the transaction scope:

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions { Timeout = 5000 }
);

KahunaKeyValue alice = await session.GetKeyValue("accounts/alice");
KahunaKeyValue bob = await session.GetKeyValue("accounts/bob");

long aliceBalance = long.Parse(alice.ValueAsString ?? "0");
long bobBalance = long.Parse(bob.ValueAsString ?? "0");

if (aliceBalance >= 100)
{
   await session.SetKeyValue("accounts/alice", (aliceBalance - 100).ToString());
    await session.SetKeyValue("accounts/bob", (bobBalance + 100).ToString());
    await session.Commit();
```

```
}
else
{
    await session.Rollback();
}
```

This transaction reads two balances, checks a constraint in application code, and writes updated balances. The reads and writes are part of a single atomic unit. If another client modifies either account between the reads and the commit, the transaction may be aborted (depending on the locking mode and read validation settings).

The session supports these operations:

Operation	Method
Set a key	`SetKeyValue(key, value, ...)`
Get a key	`GetKeyValue(key, ...)`
Check existence	`ExistsKeyValue(key, ...)`
Extend TTL	`ExtendKeyValue(key, expiresMs, ...)`
Delete a key	`DeleteKeyValue(key, ...)`
Delete many keys	`DeleteManyKeyValues(keys, ...)`
Get by prefix	`GetByBucket(prefixKey, ...)`
Get by range	`GetByRange(prefix, startKey, ..., endKey, ...)`
Compare-value-and-swap	`TryCompareValueAndSetKeyValue(key, value, compareValue, ...)`
Compare-revision-and-swap	`TryCompareRevisionAndSetKeyValue(key, value, compareRevision, ...)`

All operations check that the session is still in `Pending` status. If you try to operate on a committed, rolled back, or aborted session, the client throws a `KahunaException`.

7.2.3 Committing

Call `Commit` to finalize the transaction:

```
bool committed = await session.Commit();
```

The commit drives a two-phase commit protocol across all partitions that the transaction touched. If the commit succeeds, `committed` is true and the session status moves to `Committed`. All writes become visible to other clients.

If a conflict is detected (another transaction modified a key that this transaction read or wrote), the commit throws a `KahunaException` with an `Aborted` error code. The session moves to the `Aborted` status. This is terminal: you cannot retry a commit on an aborted session. Start a new session instead.

If a transient error occurs (network timeout, temporary leader unavailability), the session returns to `Pending` status. You can retry the commit.

7.2.4 Rolling Back

Call `Rollback` to discard all changes:

```
await session.Rollback();
```

After a rollback, the session status is `Rolledback`. All acquired locks are released and no writes from this session are applied.

If you do not call either `Commit` or `Rollback`, the `await using` block calls `Rollback` automatically when the session is disposed.

7.2.5 Transaction Status

A session moves through these states:

Status	Meaning
`Pending`	The session is active. Operations, commit, and rollback are allowed.
`Finalizing`	A commit or rollback is in flight. No new operations are accepted. If the finalize fails, the session returns to `Pending`.
`Committed`	The transaction committed. Terminal state.
`Rolledback`	The transaction rolled back. Terminal state.
`Aborted`	A commit was definitively rejected (conflict or permanent 2PC failure). Terminal state.

7.3 Locking Modes

Kahuna interactive sessions support two locking strategies: pessimistic (the default) and optimistic.

7.3.1 Pessimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Pessimistic
}
```

In pessimistic mode, the session acquires an exclusive lock on every key it reads or writes. This means:

- `GetKeyValue("k")` acquires an exclusive lock on `k` before reading.
- `SetKeyValue("k", "v")` acquires an exclusive lock on `k` before writing.
- `GetByBucket("prefix/")` acquires an exclusive prefix lock before scanning.
- `GetByRange(...)` acquires an exclusive range lock before scanning.

No other transaction can read or write a locked key until this transaction commits or rolls back. This prevents conflicts at the cost of concurrency: if two transactions touch the same key, one waits (or aborts) while the other holds the lock.

Pessimistic locking is the safer default. Use it when conflicts are common or when you cannot tolerate aborted transactions.

7.3.2 Optimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic
}
```

In optimistic mode, the session acquires exclusive locks only on writes, not on reads:

- `GetKeyValue("k")` reads without locking.
- `SetKeyValue("k", "v")` acquires an exclusive lock on `k` before writing.

This allows higher concurrency for read-heavy workloads. Multiple transactions can read the same keys simultaneously. Conflicts are detected at commit time: if a key that this transaction read was modified by another transaction after the read, the commit is rejected.

Optimistic locking works best when conflicts are rare. If conflicts are frequent, transactions abort and retry repeatedly, which wastes work.

7.4 Read Validation and Write-Skew Detection

By default (`ReadValidation.None`), the commit does not check whether keys that the transaction read were modified by other transactions. This is efficient but allows a class of anomaly called write-skew.

7.4.1 The Write-Skew Problem

Consider two doctors on call. A business rule says at least one doctor must remain on call. Both doctors check the on-call roster, see the other is on call, and each removes themselves. Both transactions commit because neither wrote to the same key. The result: zero doctors on call.

7.4.2 Enabling Read Validation

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic,
    ReadValidation = ReadValidation.TrackAndValidate
}
```

With `TrackAndValidate`, the session records every key it reads (the read set). At commit time, the server checks whether any key in the read set was modified after this transaction read it. If so, the commit is aborted. This prevents write-skew at the cost of additional validation work during commit.

`TrackAndValidate` is most useful with optimistic locking. Pessimistic locking already prevents concurrent modifications through exclusive locks, so read validation adds little value in that mode.

7.5 Conflict Policy

By default, a live write intent blocks every other writer until the transaction commits, rolls back, or expires. This is the `Normal` conflict policy. Kahuna also provides a `Yield` policy for background work that must never block foreground traffic.

7.5.1 Yielding Transactions

```
await using KahunaTransactionSession session = await client.StartTransactionSession(
    new KahunaTransactionOptions
    {
        Timeout = 30000,
        Locking = KeyValueTransactionLocking.Pessimistic,
        ConflictPolicy = TransactionConflictPolicy.Yield
    }
);
```

A yielding transaction gives way to normal transactions. When a `Normal` transaction (or a plain write with no transaction) meets an un-pinned yielding intent on a key, the normal writer takes the key over. The yielding transaction aborts at its next touch of that key or at commit.

Yielding transactions are designed for maintenance operations: bulk rewrites, backfills, and compaction sweeps. These operations should not make foreground work fail or wait.

Two constraints apply to yielding sessions:

1. Prefix locks and range locks are not allowed. The takeover rule covers point-key intents only. A prefix or range lock acquire in a yielding session is refused.
2. Two yielding transactions conflict with each other the same way two normal transactions do.

The conflict policy is separate from `TransactionPriority`. Priority governs admission order only. The conflict policy changes locking semantics.

7.6 RetryableTransaction

Many applications follow the same pattern: start a session, do work, commit, and retry on conflict. Kahuna provides a helper that encapsulates this pattern:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
        KahunaKeyValue counter = await session.GetKeyValue("stats/visits");
        long count = long.Parse(counter.ValueAsString ?? "0");
        await session.SetKeyValue("stats/visits", (count + 1).ToString());
        await session.Commit(ct);
```

```
    }
);
```

`RetryableTransaction` starts a session, runs your callback, and handles retry logic automatically. If the callback throws a `KahunaException` with an `Aborted`, `MustRetry`, or `AlreadyLocked` error code, the method waits and retries with a new session.

The retry strategy uses decorrelated jitter backoff:

- Up to 10 retries.
- The first retry delay has a median of 50 milliseconds.
- Each subsequent delay grows with jitter but is capped at 500 milliseconds.
- If all 10 retries fail, the method throws a `KahunaException` with an `Aborted` error code.

Any exception that is not `Aborted`, `MustRetry`, or `AlreadyLocked` propagates immediately without retry. This includes application logic errors, cancellation, and permanent failures.

7.7 Transaction Options Reference

Option	Type	Default	Description
`Timeout`	`int`	5000 ms	How long the transaction may live once started. The server releases the session and its locks when the timeout elapses.
`AdmissionWaitMs`	`int`	0 (server default)	How long the client waits for an admission slot when the server is at its session ceiling. If the wait is exhausted, the call fails with `AdmissionRefused`.
`LockingKeyValueTransactionLocking`		`Pessimistic`	Locking strategy: `Pessimistic` locks on reads and writes, `Optimistic` locks on writes only.
`AutoCommit`	`bool`	`true`	Whether the server should auto-commit the transaction on dispose.
`ReadValidation`	`ReadValidation`	`None`	`None` skips read-set validation. `TrackAndValidate` checks for write-skew at commit time.
`DecisionDurability`	`DecisionDurability`	`BestEffort`	`BestEffort` returns the outcome to the client before the 2PC decision record is durably replicated. `Durable` waits until the decision is committed through Raft.
`ReadTimestamp`	`HLCTimestamp`	0 (latest)	A snapshot timestamp for reads. Zero means reads observe the current committed state. A non-zero value pins reads to that point in time.
`Priority`	`TransactionPriority`	`Normal`	Admission priority when the server is saturated. Levels: `Background`, `Low`, `Normal`, `High`, `Critical`.

7.7.1 Decision Durability

The `DecisionDurability` option controls how the server handles the 2PC decision record:

- **BestEffort** (default): The server tells the client the outcome as soon as the two-phase commit protocol completes in memory. The decision is persisted asynchronously. This is

faster, but in a narrow crash window the decision could be lost and the transaction outcome becomes unknown.

- **Durable**: The server persists the decision record through Raft consensus before returning the outcome to the client. This adds latency but guarantees that the outcome survives a crash.

For most workloads, `BestEffort` is sufficient. Use `Durable` when you need an external guarantee that a committed transaction will remain committed even if the coordinator node crashes immediately after the commit response.

7.7.2 Snapshot Reads

The `ReadTimestamp` option pins all reads in the session to a specific point in time. This is useful for reporting queries that must see a consistent snapshot while other transactions continue to write:

```
await using KahunaTransactionSession session = await client.StartTransactionSession(
    new KahunaTransactionOptions
    {
        Timeout = 10000,
        Locking = KeyValueTransactionLocking.Optimistic,
        ReadTimestamp = snapshotTimestamp
    }
);

KahunaKeyValue balance = await session.GetKeyValue("accounts/alice");
```

When `ReadTimestamp` is non-zero, the server serves the value as of that timestamp without recording a read dependency. This means the read does not participate in conflict detection, because it reads historical data that cannot be changed.

7.8 Common Patterns

7.8.1 Read-Modify-Write

The most common transaction pattern: read a value, compute a new value, and write it back.

```
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue item = await session.GetKeyValue("inventory/widget-a");
        int stock = int.Parse(item.ValueAsString ?? "0");

        if (stock < 1)
            throw new InvalidOperationException("Out of stock.");

       await session.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
        await session.Commit(ct);
```

```
    }
);
```

Wrapping this in `RetryableTransaction` ensures that if another client modifies the same key concurrently, the transaction retries with fresh data.

7.8.2 Multi-Key Transfer

Move a value from one key to another atomically:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue from = await session.GetKeyValue("accounts/alice");
        KahunaKeyValue to = await session.GetKeyValue("accounts/bob");

        long fromBalance = long.Parse(from.ValueAsString ?? "0");
        long toBalance = long.Parse(to.ValueAsString ?? "0");
        long amount = 100;

        if (fromBalance < amount)
            throw new InvalidOperationException("Insufficient funds.");

              await session.SetKeyValue("accounts/alice", (fromBalance -
amount).ToString());
                 await session.SetKeyValue("accounts/bob", (toBalance +
amount).ToString());
        await session.Commit(ct);
    }
);
```

If `accounts/alice` and `accounts/bob` live on different partitions, Kahuna uses 2PC to commit both writes atomically.

7.8.3 Conditional Insert

Insert a key only if it does not already exist, as part of a larger transaction:

```
await using KahunaTransactionSession session = await
client.StartTransactionSession(
    new KahunaTransactionOptions { Timeout = 5000 }
);

KahunaKeyValue existing = await session.GetKeyValue("users/alice@example.com");

if (existing.Success)
{
    Console.WriteLine("User already exists.");
    await session.Rollback();
}
else
```

```
{
    await session.SetKeyValue("users/alice@example.com", "Alice");
    await session.SetKeyValue("user-count", "1");
    await session.Commit();
}
```

7.9 Summary

Kahuna provides two transaction models. Script transactions send a self-contained script to the server for atomic execution. Interactive transactions open a session and let the client make decisions based on intermediate results. Both models use 2PC for multi-partition atomicity.

Pessimistic locking (the default) acquires exclusive locks on every read and write, preventing conflicts at the cost of concurrency. Optimistic locking acquires locks only on writes, allowing higher concurrency but detecting conflicts at commit time. Read validation with `TrackAndValidate` prevents write-skew anomalies by checking the read set at commit time.

`RetryableTransaction` wraps the common start, work, commit, retry loop with decorrelated jitter backoff.

The next chapter covers the transaction script language in full: variables, conditionals, loops, built-in functions, and error handling.

8 The Script Language

Chapter 6 introduced script transactions: self-contained programs that the server executes atomically. This chapter is the complete reference for the script language. It covers the syntax, data types, commands, control flow, expressions, built-in functions, and transaction options.

8.1 Language Overview

The Kahuna script language is a small, domain-specific language designed for one purpose: reading and writing key-value data inside a transaction. It is not a general-purpose programming language. It has no file I/O, no networking, and no classes. What it does have is direct access to Kahuna's key-value operations, conditional logic, loops, and built-in functions for type checking and data manipulation.

Scripts are case-insensitive. `SET`, `set`, and `Set` are the same command. String literals use single quotes (`'hello'`) or double quotes (`"hello"`). A string literal may span multiple lines; the line breaks are kept in the value. Identifiers that collide with reserved words can be escaped with backticks (`` `delete` ``). Backtick identifiers must stay on one line.

A script is a sequence of statements. The server parses the script into an abstract syntax tree, executes the statements in order, and returns the result of the last statement (or the value passed to `RETURN`).

8.2 Data Types

The language supports five data types:

Type	Examples	Notes
Integer	`42`, `-7`, `0`	64-bit signed integer (`long`)
Float	`3.14`, `-0.5`, `1.0`	64-bit double-precision floating point
String	`'hello'`, `"world"`	UTF-8 strings; may span multiple lines
Boolean	`true`, `false`	Case-insensitive
Null	`null`	Represents the absence of a value

Arrays are created with the range operator (`1..10`) or returned by commands like `GET BY BUCKET` and `SCAN BY PREFIX`. You cannot construct an array literal directly.

8.3 Variables

Use `LET` to assign a value to a variable:

```
LET x = 42
LET name = 'Alice'
LET total = x + 10
LET found = true
```

Variables are dynamically typed. You can reassign a variable to a different type:

```
LET x = 42
LET x = 'now a string'
```

Variable names follow identifier rules: letters, digits, and underscores, starting with a letter or underscore.

8.4 Key-Value Commands

8.4.1 SET

`SET` writes a value to a key:

```
SET mykey 'hello world'
```

The key can be an identifier, a string literal, or a placeholder:

```
SET 'users/alice' 'active'
SET @key @value
```

8.4.1.1 SET Flags

Flags modify the behavior of `SET`. You can combine multiple flags on one command.

NX (Not Exists): Write only if the key does not exist yet.

```
SET mykey 'first' NX
```

If `mykey` already exists, the SET does not modify it.

XX (Exists): Write only if the key already exists.

```
SET mykey 'updated' XX
```

If `mykey` does not exist, the SET does nothing.

CMP (Compare Value): Write only if the current value equals the given expression.

```
SET counter 'closed' CMP 'open'
```

This sets `counter` to `'closed'` only if its current value is `'open'`.

CMPREV (Compare Revision): Write only if the current revision equals the given number.

```
SET config 'new-value' CMPREV 5
```

This sets `config` only if its revision is exactly 5. This is useful for optimistic concurrency: read the revision, do your work, then write only if nobody else changed the key.

EX (Expires): Set a TTL in milliseconds.

```
SET session 'token-abc' EX 30000
```

The key expires and is deleted after 30 seconds.

NOREV (No Revision): Do not track the revision for this write.

```
SET cache/item 'data' NOREV
```

This is useful for cache-like workloads where you do not need version tracking.

Combining flags:

```
SET mykey 'value' NX EX 60000
```

This creates the key only if it does not exist, with a 60-second TTL.

8.4.2 ESET

`ESET` is the ephemeral variant of `SET`. It works the same way but stores the value in memory only, without Raft replication. Ephemeral data does not survive a node restart.

```
ESET cache/user 'data' EX 5000
```

`ESET` supports all the same flags as `SET`: `NX`, `XX`, `CMP`, `CMPREV`, `EX`, `NOREV`.

8.4.3 GET

`GET` reads a value by key:

```
GET mykey
```

To capture the result in a variable, use `LET`:

```
LET value = GET mykey
```

8.4.3.1 Reading a Specific Revision

Read the value at a specific revision number:

```
LET old = GET mykey AT 3
```

This returns the value that `mykey` had at revision 3.

8.4.3.2 Snapshot Reads

Read the value as it was at a specific point in time (Unix timestamp in milliseconds):

```
LET snapshot = GET mykey AS OF 1700000000000
```

This returns the value that `mykey` had at the given timestamp.

8.4.4 EGET

`EGET` reads from ephemeral storage:

```
LET cached = EGET cache/user
```

`EGET` supports the same `AT` and `AS OF` variants as `GET`.

8.4.5 EXISTS

Check whether a key exists:

```
LET found = EXISTS mykey
```

`EXISTS` also supports `AT` (revision) and `AS OF` (timestamp) variants.

8.4.6 EEXISTS

Check whether an ephemeral key exists:

```
LET found = EEXISTS cache/user
```

8.4.7 DELETE

Delete a key:

```
DELETE mykey
```

8.4.8 EDELETE

Delete an ephemeral key:

```
EDELETE cache/user
```

8.4.9 EXTEND

Extend the TTL of a key by a given number of milliseconds:

```
EXTEND mykey 30000
```

This resets the expiry clock. The key will live for another 30 seconds from now.

8.4.10 EEXTEND

Extend the TTL of an ephemeral key:

```
EEXTEND cache/user 5000
```

8.4.11 GET BY BUCKET

Retrieve all keys that share a common prefix (bucket). The prefix is everything up to and including the last `/` in the key name:

```
LET items = GET BY BUCKET 'users/'
```

This returns an array of key-value results for all keys whose names start with `users/`. You can iterate over the results with a `FOR` loop.

`GET BY BUCKET` supports `AS OF` for snapshot reads:

```
LET items = GET BY BUCKET 'users/' AS OF 1700000000000
```

8.4.12 EGET BY BUCKET

The ephemeral variant:

```
LET items = EGET BY BUCKET 'cache/'
```

8.4.13 SCAN BY PREFIX

Scan all keys that match a given prefix:

```
LET results = SCAN BY PREFIX 'config/'
```

Like `GET BY BUCKET`, this returns an array. It supports `AS OF` for snapshot reads.

8.4.14 ESCAN BY PREFIX

The ephemeral variant:

```
LET results = ESCAN BY PREFIX 'cache/config/'
```

8.5 Placeholders

Placeholders let you pass values into a script from the calling application. A placeholder starts with `@`:

```
SET @key @value EX @ttl
```

In C#, pass placeholders as a list of `KeyValueParameter` objects:

```
string script = "SET @key @value EX @ttl";

var result = await client.ExecuteKeyValueTransactionScript(
    script,
    parameters: [
        new() { Key = "@key", Value = "users/alice" },
        new() { Key = "@value", Value = "active" },
        new() { Key = "@ttl", Value = "30000" }
    ]
);
```

Placeholders prevent injection and let you reuse the same compiled script with different values. When you use `LoadTransactionScript` (covered in Chapter 6), the server caches the parsed AST and reuses it across calls. Only the placeholder values change.

8.6 Expressions

The language supports arithmetic, comparison, and logical expressions.

8.6.1 Arithmetic

```
LET a = 10 + 5      // 15
LET b = 10 - 3      // 7
LET c = 4 * 3       // 12
LET d = 10 / 2      // 5
```

Arithmetic works on integers and floats. Mixing types promotes the result to float.

8.6.2 Comparison

```
LET eq  = (x = 5)        // equality (single = or ==)
LET neq = (x != 5)       // not equal (or <>)
LET lt  = (x < 10)       // less than
LET gt  = (x > 0)        // greater than
LET lte = (x <= 100)     // less than or equal
LET gte = (x >= 1)       // greater than or equal
```

Both `=` and `==` test equality. Both `!=` and `<>` test inequality.

8.6.3 Logical Operators

```
LET both = (a > 0) && (b > 0)   // AND
LET either = (a > 0) || (b > 0) // OR
LET no = !(a > 0)               // NOT
```

You can also use the word forms: `AND`, `OR`, `NOT`.

```
LET both = (a > 0) AND (b > 0)
```

8.6.4 Ranges

The `..` operator creates an array of integers:

```
LET nums = 1..10
```

This creates an array `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

8.6.5 Array Indexing

Access array elements by index (zero-based):

```
LET nums = 1..5
LET first = nums[0]    // 1
LET third = nums[2]    // 3
```

8.6.6 Special Expressions

NOT FOUND: Tests whether the last read statement (`GET`, `EXISTS`, `EGET`, `EEXISTS`, `GET BY BUCKET`, `SCAN BY PREFIX`, and their ephemeral forms) found no data. An `EXISTS` that finds a key counts as found. If no read statement ran before the guard, the script raises an error.

```
LET value = GET mykey
IF value = NOT FOUND THEN
    SET mykey 'initialized'
END
```

NOT SET: Tests whether the last write statement (`SET`, `ESET`) did not take effect (for example, a `SET ... NX` on a key that already exists). If no write statement ran before the guard, the script raises an error.

```
SET mykey 'first' NX
IF NOT SET THEN
    RETURN 'key already existed'
END
```

NOT DELETED: Tests whether the last `DELETE` or `EDELETE` found nothing to delete. If no delete statement ran before the guard, the script raises an error.

NOT EXTENDED: Tests whether the last `EXTEND` or `EEXTEND` found nothing to extend. If no extend statement ran before the guard, the script raises an error.

8.7 Control Flow

8.7.1 IF / THEN / ELSE / END

Conditional execution:

```
LET status = GET account/status

IF status = 'active' THEN
    SET account/last-login current_time()
END
```

With an else branch:

```
LET balance = GET account/balance

IF to_long(balance) >= 100 THEN
    SET account/balance to_string(to_long(balance) - 100)
ELSE
    THROW 'Insufficient balance'
END
```

Conditions can use any expression. Nested `IF` statements are allowed.

8.7.2 FOR / IN / DO / END

Loop over a range or an array:

```
FOR i IN 1..5 DO
    SET concat('key/', to_string(i)) to_string(i * 10)
END
```

This creates five keys: `key/1` through `key/5` with values `10` through `50`.

You can loop over the results of a bucket query:

8.7.3 SWITCH / CASE / ELSE / END

`SWITCH` evaluates its subject once and runs the body of the first `CASE` whose value equals it. There is no fall-through: only one body runs per match. A `CASE` can list several values, separated by commas. An optional `ELSE` runs when no `CASE` matches. Values are expressions, not only literals.

```
LET tier = GET 'account/tier'

SWITCH tier
    CASE 'free' THEN
        SET 'account/limit' '100'
    CASE 'pro', 'enterprise' THEN
        SET 'account/limit' '10000'
    ELSE
        SET 'account/limit' '500'
END
```

In a pessimistic transaction, the server locks the keys of every `CASE` body and the `ELSE` body, the same way it locks both branches of an `IF`.

You can also loop over the results of a bucket query:

```
LET items = GET BY BUCKET 'users/'

FOR item IN items DO
    SET concat('backup/', item) item
END
```

8.8 Transactions in Scripts

8.8.1 Implicit Transactions

When a script contains multiple statements without a `BEGIN` block, the server wraps the entire script in an auto-commit transaction. If any statement fails, the entire script rolls back.

```
SET account/a '900'
SET account/b '1100'
```

Both writes succeed together or fail together.

8.8.2 Explicit Transactions

Use `BEGIN` and `END` to define a transaction block explicitly. Inside the block, you can use `COMMIT` or `ROLLBACK`:

```
BEGIN
    LET balance = GET account/a
    LET amount = 100

    IF to_long(balance) >= amount THEN
        SET account/a to_string(to_long(balance) - amount)

        LET target = GET account/b
        SET account/b to_string(to_long(target) + amount)
        COMMIT
    ELSE
        ROLLBACK
    END
END
```

If you call `ROLLBACK`, all writes inside the `BEGIN` block are discarded. If you call `COMMIT`, the writes become durable. If you reach `END` without calling either, the transaction commits automatically.

8.8.3 Transaction Options

`BEGIN` accepts options in parentheses:

```
BEGIN (locking = optimistic, timeout = 5000)
    LET a = GET account/a
    LET b = GET account/b
    SET account/a to_string(to_long(a) - 50)
    SET account/b to_string(to_long(b) + 50)
END
```

Available options:

Option	Values	Default	Description
`locking`	`pessimistic`, `optimistic`	`pessimistic`	Lock acquisition strategy
`autoCommit`	`true`, `false`	`true`	Commit automatically when the block ends without an explicit COMMIT or ROLLBACK
`asyncRelease`	`true`, `false`	`false`	Release locks asynchronously after commit
`timeout`	integer (ms)	Server default	Maximum execution time for the transaction
`admissionWait`	integer (ms)	Server default	Maximum time to wait for a transaction slot
`readValidation`	`true`, `false`	`true`	Validate the read set at commit time; disable to skip read-set checks for write-only scripts
`decisionDurability`	`persistent`, `ephemeral`	`persistent`	Durability level of the transaction decision record
`snapshot`	integer (Unix ms)	0 (disabled)	Read all keys as of this timestamp
`priority`	`background`, `low`, `normal`, `high`, `critical`	`normal`	Transaction scheduling priority

8.9 RETURN

`RETURN` ends the script and returns a value to the caller:

```
LET x = GET mykey

IF x = NOT SET THEN
    RETURN 'not found'
END

RETURN concat('found: ', x)
```

Without a `RETURN`, the script returns the result of the last executed statement.

`RETURN` without a value ends the script immediately:

```
LET x = GET mykey
IF x = NOT SET THEN
    RETURN
END
SET mykey to_string(to_long(x) + 1)
```

8.10 SLEEP

`SLEEP` pauses execution for a given number of milliseconds:

```
SLEEP 1000
```

Use `SLEEP` with caution. It holds the transaction open during the pause. The primary use case is testing and debugging, not production workloads.

8.11 THROW

`THROW` raises an error and aborts the script:

```
LET balance = GET account/balance

IF to_long(balance) < 0 THEN
    THROW 'Balance cannot be negative'
END
```

The error message is returned to the caller in the transaction result's `Reason` field.

8.12 Built-in Functions

The language provides built-in functions for type checking, type conversion, string manipulation, math, and metadata access.

8.12.1 Type Checking Functions

These functions test the type of a value and return a boolean:

Function	Description
`is_long(x)` / `is_int(x)` / `is_integer(x)`	True if x is an integer
`is_float(x)` / `is_double(x)`	True if x is a float
`is_string(x)` / `is_str(x)`	True if x is a string
`is_bool(x)` / `is_boolean(x)`	True if x is a boolean
`is_null(x)`	True if x is null
`is_array(x)`	True if x is an array

```
LET x = 42
RETURN is_long(x)    // true

LET y = 'hello'
RETURN is_string(y)  // true

LET z = 1..10
RETURN is_array(z)   // true
```

8.12.2 Type Conversion Functions

These functions convert a value from one type to another:

Function	Description
`to_long(x)` / `to_int(x)` / `to_integer(x)` / `to_number(x)`	Convert to integer
`to_float(x)` / `to_double(x)`	Convert to float
`to_string(x)` / `to_str(x)`	Convert to string
`to_bool(x)` / `to_boolean(x)`	Convert to boolean
`to_json(x)`	Serialize to a JSON string

Values stored in Kahuna are byte arrays. When you read a value with `GET`, the result is a string. To do arithmetic on it, convert it to an integer or float first:

```
LET raw = GET counter
LET n = to_long(raw)
SET counter to_string(n + 1)
```

Conversion rules:

- `to_long(1.7)` truncates to `1`.
- `to_long(true)` returns `1`. `to_long(false)` returns `0`.
- `to_float(true)` returns `1`. `to_float(false)` returns `0`.
- `to_bool(0)` returns `false`. `to_bool(1)` returns `true`.
- `to_string(null)` returns an empty string.
- `to_json` works on integers, floats, strings, and arrays. It produces a JSON representation of the value.

8.12.3 String Functions

Function	Description
`upper(x)`	Convert string to uppercase
`lower(x)`	Convert string to lowercase
`concat(a, b)`	Concatenate two strings
`len(x)` / `length(x)`	Return the length of a string
`substring(s, start, length)`	Return a substring starting at `start` with the given `length`
`starts_with(s, prefix)`	True if string `s` starts with `prefix`
`ends_with(s, suffix)`	True if string `s` ends with `suffix`
`index_of(s, sub)`	Return the zero-based index of `sub` in `s`, or −1 if not found
`split(s, delimiter)`	Split `s` by `delimiter` and return an array of strings
`trim(s)`	Remove leading and trailing whitespace from `s`

```
LET name = 'Alice'
RETURN upper(name)         // 'ALICE'
RETURN lower(name)         // 'alice'
```

```
RETURN concat('Hi, ', name) // 'Hi, Alice'
RETURN len(name)            // 5
```

`concat` takes exactly two string arguments. To concatenate more than two values, nest the calls:

```
LET full = concat(concat(first, ' '), last)
```

8.12.4 Math Functions

Function	Description
`abs(x)`	Absolute value
`pow(x, y)`	x raised to the power y
`round(x)`	Round to nearest integer
`ceil(x)`	Round up
`floor(x)`	Round down
`min(a, b)`	Smaller of two values
`max(a, b)`	Larger of two values

```
LET a = abs(-5)        // 5
LET b = pow(2, 10)     // 1024
LET c = round(3.7)     // 4
LET d = ceil(3.1)      // 4
LET e = floor(3.9)     // 3
LET f = min(10, 20)    // 10
LET g = max(10, 20)    // 20
```

8.12.5 Metadata Functions

Function	Description
`revision(x)` / `rev(x)`	Return the revision number of a GET result
`expires(x)`	Return the expiry timestamp of a GET result
`current_time()`	Return the current UTC time as Unix milliseconds
`hlc()`	Return the node's current HLC timestamp as an integer
`hlc_counter()`	Return the logical counter portion of the current HLC
`count(x)`	Return the number of elements in an array

```
LET value = GET mykey
LET r = revision(value)
LET e = expires(value)
LET now = current_time()
```

`revision` and `expires` are useful for conditional logic based on a key's metadata:

```
LET value = GET mykey
IF revision(value) > 10 THEN
    SET mykey 'stable'
END
```

`count` works only on arrays:

```
LET items = GET BY BUCKET 'users/'
LET total = count(items)
RETURN to_string(total)
```

8.13 Complete Example: Balance Transfer

This example combines many features: variables, GET, SET with conditions, type conversion, explicit transactions, and error handling.

```
BEGIN (locking = pessimistic, timeout = 5000)
    LET source_balance = GET @source
    LET target_balance = GET @target

    IF source_balance = NOT SET THEN
        THROW concat('Source account not found: ', @source)
    END

    IF target_balance = NOT SET THEN
        THROW concat('Target account not found: ', @target)
    END

    LET amount = to_long(@amount)
    LET src = to_long(source_balance)
    LET tgt = to_long(target_balance)

    IF src < amount THEN
        THROW 'Insufficient balance'
    END

    SET @source to_string(src - amount)
    SET @target to_string(tgt + amount)
    COMMIT
END
```

Call this script from C#:

```
var result = await client.ExecuteKeyValueTransactionScript(
    script,
    parameters: [
        new() { Key = "@source", Value = "account/alice" },
        new() { Key = "@target", Value = "account/bob" },
        new() { Key = "@amount", Value = "500" }
    ]
);
```

The transaction reads both balances, checks the source has enough funds, updates both accounts, and commits. If any step fails or the source balance is too low, the script throws and no writes take effect.

8.14 Complete Example: Batch Initialization

This example uses a `FOR` loop to initialize a set of keys:

```
FOR i IN 0..9 DO
    LET key = concat('sensor/', to_string(i))
    LET exists_result = EXISTS key

    IF exists_result = NOT SET THEN
        SET key '0' EX 60000
    END
END
```

This creates keys `sensor/0` through `sensor/9`, each with a 60-second TTL, but only if the key does not exist yet.

8.15 Complete Example: Snapshot Audit

Read the state of several keys as they were at a specific moment:

```
BEGIN (snapshot = @timestamp)
    LET a = GET account/a
    LET b = GET account/b
    LET c = GET account/c
    LET total = to_long(a) + to_long(b) + to_long(c)
    RETURN to_string(total)
END
```

The snapshot option ensures all three reads see a consistent view of the data at the given timestamp, even if the keys changed after that point.

8.16 Script Caching

When you send a script to the server, the server parses it into an AST (abstract syntax tree). Parsing is fast, but for scripts that run frequently, you can avoid the overhead by using `LoadTransactionScript` (covered in Chapter 6). The server hashes the script with Blake3, caches the AST, and reuses it on subsequent calls with the same hash. Placeholder values change between calls, but the structure stays cached.

The server evicts cached scripts when they have not been used within the cache TTL. The TTL is configured with `--script-cache-expiration` (default: 600 seconds).

8.17 Reserved Words

These words are reserved by the language. You cannot use them as bare key names or variable names without backtick escaping:

`SET`, `GET`, `LET`, `IF`, `THEN`, `ELSE`, `END`, `FOR`, `DO`, `IN`, `BEGIN`, `COMMIT`, `ROLLBACK`, `RETURN`, `SLEEP`, `DELETE`, `DEL`, `EXTEND`, `EXISTS`, `ESET`, `EGET`, `EDELETE`, `EDEL`, `EEXTEND`, `EEXISTS`, `SWITCH`, `CASE`, `TRUE`, `FALSE`,

`NULL`, `NX`, `XX`, `EX`, `CMP`, `CMPREV`, `NOREV`, `THROW`, `FOUND`, `NOT`, `AT`, `AS`, `OF`, `SCAN`, `ESCAN`, `BY`, `BUCKET`, `PREFIX`, `AND`, `OR`

If your key name collides with a reserved word, use a string literal or backtick escaping:

```
SET 'delete' 'some value'
SET `delete` 'some value'
```

8.18 Operator Precedence

Operators follow this precedence, from lowest to highest:

Precedence	Operator	Description
1 (lowest)	`..`	Range
2	`OR`, `||`	Logical OR
3	`AND`, `&&`	Logical AND
4	`=`, `==`, `!=`, `<>`	Equality
5	`<`, `>`, `<=`, `>=`	Comparison
6	`+`, `-`	Addition, subtraction
7	`*`, `/`	Multiplication, division
8	`!`, `NOT`	Logical NOT (right-associative)
9 (highest)	`[]`	Array indexing

Use parentheses to override the default order:

```
LET result = (a + b) * c
```

8.19 User-Defined Functions

The server supports user-defined functions that scripts can call. There are two ways to register functions.

8.19.1 Embedded Registration

When the application hosts Kahuna as an embedded node, register functions directly on the `KahunaFunctionRegistry` through `EmbeddedKahunaOptions`:

```
var options = new EmbeddedKahunaOptions();
options.Functions.Register("my_hash", (context, args) =>
{
    string input = args[0].AsString();
    return new KahunaValue(ComputeHash(input));
});
```

8.19.2 Extension Assemblies

When the application runs the shipped server binary, load functions from an extension assembly. The assembly must contain a public class that implements `IKahunaFunctionProvider`:

```
public class MyFunctions : IKahunaFunctionProvider
{
```

```
    public void Register(KahunaFunctionRegistry registry)
    {
        registry.Register("my_hash", (context, args) =>
        {
            string input = args[0].AsString();
            return new KahunaValue(ComputeHash(input));
        });
    }
}
```

Start the server with the `--extension-assemblies` flag to load the assembly:

```
kahuna-server --extension-assemblies /path/to/MyExtension.dll
```

8.19.3 Registry Behavior

The function registry is frozen after the node starts. No functions can be added or removed at runtime. This freeze ensures that every script sees the same set of functions for the lifetime of the node.

The server logs a fingerprint of all registered functions at startup. The fingerprint helps operators verify that all nodes in a cluster have the same function set.

Functions that take longer than `--function-slow-warn-ms` (default: 100 ms) are logged as slow calls. A function that throws a `KahunaFunctionException` aborts the script with the exception message.

8.20 Summary

The Kahuna script language provides the building blocks for server-side transaction logic: key-value commands with conditional flags, variables, arithmetic and logical expressions, control flow with `IF`, `FOR`, and `SWITCH`, explicit transaction blocks with configurable options, and built-in functions for type checking, conversion, string manipulation, and math. Placeholders separate data from logic, and script caching amortizes parsing cost.

Each persistent command (`SET`, `GET`, `EXISTS`, `DELETE`, `EXTEND`) has an ephemeral counterpart (`ESET`, `EGET`, `EEXISTS`, `EDELETE`, `EEXTEND`) for in-memory workloads. Bulk reads use `GET BY BUCKET` and `SCAN BY PREFIX`.

The next chapter covers real-world patterns: how to combine these primitives to build distributed coordination workflows, leader election, and idempotent processing.

9 Leader Election and Job Coordination

Many distributed applications need exactly one active process to do a specific job. A scheduler that sends reminder emails, a worker that processes a payment queue, a service that aggregates metrics: each of these must run on one node at a time. If two instances run simultaneously, they send duplicate emails, process payments twice, or produce incorrect aggregates.

On a single machine, this is simple. You run one process. If it crashes, a supervisor restarts it. In a distributed system, you run multiple instances for availability, and you need a way to decide which one is the active leader. The others stand by and take over if the leader fails.

This chapter shows how to build leader election and job coordination patterns with Kahuna's locks, fencing tokens, and key-value operations.

9.1 The Simplest Leader Election

A distributed lock is a natural fit for leader election. The process that holds the lock is the leader. All other processes are followers. When the leader fails, the lock's lease expires and a follower acquires it.

Here is the simplest version:

```
while (!stoppingToken.IsCancellationRequested)
{
    await using KahunaLock lockHandle = await client.GetOrCreateLock(
        resource: "leader/email-scheduler",
        expiry: TimeSpan.FromSeconds(15),
        wait: TimeSpan.FromSeconds(10),
        retry: TimeSpan.FromMilliseconds(500),
        durability: LockDurability.Persistent,
        cancellationToken: stoppingToken
    );

    if (lockHandle.IsAcquired)
    {
                Console.WriteLine($"I  am  the  leader.  Fencing  token:
{lockHandle.FencingToken}");
        await RunSchedulerLoop(lockHandle, stoppingToken);
    }
    else
    {
        Console.WriteLine("Another instance is the leader. Retrying...");
        await Task.Delay(2000, stoppingToken);
    }
}
```

Each instance tries to acquire the lock. The winner becomes the leader and runs the scheduler loop. The losers wait and try again. If the leader crashes, the 15-second lease expires and a follower acquires the lock on the next attempt.

This version works, but it has a problem. The leader does work for up to 15 seconds and then the lock expires. If the work takes longer than 15 seconds, another instance acquires the lock and both instances run simultaneously.

9.2 Heartbeat Renewal

The solution is a heartbeat: the leader renews its lease periodically while it does work. The lease duration is a safety net for crashes, not a cap on how long the leader can run.

```
async Task RunAsLeader(KahunaLock lockHandle, CancellationToken stoppingToken)
{
                        using      CancellationTokenSource      leaderCts      =
CancellationTokenSource.CreateLinkedTokenSource(stoppingToken);

    Task renewalTask = RunHeartbeat(lockHandle, leaderCts);

    try
    {
        await DoLeaderWork(lockHandle.FencingToken, leaderCts.Token);
    }
    finally
    {
        leaderCts.Cancel();
        await renewalTask;
    }
}

async   Task   RunHeartbeat(KahunaLock   lockHandle,   CancellationTokenSource
leaderCts)
{
    try
    {
        while (!leaderCts.Token.IsCancellationRequested)
        {
            await Task.Delay(5000, leaderCts.Token);

                                    (bool   extended,   long   token)   =   await
lockHandle.TryExtend(TimeSpan.FromSeconds(15));

            if (!extended)
            {
                Console.WriteLine("Lost leadership. Lease renewal failed.");
                leaderCts.Cancel();
                return;
            }
        }
    }
    catch (OperationCanceledException)
    {
```

```
        // Normal shutdown.
    }
}
```

The heartbeat task renews the lease every 5 seconds. The lease duration is 15 seconds. This gives the leader three chances to renew before the lease expires. If a single renewal fails (network blip, temporary leader unavailability), the leader has 10 more seconds before it loses the lock.

A good rule of thumb: renew at one-third of the lease duration. This gives you two full renewal attempts as a buffer.

9.2.1 What Happens When Renewal Fails

If `TryExtend` returns false, one of these things occurred:

1. The lease expired before the renewal reached the server (network delay, GC pause, slow disk).
2. Another process acquired the lock after the lease expired.

In both cases, the leader must stop its work immediately. The `CancellationTokenSource` propagates the cancellation to all ongoing work. Any downstream operation that respects the cancellation token will stop.

The leader should not attempt to re-acquire the lock in the same iteration. It should release the lock handle, return to the outer loop, and compete for leadership again from scratch.

9.3 The Leader Election Loop

Here is the complete pattern that combines acquisition, heartbeat renewal, and graceful shutdown:

```
public class LeaderElectionService : BackgroundService
{
    private readonly KahunaClient _client;
    private readonly string _resource;
    private readonly TimeSpan _leaseDuration;
    private readonly TimeSpan _renewalInterval;

    public LeaderElectionService(KahunaClient client, string resource)
    {
        _client = client;
        _resource = resource;
        _leaseDuration = TimeSpan.FromSeconds(15);
        _renewalInterval = TimeSpan.FromSeconds(5);
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using KahunaLock lockHandle = await _client.GetOrCreateLock(
                resource: _resource,
```

```
                expiry: _leaseDuration,
                wait: TimeSpan.FromSeconds(30),
                retry: TimeSpan.FromMilliseconds(500),
                durability: LockDurability.Persistent,
                cancellationToken: stoppingToken
            );

            if (!lockHandle.IsAcquired)
            {
                await Task.Delay(2000, stoppingToken);
                continue;
            }

                            Console.WriteLine($"Elected as leader. Token:
{lockHandle.FencingToken}");

            using CancellationTokenSource leaderCts =
               CancellationTokenSource.CreateLinkedTokenSource(stoppingToken);

            Task heartbeat = RunHeartbeat(lockHandle, leaderCts);

            try
            {
                await DoLeaderWork(lockHandle.FencingToken, leaderCts.Token);
            }
                                catch (OperationCanceledException) when
(leaderCts.IsCancellationRequested)
            {
                Console.WriteLine("Leadership ended.");
            }
            finally
            {
                leaderCts.Cancel();
                await heartbeat;
            }
        }
    }

              private async Task RunHeartbeat(KahunaLock lockHandle,
CancellationTokenSource leaderCts)
    {
        try
        {
            while (!leaderCts.Token.IsCancellationRequested)
            {
                await Task.Delay(_renewalInterval, leaderCts.Token);
                (bool ok, _) = await lockHandle.TryExtend(_leaseDuration);
```

```
                if (!ok)
                {
                    leaderCts.Cancel();
                    return;
                }
            }
        }
        catch (OperationCanceledException) { }
    }

    private async Task DoLeaderWork(long fencingToken, CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            await ProcessNextBatch(fencingToken, ct);
        }
    }

    private async Task ProcessNextBatch(long fencingToken, CancellationToken ct)
    {
        // Application-specific work goes here.
        // Pass fencingToken to downstream systems.
        await Task.Delay(1000, ct);
    }
}
```

This pattern uses .NET's `BackgroundService` as a host. You can register it in your dependency injection container. It runs for the lifetime of the application.

9.3.1 State Machine

A process in this pattern moves through three states:

1. **Candidate.** The process tries to acquire the lock. It waits and retries until it succeeds or the application shuts down.
2. **Leader.** The process holds the lock and does work. A background task renews the lease. The fencing token identifies this leadership term.
3. **Follower.** The process lost or failed to acquire the lock. It returns to the candidate state and tries again.

The transitions are:

- Candidate to Leader: `GetOrCreateLock` returns `IsAcquired = true`.
- Leader to Follower: `TryExtend` returns false, or the application requests shutdown.
- Follower to Candidate: the outer loop iterates and calls `GetOrCreateLock` again.

9.4 Why Fencing Tokens Matter

Chapter 4 introduced fencing tokens. In leader election, they are essential. Here is why.

9.4.1 The Stale Leader Problem

Consider this sequence of events:

1. Process A acquires the leadership lock. Fencing token: 5.
2. Process A starts processing jobs from a queue.
3. Process A experiences a long garbage collection pause (20 seconds).
4. The lease (15 seconds) expires during the pause. Process A does not know this.
5. Process B acquires the lock. Fencing token: 6.
6. Process B starts processing jobs from the same queue.
7. Process A resumes. It still believes it is the leader. It processes the same jobs that process B is already handling.

The result: duplicate processing. Both processes act as leader simultaneously.

9.4.2 Leader Election Without Fencing (The Wrong Way)

This implementation looks correct but is broken:

```
// INCORRECT: no fencing token check
async Task ProcessJob(KahunaClient client, string jobId)
{
    KahunaKeyValue job = await client.GetKeyValue($"jobs/{jobId}");

    if (job.ValueAsString == "pending")
    {
        await ExecuteJob(jobId);
        await client.SetKeyValue($"jobs/{jobId}", "completed");
    }
}
```

A stale leader (process A) can execute this code after its lease expired. It reads the job, sees "pending", executes it, and marks it "completed." Meanwhile, the new leader (process B) also reads the same job as "pending" before A marks it complete. Both execute the job.

9.4.3 Leader Election With Fencing (The Correct Way)

Pass the fencing token to every operation that the leader performs. The downstream system rejects writes from stale leaders:

```
async Task ProcessJob(KahunaClient client, long fencingToken, string jobId)
{
    KahunaKeyValue job = await client.GetKeyValue($"jobs/{jobId}");

    if (job.ValueAsString == "pending")
    {
        await ExecuteJob(jobId);

        // Store the fencing token alongside the job status.
        await client.SetKeyValue(
            $"jobs/{jobId}",
            $"completed|token={fencingToken}"
        );
    }
}
```

For systems that do not natively support fencing (most databases and message queues), you can implement a fence check at the application level:

```
async Task<bool> TryClaimJob(KahunaClient client, long fencingToken, string
jobId)
{
    // Use compare-and-swap to claim the job atomically.
    KahunaKeyValue result = await client.SetKeyValue(
        $"jobs/{jobId}",
        $"running|token={fencingToken}",
        flags: KeyValueFlags.SetIfEqualToValue,
        durability: KeyValueDurability.Persistent
    );

    return result.Success;
}
```

If a stale leader with token 5 tries to claim a job that the new leader (token 6) already claimed, the compare-and-swap fails because the current value does not match "pending."

9.5 Publishing Leader Identity

Sometimes other services need to know who the current leader is. You can publish leader identity in a key-value entry alongside the lock:

```
if (lockHandle.IsAcquired)
{
    string identity = JsonSerializer.Serialize(new
    {
        host = Environment.MachineName,
        processId = Environment.ProcessId,
        fencingToken = lockHandle.FencingToken,
        electedAt = DateTimeOffset.UtcNow
    });

    await client.SetKeyValue(
        "leader/email-scheduler/identity",
        identity,
        expiryTime: 20000
    );
}
```

Other services can read this key to discover the current leader. Set a TTL slightly longer than the lease duration so the identity key expires shortly after the lock.

Update the identity key during each heartbeat cycle to keep the TTL fresh:

```
while (!leaderCts.Token.IsCancellationRequested)
{
    await Task.Delay(5000, leaderCts.Token);

    (bool ok, _) = await lockHandle.TryExtend(TimeSpan.FromSeconds(15));
```

```
    if (!ok)
    {
        leaderCts.Cancel();
        return;
    }

    // Refresh the identity TTL.
    await client.ExtendKeyValue("leader/email-scheduler/identity", 20000);
}
```

9.6 Distributed Task Queue

Leader election puts one process in charge. A task queue distributes work across many processes. Each worker claims a task, processes it, and marks it complete. No two workers process the same task.

9.6.1 Task Structure

Store tasks as key-value entries. Use a sequence for unique task IDs:

```
await client.CreateSequence("task-ids");

async Task<string> EnqueueTask(KahunaClient client, string payload)
{
    long taskId = await client.NextSequenceValue("task-ids");
    string key = $"tasks/{taskId}";

    await client.SetKeyValue(key, JsonSerializer.Serialize(new
    {
        id = taskId,
        payload,
        status = "pending",
        createdAt = DateTimeOffset.UtcNow
    }));

    return key;
}
```

9.6.2 Claiming a Task

A worker claims a task by acquiring a lock on that task's key. The lock lease acts as a processing timeout:

```
async Task<bool> TryProcessTask(KahunaClient client, string taskKey)
{
    await using KahunaLock taskLock = await client.GetOrCreateLock(
        resource: $"lock/{taskKey}",
        expiry: TimeSpan.FromSeconds(60)
    );

    if (!taskLock.IsAcquired)
```

```
        return false;

    KahunaKeyValue task = await client.GetKeyValue(taskKey);
    var taskData = JsonSerializer.Deserialize<TaskRecord>(task.ValueAsString ??
"{}");

    if (taskData?.Status != "pending")
        return false;

    try
    {
        await ExecuteTask(taskData, taskLock.FencingToken);

       await client.SetKeyValue(taskKey, JsonSerializer.Serialize(taskData with
        {
            Status = "completed",
            CompletedAt = DateTimeOffset.UtcNow
        }));

        return true;
    }
    catch (Exception ex)
    {
       await client.SetKeyValue(taskKey, JsonSerializer.Serialize(taskData with
        {
            Status = "failed",
            Error = ex.Message
        }));

        return false;
    }
}
```

If the worker crashes while processing, the lock lease expires after 60 seconds. Another worker can then claim the task. The task stays in "pending" status because the crashed worker never updated it.

9.6.3 Worker Loop

Each worker scans for pending tasks and claims them:

```
async Task RunWorker(KahunaClient client, CancellationToken ct)
{
    while (!ct.IsCancellationRequested)
    {
        List<KahunaKeyValue> tasks = await client.GetByBucket(
            "tasks/",
            KeyValueDurability.Persistent
        );
```

```
        bool processed = false;

        foreach (KahunaKeyValue task in tasks)
        {
         var data = JsonSerializer.Deserialize<TaskRecord>(task.ValueAsString ??
"{}");

            if (data?.Status == "pending")
            {
                bool claimed = await TryProcessTask(client, task.Key);
                if (claimed) processed = true;
            }
        }

        if (!processed)
        {
            // No work available. Wait before scanning again.
            await Task.Delay(2000, ct);
        }
    }
}
```

Multiple workers can run this loop concurrently. The lock on each task ensures that only one worker processes it. If two workers try to claim the same task, one gets `IsAcquired = false` and moves on.

9.6.4 Extending Task Leases

For long tasks, extend the lock lease during processing (the same pattern as the leader heartbeat):

```
async Task ExecuteLongTask(KahunaLock taskLock, TaskRecord task,
CancellationToken ct)
{
    foreach (var step in task.Steps)
    {
        await ProcessStep(step, ct);

        (bool ok, _) = await taskLock.TryExtend(TimeSpan.FromSeconds(60));

        if (!ok)
        {
            Console.WriteLine("Lost task lock. Another worker may take over.");
            return;
        }
    }
}
```

9.7 Failure Scenarios

9.7.1 Leader Crashes

1. The leader process terminates (OOM kill, hardware failure, unhandled exception).
2. The `DisposeAsync` finalizer does not run because the process is gone.
3. The lock lease expires after 15 seconds.
4. A follower acquires the lock on its next attempt.
5. The new leader gets a higher fencing token.

No manual intervention is required. The lease is the recovery mechanism.

9.7.2 Slow Leader

1. The leader encounters a slow database query, a full GC, or a network partition.
2. The heartbeat task cannot renew the lease in time.
3. The lease expires. A follower acquires the lock.
4. The slow leader's heartbeat eventually executes and `TryExtend` returns false.
5. The leader cancels its work through the `CancellationTokenSource`.

The window of danger is the time between the lease expiry and the moment the old leader detects the failure. During this window, both the old and new leaders may act. Fencing tokens prevent the old leader from corrupting shared state.

9.7.3 Network Partition

1. The leader can still reach the work (database, queue) but cannot reach Kahuna.
2. The heartbeat fails. The lease expires on the Kahuna side.
3. A follower on the other side of the partition acquires the lock.
4. The old leader continues doing work because it has not detected the partition.

This is the scenario where fencing is critical. Without fencing, both leaders write to the shared resource. With fencing, the shared resource rejects writes from the old leader's stale token.

9.7.4 Worker Crashes Mid-Task

1. A worker acquires a task lock and starts processing.
2. The worker crashes.
3. The task lock expires after 60 seconds.
4. Another worker scans for pending tasks, finds this one, and claims it.
5. The task is processed to completion by the second worker.

For this to work correctly, the task processing must be idempotent or the task must record progress. If the first worker completed half the work before crashing, the second worker must be able to resume or redo the work safely.

9.8 Tuning Leader Election

9.8.1 Lease Duration

Short leases (5 to 10 seconds) provide fast failover but require frequent renewals. Each renewal is a network round trip to Kahuna. If the network is unreliable, short leases increase the risk of losing leadership during a transient failure.

Long leases (30 to 60 seconds) tolerate network blips but delay failover after a crash. A crashed leader blocks progress for the full lease duration.

9.8.2 Renewal Interval

Renew at one-third of the lease duration. This gives two full buffer attempts:

Lease Duration	Renewal Interval	Buffer Before Expiry
9 seconds	3 seconds	6 seconds (2 retries)
15 seconds	5 seconds	10 seconds (2 retries)
30 seconds	10 seconds	20 seconds (2 retries)

9.8.3 Wait and Retry

When a follower tries to acquire the lock, the `wait` parameter controls how long it blocks. A long wait (30 to 60 seconds) keeps the follower ready to take over quickly. A short wait (5 to 10 seconds) with external polling gives you more control over shutdown behavior.

The `retry` interval (the polling frequency during the wait period) should be short enough to detect a released lock quickly but long enough to avoid excessive load. Values of 200 to 500 milliseconds work well for most workloads.

9.9 Combining Leader Election with Task Queues

A common architecture uses both patterns together: a single leader distributes work, and multiple workers process it.

```
async Task DoLeaderWork(long fencingToken, CancellationToken ct)
{
    while (!ct.IsCancellationRequested)
    {
        // The leader generates tasks.
        List<string> pendingItems = await FetchNewItemsFromExternalSource(ct);

        foreach (string item in pendingItems)
        {
            long taskId = await _client.NextSequenceValue("task-ids");

                                    await   _client.SetKeyValue($"tasks/{taskId}",
JsonSerializer.Serialize(new
            {
                id = taskId,
                payload = item,
                status = "pending",
                assignedBy = fencingToken
            }));
        }

        await Task.Delay(5000, ct);
```

```
    }
}
```

The leader creates tasks. Workers (which do not need to win an election) claim and process tasks independently. If the leader crashes, a new leader takes over task creation. The workers are unaffected because they operate on the task queue, not on the leadership lock.

9.10 Summary

Leader election in Kahuna uses a distributed lock as the coordination primitive. The process that holds the lock is the leader. A background heartbeat task renews the lease to keep leadership alive. When the leader fails, the lease expires and a follower acquires the lock automatically.

Fencing tokens prevent stale leaders from corrupting shared state during the window between lease expiry and failure detection. Every downstream write should carry the fencing token and reject stale values.

The same lock primitive supports task queues: each task gets its own lock, workers claim tasks by acquiring the lock, and crashed workers release their tasks through lease expiry.

The next chapter covers idempotency and duplicate prevention: how to make operations safe to retry without side effects.

10 Idempotency and Duplicate Prevention

In a distributed system, retries are not optional. They are inevitable. A client sends a request to the server. The network drops the response. The client does not know whether the server processed the request or not. It retries.

If the operation was "set the status to completed," the retry is harmless. The status was already "completed," and setting it again changes nothing. But if the operation was "add 100 to the balance," the retry doubles the effect. The balance increases by 200 instead of 100.

An operation is idempotent when executing it multiple times produces the same result as executing it once. This chapter shows how to build idempotent operations with Kahuna's key-value primitives, conditional writes, fencing tokens, and sequence-based deduplication.

10.1 The Retry Problem

Consider a simple counter increment:

```
// INCORRECT: not idempotent
KahunaKeyValue current = await client.GetKeyValue("stats/page-views");
long count = long.Parse(current.ValueAsString ?? "0");
await client.SetKeyValue("stats/page-views", (count + 1).ToString());
```

This code reads the counter, adds 1, and writes the new value. If the client retries (because the response to the write was lost), it reads the already-incremented counter, adds 1 again, and writes a doubly-incremented value.

The problem is not the retry itself. The problem is that the operation has no way to detect that it already succeeded.

10.2 Delivery Semantics

Distributed systems describe three levels of delivery guarantee:

- **At-most-once.** The operation executes zero or one time. If it fails, the system does not retry. Data may be lost, but it is never duplicated.
- **At-least-once.** The operation executes one or more times. The system retries on failure. Data is never lost, but it may be duplicated.
- **Exactly-once.** The operation executes exactly one time. This is the ideal, but it is impossible to achieve in the general case without additional mechanisms.

Most systems provide at-least-once delivery by default: the client retries until it receives a success response. To get effectively-exactly-once behavior, the operation must be idempotent. A retry of an idempotent operation produces no additional effect, so "at least once" becomes equivalent to "exactly once" in practice.

10.3 Naturally Idempotent Operations

Some operations are idempotent by nature. No extra work is required:

- **Unconditional SET.** `SetKeyValue("config/mode", "maintenance")` can be called any number of times. The result is always the same: the key holds the value "maintenance."
- **DELETE.** `DeleteKeyValue("temp/session-abc")` succeeds on the first call and returns "not found" on subsequent calls. Either way, the key is gone.

- **SetIfNotExists.** `SetKeyValue("users/alice", "...", flags: KeyValueFlags.SetIfNotExists)` creates the key once. Retries see the key already exists and return `Success = false`. No duplicate is created.

These operations are safe to retry without any guarding mechanism.

10.4 Non-Idempotent Operations

These operations are dangerous to retry:

- **Read-modify-write.** Read a value, compute a new value, write it back. A retry reads the already-modified value and modifies it again.
- **Append.** Add an item to a list or log. A retry adds the item twice.
- **Increment.** Add a number to a counter. A retry adds the number twice.
- **Side effects.** Send an email, charge a credit card, enqueue a message. A retry repeats the side effect.

Each of these needs an explicit mechanism to make it safe for retries.

10.5 Revision-Based Idempotency (CRAS)

Every key in Kahuna has a revision number. The revision starts at 0 and increases by 1 on every successful write. You can use this revision as a guard: "write this value, but only if the revision is still what I saw when I read it."

This is the Compare-Revision-And-Swap (CRAS) operation:

```
// Step 1: Read the current value and its revision.
KahunaKeyValue current = await client.GetKeyValue("stats/page-views");
long count = long.Parse(current.ValueAsString ?? "0");
long revision = current.Revision;

// Step 2: Write the new value, but only if the revision has not changed.
KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
    "stats/page-views",
    (count + 1).ToString(),
    compareRevision: revision
);

if (result.Success)
    Console.WriteLine("Counter incremented.");
else
    Console.WriteLine("Conflict: another write changed the key. Retry from
step 1.");
```

How this prevents double-counting:

1. The client reads revision 5 and value "100."
2. The client writes "101" with `compareRevision: 5`.
3. The write succeeds. The revision becomes 6.
4. The client retries (response was lost). It writes "101" with `compareRevision: 5` again.
5. The revision is now 6, not 5. The compare fails. The retry is rejected.

The retry does not corrupt the data. The client can re-read the key, see that the value is already "101," and conclude that the original write succeeded.

10.5.1 When CRAS Is Not Enough

CRAS protects against concurrent writes and duplicate retries from the same read. But if the client crashes after reading and before writing, it restarts with no memory of the previous read. The new read sees the current state, and the new write succeeds normally. This is correct behavior, not a bug, because the first attempt never wrote anything.

The risk arises when the operation has external side effects. If the client charged a credit card, crashed, and retried, the charge happened but the status update did not. CRAS does not help here because the external side effect is outside Kahuna's control. For that, you need a request-level idempotency key (covered later in this chapter).

10.6 Value-Based Idempotency (CVAS)

Compare-Value-And-Swap (CVAS) writes a new value only if the current value matches an expected value:

```
KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
    "orders/abc/status",
    value: "shipped",
    compareValue: "paid"
);

if (result.Success)
    Console.WriteLine("Order marked as shipped.");
else
    Console.WriteLine("Order is not in 'paid' status. No change made.");
```

This is idempotent for state-machine transitions. If the order is already "shipped" (from a previous attempt), the compare against "paid" fails and the retry is a no-op.

CVAS is useful when:

- The key follows a state machine (pending, paid, shipped, delivered).
- Each transition is valid from exactly one source state.
- A retry from the same source state is harmless because the transition already happened.

10.7 SetIfNotExists as an Idempotency Guard

The simplest idempotency guard stores a record of the completed operation. Before doing work, check whether the record exists. If it does, skip the work.

```
async Task<bool> ProcessOrderIdempotently(KahunaClient client, string orderId,
OrderDetails details)
{
    // Try to create a completion record. If it already exists, we already
processed this order.
    KahunaKeyValue guard = await client.SetKeyValue(
        $"processed/{orderId}",
        "done",
```

```
        flags: KeyValueFlags.SetIfNotExists,
        durability: KeyValueDurability.Persistent
    );

    if (!guard.Success)
    {
        Console.WriteLine($"Order {orderId} was already processed. Skipping.");
        return false;
    }

    // First time: do the actual work.
    await ChargePayment(details);
    await UpdateInventory(details);
    await SendConfirmationEmail(details);

    return true;
}
```

The `SetIfNotExists` flag ensures that only one call succeeds. All retries see the existing key and skip the work.

10.7.1 The Crash Window

There is a gap between the guard write and the actual work. If the process crashes after writing the guard but before finishing the work, the retried call sees the guard and skips the work. The result: the work is never completed.

To close this gap, use a two-phase approach:

```
async Task ProcessOrderSafely(KahunaClient client, string orderId, OrderDetails
details)
{
    string guardKey = $"processed/{orderId}";

    KahunaKeyValue existing = await client.GetKeyValue(guardKey);

    if (existing.ValueAsString == "completed")
        return;

    // Mark as in-progress (not yet completed).
    await client.SetKeyValue(
        guardKey,
        "in-progress",
        flags: KeyValueFlags.SetIfNotExists
    );

    // Do the work.
    await ChargePayment(details);
    await UpdateInventory(details);
```

```
    // Mark as completed.
    await client.TryCompareValueAndSetKeyValue(
        guardKey,
        value: "completed",
        compareValue: "in-progress"
    );
}
```

On retry:

- If the guard is "completed," skip. The work finished successfully.
- If the guard is "in-progress," the previous attempt started but did not finish. You can resume or redo the work (if the work itself is idempotent).
- If the guard does not exist, this is a fresh attempt.

10.8 Fencing-Based Exactly-Once Writes

Chapter 4 and Chapter 8 showed how fencing tokens prevent stale lock holders from corrupting data. The same mechanism provides exactly-once writes.

The pattern:

1. Acquire a lock on the logical operation. Each lock acquisition gets a unique, monotonically increasing fencing token.
2. Perform the work.
3. Write the result along with the fencing token.
4. The downstream system rejects writes with a fencing token lower than or equal to the highest token it has already seen.

```
async Task ProcessPaymentWithFencing(KahunaClient client, string paymentId)
{
    await using KahunaLock lockHandle = await client.GetOrCreateLock(
        $"payment/{paymentId}",
        expiry: TimeSpan.FromSeconds(30),
        wait: TimeSpan.FromSeconds(10),
        retry: TimeSpan.FromMilliseconds(200)
    );

    if (!lockHandle.IsAcquired)
        throw new TimeoutException("Could not acquire payment lock.");

    long token = lockHandle.FencingToken;

    // Read the current state.
    KahunaKeyValue state = await client.GetKeyValue($"payment/{paymentId}/
state");

    if (state.ValueAsString == "completed")
        return;

    // Charge the payment (external side effect).
```

```
    await ChargePaymentGateway(paymentId);

    // Write the result with the fencing token.
    // Only succeeds if no higher token has written to this key.
    await client.SetKeyValue(
        $"payment/{paymentId}/state",
        $"completed|token={token}"
    );
}
```

If the client retries after a timeout:

- If the lock is still held by the original attempt, the retry waits until it expires.
- When the retry acquires the lock, it gets a higher fencing token.
- The retry reads the state. If "completed," it skips. If not, it processes and writes with the new token.

Fencing tokens are especially valuable when the operation involves an external system (a payment gateway, an email service) that cannot be rolled back. The token guarantees that only the most recent lock holder's writes persist.

10.9 Sequence-Based Deduplication

Kahuna's sequencer supports idempotency keys natively (Chapter 5). When you allocate a value with an idempotency key, retrying with the same key returns the same value:

```
long orderId = await client.NextSequenceValue(
    "order-ids",
    idempotencyKey: $"checkout-{requestId}"
);
```

If the response is lost and the client retries with the same `requestId`, Kahuna returns the same `orderId`. No new value is consumed. This is server-side deduplication: Kahuna stores a map of recent idempotency keys and their results.

The same applies to range reservations:

```
KahunaSequenceRange batch = await client.ReserveSequenceRange(
    "event-offsets",
    count: 50,
    idempotencyKey: $"import-batch-{batchId}"
);
```

A retry with the same key and count returns the identical range. A retry with the same key but a different count returns an error, because changing the parameters after the first call is ambiguous.

10.9.1 Retention Limits

Idempotency entries are not stored forever. Two server settings control retention:

Setting	Default	Description
`--sequencer-idempotency-retention-max`	256	Maximum entries per sequence
`--sequencer-idempotency-retention-ttl`	600 seconds	Time window for replay

After an entry is evicted (by count or age), a retry with that key allocates fresh values. Set these limits based on how long your retry windows last.

10.10 Transaction Operation IDs

Inside an interactive transaction session (Chapter 6), each operation carries a `TransactionOperationId`: a 128-bit random identifier. The client generates one automatically for every operation.

If a network timeout leaves the outcome of an operation unknown, the client can resubmit the same operation with the same `TransactionOperationId`. The server detects the duplicate and replays the original result instead of executing the operation again.

This deduplication is transparent. You do not need to manage operation IDs yourself. The `KahunaTransactionSession` generates a fresh random ID for every call you make. The server uses these IDs internally to handle retransmissions during the session's lifetime.

The IDs also support derived sub-operations. When an operation spans multiple pages (as in a paginated scan), the client derives deterministic sub-IDs from the original. If a page request is retried, the derived ID is the same, and the server replays the result.

10.11 Building an Idempotent Pipeline

Real applications often chain multiple steps. Each step must be idempotent independently, because a failure can happen between any two steps.

Consider an order processing pipeline:

```
async Task ProcessOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Step 1: Validate and reserve inventory.
    await ReserveInventory(client, orderId, order);

    // Step 2: Charge payment.
    await ChargePayment(client, orderId, order);

    // Step 3: Confirm order.
    await ConfirmOrder(client, orderId, order);
}
```

Each step uses a state key to track progress:

```
async Task ReserveInventory(KahunaClient client, string orderId, OrderDetails
order)
```

```
{
    string stepKey = $"orders/{orderId}/step/reserve";
    KahunaKeyValue status = await client.GetKeyValue(stepKey);

    if (status.ValueAsString == "done")
        return;

    // Do the reservation.
    foreach (var item in order.Items)
    {
        await client.RetryableTransaction(
            new KahunaTransactionOptions { Timeout = 5000 },
            async (session, ct) =>
            {
                 KahunaKeyValue stock = await session.GetKeyValue($"inventory/
{item.Sku}");
                int available = int.Parse(stock.ValueAsString ?? "0");

                if (available < item.Quantity)
                      throw new InvalidOperationException($"Insufficient stock
for {item.Sku}.");

                await session.SetKeyValue(
                    $"inventory/{item.Sku}",
                    (available - item.Quantity).ToString()
                );
                await session.Commit(ct);
            }
        );
    }

    await client.SetKeyValue(stepKey, "done");
}

async Task ChargePayment(KahunaClient client, string orderId, OrderDetails
order)
{
    string stepKey = $"orders/{orderId}/step/charge";
    KahunaKeyValue status = await client.GetKeyValue(stepKey);

    if (status.ValueAsString == "done")
        return;

    // Acquire a lock with fencing to prevent duplicate charges.
    await using KahunaLock chargeLock = await client.GetOrCreateLock(
        $"charge/{orderId}",
        expiry: TimeSpan.FromSeconds(30)
    );
```

```
    if (!chargeLock.IsAcquired)
        throw new InvalidOperationException("Could not acquire charge lock.");

    // Check again after acquiring the lock (another instance may have completed
it).
    status = await client.GetKeyValue(stepKey);
    if (status.ValueAsString == "done")
        return;

    await CallPaymentGateway(order.PaymentDetails, chargeLock.FencingToken);

    await client.SetKeyValue(stepKey, "done");
}

async Task ConfirmOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Use CVAS: transition from "processing" to "confirmed."
    await client.TryCompareValueAndSetKeyValue(
        $"orders/{orderId}/status",
        value: "confirmed",
        compareValue: "processing"
    );
}
```

Each step checks a status key before doing work. If the step already completed, it returns immediately. If the process crashes and restarts, it resumes from the last incomplete step. No step executes twice.

10.12 Choosing the Right Mechanism

Mechanism	Best For	Scope
Unconditional SET	Values that can be overwritten safely	Single key
SetIfNotExists	One-time creation (guards, flags)	Single key
CRAS (Compare-Revision-And-Swap)	Read-modify-write on a single key	Single key
CVAS (Compare-Value-And-Swap)	State machine transitions	Single key
Fencing tokens	Protecting external side effects	Lock scope
Sequence idempotency keys	Preventing duplicate ID allocation	Single sequence
Transaction operation IDs	Retransmission within a session	Transaction scope
Step-based progress tracking	Multi-step pipelines	Workflow scope

For simple key-value updates, CRAS or CVAS is enough. For operations with external side effects (payment charges, email sends), use fencing tokens. For multi-step workflows, combine step-based progress tracking with per-step idempotency guards.

10.13 Common Mistakes

10.13.1 Checking Before Writing (Without Atomicity)

```
// INCORRECT: race condition between check and write
KahunaKeyValue existing = await client.GetKeyValue($"processed/{requestId}");
if (existing.Success)
    return;

await DoWork();
await client.SetKeyValue($"processed/{requestId}", "done");
```

Two concurrent requests can both read "not found" and both proceed to do the work. Use `SetIfNotExists` or CRAS to make the check-and-write atomic.

10.13.2 Relying on Client-Side State

```
// INCORRECT: client-side flag does not survive a restart
private bool _processed = false;

async Task Handle(string requestId)
{
    if (_processed) return;
    await DoWork();
```

```
    _processed = true;
}
```

Client-side state is lost on crash or restart. Store the idempotency record in Kahuna, where it survives process failures and is visible to all instances.

10.13.3 Ignoring the Gap Between Guard and Work

Writing a guard and then doing work is not atomic. If the process crashes between the two, the guard says "done" but the work did not complete. Use a two-phase status ("in-progress" then "completed") so a retry can detect and resume incomplete work.

10.14 Summary

Retries are inevitable in distributed systems. Every mutating operation must be safe to retry, either naturally or through an explicit mechanism.

Kahuna provides several building blocks for idempotency. Conditional writes (CRAS, CVAS, SetIfNotExists) make single-key updates safe. Fencing tokens protect operations that touch external systems. Sequence idempotency keys prevent duplicate ID allocation at the server level. Transaction operation IDs handle retransmission within a session transparently.

For multi-step workflows, combine step-based progress tracking with per-step idempotency guards. Each step records its completion status. On retry, the pipeline resumes from the last incomplete step.

The next chapter covers globally unique identifier generation: patterns for producing unique, sortable IDs at scale using Kahuna's distributed sequencer.

11 Globally Unique Identifiers

Most distributed applications need unique identifiers. Order numbers, event offsets, invoice IDs, user handles, log sequence numbers: each must be unique across every node, process, and data center that participates in the system.

On a single database, an auto-incrementing column solves the problem. The database serializes all inserts through one counter. In a distributed system, there is no single database. Two services inserting records into two different databases will produce colliding IDs unless they coordinate.

This chapter explores patterns for generating unique IDs with Kahuna's distributed sequencer. It compares Kahuna sequences with UUIDs, Snowflake IDs, and database sequences, and shows how to tune the sequencer for different workloads.

11.1 The ID Generation Problem

A good identifier scheme must satisfy at least two requirements:

1. **Uniqueness.** No two records in the system share the same ID.
2. **Availability.** Generating an ID must not become a bottleneck under high throughput.

Beyond these, different applications value different properties:

- **Sortability.** Can IDs be sorted chronologically? This matters for time-series data, event logs, and database indexes.
- **Human readability.** Can a support engineer read an ID over the phone? Short numeric IDs are easier to communicate than 128-bit hex strings.
- **Contiguity.** Are IDs contiguous (no gaps)? Some financial regulations require contiguous invoice numbers.
- **Predictability.** Should IDs be unpredictable? Public-facing IDs that are sequential can leak information about volume and growth.

No single scheme satisfies all of these. Each scheme makes trade-offs.

11.2 Common ID Schemes

11.2.1 UUIDs (v4)

A UUID v4 is a 128-bit random number. It requires no coordination: any node can generate one independently.

```
string id = Guid.NewGuid().ToString();
// "3f2504e0-4f89-11d3-9a0c-0305e82c3301"
```

Strengths:

- No coordination. Any process generates one locally.
- Extremely low collision probability.
- Works offline.

Weaknesses:

- Not sortable by time.
- Not human-readable (36 characters).

- Poor database index performance. Random values cause B-tree fragmentation because inserts scatter across the index instead of appending to the end.
- Not contiguous.

11.2.2 UUIDs (v7)

UUID v7 embeds a Unix timestamp in the most significant bits. This makes UUIDs roughly sortable by creation time.

```
string id = Guid.CreateVersion7().ToString();
```

This solves the sortability and B-tree fragmentation problems but does not improve human readability. The IDs are still 36 characters long.

11.2.3 Snowflake IDs

Twitter's Snowflake scheme packs a 64-bit integer from three components: a timestamp (41 bits), a machine ID (10 bits), and a per-machine sequence (12 bits). Each machine generates IDs independently.

Strengths:

- Sortable by time.
- 64-bit integer (fits in a database `bigint`).
- High throughput (4,096 IDs per millisecond per machine).

Weaknesses:

- Requires machine ID assignment. You must ensure no two machines share the same ID.
- Clock skew can cause non-monotonic IDs across machines.
- Not contiguous.
- Not human-readable (large numbers).

11.2.4 Database Auto-Increment

A single database generates sequential IDs through an auto-incrementing column.

Strengths:

- Contiguous.
- Sortable.
- Human-readable (short numbers).

Weaknesses:

- Single point of failure.
- Throughput bottleneck. Every insert must go through one database.
- Does not work across multiple databases without coordination.

11.2.5 Kahuna Sequences

Kahuna's distributed sequencer generates monotonically increasing 64-bit integers. Block-based allocation amortizes the cost of consensus across many values (Chapter 5).

Strengths:

- Unique across the entire cluster.
- Monotonically increasing (sortable).

- Human-readable (short numbers starting from a configurable initial value).
- High throughput via block allocation.
- Contiguous within a reserved range.
- Idempotent allocation with idempotency keys.

Weaknesses:

- Requires a running Kahuna cluster (not available offline).
- Gaps can occur on node failure or leadership change (unless block size is 1).
- One network round trip per block allocation.

11.3 Comparison Table

Property	UUID v4	UUID v7	Snowflake	DB Auto-Inc	Kahuna Sequence
Coordination required	No	No	Machine ID	Single DB	Kahuna cluster
Sortable by time	No	Yes	Yes	Yes	Yes
Human-readable	No	No	No	Yes	Yes
Contiguous	No	No	No	Yes	Configurable
Throughput	Unlimited	Unlimited	4,096/ms/machine	DB-limited	Tunable (block size)
Size	128 bits	128 bits	64 bits	32 or 64 bits	64 bits
Offline generation	Yes	Yes	Yes	No	No
B-tree friendly	No	Yes	Yes	Yes	Yes

Choose Kahuna sequences when you need human-readable, sortable, unique IDs and you already run a Kahuna cluster. Choose UUIDs when you need offline generation or cannot depend on a central service. Choose Snowflake when you need high throughput without any coordination service and can manage machine IDs.

11.4 Patterns for ID Generation

11.4.1 Single Sequence Per Entity Type

The simplest pattern: one sequence per type of entity.

```
await client.CreateSequence("order-ids", initialValue: 10000);
await client.CreateSequence("invoice-ids", initialValue: 100000);
await client.CreateSequence("user-ids");
```

```
// In the order service:
long orderId = await client.NextSequenceValue("order-ids");

// In the billing service:
long invoiceId = await client.NextSequenceValue("invoice-ids");

// In the user service:
long userId = await client.NextSequenceValue("user-ids");
```

Each entity type gets its own counter. The counters are independent. The order service and the billing service can allocate IDs concurrently without contention because they use different sequences.

Setting `initialValue` to a non-zero number makes early IDs more presentable. Order #10001 looks more professional than order #1.

11.4.2 Sequence Per Tenant

In a multi-tenant system, each tenant can have its own sequence:

```
async Task<long> GetNextOrderId(KahunaClient client, string tenantId)
{
    string sequenceName = $"orders/{tenantId}";

    try
    {
        return await client.NextSequenceValue(sequenceName);
    }
    catch (KahunaException ex) when (ex.Message.Contains("NotFound"))
    {
        await client.CreateSequence(sequenceName, initialValue: 1000);
        return await client.NextSequenceValue(sequenceName);
    }
}
```

Tenant A's order #1001 and tenant B's order #1001 are independent. This is useful when tenants expect their own numbering (invoices, tickets, work orders).

The trade-off is the number of sequences. If you have 10,000 tenants, you have 10,000 sequences. Each sequence occupies a small amount of memory on the sequencer actor that manages it. The server setting `--sequencer-max-sequences-per-actor` (default: 10,000) controls how many sequences one actor keeps in memory. Least recently used sequences are evicted and reloaded on demand.

11.4.3 Prefixed Composite IDs

Combine a sequence number with a prefix to create IDs that carry context:

```
long seq = await client.NextSequenceValue("invoice-ids");
string invoiceId = $"INV-{DateTime.UtcNow:yyyyMM}-{seq:D6}";
// "INV-202608-010042"
```

The prefix makes the ID self-describing. A support engineer can see at a glance that "INV-202608-010042" is an invoice from August 2026. The sequence number provides uniqueness. The zero-padded format (`D6`) makes IDs sort correctly as strings.

Be careful with this pattern. The date prefix is informational, not a uniqueness guarantee. The sequence number alone guarantees uniqueness. If you reset the sequence or create a new one each month, you must ensure old and new sequences do not overlap.

11.4.4 Batch ID Allocation

When you import data in bulk, allocating one ID at a time is inefficient. Reserve a range and assign IDs locally:

```
async Task ImportCustomers(KahunaClient client, List<CustomerRecord> records)
{
    KahunaSequenceRange range = await client.ReserveSequenceRange(
        "customer-ids",
        count: records.Count,
        idempotencyKey: $"import-{batchId}"
    );

    long id = range.Start;

    foreach (CustomerRecord record in records)
    {
        record.Id = id++;
        await SaveToDatabase(record);
    }
}
```

One network round trip reserves all the IDs. The application assigns them locally. The idempotency key ensures that retrying the import does not consume a second range.

Within a reserved range, IDs are contiguous. This is useful for data imports where contiguous numbering is a requirement.

11.4.5 Sharded ID Spaces

When multiple services need non-overlapping ID ranges, use staggered sequences:

```
int shardCount = 4;

for (int i = 0; i < shardCount; i++)
{
    await client.CreateSequence(
        $"event-ids-shard-{i}",
        initialValue: i,
        increment: shardCount
    );
}

// Shard 0 produces: 4, 8, 12, 16, ...
```

```
// Shard 1 produces: 5, 9, 13, 17, ...
// Shard 2 produces: 6, 10, 14, 18, ...
// Shard 3 produces: 7, 11, 15, 19, ...
```

Each shard generates IDs independently. The increment ensures the ranges never overlap. The shard number is encoded in the last bits of every ID (`id % shardCount == shardIndex`).

This pattern is useful when each shard writes to its own database partition and you need globally unique IDs without cross-shard coordination at write time.

11.4.6 Gap-Free Invoice Numbers

Some jurisdictions require contiguous invoice numbers with no gaps. Set the block size to 1:

```
kahuna-server --sequencer-block-size 1
```

With a block size of 1, every allocation requires a Raft round trip. This is slower (roughly 1,000 times slower than the default block size of 1,000) but guarantees no gaps from unused block remainders.

Even with block size 1, an idempotent allocation with a lost response and no retry can appear as a gap to the application. The sequence advanced, but the application never used the value. To close this gap, use an idempotency key:

```
long invoiceNumber = await client.NextSequenceValue(
    "invoice-numbers",
    idempotencyKey: $"create-invoice-{requestId}"
);
```

If the response is lost, the retry returns the same number. No value is wasted.

11.5 Tuning Block Size

The `--sequencer-block-size` setting controls the trade-off between throughput and gap size:

Block Size	Raft Commits Per 10,000 IDs	Maximum Gap on Failure	Use Case
1	10,000	0	Gap-free numbering (invoices, legal documents)
10	1,000	9	Low-gap tolerance with moderate throughput
100	100	99	Balanced for most workloads
1,000 (default)	10	999	High throughput (event offsets, log entries)
10,000	1	9,999	Maximum throughput (analytics, telemetry)

The "maximum gap on failure" column shows the worst case: a node reserves a full block and crashes before issuing any value from it. In practice, gaps are smaller because the node usually issues some values before failing.

11.5.1 Block Lease Revalidation

Each block has a lease (default: 5 seconds, configurable with `--sequencer-block-lease`). If a block sits in memory longer than its lease without being revalidated, the sequencer checks the durable record before serving more values. This prevents a stale leader from issuing values that conflict with a new leader's allocations.

Shorter leases reduce stale-leader exposure but increase revalidation overhead. For most workloads, the 5-second default is appropriate.

11.6 Failure Scenarios

11.6.1 Node Failure Mid-Block

1. A sequencer actor reserves block [5001, 6000].
2. It issues values 5001 through 5042 to clients.
3. The node crashes.
4. A new leader takes over. It reserves block [6001, 7000].
5. Values 5043 through 6000 are never issued. The sequence jumps from 5042 to 6001.

This is the fundamental gap trade-off of block allocation. The gap is bounded by the block size. With the default block size of 1,000, the maximum gap from a single failure is 999 values.

11.6.2 Sequence Exhaustion

If a sequence has a maximum value and the high-water mark reaches it, further allocations fail:

```
try
{
    long id = await client.NextSequenceValue("limited-ids");
}
catch (KahunaException ex)
{
    Console.WriteLine($"Sequence exhausted: {ex.Message}");
}
```

Plan for exhaustion in advance. Monitor the current high-water mark with `GetSequence` and alert when it approaches the maximum:

```
KahunaSequence? seq = await client.GetSequence("limited-ids");

if (seq != null && seq.MaxValue > 0)
{
    double usedPercent = (double)seq.CurrentValue / seq.MaxValue * 100;

    if (usedPercent > 90)
       Console.WriteLine($"WARNING: Sequence is {usedPercent:F1}% exhausted.");
}
```

The `CurrentValue` property is the high-water mark (the highest reserved value), not the last value issued. It may be higher than the last value a client received.

11.6.3 Leadership Change

During a Raft leadership change, the old leader's in-memory block is discarded. The new leader starts fresh by reserving a new block. This causes a gap equal to the unused portion of the old block.

The gap preserves correctness: the new leader cannot reuse values from the old block because it does not know which values were already issued. Starting from the next block boundary guarantees uniqueness.

11.7 When Not to Use Kahuna Sequences

Kahuna sequences are a poor fit in some scenarios:

- **Offline ID generation.** If the client cannot reach a Kahuna cluster, it cannot allocate IDs. Use UUIDs or Snowflake IDs for offline-capable systems.
- **Unpredictable IDs.** Sequential IDs leak information. If a user sees order #10042, they know roughly how many orders exist. For public-facing IDs where this matters, generate a random ID or hash the sequential one.
- **Extremely high throughput with zero coordination.** If each node needs millions of IDs per second and cannot tolerate a network round trip per block, use local Snowflake-style generation.
- **Cross-cluster uniqueness.** Kahuna sequences are unique within one cluster. If you run multiple independent clusters, sequences can collide. Use a cluster prefix or a UUID for cross-cluster uniqueness.

11.8 A Complete Example

This example creates a service that generates formatted order IDs with a prefix, using batch allocation for throughput:

```
public class OrderIdGenerator
{
    private readonly KahunaClient _client;
    private readonly string _sequenceName = "order-ids";
    private readonly int _batchSize = 100;
    private long _nextId;
    private long _maxId;
    private readonly SemaphoreSlim _lock = new(1, 1);

    public OrderIdGenerator(KahunaClient client)
    {
        _client = client;
    }

    public async Task Initialize()
    {
        try
        {
            await _client.CreateSequence(_sequenceName, initialValue: 100000);
        }
        catch (KahunaException)
        {
            // Sequence already exists.
        }
    }

    public async Task<string> NextOrderId()
    {
        await _lock.WaitAsync();
```

```
        try
        {
            if (_nextId >= _maxId)
            {
                KahunaSequenceRange range = await _client.ReserveSequenceRange(
                    _sequenceName,
                    count: _batchSize
                );

                _nextId = range.Start;
                _maxId = range.End + 1;
            }

            long id = _nextId++;
            return $"ORD-{id}";
        }
        finally
        {
            _lock.Release();
        }
    }
}
```

Usage:

```
var generator = new OrderIdGenerator(client);
await generator.Initialize();

string id1 = await generator.NextOrderId();  // "ORD-100001"
string id2 = await generator.NextOrderId();  // "ORD-100002"
```

The generator reserves 100 IDs at a time and serves them from memory. After 100 IDs, it reserves another batch. This reduces network round trips to one per 100 IDs.

The `SemaphoreSlim` protects the in-memory range from concurrent access. If multiple threads call `NextOrderId` simultaneously, only one reserves a new batch while the others wait.

If the process crashes with unused IDs in the batch, those IDs are lost (a gap of at most 99 values). For most order numbering, this is acceptable.

11.9 Summary

Kahuna's distributed sequencer provides human-readable, sortable, unique IDs without a single-database bottleneck. Block allocation delivers high throughput by amortizing Raft consensus across many values.

Choose the block size based on your tolerance for gaps: block size 1 for gap-free numbering, larger blocks for higher throughput. Use range reservation for batch imports. Use staggered sequences for sharded ID spaces. Use idempotency keys to prevent duplicate allocation on retries.

For workloads that need offline generation, unpredictable IDs, or cross-cluster uniqueness, consider UUIDs or Snowflake IDs instead.

The next chapter covers service coordination: distributed configuration, service presence detection, and session management.

12 Coordinating Services

Backend services in a distributed system need shared state. A feature flag must be visible to every instance of the API. A scheduler must know which workers are alive. A game server must track which players are in a lobby. Each of these problems requires state that is consistent, fault-tolerant, and observable across multiple processes.

This chapter shows five coordination patterns built on Kahuna's key-value store: distributed configuration, service presence detection, session management, game lobby coordination, and distributed rate limiting.

12.1 Distributed Configuration

Many applications store configuration in a file or environment variable. This works until you need to change a value without restarting the application. A distributed configuration registry stores configuration in Kahuna, where any service instance can read the current value and where updates take effect immediately.

12.1.1 Storing Configuration

Use a key prefix to organize configuration by domain:

```
await client.SetKeyValue("config/api/rate-limit", "1000");
await client.SetKeyValue("config/api/timeout-ms", "5000");
await client.SetKeyValue("config/features/dark-mode", "true");
await client.SetKeyValue("config/features/beta-signup", "false");
```

Each configuration entry is a key-value pair. The prefix (`config/api/`, `config/features/`) groups related entries.

12.1.2 Reading Configuration

Read a single value:

```
KahunaKeyValue entry = await client.GetKeyValue("config/api/rate-limit");
int rateLimit = int.Parse(entry.ValueAsString ?? "500");
```

Read all entries in a group:

```
List<KahunaKeyValue> apiConfig = await client.GetByBucket(
    "config/api/",
    KeyValueDurability.Persistent
);

foreach (KahunaKeyValue kv in apiConfig)
{
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString}");
}
```

`GetByBucket` returns up to 4,096 entries that match the prefix. For configuration registries, this limit is rarely a concern. If you have more than 4,096 configuration entries, use `ScanAllByPrefix` instead, which streams all matching entries:

```
await foreach (KahunaKeyValue kv in client.ScanAllByPrefix("config/"))
{
```

```
    Console.WriteLine($"{kv.Key} = {kv.ValueAsString}");
}
```

12.1.3 Updating Configuration Safely

When two operators update the same configuration key at the same time, one update can overwrite the other. Use Compare-Revision-And-Swap (CRAS) to prevent lost updates:

```
async Task<bool> UpdateConfig(KahunaClient client, string key, string newValue)
{
    KahunaKeyValue current = await client.GetKeyValue(key);

    KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
        key,
        newValue,
        compareRevision: current.Revision
    );

    return result.Success;
}
```

If another update changed the key between the read and the write, the revision does not match and the write fails. The caller can retry with the fresh value.

12.1.4 Polling for Changes

Kahuna does not provide a push-based watch mechanism (unlike etcd's watch API). Services must poll for changes. A simple polling loop reads configuration periodically and applies changes:

```
public class ConfigPoller
{
    private readonly KahunaClient _client;
    private readonly string _prefix;
     private readonly Dictionary<string, (string Value, long Revision)> _cache
= new();

    public ConfigPoller(KahunaClient client, string prefix)
    {
        _client = client;
        _prefix = prefix;
    }

    public async Task PollOnce()
    {
        List<KahunaKeyValue> entries = await _client.GetByBucket(
            _prefix,
            KeyValueDurability.Persistent
        );

        foreach (KahunaKeyValue entry in entries)
```

```
        {
            if (_cache.TryGetValue(entry.Key, out var cached) && cached.Revision
== entry.Revision)
                continue;

            _cache[entry.Key] = (entry.ValueAsString ?? "", entry.Revision);
            OnConfigChanged(entry.Key, entry.ValueAsString);
        }
    }

    private void OnConfigChanged(string key, string? value)
    {
        Console.WriteLine($"Config changed: {key} = {value}");
    }
}
```

The poller compares revisions to detect changes. If the revision for a key has not changed since the last poll, the value is the same and no action is needed. This avoids unnecessary processing.

A poll interval of 5 to 30 seconds is appropriate for most configuration use cases. Critical settings (rate limits, circuit breaker thresholds) benefit from shorter intervals. Static settings (feature flags, display text) can tolerate longer intervals.

12.1.5 Configuration with TTL

Some configuration entries should expire automatically. A maintenance window flag, for example, should turn itself off after a set duration:

```
await client.SetKeyValue(
    "config/maintenance-mode",
    "true",
    expiryTime: 3600000  // 1 hour in milliseconds
);
```

After one hour, the key expires. Services that poll for this key will see it disappear and exit maintenance mode. No manual cleanup is required.

12.2 Service Presence Detection

In a microservice architecture, services need to know which instances of other services are alive. A load balancer needs a list of healthy backends. A job scheduler needs to know which workers are available.

Kahuna's TTL-based keys provide a simple presence mechanism. Each service instance writes an ephemeral key with a short TTL. The key acts as a heartbeat. If the instance crashes, the key expires and the instance disappears from the registry.

12.2.1 Registering Presence

Each service instance registers itself on startup and renews its registration periodically:

```
public class PresenceService : BackgroundService
{
```

```
    private readonly KahunaClient _client;
    private readonly string _serviceName;
    private readonly string _instanceId;
    private readonly int _ttlMs = 15000;
    private readonly int _renewalMs = 5000;

    public PresenceService(KahunaClient client, string serviceName)
    {
        _client = client;
        _serviceName = serviceName;
        _instanceId = Guid.NewGuid().ToString("N")[..8];
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        string key = $"presence/{_serviceName}/{_instanceId}";

        string info = JsonSerializer.Serialize(new
        {
            instanceId = _instanceId,
            host = Environment.MachineName,
            port = 8080,
            startedAt = DateTimeOffset.UtcNow
        });

        while (!stoppingToken.IsCancellationRequested)
        {
            await _client.SetKeyValue(key, info, expiryTime: _ttlMs);
            await Task.Delay(_renewalMs, stoppingToken);
        }
    }
}
```

The key includes the service name and a unique instance ID. The value contains connection details that other services need (host, port). The TTL is 15 seconds. The renewal runs every 5 seconds (one-third of the TTL), which gives two buffer attempts before the key expires.

12.2.2 Discovering Instances

Other services query the presence registry to find healthy instances:

```
async Task<List<ServiceInstance>> DiscoverInstances(KahunaClient client, string
serviceName)
{
    List<KahunaKeyValue> entries = await client.GetByBucket(
        $"presence/{serviceName}/",
        KeyValueDurability.Persistent
    );

    List<ServiceInstance> instances = new();
```

```
    foreach (KahunaKeyValue entry in entries)
    {
                                                    var       instance       =
JsonSerializer.Deserialize<ServiceInstance>(entry.ValueAsString ?? "{}");
        if (instance != null)
            instances.Add(instance);
    }

    return instances;
}
```

Only instances with unexpired keys appear in the result. A crashed instance's key expires after 15 seconds. The discovery query returns only live instances.

12.2.3 Ephemeral Keys for Presence

For presence data that does not need Raft replication, use ephemeral durability:

```
await client.SetKeyValue(
    $"presence/{serviceName}/{instanceId}",
    info,
    expiryTime: 15000,
    durability: KeyValueDurability.Ephemeral
);
```

Ephemeral keys are stored in memory on the receiving node. They are faster to write (no Raft round trip) but do not survive a node restart. If the Kahuna node holding the ephemeral key crashes, all presence records on that node vanish instantly.

Use ephemeral keys when:

- The presence data is recreated quickly (instances re-register on their next heartbeat cycle).
- Speed matters more than durability.
- Losing presence records temporarily is acceptable (the system recovers within one heartbeat cycle).

Use persistent keys when:

- The presence data is expensive to recreate.
- You need the registry to survive a Kahuna node restart.
- Accuracy during Kahuna cluster maintenance is important.

12.2.4 Graceful Deregistration

When a service shuts down gracefully, it should delete its presence key immediately instead of waiting for the TTL:

```
public override async Task StopAsync(CancellationToken cancellationToken)
{
    string key = $"presence/{_serviceName}/{_instanceId}";
    await _client.DeleteKeyValue(key);
    await base.StopAsync(cancellationToken);
}
```

This makes the instance disappear from the registry immediately. Other services see the updated list on their next discovery query. Without graceful deregistration, the stale key persists for up to 15 seconds after the process exits.

12.3 Session Management

Web applications and APIs often need server-side session state: a shopping cart, user preferences, or authentication tokens. Storing session state in Kahuna provides fault tolerance (the session survives a process restart) and consistency (multiple API instances can read the same session).

12.3.1 Creating a Session

```
async Task<string> CreateSession(KahunaClient client, string userId)
{
    string sessionId = Guid.NewGuid().ToString("N");
    string key = $"sessions/{sessionId}";

    string sessionData = JsonSerializer.Serialize(new
    {
        userId,
        createdAt = DateTimeOffset.UtcNow,
        cart = new List<string>()
    });

    await client.SetKeyValue(key, sessionData, expiryTime: 1800000);  // 30
minutes

    return sessionId;
}
```

The session key includes a random ID. The TTL (30 minutes) defines the session timeout. If the user does not interact within 30 minutes, the session expires automatically.

12.3.2 Reading and Updating a Session

```
async Task<SessionData?> GetSession(KahunaClient client, string sessionId)
{
    KahunaKeyValue entry = await client.GetKeyValue($"sessions/{sessionId}");

    if (!entry.Success)
        return null;

    // Extend the session TTL on each access (sliding expiration).
    await client.ExtendKeyValue($"sessions/{sessionId}", 1800000);

   return JsonSerializer.Deserialize<SessionData>(entry.ValueAsString ?? "{}");
}

async Task UpdateSession(KahunaClient client, string sessionId, SessionData
data)
```

```
{
    string key = $"sessions/{sessionId}";

    KahunaKeyValue current = await client.GetKeyValue(key);

    if (!current.Success)
        throw new InvalidOperationException("Session not found.");

    KahunaKeyValue result = await client.TryCompareRevisionAndSetKeyValue(
        key,
        JsonSerializer.Serialize(data),
        compareRevision: current.Revision
    );

    if (!result.Success)
        throw new InvalidOperationException("Session was modified concurrently.
Retry.");

    await client.ExtendKeyValue(key, 1800000);
}
```

Key details:

- `ExtendKeyValue` resets the TTL from the current time. This provides sliding expiration: the session stays alive as long as the user keeps interacting.
- `TryCompareRevisionAndSetKeyValue` prevents lost updates. If two API instances update the same session simultaneously, one succeeds and the other gets `Success = false`.

12.3.3 Session Cleanup

Sessions expire automatically through TTL. No background cleanup is needed. When the TTL elapses, Kahuna removes the key.

For an explicit logout:

```
async Task DestroySession(KahunaClient client, string sessionId)
{
    await client.DeleteKeyValue($"sessions/{sessionId}");
}
```

12.3.4 Listing Active Sessions

To list all active sessions (for an admin dashboard, for example):

```
await foreach (KahunaKeyValue session in client.ScanAllByPrefix("sessions/"))
{
    var data = JsonSerializer.Deserialize<SessionData>(session.ValueAsString ??
"{}");
    Console.WriteLine($"Session: {session.Key}, User: {data?.UserId}");
}
```

`ScanAllByPrefix` streams results with no upper limit, so it handles large numbers of active sessions.

12.4 Game Lobby Coordination

Online multiplayer games need coordination infrastructure: creating game rooms, matching players, tracking who is in a lobby, and managing game state transitions. Kahuna's primitives (locks, TTL keys, prefix queries, transactions) combine to support these patterns.

12.4.1 Room Creation

Use a lock to ensure that room creation is exclusive:

```
async Task<string> CreateRoom(KahunaClient client, string gameMode, string
hostPlayer)
{
    long roomId = await client.NextSequenceValue("room-ids");
    string roomKey = $"rooms/{roomId}";

    string roomData = JsonSerializer.Serialize(new
    {
        id = roomId,
        gameMode,
        host = hostPlayer,
        status = "waiting",
        maxPlayers = 4,
        createdAt = DateTimeOffset.UtcNow
    });

    await client.SetKeyValue(roomKey, roomData);

    return roomKey;
}
```

The sequencer generates a unique room ID. The room starts in the "waiting" status.

12.4.2 Player Presence in a Room

Each player registers their presence in the room with a TTL key:

```
async Task JoinRoom(KahunaClient client, string roomKey, string playerId)
{
    string presenceKey = $"{roomKey}/players/{playerId}";

    string playerInfo = JsonSerializer.Serialize(new
    {
        playerId,
        joinedAt = DateTimeOffset.UtcNow,
        ready = false
    });

    await client.SetKeyValue(presenceKey, playerInfo, expiryTime: 10000);
}

async Task SendHeartbeat(KahunaClient client, string roomKey, string playerId)
```

```
{
    await client.ExtendKeyValue($"{roomKey}/players/{playerId}", 10000);
}
```

The player's presence key has a 10-second TTL. A heartbeat task renews it every 3 seconds. If the player disconnects (closes the browser, loses network), the key expires and the player disappears from the room.

12.4.3 Listing Players in a Room

```
async Task<List<PlayerInfo>> GetPlayersInRoom(KahunaClient client, string roomKey)
{
    List<KahunaKeyValue> entries = await client.GetByBucket(
        $"{roomKey}/players/",
        KeyValueDurability.Persistent
    );

    return entries
        .Select(e => JsonSerializer.Deserialize<PlayerInfo>(e.ValueAsString ?? "{}"))
        .Where(p => p != null)
        .ToList()!;
}
```

Only players with unexpired keys appear. A player who disconnected 10 seconds ago is already gone from the list.

12.4.4 Room State Transitions

A game room moves through states: waiting, matching, playing, finished. Use CVAS (Compare-Value-And-Swap) to ensure state transitions are valid:

```
async Task<bool> StartGame(KahunaClient client, string roomKey)
{
    // Only transition from "waiting" to "playing."
    KahunaKeyValue room = await client.GetKeyValue(roomKey);
    var roomData = JsonSerializer.Deserialize<RoomData>(room.ValueAsString ?? "{}");

    if (roomData?.Status != "waiting")
        return false;

    var updated = roomData with { Status = "playing", StartedAt = DateTimeOffset.UtcNow };

    KahunaKeyValue result = await client.TryCompareValueAndSetKeyValue(
        roomKey,
        JsonSerializer.Serialize(updated),
        compareValue: room.ValueAsString!
    );
```

```
    return result.Success;
}
```

If two players press "Start" at the same moment, only one CVAS succeeds. The other gets `Success = false`. The room transitions exactly once.

12.4.5 Matchmaking

A simple matchmaking system scans for rooms in the "waiting" state and adds the player to the first room with available slots:

```
async Task<string?> FindMatch(KahunaClient client, string playerId, string gameMode)
{
    List<KahunaKeyValue> rooms = await client.GetByBucket(
        "rooms/",
        KeyValueDurability.Persistent
    );

    foreach (KahunaKeyValue entry in rooms)
    {
        var room = JsonSerializer.Deserialize<RoomData>(entry.ValueAsString ?? "{}");

        if (room?.Status != "waiting" || room.GameMode != gameMode)
            continue;

        List<PlayerInfo> players = await GetPlayersInRoom(client, entry.Key);

        if (players.Count >= room.MaxPlayers)
            continue;

        // Try to join. The TTL-based presence handles the race:
        // if two players join and exceed the limit, the host can
        // enforce the cap before starting the game.
        await JoinRoom(client, entry.Key, playerId);
        return entry.Key;
    }

    return null;
}
```

This is a simple linear scan. For production systems with thousands of rooms, partition rooms by game mode (use a prefix like `rooms/{gameMode}/`) and scan only the relevant partition.

12.4.6 Room Cleanup

Finished game rooms can be deleted explicitly:

```
async Task CleanupRoom(KahunaClient client, string roomKey)
{
```

```
    // Delete all player presence keys.
    List<KahunaKeyValue> players = await client.GetByBucket(
        $"{roomKey}/players/",
        KeyValueDurability.Persistent
    );

    foreach (KahunaKeyValue player in players)
    {
        await client.DeleteKeyValue(player.Key);
    }

    // Delete the room itself.
    await client.DeleteKeyValue(roomKey);
}
```

Alternatively, create rooms with a TTL so they expire automatically after the maximum game duration:

```
await client.SetKeyValue(roomKey, roomData, expiryTime: 3600000);  // 1 hour
```

12.5 Failure Scenarios

12.5.1 Service Crash

A service instance crashes. Its presence key has a 15-second TTL. After 15 seconds, the key expires. The next discovery query omits the crashed instance. Other instances continue to serve traffic. When the crashed instance restarts, it re-registers with a new presence key.

12.5.2 Configuration Update Race

Two operators update the same configuration key at the same moment.

1. Operator A reads the key at revision 5.
2. Operator B reads the key at revision 5.
3. Operator A writes a new value with `compareRevision: 5`. The write succeeds. The revision becomes 6.
4. Operator B writes a different value with `compareRevision: 5`. The compare fails because the revision is now 6.

Operator B must re-read the key and decide whether to retry. No data is lost and no update is silently overwritten.

12.5.3 Player Disconnect

A player's network connection drops. The player's heartbeat stops. After 10 seconds, the presence key expires. The room's player list no longer includes the disconnected player. The host (or the game logic) can detect the change on the next player list query and handle it: pause the game, assign the player's slot to an AI, or end the match.

If the player reconnects within 10 seconds and resumes heartbeats, the key is refreshed and the player remains in the room.

12.6 Choosing Between Persistent and Ephemeral

Concern	Persistent	Ephemeral
Raft consensus	Yes (durable, replicated)	No (in-memory, single node)
Write latency	Higher	Lower
Survives node restart	Yes	No
Use case	Configuration, sessions, game state	Presence heartbeats, rate counters
Consistency	Strong (linearizable)	Best-effort (single node)

A good default: use persistent durability for data that matters (configuration, session state, room records) and ephemeral durability for data that is rebuilt quickly (presence heartbeats, temporary counters).

12.7 Distributed Rate Limiting

A .NET rate limiter counts the requests of one process. When an application runs as several replicas, each replica keeps its own budget. A limit of 100 requests per minute admits 100 per minute per replica. The Kahuna rate limiters keep the budget in the cluster, so every replica that names the same key spends one shared budget.

The `Kahuna.Client.RateLimiting` namespace provides four limiter types that implement the `System.Threading.RateLimiting` contract. Each decision runs as one script transaction: two replicas cannot both take the last permit. Time comes from `hlc()`, so all replicas agree on window boundaries and refills.

12.7.1 Limiter Types

Fixed window. At most `PermitLimit` requests per `Window`. Windows are aligned to the cluster clock. The budget resets on each window boundary.

```
var limiter = new KahunaFixedWindowRateLimiter(client, new()
{
    Key = "api/global",
    PermitLimit = 1000,
    Window = TimeSpan.FromMinutes(1)
});
```

Sliding window. At most `PermitLimit` requests in any `Window`. The window is divided into `SegmentsPerWindow` segments (maximum 1000). Permits of a segment come back when that segment slides out of the window.

```
var limiter = new KahunaSlidingWindowRateLimiter(client, new()
{
    Key = "api/global",
    PermitLimit = 1000,
    Window = TimeSpan.FromMinutes(1),
    SegmentsPerWindow = 6
});
```

Token bucket. Bursts up to `TokenLimit` tokens. The bucket refills by `TokensPerPeriod` tokens every `ReplenishmentPeriod`. No process runs a refill timer. Each decision computes the refill from the cluster clock.

```
var limiter = new KahunaTokenBucketRateLimiter(client, new()
{
    Key = "api/global",
    TokenLimit = 50,
    ReplenishmentPeriod = TimeSpan.FromSeconds(10),
    TokensPerPeriod = 10
});
```

Concurrency. At most `PermitLimit` requests in progress at once, across every replica. A permit is a lease key that expires after `LeaseDuration` (default 30 seconds). The holder renews it automatically. When the holder dies, the lease expires and the permit returns to the pool.

```
var limiter = new KahunaConcurrencyLimiter(client, new()
{
    Key = "api/concurrent",
    PermitLimit = 20,
    LeaseDuration = TimeSpan.FromSeconds(30)
});
```

12.7.2 Common Options

All four types share these settings through the `KahunaRateLimiterOptions` base class:

Option	Default	Description
`Key`	(required)	Kahuna key that holds the limiter state; every process that names the same key spends one budget
`Durability`	`Ephemeral`	`Persistent` replicates every admission; `Ephemeral` keeps the state in memory
`QueueLimit`	`0`	How many permits may wait in this process for the budget to free up; zero refuses at once
`QueueProcessingOrder`	`OldestFirst`	Which waiter is served first
`FailureMode`	`Throw`	What happens when the cluster is unreachable: `Throw`, `Allow` (admit the request), or `Deny` (refuse it)
`MaxRetries`	`8`	Retry count for a decision that aborted on a contended counter

`AttemptAcquire` always refuses because a decision needs a cluster round trip. The middleware calls `AcquireAsync` and gets the real decision.

12.7.3 Partitioned Limiting

`KahunaRateLimitPartition` builds a separate budget for each partition key (for example, each user or each client address). Use `KeyFor` to name the Kahuna key of a partition:

```
string key = KahunaRateLimitPartition.KeyFor("login_attempts", userEmail);
// produces "rate-limit/login_attempts/alice@example.com"
```

12.7.4 ASP.NET Core Integration

The `Kahuna.Client.AspNetCore` package adds extension methods to `RateLimiterOptions`. These register Kahuna-backed policies that the ASP.NET Core rate-limiting middleware enforces.

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // One budget for all replicas
    options.AddKahunaFixedWindowLimiter("global_api", o =>
    {
        o.PermitLimit = 1000;
        o.Window = TimeSpan.FromMinutes(1);
    });

    // One budget per user
    options.AddKahunaTokenBucketLimiter("login_attempts",
        context => context.Request.Query["email"].ToString(),
        o =>
        {
            o.TokenLimit = 5;
            o.ReplenishmentPeriod = TimeSpan.FromMinutes(5);
            o.TokensPerPeriod = 1;
        });
});
```

Each method has two forms. The form without a `partitionBy` function gives the policy one budget for all requests. The form with `partitionBy` gives each partition its own budget. The `KahunaClient` can be passed directly or resolved from the request services.

12.8 Summary

Kahuna's key-value primitives support five common coordination patterns. A configuration registry stores shared settings that all service instances can read. Prefix queries (`GetByBucket`, `ScanAllByPrefix`) retrieve groups of related entries. CRAS prevents lost updates.

Presence detection uses TTL-based keys as heartbeats. A service instance writes a key with a short TTL and renews it periodically. When the instance crashes, the key expires and the instance disappears from the registry.

Session management stores user state in keyed entries with sliding expiration. `ExtendKeyValue` resets the TTL on each access. CRAS prevents concurrent session updates from overwriting each other.

Game lobby coordination combines sequences (room IDs), TTL keys (player presence), prefix queries (player lists), and CVAS (state transitions) into a cohesive system.

Distributed rate limiting keeps a shared budget in the cluster so that every replica of an application enforces one limit. Four limiter types (fixed window, sliding window, token bucket, and concurrency) integrate with the `System.Threading.RateLimiting` contract and the ASP.NET Core middleware.

The next chapter covers transactional workflows: how to decompose multi-step business operations into atomic transactions.

13 Transactional Workflows

Chapter 6 introduced transactions: script transactions that execute atomically on the server, and interactive sessions that let the client make decisions between operations. Chapter 9 showed how to make individual operations idempotent.

This chapter brings both ideas together. Real business operations span multiple keys, multiple decisions, and sometimes multiple transactions. An order that reserves inventory, charges a payment, and sends a confirmation must either complete fully or leave no partial trace. This chapter shows how to design these workflows, where to draw transaction boundaries, and how to handle failures at each stage.

13.1 The Problem with Separate Writes

Consider this order placement code:

```
// INCORRECT: two separate writes, no transaction
await client.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
await client.SetKeyValue("orders/123/status", "confirmed");
```

If the process crashes between the two writes, the inventory is decremented but the order is never confirmed. The customer does not get their order. The inventory count is wrong.

Making both writes unconditional does not help either. If the second write fails (network timeout, server error), retrying the entire operation decrements inventory a second time.

The solution is a transaction. Both writes succeed together or fail together. No intermediate state is visible to other clients.

13.2 Single-Transaction Workflows

13.2.1 Script Transaction

When the workflow logic is a simple sequence of reads and writes with straightforward conditions, a script transaction is the best fit:

```
string script = @"
    LET stock = GET inventory/widget-a
    IF stock = NOT FOUND THEN
        THROW 'Item not found'
    END

    LET count = TO_INT(stock)
    IF count < 1 THEN
        THROW 'Out of stock'
    END

    SET inventory/widget-a TO_STRING(count - 1)
    SET orders/@orderId/status 'confirmed'
    SET orders/@orderId/item 'widget-a'
";

KahunaKeyValueTransactionResult      result      =
```

```
await client.ExecuteKeyValueTransactionScript(
    script,
    parameters: new()
    {
        new() { Key = "@orderId", Value = orderId }
    }
);
```

The server executes the entire script as one atomic unit. If the stock check fails, no writes happen. If both writes succeed, they are visible together. No other client can see the inventory decremented without the order confirmed.

Script transactions are best when:

- All the data needed for decisions is inside Kahuna.
- The logic is expressible with the script language (conditions, loops, string operations).
- No external service calls are needed during the transaction.

13.2.2 Interactive Session

When the workflow requires application logic that the script language cannot express (calling an external API, running a complex computation, making a decision based on business rules), use an interactive session:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
            KahunaKeyValue stockEntry = await session.GetKeyValue("inventory/
widget-a");
        int stock = int.Parse(stockEntry.ValueAsString ?? "0");

        if (stock < 1)
            throw new InvalidOperationException("Out of stock.");

        decimal price = await GetCurrentPrice("widget-a");

        if (price > maxBudget)
            throw new InvalidOperationException("Price exceeds budget.");

       await session.SetKeyValue("inventory/widget-a", (stock - 1).ToString());
        await session.SetKeyValue($"orders/{orderId}/status", "confirmed");
       await session.SetKeyValue($"orders/{orderId}/price", price.ToString());
        await session.Commit(ct);
    }
);
```

The `GetCurrentPrice` call is application logic that cannot run inside a script. The interactive session lets you interleave reads from Kahuna, external calls, and writes within one atomic boundary.

`RetryableTransaction` wraps the session in a retry loop. If a concurrent transaction modifies the same keys, the commit fails and the entire callback runs again with a fresh session and fresh reads.

13.3 Choosing a Locking Mode

The locking mode determines how the transaction handles concurrent access.

13.3.1 Pessimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Pessimistic
}
```

Pessimistic locking acquires an exclusive lock on every key the session reads or writes. Other transactions that touch the same keys wait until this transaction commits or rolls back.

Use pessimistic locking when:

- Conflicts are frequent. Multiple workers process the same queue. Multiple users update the same account.
- The transaction does expensive work (external API calls, long computations). Aborting and retrying wastes that work.
- You prefer to wait for access rather than retry on conflict.

13.3.2 Optimistic Locking

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic
}
```

Optimistic locking acquires locks only on writes. Reads are lock-free. Conflicts are detected at commit time: if a key that this transaction read was modified by another transaction, the commit is rejected.

Use optimistic locking when:

- Conflicts are rare. Most transactions touch different keys.
- The transaction is short (a few reads and writes, no external calls).
- You prefer higher concurrency over guaranteed success.

13.3.3 Adding Read Validation

With optimistic locking, enable `TrackAndValidate` to detect write-skew anomalies:

```
new KahunaTransactionOptions
{
    Locking = KeyValueTransactionLocking.Optimistic,
    ReadValidation = ReadValidation.TrackAndValidate
}
```

The server records every key the session reads. At commit time, it checks whether any read key was modified after the read. If so, the commit is rejected. This prevents a class of bug where two transactions read overlapping data, make independent decisions, and both commit without conflict.

13.4 Multi-Transaction Workflows

Not every workflow fits in a single transaction. A workflow that calls an external payment gateway cannot roll back the payment if a later step fails. The payment gateway does not participate in Kahuna's transaction protocol.

For these workflows, decompose the work into multiple transactions with explicit progress tracking between them.

13.4.1 The Saga Pattern

A saga is a sequence of transactions, each with a compensating action. If a step fails, the saga runs the compensating actions for all previously completed steps.

Here is an order placement saga with three steps:

```
async Task PlaceOrder(KahunaClient client, string orderId, OrderDetails order)
{
    // Step 1: Reserve inventory (within a Kahuna transaction).
    await ReserveInventory(client, orderId, order);

    try
    {
        // Step 2: Charge payment (external API call, guarded by a lock).
        await ChargePayment(client, orderId, order);
    }
    catch
    {
        // Compensate step 1: release the reserved inventory.
        await ReleaseInventory(client, orderId, order);
        throw;
    }

    // Step 3: Confirm the order (within a Kahuna transaction).
    await ConfirmOrder(client, orderId);
}
```

Each step is a separate transaction (or a lock-protected external call). If the payment fails, the compensation releases the inventory. The order is never left in an inconsistent state.

13.4.2 Step 1: Reserve Inventory

```
async Task ReserveInventory(KahunaClient client, string orderId, OrderDetails
order)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
```

```
        {
            foreach (var item in order.Items)
            {
                KahunaKeyValue stock = await session.GetKeyValue($"inventory/
{item.Sku}");
                int available = int.Parse(stock.ValueAsString ?? "0");

                if (available < item.Quantity)
                     throw new InvalidOperationException($"Insufficient stock
for {item.Sku}.");

                await session.SetKeyValue(
                    $"inventory/{item.Sku}",
                    (available - item.Quantity).ToString()
                );
            }

              await session.SetKeyValue($"orders/{orderId}/step", "inventory-
reserved");
            await session.Commit(ct);
        }
    );
}
```

13.4.3 Step 2: Charge Payment

```
async Task ChargePayment(KahunaClient client, string orderId, OrderDetails
order)
{
    await using KahunaLock chargeLock = await client.GetOrCreateLock(
        $"charge/{orderId}",
        expiry: TimeSpan.FromSeconds(30)
    );

    if (!chargeLock.IsAcquired)
        throw new InvalidOperationException("Could not acquire charge lock.");

     KahunaKeyValue stepCheck = await client.GetKeyValue($"orders/{orderId}/
step");

    if (stepCheck.ValueAsString == "payment-charged")
        return;

    await CallPaymentGateway(order.PaymentDetails, chargeLock.FencingToken);

    await client.SetKeyValue($"orders/{orderId}/step", "payment-charged");
}
```

The lock prevents duplicate charges on retry. The step check provides idempotency: if the step already completed, the method returns without charging again. The fencing token protects against stale lock holders (Chapter 9).

13.4.4 Step 3: Confirm Order

```
async Task ConfirmOrder(KahunaClient client, string orderId)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            await session.SetKeyValue($"orders/{orderId}/status", "confirmed");
            await session.SetKeyValue($"orders/{orderId}/step", "completed");
            await session.Commit(ct);
        }
    );
}
```

13.4.5 Compensation: Release Inventory

```
async Task ReleaseInventory(KahunaClient client, string orderId, OrderDetails
order)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            foreach (var item in order.Items)
            {
                KahunaKeyValue stock = await session.GetKeyValue($"inventory/
{item.Sku}");
                int current = int.Parse(stock.ValueAsString ?? "0");

                await session.SetKeyValue(
                    $"inventory/{item.Sku}",
                    (current + item.Quantity).ToString()
                );
            }

            await session.SetKeyValue($"orders/{orderId}/step", "cancelled");
            await session.Commit(ct);
        }
    );
}
```

13.5 Transaction Boundaries

Choosing where to draw transaction boundaries is a design decision. Two principles guide it.

13.5.1 Principle 1: Keep Transactions Short

A transaction holds locks (in pessimistic mode) or risks conflicts (in optimistic mode) for its entire duration. The longer the transaction, the more contention it creates.

Do not put external calls (HTTP requests, database queries, message publishes) inside a Kahuna transaction. These calls add latency and can time out, which extends the lock duration and increases the chance of a transaction timeout.

Instead, separate the external call from the transaction:

```
// Read data in a transaction.
string price;
await client.RetryableTransaction(
    new KahunaTransactionOptions { Timeout = 5000 },
    async (session, ct) =>
    {
        KahunaKeyValue entry = await session.GetKeyValue("products/widget-a");
        price = entry.ValueAsString ?? "0";
        await session.Commit(ct);
    }
);

// External call outside the transaction.
bool approved = await PaymentGateway.Charge(price);

// Write the result in a second transaction.
if (approved)
{
    await client.RetryableTransaction(
        new KahunaTransactionOptions { Timeout = 5000 },
        async (session, ct) =>
        {
            await session.SetKeyValue("orders/123/status", "paid");
            await session.Commit(ct);
        }
    );
}
```

13.5.2 Principle 2: Group Related Writes

Writes that must be consistent with each other belong in the same transaction. If decrementing inventory and recording the order must happen together, they go in one transaction. If they can tolerate temporary inconsistency (inventory decremented but order not yet recorded), they can go in separate transactions.

Ask this question: "If the process crashes between these two operations, is the data in a valid state?" If the answer is no, the operations belong in the same transaction.

13.6 Handling Conflicts

When two transactions modify the same keys concurrently, one succeeds and the other is rejected.

13.6.1 With Pessimistic Locking

The second transaction waits for the first to release its locks. If the wait exceeds the transaction timeout, the second transaction is aborted with a timeout error. The client can start a new session and retry.

13.6.2 With Optimistic Locking

Both transactions proceed without waiting. At commit time, the server detects that one transaction read a key that the other modified. The second transaction to commit is rejected with an `Aborted` error. The client starts a new session and retries with fresh reads.

13.6.3 The Retry Loop

`RetryableTransaction` handles both cases automatically. It catches `Aborted`, `MustRetry`, and `AlreadyLocked` errors, waits with decorrelated jitter, and retries with a new session:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Optimistic,
        ReadValidation = ReadValidation.TrackAndValidate
    },
    async (session, ct) =>
    {
        KahunaKeyValue a = await session.GetKeyValue("accounts/alice");
        KahunaKeyValue b = await session.GetKeyValue("accounts/bob");

        long balanceA = long.Parse(a.ValueAsString ?? "0");
        long balanceB = long.Parse(b.ValueAsString ?? "0");

       await session.SetKeyValue("accounts/alice", (balanceA - 50).ToString());
        await session.SetKeyValue("accounts/bob", (balanceB + 50).ToString());
        await session.Commit(ct);
    }
);
```

If another transaction modifies Alice's or Bob's balance between the reads and the commit, this transaction is rejected and retried. The retry reads fresh balances and recomputes the transfer.

The retry strategy uses up to 10 attempts with a jitter cap of 500 milliseconds. If all 10 attempts fail, the method throws a `KahunaException`.

13.7 Script Transactions for Complex Logic

Some workflows fit entirely in a script with no external calls. Script transactions are faster (one round trip instead of many) and simpler (no session management, no retry loop).

13.7.1 Conditional Batch Update

```
string script = @"
    LET total = 0

    FOR key IN 'cart/user-42/' DO
        LET item = GET key
        IF item != NOT FOUND THEN
            LET price = TO_INT(item)
            LET total = total + price
        END
    END

    IF total > 10000 THEN
        SET orders/user-42/discount '10'
    ELSE
        SET orders/user-42/discount '0'
    END

    SET orders/user-42/total TO_STRING(total)
";

await client.ExecuteKeyValueTransactionScript(script);
```

This script scans all items in a user's cart, computes the total, applies a discount rule, and writes the result. The entire operation is atomic.

13.7.2 Multi-Key Transfer with Validation

```
string script = @"
    LET from_balance = TO_INT(GET accounts/@from)
    LET to_balance = TO_INT(GET accounts/@to)
    LET amount = TO_INT(@amount)

    IF from_balance < amount THEN
        THROW 'Insufficient funds'
    END

    SET accounts/@from TO_STRING(from_balance - amount)
    SET accounts/@to TO_STRING(to_balance + amount)
";

KahunaTransactionScript transferScript = client.LoadTransactionScript(script);

await transferScript.Run(parameters: new()
{
    new() { Key = "@from", Value = "alice" },
    new() { Key = "@to", Value = "bob" },
    new() { Key = "@amount", Value = "100" }
});
```

The pre-hashed script caches the parsed plan on the server. Subsequent calls with different parameters skip parsing.

13.8 Failure Scenarios

13.8.1 Partial Failure Mid-Workflow

1. The saga reserves inventory (step 1 commits).
2. The payment gateway call times out (step 2 fails).
3. The saga runs compensation: releases the reserved inventory.
4. The order is not created. Inventory is back to its original state.

Without compensation, the inventory stays decremented and the order does not exist. The system is inconsistent.

13.8.2 Concurrent Workflow Conflict

1. Two users try to buy the last unit of the same item.
2. User A's transaction reads stock = 1 and decrements to 0.
3. User B's transaction reads stock = 1 (before A commits, in optimistic mode) and tries to decrement to 0.
4. User A commits first.
5. User B's commit is rejected (the stock key was modified after the read).
6. User B retries. The fresh read sees stock = 0. The transaction throws "Out of stock."

The conflict is detected and resolved correctly. No stock is oversold.

13.8.3 Timeout During Commit

1. The client sends a commit request.
2. The network drops the response.
3. The client does not know whether the commit succeeded.

What happens next depends on the transaction model:

- **Script transactions.** The server executes the script atomically. The client can retry with the same idempotency key (if the script is designed for it) or check the state by reading the keys.
- **Interactive sessions.** The server is the transaction coordinator. If the commit request reached the server and the server decided to commit, the writes are applied regardless of the client's network failure. The client should read the affected keys to determine the outcome.

In both models, the server drives the commit decision. A client crash after sending the commit request does not prevent the transaction from completing.

13.9 Workflow Design Checklist

Before building a multi-step workflow, answer these questions:

1. **Can the entire workflow fit in one transaction?** If yes, use a single script or session transaction. This is simpler and avoids partial failure.
2. **Does the workflow call external services?** If yes, those calls cannot be inside a Kahuna transaction. Split the workflow into multiple transactions with progress tracking between them.

3. **Does each step need a compensating action?** If a later step fails, can you undo the earlier steps? Define compensation for each step.
4. **Is each step idempotent?** On retry, does the step produce the same result? Use step-tracking keys (Chapter 9) to guard against duplicate execution.
5. **What locking mode fits?** High contention favors pessimistic. Rare conflicts favor optimistic. When unsure, start with pessimistic and measure.
6. **How long is the transaction?** Keep each transaction under 5 seconds. If work takes longer, split it into shorter transactions.

13.10 Summary

Transactional workflows ensure that multi-key operations are atomic: all writes commit or none do. Script transactions handle workflows where all logic runs on the server. Interactive sessions handle workflows that need application logic between reads and writes.

For workflows that involve external services, decompose the work into multiple transactions with the saga pattern. Each step records its progress. Compensation actions undo completed steps if a later step fails.

Keep transactions short. Do not put external calls inside a transaction. Group writes that must be consistent in the same transaction. Use `RetryableTransaction` to handle conflicts automatically.

The next chapter covers failure handling and retries: how to classify errors, configure timeouts, and build resilient applications.

14 Handling Failures and Retries

Every distributed system fails. Networks drop packets. Nodes crash. Leaders change. Disks fill up. An application that ignores these failures will eventually lose data, hang indefinitely, or corrupt state.

This chapter catalogs every failure mode a Kahuna client can encounter. For each failure, it explains what happened, whether the operation is safe to retry, and what the application should do.

14.1 Error Classification

Kahuna operations return a response type that indicates the outcome. When an operation fails, the client throws a `KahunaException`. The exception carries an error code that tells you what went wrong and whether you can retry.

The error codes fall into five categories:

14.1.1 Transient Errors (Retry Immediately)

`MustRetry` means the operation did not execute and is safe to retry. Common causes:

- A leader change occurred while the operation was in flight. The request reached a node that is no longer the leader for the target partition.
- The node has not yet reached a safe timestamp for the requested read.
- The routing table is stale. The client sent the request to the wrong partition.

The operation had no effect. Retry it immediately or after a short delay. The client's connection layer will route the retry to the current leader.

14.1.2 Conflict Errors (Retry with Fresh State)

`Aborted` means the operation executed partially but was rejected at commit time. The most common cause is a transaction conflict: another transaction modified a key that this transaction read or wrote.

The transaction had no effect (all writes were rolled back). You can retry, but you must start a new transaction session and re-read all keys. Do not retry the commit on the same session.

`AlreadyLocked` means a key the transaction tried to lock is held by another transaction. In pessimistic mode, this happens when the lock wait exceeds the transaction timeout. You can retry with a new session.

14.1.3 Overload Errors (Back Off and Retry)

`AdmissionRefused` means the server rejected the request because it is at its concurrency ceiling. The server is not broken. It is protecting itself from overload.

Wait before retrying. Use exponential backoff with jitter. If `AdmissionRefused` errors persist, the server is saturated and needs more capacity or the workload needs throttling.

You can influence how long the client waits for an admission slot with the `AdmissionWaitMs` transaction option. A non-zero value tells the client to wait up to that many milliseconds for a slot before failing.

14.1.4 Permanent Errors (Do Not Retry)

`Errored` means the operation failed permanently. Something is wrong at the server level (storage corruption, internal bug, unrecoverable state). Retrying will produce the same error.

Log the error and alert. Do not retry.

`InvalidInput` means the client sent a malformed request: an empty key, an invalid flag combination, a script syntax error. This is a bug in the application code. Fix the input.

14.1.5 Not-Found Responses

`DoesNotExist` (for keys, locks, and sequences) and **`NotSet`** (for conditional writes that failed their condition) are not errors. They are normal outcomes that indicate the resource does not exist or the condition was not met. The application should handle these in its logic, not in its error handler.

14.2 The Error Decision Table

Error Code	Meaning	Safe to Retry?	Action
`MustRetry`	Transient (leader change, stale route)	Yes, immediately	Retry the same operation
`Aborted`	Transaction conflict	Yes, with fresh state	Start a new session, re-read, retry
`AlreadyLocked`	Key locked by another transaction	Yes, with fresh state	Start a new session, retry
`AdmissionRefused`	Server overloaded	Yes, after backoff	Wait with exponential backoff, then retry
`Errored`	Permanent server error	No	Log and alert
`InvalidInput`	Malformed request	No	Fix the application code
`DoesNotExist`	Resource not found	N/A	Handle in application logic
`NotSet`	Conditional write failed	N/A	Handle in application logic

14.3 Retry Strategies

14.3.1 Simple Retry with Backoff

For standalone key-value operations (not inside a transaction), implement a retry loop with exponential backoff and jitter:

```
async Task<KahunaKeyValue> SetWithRetry(
    KahunaClient client, string key, string value, int maxRetries = 5)
{
    int attempt = 0;

    while (true)
    {
        try
        {
            return await client.SetKeyValue(key, value);
        }
        catch (KahunaException ex) when (
            ex.KeyValueErrorCode == KeyValueResponseType.MustRetry ||
            ex.KeyValueErrorCode == KeyValueResponseType.AdmissionRefused)
        {
            attempt++;

            if (attempt >= maxRetries)
                throw;

            int delayMs = Math.Min(50 * (1 << attempt), 2000);
            int jitter = Random.Shared.Next(0, delayMs / 2);
            await Task.Delay(delayMs + jitter);
        }
    }
}
```

The delay doubles on each attempt (50, 100, 200, 400, 800 ms) with a cap of 2 seconds. The jitter prevents multiple clients from retrying at the same instant.

For `MustRetry`, the delay can be shorter (the error is transient and the next attempt often succeeds immediately). For `AdmissionRefused`, a longer delay gives the server time to recover.

14.3.2 RetryableTransaction

For transactions, use the built-in `RetryableTransaction` helper (Chapter 6). It handles retry logic automatically:

```
await client.RetryableTransaction(
    new KahunaTransactionOptions
    {
        Timeout = 5000,
        Locking = KeyValueTransactionLocking.Pessimistic
    },
    async (session, ct) =>
    {
        KahunaKeyValue counter = await session.GetKeyValue("stats/visits");
        long count = long.Parse(counter.ValueAsString ?? "0");
        await session.SetKeyValue("stats/visits", (count + 1).ToString());
        await session.Commit(ct);
```

```
    }
);
```

`RetryableTransaction` retries on `Aborted`, `MustRetry`, and `AlreadyLocked`. Its retry strategy:

- Up to 10 retries.
- Decorrelated jitter backoff with a median first delay of 50 milliseconds.
- Each delay is capped at 500 milliseconds.
- After 10 failed attempts, it throws a `KahunaException` with `Aborted`.

Any exception that is not `Aborted`, `MustRetry`, or `AlreadyLocked` propagates immediately. This includes `InvalidInput`, `Errored`, application logic exceptions, and cancellation.

14.3.3 When Not to Retry

Do not retry when:

- The error is `InvalidInput`. The request is malformed. Retrying sends the same bad request.
- The error is `Errored`. The server has a permanent problem. Retrying will not fix it.
- The operation has non-reversible side effects. If you called an external payment gateway before the Kahuna operation failed, retrying the entire workflow may charge the customer twice. Use idempotency guards (Chapter 9).

14.4 Timeout Configuration

Three layers of timeout apply to a Kahuna operation:

14.4.1 Operation Timeout

The `DefaultOperationTimeout` on the `KahunaOptions` client configuration sets a deadline for every operation that does not supply its own `CancellationToken`. The default is 30 seconds.

```
var client = new KahunaClient("https://localhost:2070", new KahunaOptions
{
    DefaultOperationTimeout = TimeSpan.FromSeconds(10)
});
```

If the operation does not complete within this timeout, the client throws an `OperationCanceledException`. The server may still process the request. The client does not know whether the operation succeeded.

14.4.2 Transaction Timeout

The `Timeout` option on `KahunaTransactionOptions` controls how long a transaction session can live on the server:

```
new KahunaTransactionOptions { Timeout = 5000 }  // 5 seconds
```

When the timeout elapses, the server releases all locks held by the session and aborts any uncommitted transaction. This prevents a slow or crashed client from holding locks indefinitely.

Set the transaction timeout based on how long the transaction's work takes. A 5-second timeout is appropriate for most read-modify-write patterns. Long-running transactions (batch imports, multi-step workflows) may need 10 to 30 seconds.

14.4.3 Lock Wait Timeout

The `wait` parameter on `GetOrCreateLock` controls how long the client retries before giving up:

```
await client.GetOrCreateLock(
    "jobs/send-email",
    expiry: TimeSpan.FromSeconds(15),
    wait: TimeSpan.FromSeconds(10),
    retry: TimeSpan.FromMilliseconds(200)
);
```

If the lock is not acquired within 10 seconds, `IsAcquired` is false. No exception is thrown.

14.4.4 Timeout Hierarchy

Timeout	Scope	Default	Set By
Operation timeout	Single RPC call	30 seconds	`KahunaOptions.DefaultOperationTimeout`
Transaction timeout	Entire session	5,000 ms (typical)	`KahunaTransactionOptions.Timeout`
Lock wait	Lock acquisition	0 (no wait)	`GetOrCreateLock` `wait` parameter
Admission wait	Transaction admission	0 (server default)	`KahunaTransactionOptions.AdmissionWaitMs`

14.5 Failure Scenarios

14.5.1 Leader Change During a KV Operation

1. The client sends a `SetKeyValue` request to node A (the current leader for the target partition).
2. A leader election starts. Node B becomes the new leader.
3. Node A receives the request but is no longer the leader. It rejects the request with `MustRetry`.
4. The client retries. The routing layer discovers the new leader (node B) and sends the request there.
5. The operation succeeds on node B.

The key point: `MustRetry` means the operation did not execute. The retry is safe. No data was written by the failed attempt.

14.5.2 Leader Change During a Transaction

1. The client opens a transaction session on node A.
2. The client sends several operations (reads and writes) within the session.
3. A leader change occurs. Node A is no longer the leader.
4. The next operation in the session returns `MustRetry` or `Aborted`.
5. The session is no longer valid. The client must start a new session.

When wrapped in `RetryableTransaction`, this happens automatically: the callback is invoked again with a fresh session, and all reads and writes are repeated from scratch.

14.5.3 Network Partition

1. The client cannot reach any Kahuna node.
2. Every operation times out with an `OperationCanceledException` or a gRPC `Unavailable` error.
3. The client should back off and retry. If the partition persists, the application must decide how to degrade.

Options for degradation:

- Return a cached value (if staleness is acceptable).
- Queue the operation for later retry.
- Return an error to the caller.
- Switch to a fallback data source.

14.5.4 Node Crash

1. The client is connected to node A, which crashes.
2. The gRPC connection drops. The client receives a transport-level error.
3. The client reconnects to another node in its URL list.
4. Operations resume on the surviving nodes.

If the client was constructed with multiple URLs, it distributes requests across available nodes automatically:

```
var client = new KahunaClient(new[]
{
    "https://node1:2070",
    "https://node2:2070",
    "https://node3:2070"
});
```

If a node is unreachable, the client skips it and sends the request to the next node in the round-robin rotation.

14.5.5 Transaction Conflict Under High Contention

1. Ten workers update the same counter concurrently.
2. Each starts a transaction, reads the counter, increments it, and commits.
3. Only one commit succeeds per round. The other nine receive `Aborted`.
4. Each retries with `RetryableTransaction`. The jitter spreads retries over time.
5. Eventually all ten updates complete.

High contention causes many retries. Two strategies reduce contention:

- **Pessimistic locking.** The first transaction locks the key. Others wait instead of reading and aborting. Less wasted work, but lower concurrency.
- **Reduce the conflict window.** Keep transactions short. Read, compute, write, commit. Do not hold a session open while doing unrelated work.

14.5.6 Admission Refused Under Load

1. The server has reached its maximum concurrent transaction count.
2. A new transaction request arrives.
3. The server rejects it with `AdmissionRefused`.

4. The client waits and retries.

To reduce `AdmissionRefused` errors:

- Set `AdmissionWaitMs` to a non-zero value. The server queues the request for up to that many milliseconds before rejecting it.
- Increase the server's admission ceiling (if the hardware supports it).
- Reduce the number of concurrent transactions from the application side (use a semaphore or work queue).

14.5.7 Timeout with Unknown Outcome

1. The client sends a `Commit` request.
2. The network drops the response.
3. The client's operation timeout fires. The client does not know whether the commit succeeded.

This is the most difficult failure mode. The server may have committed the transaction. The client cannot know without checking.

What to do:

- Read the keys that the transaction wrote. If they have the expected values, the commit succeeded.
- If the operation is idempotent (Chapter 9), retry the entire workflow. The idempotency guard prevents duplicate effects.
- If the operation is not idempotent and you cannot determine the outcome, log the uncertainty and alert. A human or automated reconciliation process must resolve the ambiguity.

The server is the transaction coordinator. If the commit request reached the server and the server decided to commit, the writes are applied even if the client never receives the response. The data is consistent. The only problem is that the client does not know it.

14.6 Application-Level Circuit Breaker

When Kahuna is unreachable for an extended period, retrying every operation wastes resources and adds latency. A circuit breaker stops sending requests after a threshold of consecutive failures and periodically probes to detect recovery.

```
public class KahunaCircuitBreaker
{
    private int _failureCount;
    private DateTime _openUntil = DateTime.MinValue;
    private readonly int _threshold;
    private readonly TimeSpan _openDuration;

    public KahunaCircuitBreaker(int threshold = 5, int openSeconds = 30)
    {
        _threshold = threshold;
        _openDuration = TimeSpan.FromSeconds(openSeconds);
    }

    public bool IsOpen => DateTime.UtcNow < _openUntil;
```

```
    public void RecordSuccess()
    {
        _failureCount = 0;
        _openUntil = DateTime.MinValue;
    }

    public void RecordFailure()
    {
        _failureCount++;

        if (_failureCount >= _threshold)
            _openUntil = DateTime.UtcNow + _openDuration;
    }
}
```

Usage:

```
if (circuitBreaker.IsOpen)
{
    // Fail fast. Do not send the request to Kahuna.
    return GetCachedValue(key);
}

try
{
    KahunaKeyValue result = await client.GetKeyValue(key);
    circuitBreaker.RecordSuccess();
    return result.ValueAsString;
}
catch (Exception)
{
    circuitBreaker.RecordFailure();
    return GetCachedValue(key);
}
```

After 5 consecutive failures, the circuit opens for 30 seconds. During that window, requests bypass Kahuna and use a fallback. After 30 seconds, the next request probes Kahuna. If it succeeds, the circuit closes. If it fails, the circuit stays open for another 30 seconds.

14.7 Error Handling Checklist

1. **Catch `KahunaException` and check the error code.** Do not catch the base `Exception` type and treat all errors the same.
2. **Retry `MustRetry` immediately.** The operation did not execute. The next attempt will likely succeed.
3. **Retry `Aborted` and `AlreadyLocked` with a new session.** Re-read all data. Do not reuse the old session.
4. **Back off on `AdmissionRefused`.** The server needs time to catch up.
5. **Do not retry `Errored` or `InvalidInput`.** These indicate a permanent problem.

6. **Use `RetryableTransaction` for transactions.** It handles retry, backoff, and session management.
7. **Set appropriate timeouts.** Operation timeout for individual calls, transaction timeout for sessions, admission wait for overloaded servers.
8. **Handle unknown outcomes explicitly.** When a timeout leaves the result unknown, check the state or use idempotency guards.

14.8 Summary

Kahuna errors fall into five categories: transient (`MustRetry`), conflict (`Aborted`, `AlreadyLocked`), overload (`AdmissionRefused`), permanent (`Errored`), and client bugs (`InvalidInput`). Each category demands a different response.

For standalone operations, implement a retry loop with exponential backoff and jitter. For transactions, use `RetryableTransaction`, which retries up to 10 times with decorrelated jitter backoff.

Configure timeouts at three levels: operation timeout (30 seconds default), transaction timeout (set per session), and lock wait (set per acquisition). A timeout with unknown outcome is the hardest failure mode: check the state or rely on idempotency guards.

A circuit breaker protects the application when Kahuna is unreachable for an extended period. It fails fast during the outage and probes periodically for recovery.

This chapter concludes Part II. The next chapter begins Part III (Understanding Kahuna Internals) with an architecture overview: the project layout, the component hierarchy, and how the three subsystems share a common structure.

15 Architecture Overview

Part II showed how to use Kahuna from the outside: keys, locks, sequences, transactions, and patterns for building distributed applications. Part III moves inside. The next ten chapters trace requests through the system, explain how data is partitioned and replicated, and describe how each subsystem works at the implementation level.

This chapter provides the map. It introduces the project layout, the component hierarchy, the shared patterns across subsystems, and the external libraries that Kahuna depends on. Every subsequent internals chapter zooms in on one part of this map.

15.1 Project Layout

The Kahuna repository contains several projects. Each has a distinct role:

Project	Description
`Kahuna.Core`	The embeddable engine. Contains all subsystem managers (key-value, locks, sequencer), persistence backends, the actor-based processing pipeline, composition, and inter-node communication.
`Kahuna.Server`	The server executable (`kahuna-server`). Hosts gRPC and REST endpoints, parses command-line options, and wires `Kahuna.Core` into an ASP.NET host.
`Kahuna.Client`	The .NET client library. Provides `KahunaClient`, gRPC and REST transport, request batching, and session management.
`Kahuna.Shared`	Types shared between client and server: enums (`KeyValueResponseType`, `LockResponseType`, `SequenceResponseType`), request and response DTOs, and protocol types.
`Kahuna.Control`	The command-line client (`kahuna-cli`). A thin wrapper over `Kahuna.Client` for interactive use.

Test and benchmark projects (`Kahuna.Client.Tests`, `Kahuna.Server.Tests`, `Kahuna.Benchmark`, `Kahuna.Microbenchmarks`) round out the repository.

The two most important projects for understanding internals are `Kahuna.Core` (where the engine lives) and `Kahuna.Server` (where the engine is hosted).

15.2 External Libraries

Kahuna depends on two libraries that are central to its architecture:

15.2.1 Kommander (Raft Consensus)

Kommander is the Raft consensus library. It provides:

- **`IRaft`**: the interface for proposing log entries, querying leader status, and registering replication callbacks.
- **Leader election**: heartbeat-based leader election within each Raft group.

- **Log replication**: append-only log with majority-based commit.
- **Hybrid Logical Clocks (HLC)**: timestamps that combine physical time with a logical counter for causal ordering.
- **Write-ahead log (WAL)**: durable log storage that Raft entries are written to before replication.
- **State transfer**: snapshot-based catch-up for followers that fall behind log compaction.

Kahuna does not implement consensus itself. Every durable write goes through Kommander's Raft protocol. Kommander handles leader election, log replication, and commit notification. Kahuna registers callbacks to apply committed entries to its own state.

15.2.2 Nixie (Actor System)

Nixie is the actor framework. It provides:

- **`ActorSystem`**: the container that manages actor lifecycles.
- **`IActor<TRequest, TResponse>`**: the interface that all Kahuna actors implement. Each actor processes messages one at a time in a single-threaded loop.
- **`IActorRef<TActor, TRequest>`**: a handle for sending messages to an actor.

Actors are Kahuna's concurrency model. Each key-value partition, lock partition, and sequencer partition has its own set of actors. Because each actor processes one message at a time, there is no need for locks on the in-memory state within an actor. Concurrency comes from having many actors, not from shared-memory parallelism within one.

15.3 The IKahuna Facade

The `IKahuna` interface is the contract between the server layer and the core engine. It declares every operation that Kahuna supports, organized into categories:

- **Key-value operations**: set, get, exists, delete, extend, batch variants, prefix queries, range scans.
- **Lock operations**: acquire, extend, release, query.
- **Sequence operations**: create, next value, reserve range, get, delete.
- **Transaction lifecycle**: start session, commit, rollback, close.
- **Transaction internals**: acquire exclusive locks, prepare mutations (2PC phase 1), commit mutations (2PC phase 2), rollback mutations, write-intent checks.
- **Script execution**: parse and execute transaction scripts.
- **Replication callbacks**: log restored, replication received, replication error, leader changed.
- **Range management**: register and unregister key ranges, state transfer for range splits.
- **Persistence**: flush pending writes, bootstrap from backup.

Every routed operation has two variants. The `LocateAnd*` variant (for example, `LocateAndTrySetKeyValue`) resolves which partition owns the key, checks whether this node is the leader for that partition, and either dispatches locally or forwards to the correct node. The direct `Try*` variant (for example, `TrySetKeyValue`) operates locally when the caller already knows that this node is the correct leader.

15.4 KahunaManager

`KahunaManager` is the class that implements `IKahuna`. It is the root of the component hierarchy. Its constructor calls `KahunaNodeComposer.Build()`, which assembles all internal components in a specific order:

1. Wrap the raw persistence backend in an `UnflushedOverlayPersistenceBackend`. This overlay caches writes that are committed in Raft but not yet flushed to disk, so reads do not miss recently committed values.
2. Create I/O schedulers (`FairReadScheduler`) for backend reads and writes.
3. Create internal stores: `SnapshotFloorStore`, `CompletionReceiptStore`, `TransactionRecordStore`, `PreparedIntentStore`.
4. Spawn the `BackgroundWriterActor`. This Nixie actor flushes committed Raft entries to the persistence backend asynchronously.
5. Create `LockManager`.
6. Create `KeyValuesManager`.
7. Wire the flush notification sink so the background writer can notify the key-value subsystem when writes are durable.
8. Create `SequencerManager`. It depends on `KeyValuesManager` because sequences store their durable state as key-value entries.
9. Register state transfer hooks with Raft for recovery and follower catch-up.

The result is a tree of components:

```
KahunaManager
├── KeyValuesManager
│   ├── KeyValueLocator (routes keys to partitions)
│   ├── KeyValueActor (per-partition, handles reads)
│   ├── PartitionWriteAggregatorActor (batches writes)
│   ├── DurableProposalSubmission (submits Raft proposals)
│   └── KeyValueReplicator (applies committed entries)
├── LockManager
│   ├── LockLocator (routes lock resources to partitions)
│   ├── LockActor (per-partition lock state)
│   ├── LockProposalActor (submits Raft proposals)
│   └── LockReplicator (applies committed entries)
├── SequencerManager
│   ├── SequenceLocator (routes sequence names to partitions)
│   └── SequenceActor (per-partition sequence state)
├── BackgroundWriterActor (async persistence)
├── PartitionPlacementCoordinator
└── IPersistenceBackend (RocksDB, SQLite, or Memory)
```

15.5 The Shared Pattern: Manager, Locator, Actor

All three subsystems follow the same layered architecture. Understanding this pattern once makes every subsystem easier to follow.

15.5.1 Manager

The manager is the entry point. `KeyValuesManager`, `LockManager`, and `SequencerManager` each own their locator, actors, and configuration. The `KahunaManager` delegates `IKahuna` method calls to the appropriate subsystem manager.

Manager classes are split into partial classes by responsibility. `KeyValuesManager`, for example, has separate files for admin operations, routed operations (the `LocateAnd*` variants), and local operations (the direct `Try*` variants).

15.5.2 Locator

The locator resolves a key (or lock resource, or sequence name) to the correct partition and then to the current leader for that partition. It uses the `DataPartitionRouter`, which maps key hashes to partition numbers and partition numbers to leader node addresses.

If this node is the leader for the target partition, the locator dispatches the request locally to the actor. If another node is the leader, the locator forwards the request over `IInterNodeCommunication`.

15.5.3 Actor

The actor holds the in-memory state for a partition. `KeyValueActor` holds the key-value entries for its partition. `LockActor` holds the lock states (owner, expiry, fencing token). `SequenceActor` holds the sequence states (high-water mark, block cache, idempotency entries).

Each actor implements `IActor<TRequest, TResponse>` from Nixie. It processes one message at a time. This single-threaded guarantee means the actor's state does not need locks or concurrent data structures. Within a partition, the actor is consistent by construction.

Multiple actors can exist per partition (determined by a consistent-hash ring), allowing parallelism within a partition for non-overlapping keys.

15.5.4 Proposal and Replication

When an actor needs to make a durable write, it submits a Raft proposal. The mechanism differs slightly by subsystem:

- **Locks** use a `LockProposalActor` that submits one proposal per lock mutation. This is straightforward because lock operations are individually small.
- **Key-values** use a `PartitionWriteAggregatorActor` that batches multiple writes into a single Raft proposal. This is important for throughput: one Raft round trip commits many key-value writes.
- **Sequences** store their durable state through the key-value subsystem. When a sequence needs to advance its high-water mark, it writes a key-value entry, which flows through the KV write pipeline.

After Raft commits a log entry, each subsystem has a replicator (`KeyValueReplicator`, `LockReplicator`) that applies the committed entry back to the actor's in-memory state. This is how followers stay in sync: they receive committed entries from the leader and apply them through the same replicator path.

15.6 Inter-Node Communication

When a request arrives at a node that is not the leader for the target partition, the locator forwards it to the correct node. Two implementations exist:

- **`GrpcInterNodeCommunication`**: the production implementation. Nodes communicate over gRPC. This is the same transport the client uses, but the inter-node variant carries internal metadata (partition ID, leader hint) that the client does not send.
- **`MemoryInterNodeCommunication`**: the test implementation. All nodes run in the same process. Messages are passed through in-memory queues. This allows integration tests to simulate a multi-node cluster without network overhead.

The interface (`IInterNodeCommunication`) abstracts the transport. The subsystem code does not know or care whether it is running in a production cluster or a test harness.

15.7 Persistence Backends

Kahuna supports three persistence backends, all implementing `IPersistenceBackend`:

Backend	Use Case
`RocksDbPersistenceBackend`	Production default. LSM-tree storage with high write throughput.
`SqlitePersistenceBackend`	Alternative for smaller deployments. Single-file database.
`MemoryPersistenceBackend`	Testing. All data in memory. Lost on restart.

An `UnflushedOverlayPersistenceBackend` wraps the chosen backend. This overlay solves a timing problem: a Raft entry is committed (replicated to a majority) before the `BackgroundWriterActor` flushes it to the persistence backend. Without the overlay, a read immediately after a committed write could miss the value because it is not yet on disk. The overlay caches committed-but-unflushed writes and serves them on reads.

The `BackgroundWriterActor` runs as a Nixie actor. It receives batches of committed entries and writes them to the persistence backend asynchronously. This decouples the Raft commit path (fast, in-memory) from the disk I/O path (slower, batched).

15.8 The Meta Partition

Partition 0 is the meta partition. It stores cluster-wide metadata that all nodes need:

- **Range map**: the mapping from key ranges to partition numbers. When the cluster splits or merges key ranges, the updated map is replicated through partition 0.
- **Snapshot floor**: the MVCC retention boundary. Reads older than this timestamp are not guaranteed to find data.
- **Transaction coordinator decisions**: the 2PC decision records for committed or aborted transactions.

The meta partition uses the same Raft group as any data partition: it has a leader, replicates to a majority, and supports state transfer for catch-up. Its special status comes from its content, not from a different mechanism.

15.9 Client Communication

The client communicates with the server through two protocols:

- **gRPC**: the primary protocol. Supports streaming, multiplexing, and efficient binary serialization via Protocol Buffers.
- **REST**: an alternative for clients that cannot use gRPC. The REST API covers the same operations but without streaming.

The `KahunaClient` distributes requests across server URLs in round-robin order. When constructed with multiple URLs, it rotates through them. If a node is unreachable, the client skips it on the next rotation.

The client also batches requests. The `GrpcBatcher` coalesces multiple operations headed for the same server into a single gRPC call. The `BatchCoalescingThreshold` and `BatchCoalescingDelayMs` options on `KahunaOptions` control when the batcher dispatches.

15.10 Summary

Kahuna's architecture is a layered system of managers, locators, and actors, built on Kommander for consensus and Nixie for concurrency.

Every request follows the same path: the client sends it to a server node. The server's `KahunaManager` delegates to the appropriate subsystem manager. The manager's locator resolves the key to a partition and a leader. If this node is the leader, the request is dispatched to the partition's actor. If not, it is forwarded to the correct node. Durable writes go through Raft proposals. Committed entries are applied by replicators and flushed to disk by the background writer.

The three subsystems (key-value, locks, sequences) share this pattern but differ in their write pipelines: key-values batch writes for throughput, locks submit individual proposals, and sequences piggyback on the key-value pipeline.

The next chapter traces a single request through every layer of this architecture, from the client's `SetKeyValue` call to the persistence backend and back.

16 The Request Lifecycle

Chapter 14 introduced Kahuna's architecture: the project layout, the component hierarchy, and the shared Manager, Locator, Actor pattern across subsystems. This chapter traces a single request through every layer of that architecture, from the client's `SetKeyValue` call to the persistence backend and back.

The goal is to build a complete picture of where each component sits in the request path. Every subsequent internals chapter zooms in on one part of this path. This chapter is the map that connects them all.

16.1 The Write Path: Setting a Key

A `SetKeyValue` call passes through seven stages before the value is durable: client batching, server dispatch, partition routing, actor processing, Raft proposal, replication, and persistence. Each stage is a distinct component with a distinct responsibility.

16.1.1 Stage 1: Client Batching

The application calls `KahunaClient.SetKeyValue`. The client selects a server URL through its routing mode. The default mode is `Auto`, which resolves to `Learned` when the client was given more than one endpoint and `RoundRobin` when it was given exactly one.

Four routing modes are available:

- **RoundRobin**: rotate over the configured endpoints. Every request lands on an arbitrary node. If that node is not the leader for the key's partition, it forwards the request internally. This was the only behavior before client routing existed.
- **Learned**: reuse the endpoint that a previous response reported for the same resource. The server attaches a routing hint to each response. The client stores the hint in a `RouteCache` and sends future requests for the same key directly to the hinted node. A cache miss falls back to round-robin.
- **Metadata**: learned routing plus a one-time resolution of routing metadata per key space. The client queries the cluster's routing metadata API, which returns partition assignments and the hash placement rule. A key the client has never seen before can go to the correct node without a cache-miss forward. The client falls back to learned behavior when the metadata is missing or stale.
- **Auto** (default): selects `Learned` for multi-endpoint clients and `RoundRobin` for single-endpoint clients.

The routing mode is set in `KahunaOptions.RoutingMode`. In all modes, correctness is the same: every node accepts every operation and forwards it to the owner if necessary. The routing mode changes efficiency, not behavior.

After the client selects a target endpoint, it creates a `GrpcTrySetKeyValueRequest` and passes it to the `GrpcBatcher` for that URL.

The `GrpcBatcher` does not send each request individually. It uses gRPC bidirectional streaming to multiplex many operations over a single long-lived HTTP/2 stream. When a request arrives, the batcher enqueues it in a `ConcurrentQueue` and creates a `TaskCompletionSource` that the caller awaits. A single dispatch loop drains the queue and sends all pending items over the shared stream.

Two settings control coalescing behavior:

- `BatchCoalescingThreshold` (default 10): when fewer than this many requests are ready, the batcher waits briefly before sending.
- `BatchCoalescingDelayMs` (default 2): the maximum wait in milliseconds, randomized to avoid synchronization.

This batching is transparent to the caller. The caller awaits a task. The batcher groups that request with other concurrent requests, sends them over the stream, and completes the task when the server responds.

If a transport-level error occurs (gRPC connection drops), the batcher invalidates the shared connection and retries once. The `GrpcCommunication` layer above the batcher adds its own retry logic: up to 5 retries on `MustRetry` responses and 2 retries on transport failures.

16.1.2 Stage 2: Server Dispatch

The request arrives at the server's gRPC service. `KeyValuesService` (which extends the generated `KeyValuer.KeyValuerBase`) receives the call in its `TrySetKeyValue` method. The method validates the input (non-empty key, non-negative expiry), extracts the key, value, flags, and durability from the protobuf request, and calls `keyValues.LocateAndTrySetKeyValue` on the `IKahuna` facade (implemented by `KahunaManager`).

`KahunaManager` delegates to `KeyValuesManager`, which delegates to `KeyValueLocator.LocateAndTrySetKeyValue`. The delegation chain is short: facade to subsystem manager to locator.

16.1.3 Stage 3: Partition Routing

`KeyValueLocator.LocateAndTrySetKeyValue` is the routing core. It determines which partition owns the key and whether this node is the leader for that partition.

The method starts with input validation. If the key is empty or the TTL is negative, it returns `InvalidInput` immediately. Then it resolves the partition:

```
(partitionId, generation, isKeyRange, descriptor) = LocateRangeWithMode(key)
```

`LocateRangeWithMode` uses `RangeRouting.Locate`, which hashes the key through the `KeySpaceRegistry` and the `DataPartitionRouter` to find the partition number. For key-range spaces (range-partitioned keys), it also checks whether the range is quiesced (in the middle of a split or merge). A quiesced range rejects writes with `MustRetry`.

After resolving the partition, the locator checks whether this node is the leader:

```
if (await raft.AmILeaderIfHosted(partitionId, cancellationToken))
{
    return await manager.TrySetKeyValue(...);
}
```

If this node is the leader, the request is dispatched locally. If not, the locator resolves the leader address:

```
string? leader = await TryWaitForLeader(partitionId, cancellationToken);
```

`TryWaitForLeader` calls `raft.TryResolveLeader`. If no leader is found (the election is still in progress, the partition is not hosted, or a `RaftException` occurs), the method returns null. The locator then returns `MustRetry` to the client.

If the leader is found and it is a remote node, the locator forwards the request over `IInterNodeCommunication`:

```
response = await interNodeCommunication.TrySetKeyValue(leader, ...);
```

The forwarded request carries the same parameters as the original, plus a `routedGeneration` that tracks the range map version at the coordinating node. The remote node uses this generation to detect stale routes.

Each forwarded request also carries a hop counter (`ForwardHops`). The counter starts at zero on the client's request. Each inter-node forward increments the counter by one. When the counter reaches `MaxForwardHops` (2), the receiver returns `MustRetry` instead of forwarding again. This hop budget prevents infinite ping-pong between nodes during range map convergence.

16.1.4 Stage 4: Actor Processing

When the request is dispatched locally, `KeyValuesManager.TrySetKeyValue` sends it to the `KeyValueActor` for the target partition through a consistent-hash router. The router selects one of several actors within the partition based on the key hash.

The actor processes one message at a time. It checks conditions: compare-and-swap revision, compare-and-swap value, flags (`SetIfNotExists`, `SetIfExists`), and MVCC write intents from concurrent transactions. If the conditions pass, the actor prepares the write.

For persistent (durable) writes, the actor creates a Raft proposal. It serializes the key, value, revision, timestamps, and flags into a `RaftProposalEntry` and submits it to the write pipeline.

16.1.5 Stage 5: Write Aggregation and Raft Proposal

The proposal enters the `PartitionWriteAggregatorActor`. This is where key-value writes differ from lock writes. Lock writes submit one Raft proposal per operation. Key-value writes are batched: the aggregator collects multiple proposals for the same partition and submits them as a single Raft log entry.

The aggregator is a Nixie actor with per-partition state. When a proposal arrives, the aggregator enqueues it in the partition's buffer. Two triggers cause the buffer to flush:

1. The buffer reaches `MaxBatchItems` or `MaxBatchBytes`.
2. The linger timer fires (a configurable delay that gives more proposals time to arrive).

When the buffer flushes, the aggregator selects a batch and dispatches it. Before dispatch, each item is checked for staleness: if the range map moved since the item was admitted, or if the item exceeded its maximum queue age, it is released with a retryable failure.

Valid items are flattened into a single `RaftProposalEntry[]` array. The aggregator calls `executor.ReplicateAsync(partitionId, entries, cancellationToken)`, which submits the batch to Raft as one proposal.

Raft replicates the log entry to a majority of nodes. Each node appends the entry to its write-ahead log. When a majority acknowledges the append, the entry is committed. The `RaftBatchReplicationResult` reports whether each entry committed or failed.

The aggregator maps the result back to individual submissions. Each `KeyValueProposalRequest` carries a reference to the originating actor and its reply promise. When Raft commits, the aggregator calls `Complete` on each request, which sends a `CompleteProposal` message back to the actor. The actor clears the replication intent, applies the committed state to the entry, and resolves the response promise. On failure, the actor returns `MustRetry` or `Aborted`.

By default, only one batch per partition is in flight at any time. While a batch is in Raft, new proposals accumulate in the buffer behind it. When the in-flight batch completes, the aggregator immediately dispatches the buffered items as the next batch.

The `--kv-write-max-in-flight-batches` flag (default 1) allows multiple batches per partition to await Raft concurrently. With a value of 2, the aggregator dispatches a second batch while the first is still in Raft. FIFO ordering is preserved: batches commit in the order they were submitted. Pipelined batching increases throughput on partitions with sustained write load by overlapping Raft replication rounds.

16.1.6 Stage 6: Replication and State Application

After Raft commits the log entry, the `KeyValueReplicator` applies it to the node's state. The replicator runs on every node: the leader and all followers.

The replicator deserializes the committed log entry with `ReplicationSerializer.UnserializeKeyValueMessage`. Based on the operation type (`TrySet`, `TryDelete`, `TryExtend`), it performs three actions in order:

1. **Record in the unflushed overlay.** `UnflushedKeyValueWritesIndex.Record` caches the committed write so that reads can see it before the background writer flushes it to disk.
2. **Enqueue for persistence.** The replicator sends a `QueueStoreKeyValue` message to the `BackgroundWriterActor`. This enqueues the write for asynchronous disk flush.
3. **Update the in-memory actor state.** The replicator sends an `InvalidateOrApply` message to the `KeyValueActor` through the persistent router. The actor either applies the new value (on followers, where the actor did not process the original request) or invalidates its cached entry (on the leader, where the actor already has the value from processing the request).

The replicator also records a completion receipt. This receipt lets the server answer re-commits for the same transaction with `Committed` instead of `MustRetry`, even after the write intent and MVCC snapshot are gone.

16.1.7 Stage 7: Persistence

The `BackgroundWriterActor` is a Nixie actor that runs a periodic flush timer. The default interval is configurable (5 seconds by default). On each tick, it drains the dirty queue and writes batches to the `IPersistenceBackend` (RocksDB, SQLite, or Memory).

Writes are batched up to 1,024 items or 512 KB per batch. The actor processes lock flushes and key-value flushes separately. After each flush, it advances durability floors, runs checkpoint operations (so Raft can compact its write-ahead log), and cleans up old revisions.

If a write fails, the actor retries up to 5 times with backoff. Items that fail all retries stay in the queue for the next flush cycle.

The separation between Raft commit and disk persistence is a key design choice. A Raft commit means the value is replicated to a majority of nodes in memory and in the write-ahead log. The background flush is an optimization: it moves data from the WAL to the structured storage backend. If a node crashes before the flush, it recovers from the WAL on restart.

16.1.8 The Response Path

After the write commits in Raft, the response travels back through the layers:

1. The write aggregator sends a `CompleteProposal` message to the `KeyValueActor`. The actor clears the replication intent, applies the committed state, and resolves the `TaskCompletionSource` that the gRPC handler is awaiting.
2. The gRPC service constructs the response protobuf with the response type, revision, last-modified timestamp, and elapsed time.
3. The server sends the response over the gRPC stream.
4. The `GrpcBatcher` on the client side receives the response and resolves the matching `TaskCompletionSource`.
5. `KahunaClient.SetKeyValue` wraps the result in a `KahunaKeyValue` object and returns it to the application.

If the request was forwarded to a remote node, the inter-node communication layer receives the response from the remote leader and passes it back to the locator on the coordinating node.

16.2 The Read Path: Getting a Key

The read path is shorter than the write path. Reads do not create Raft proposals and do not go through the write aggregator. The critical difference is how the read path ensures consistency.

16.2.1 Client to Locator

The client calls `KahunaClient.GetKeyValue`. As with writes, the request goes through the `GrpcBatcher` and arrives at the server's gRPC service. The service calls `keyValues.LocateAndTryGetValue`, which delegates through `KahunaManager` and `KeyValuesManager` to `KeyValueLocator.LocateAndTryGetValue`.

The locator resolves the partition with `RouteKey(key)` (the same hash-based routing as writes).

16.2.2 Leadership Confirmation for Reads

Here the read path diverges from the write path. For writes, checking `AmILeaderIfHosted` is sufficient: if the leader belief is wrong, the Raft proposal will fail. Writes are self-validating because replication itself fails on a deposed leader.

Reads are different. A node that believes it is the leader but is actually partitioned from the majority will serve stale data as a successful response. There is no replication step to catch the stale belief.

To prevent stale reads, the locator calls `ConfirmLeadershipForRead`:

```
if (await ConfirmLeadershipForRead(partitionId, cancellationToken))
    return await manager.TryGetValue(...);
```

`ConfirmLeadershipForRead` delegates to `raft.ConfirmLeadershipIfHosted`, which performs a Raft read-index check. This check confirms that the node is still the leader by contacting a quorum. If the quorum confirms, the read proceeds. If not, the locator returns `MustRetry`.

This is the mechanism that provides linearizable reads. Every read confirms leadership before serving data, so a read always reflects all writes that committed before it started.

16.2.3 Local Read

If leadership is confirmed, the locator calls `manager.TryGetValue`, which routes the request to the `KeyValueActor` through the consistent-hash router. The actor looks up the key in its in-memory state and returns the value, revision, and timestamps.

If the key is not in the actor's memory (it was evicted or never loaded), the actor reads from the persistence backend. The `UnflushedOverlayPersistenceBackend` wraps the raw backend and intercepts reads. If the key is in the unflushed overlay (committed in Raft but not yet flushed to disk), the overlay returns the cached value. If not, the read falls through to the underlying RocksDB, SQLite, or Memory backend.

16.2.4 Remote Forward

If this node is not the leader, the locator calls `TryWaitForLeader` to find the leader's address. If the leader is found, the request is forwarded:

```
response = await interNodeCommunication.TryGetValue(leader, ...);
```

The remote leader runs the same leadership confirmation and local read. The response travels back through the inter-node communication layer to the coordinating node, and from there back to the client.

16.3 Read Path vs Write Path

The two paths share the same routing and dispatch layers. They diverge at two points:

Aspect	Write Path	Read Path
Leadership check	`AmILeaderIfHosted` (belief-based)	`ConfirmLeadershipForRead` (quorum-confirmed)
Raft involvement	Submits a proposal, waits for majority commit	No proposal (read-only, confirmed via read-index)
Write aggregator	Batches proposals per partition	Not involved
Replicator	Applies committed entries on all nodes	Not involved
Persistence	Background flush after commit	Reads from overlay or backend

Writes are self-validating: a proposal on a deposed leader fails at the replication step, so a belief-based check is sufficient. Reads need the quorum confirmation because there is no replication step to catch a stale leader.

16.4 Synchronous vs Asynchronous Boundaries

The request lifecycle has two asynchronous boundaries where the request "detaches" from the caller and continues independently:

1. **Raft replication.** The write aggregator submits a Raft proposal and awaits the result. The caller's thread is free while Raft replicates the entry to followers. The `TaskCompletionSource` on the `DurableProposalSubmission` completes when Raft commits.
2. **Background persistence.** After Raft commits, the `KeyValueReplicator` enqueues the write for disk flush but does not wait for it. The response returns to the client before the value is on disk. The unflushed overlay ensures reads see the committed value in the interim.

Everything else is synchronous from the caller's perspective: the client awaits the response, the server processes the request inline, and the locator routes it within the same async call chain.

16.5 Where Latency Comes From

Each stage adds latency. In a healthy cluster, the dominant cost is Raft replication (network round trips to followers).

Stage	Typical Contribution
Client batching	0 to 2 ms (coalescing delay)
Network to server	Network dependent
Server dispatch and routing	Microseconds
Actor processing	Microseconds
Write aggregation linger	0 to linger setting (default: configurable)
Raft replication	1 network round trip to majority
Response to client	Network dependent

For reads, the Raft replication step is replaced by the read-index confirmation, which also requires a quorum round trip. Reads that hit the in-memory actor cache avoid disk I/O. Reads that miss the cache pay the additional cost of a backend lookup.

Inter-node forwarding adds one extra network hop. A request that arrives at the wrong node travels: client to coordinating node, coordinating node to leader, leader back to coordinating node, coordinating node back to client. This is why the client distributes requests across all nodes: most requests hit the correct leader on the first hop.

16.6 Failure Scenarios

16.6.1 Leader Not Found

The locator calls `TryWaitForLeader` and receives null. The node may not host this partition, or an election may be in progress. The locator returns `MustRetry`. The client retries on another node.

No data was read or written. The retry is safe.

16.6.2 Leader Changes Mid-Request

The request arrives at the leader. A new election starts. The Raft proposal fails because the node is no longer the leader. The write aggregator releases the proposal with a transient failure. The actor returns `MustRetry`.

On the write path, the proposal did not commit. No data was written. On the read path, the leadership confirmation fails, and the locator returns `MustRetry` before any data is read.

16.6.3 Background Writer Lag

Under heavy write load, the `BackgroundWriterActor` may fall behind. Committed values accumulate in the unflushed overlay. Reads still succeed because the overlay serves committed-but-unflushed values. The write-ahead log grows until the background writer catches up.

If the dirty queue is not empty after a flush cycle, the background writer sends itself another flush message immediately, without waiting for the next periodic timer tick. This self-scheduling drains the backlog as fast as the backend allows.

16.7 Summary

A key-value write passes through seven stages: client batching, server dispatch, partition routing, actor processing, write aggregation, Raft replication, and background persistence. The client batches requests over a shared gRPC stream. The server routes each request to the correct partition leader. The write aggregator batches multiple proposals into one Raft entry for throughput. Raft replicates the entry to a majority. The replicator applies the committed entry to all nodes. The background writer flushes data to disk asynchronously.

A key-value read follows the same routing path but diverges at the leadership check. Reads use a quorum-confirmed read-index check instead of a belief-based leader check. This prevents a partitioned leader from serving stale data. Reads do not create Raft proposals and do not go through the write aggregator.

The two asynchronous boundaries are Raft replication (the caller awaits the commit) and background persistence (the response returns before the disk flush). The unflushed overlay bridges the gap between Raft commit and disk flush.

The next chapter examines how Kahuna partitions data across nodes: how keys map to partitions, how partitions map to nodes, and how the system rebalances when nodes join or leave.

17 Partitioning and Data Distribution

Chapter 15 traced a single request through every layer of the architecture. One step in that path was partition routing: the locator resolved a key to a partition number and found the leader for that partition. This chapter explains how that routing works, how partitions are created and split, and how the system rebalances data as it grows.

Partitioning determines three properties of the system: scalability (how much data and throughput a cluster can handle), data locality (which keys are stored together), and failure isolation (how much data is affected when a node goes down). A single partition is a bottleneck. Too many partitions waste resources. The right partitioning strategy depends on the workload.

17.1 Two Routing Modes

Kahuna supports two ways to map keys to partitions: hash-based routing and key-range routing. Each mode has different trade-offs for data distribution and query patterns.

17.1.1 Hash-Based Routing

Hash-based routing is the default. Every key-space prefix that is not explicitly registered for key-range routing uses hash-based routing.

The routing works as follows. The `DataPartitionRouter` extracts the key-space prefix from the key (the portion before the last `/` separator) and computes an ordinal hash of that prefix. The hash maps to one of the partitions in a fixed pool.

The pool consists of partitions numbered from 1 to `InitialPartitions`. Partition 0 is the meta partition (described later in this chapter). It is excluded from the data pool. The `PoolSize` equals the `InitialPartitions` value from the Raft configuration, which is set at cluster startup with the `--initial-cluster-partitions` flag (default 3).

```
DataPartitionRouter.Locate("orders/123/status")
  → prefix = "orders/123"
  → hash = HashUtils.InversePrefixedHash("orders/123/status", '/', PoolSize)
  → partitionId = hash + FirstUserPartitionId (1)
  → result: partition 2
```

17.1.1.1 Placement Groups

The hash does not run over the full key-space prefix. It runs over the prefix's **placement group**: the portion before the first `|` character. If the key space has no `|`, the entire key space is its own placement group.

For example, the key `t:r|i:pk/0001` has key space `t:r|i:pk` and placement group `t:r`. The key `t:r/0001` has key space `t:r` and placement group `t:r` (no `|`, so the full key space is the group). Both keys hash to the same partition because they share the placement group `t:r`.

This rule lets an application co-locate a table's rows and its index entries on the same partition. Name the row key space `t:r/rowKey` and the index key space `t:r|i:pk/indexKey`. Both key spaces share the placement group `t:r` and hash together. The key spaces remain separate for prefix scans and range descriptors.

The placement function is defined in `HashPlacement`. Both the server's routers and the client's local resolver call the same function so the two cannot drift. The function identifier (`kahuna.placement-group-jump-xxh32-v1`) is published in routing metadata. A client that implements a different function refuses to hash locally rather than approximate.

Hash-based routing has two properties:

1. **Uniform distribution.** Keys spread evenly across partitions. No partition is a hot spot unless the application writes heavily to a single placement group.
2. **No key locality.** Keys with related prefixes (such as `orders/100` and `orders/101`) may land on different partitions. Range scans across prefixes require querying multiple partitions. Keys within the same placement group co-locate, but placement groups are distributed independently.

The partition pool is fixed. It does not grow or shrink. Partitions created by key-range splits receive higher IDs and are not part of the hash pool. Hash-based routing is static: the same key always maps to the same partition.

Because the pool is fixed, hash-based routing does not need a generation fence. The `routedGeneration` for hash-routed keys is always 0.

17.1.2 Key-Range Routing

Key-range routing assigns contiguous intervals of keys to partitions. Keys that are close in sort order are stored on the same partition. This enables efficient range scans and prefix queries within a single partition.

Key-range routing is opt-in. A key space becomes key-range routed only by explicit registration through the `KeySpaceRegistry`. The registry maintains a per-node map from key-space prefixes to a `RoutingMode` value: either `Hash` (the default) or `KeyRange`.

When a key arrives, the `KeySpaceRegistry` extracts the key space (the portion before the last `/`). It looks up the routing mode for that key space. If the mode is `KeyRange`, the request is routed through the `RangeMap`. If the mode is `Hash`, the request goes through the `DataPartitionRouter` as described above.

The registry uses a `ConcurrentDictionary` with zero-allocation span lookups for performance. It reconciles its state against the replicated range map on every range map update.

17.1.3 RangeRouting: The Single Source of Truth

The `RangeRouting` static class is the single entry point for all key-to-partition resolution. Both the `KeyValueLocator` and the leader-side direct-write path call through `RangeRouting`. This ensures that every routing decision uses the same logic.

`RangeRouting.Locate` takes the registry, range map, data partition router, and key. It returns a `(partitionId, generation)` tuple. For hash-routed keys, the generation is 0. For key-range keys, the generation comes from the `RangeDescriptor` that covers the key.

`RangeRouting.LocateWithMode` returns additional information: whether the key is key-range routed and, if so, the covering `RangeDescriptor`. The locator uses this to check for quiesced ranges and to carry the generation through inter-node forwards.

17.2 The Range Map

The range map is an immutable in-memory data structure that maps key-range spaces to their partition assignments. Each entry is a `RangeDescriptor`.

17.2.1 RangeDescriptor

A `RangeDescriptor` is a sealed record with these fields:

Field	Description
`KeySpace`	The key-space prefix (for example, `"t:r"`).
`StartKey`	Inclusive lower bound of the range. Null means negative infinity.
`EndKey`	Exclusive upper bound of the range. Null means positive infinity.
`PartitionId`	The Raft group that serves this range.
`Generation`	Incremented on every split, merge, or move. Used for generation fencing.
`QuiescedUntil`	Timestamp deadline until which writes are refused (data is in transit).
`QuiesceOwner`	The move operation that opened the quiesce window.
`QuiesceStartKey`	Start of the sub-interval being quiesced.
`QuiesceEndKey`	End of the sub-interval being quiesced.

Bounds use ordinal string comparison. The interval is half-open: `[StartKey, EndKey)`. A descriptor with `StartKey = "a"` and `EndKey = "m"` contains keys from `"a"` up to but not including `"m"`.

The `Contains(key)` method checks whether a key falls within the half-open interval. The `IsQuiescedAt(key, now)` method checks whether a key falls within the quiesced sub-interval and the quiesce deadline has not lapsed.

17.2.2 RangeMap Lookups

The `RangeMap` groups descriptors by key space and sorts them by `StartKey` within each group. Lookups use binary search:

- `Find(keySpace, key)`: O(log n) search for the descriptor that covers the key.
- `FindCovering(key)`: extracts the key space from the key, then calls `Find`.
- `FindAll(keySpace)`: returns all descriptors for a key space in sorted order.
- `FindIntersecting(keySpace, startKey, endKey)`: O(log n + k) for range queries, where k is the number of matching descriptors.

The range map enforces an invariant: within each key space, descriptors must be contiguous with no gaps and no overlaps. The `Validate` method checks this invariant and is called after every mutation.

17.2.3 RangeMapStore

The `RangeMapStore` is the replicated source of truth for the range map. It wraps an immutable `RangeMap` and replicates changes through Raft on the meta partition (partition 0).

All mutations go through a single method: `MutateAsync(transform)`. This method serializes access with a semaphore, validates the result, replicates the new descriptor set through Raft, and swaps the in-memory map. Only the meta partition leader can mutate the range map.

The store uses snapshot semantics: each replicated entry carries the full descriptor set, not a delta. Replay is idempotent. On startup, the store loads a durable snapshot from disk. Periodic checkpointing (every 32 mutations by default) writes a fresh snapshot and lets Kommander trim the write-ahead log.

17.3 The Meta Partition

Partition 0 is the meta partition. Chapter 14 introduced it briefly. This chapter adds the partitioning-specific details.

The meta partition stores the authoritative range map. Every range split, merge, or move replicates through partition 0's Raft group. All nodes receive the updated range map through normal Raft replication.

The meta partition uses the same Raft protocol as data partitions. It has a leader, replicates to a majority, and supports state transfer for follower catch-up. Its special status comes from its content (the range map, transaction coordinator decisions, the MVCC retention boundary), not from a different mechanism.

Schema-log spaces (key spaces with a `/meta` suffix) are never registered as key-range routed. The `KeySpaceRegistry` enforces this rule.

17.4 Range Splits

When a key-range partition grows too large or too hot, the system splits it into two smaller ranges. A split takes a range `[S, E)` on partition P and produces two ranges: `[S, K)` on the original partition P and `[K, E)` on a new partition P'.

17.4.1 The Split Lifecycle

The `RangeSplitter` executes the split as a multi-step transaction. The steps proceed in order:

1. **Locate the covering range.** Find the `RangeDescriptor` for the range that contains the split key K.
2. **Validate bounds.** Confirm that `S < K < E` in ordinal order. Both halves must be non-empty.
3. **Check minimum size.** Probe both halves to ensure each has at least `--range-split-min-range-size` keys (default 10). This prevents splits that produce ranges too small to be useful.
4. **Create the new partition.** Call `CreatePartitionAsync` with `RaftRoutingMode.Unrouted`. The new partition P' exists in Raft but does not receive routed traffic yet.

5. **Bulk copy.** Copy all keys in `[K, E)` from P to P' at a consistent MVCC snapshot timestamp. The `KvStateMachineTransfer` handles this copy in pages of 256 entries, each checksummed.

6. **Quiesce.** Acquire an exclusive range lock on `[K, E)` within partition P. Publish a quiesce window on the descriptor with a 30-second deadline (`QuiesceTtlMs = 30_000`). While the range is quiesced, writes to keys in `[K, E)` are rejected with `MustRetry`. Settle any durable intents for keys in the moving range.

7. **Final catch-up copy.** Capture writes that occurred between the MVCC snapshot and the quiesce point. Transfer range locks for the moving sub-interval. Gather and hand off transaction state: completion receipts, transaction records, and prepared intents.

8. **Atomic cutover.** Call `RangeMapStore.MutateAsync` to replace the original descriptor with two new descriptors: `[S, K)` on P with generation+1 and `[K, E)` on P' with generation+1. This is a single Raft proposal on the meta partition. Both descriptors receive a bumped generation.

9. **Release quiesce.** Release the exclusive range lock on the original partition. The quiesce window on the descriptor expires by its deadline.

After cutover, the original partition P still contains the `[K, E)` rows. These orphan rows are unreachable through routing and are not deleted immediately. They consume storage but do not affect correctness.

The split must run on the meta partition (partition 0) leader. This ensures that range map mutations are serialized.

17.4.2 Quiesce: Two Complementary Guards

The quiesce mechanism uses two guards that work together:

1. **Exclusive range lock.** This is an actor-local lock that blocks writes at the actor processing level. It is ordered against concurrent writes: a write that arrives before the lock is acquired proceeds; a write that arrives after is rejected. This guard handles the local partition.

2. **Descriptor quiesce.** This is a replicated flag on the `RangeDescriptor`. It survives leadership changes and is visible to all nodes. It has a deadline (not a flag that must be cleared), so it expires automatically if the split fails. This guard handles forwarded requests from other nodes.

Both guards are necessary. The actor-local lock prevents races on the partition leader. The descriptor quiesce prevents a redirected request from bypassing the lock on a different node.

17.4.3 Automatic Split Triggers

The `RangeSplitTrigger` monitors key-range partitions and triggers splits automatically. It has two branches:

Count branch (slow cadence, approximately 60 seconds). The trigger samples the key count for each descriptor. When the count reaches the `--range-split-threshold` (default 1000 keys), a split is triggered.

Load branch (fast cadence, approximately 5 seconds). The trigger evaluates a predicate: operations per second must exceed `--range-split-load-threshold` AND the write-ahead log

queue depth must exceed `--range-split-load-min-queue-depth` (default 8). The predicate must hold for the full `--range-split-load-window` (default 15 seconds) before a split fires. This debounce prevents splits from transient load spikes.

Both branches enforce a settle window. After a split, the two child ranges are excluded from re-evaluation for `--range-split-settle-window` seconds (default 10). This prevents cascading splits.

An indivisibility guard prevents splits when the write-frequency histogram shows extreme imbalance (all writes target a single key). A relief guard skips load-based splits when no peer node is alive, because a single-node cluster gains no redistribution benefit from splitting.

All splits are serialized through a semaphore. Only one split runs at a time.

17.4.4 Split Key Selection

The `RangeSplitPolicy` computes the split key from an ordered sample of keys in the range:

- **Normal distribution.** The split key is the median (50th percentile). This produces two halves of equal size.
- **Monotonic-append detection.** If 75% or more of recently written keys cluster in the top 25% of the ordinal range (a common pattern for time-series or auto-incrementing keys), the split key moves to the 75th percentile. This gives the hot tail its own range.
- **Write-centroid path.** When the write-frequency histogram is warm, the policy finds the split index that best bisects cumulative writes. This balances write load rather than key count.

The policy clamps the split index so both halves have at least `--range-split-min-range-size` keys.

17.5 Range Merges

When two adjacent key-range partitions are both under a minimum size, the system merges them. A merge takes `[A, B)` on partition P1 and `[B, C)` on partition P2 and produces `[A, C)` on P1. Partition P2 is retired.

17.5.1 The Merge Lifecycle

The `RangeMerger` follows a similar pattern to the splitter:

1. **Validate adjacency.** Confirm that the two ranges are in the same key space and share a boundary (`EndKey` of the first equals `StartKey` of the second).
2. **Quiesce the source range.** Acquire an exclusive range lock on `[B, C)` and publish a quiesce window on P2's descriptor.
3. **Settle intents.** Wait for durable intents in the source range to complete.
4. **Copy data.** Transfer all keys from `[B, C)` on P2 to P1.
5. **Atomic cutover.** Replace both descriptors with a single `[A, C)` descriptor on P1 with generation+1.
6. **Release and retire.** Release the quiesce lock. The caller retires partition P2.

A merge copies in one direction only (source to destination), because merge candidates are small ranges.

17.5.2 Automatic Merge Triggers

The `RangeMergeTrigger` scans for adjacent pairs where both ranges have fewer than `--range-merge-min-size` keys (default 10). It runs on the meta partition leader. It skips warm partitions (where operations per second exceed the load threshold) to prevent merge-split oscillation: a range that was recently split due to load should not be merged back immediately.

17.6 The Generation Fence

The generation fence prevents stale routing after a split or merge. When a range splits, the generation on both child descriptors is incremented. Any request that was routed with the old generation is rejected.

17.6.1 How It Works

When a request is forwarded from one node to another, the coordinating node includes a `routedGeneration` value. This is the generation of the descriptor that the coordinating node used to route the request.

On the receiving node, `RangeRouting.ResolveForDirectWrite` checks the routed generation against the live descriptor. The check returns one of four results:

Result	Meaning
`Ok`	The generation matches. The write is admitted.
`NoDescriptor`	The range moved away from this partition. The descriptor is gone.
`GenerationFenced`	The routed generation does not match the live generation. The routing is stale.
`Quiesced`	The range is in the middle of a split or merge. Writes are temporarily refused.

When a request is fenced (`GenerationFenced`, `NoDescriptor`, or `Quiesced`), the server returns `MustRetry`. The client retries, and the routing layer discovers the updated range map and sends the request to the correct partition.

The fence is checked at two points:

1. When the locator routes the request (before dispatching locally or forwarding).
2. At the leader-side direct-write path (just before proposing to Raft).

The second check catches a race: the range map may update between the routing decision and the proposal submission.

Hash-routed keys do not use the generation fence. Their routing is static (the pool never changes), so the generation is always 0.

17.6.2 Deferred Write Staleness

The write aggregator batches multiple proposals for efficiency (Chapter 15). A proposal may sit in the aggregator's buffer while a split completes. Before flushing the buffer, the aggregator calls `RangeRouting.HasKeyRangeMovedSinceAdmission` to check whether any buffered key's range moved since the key was admitted. Stale items are released with a retryable failure.

17.7 State Transfer During Splits

The `KvStateMachineTransfer` handles the bulk copy of data during splits and merges.

17.7.1 Export

The export reads keys in the range `[startKey, endKey)` at a consistent MVCC snapshot. It pages through the data in batches of 256 entries. Each page is checksummed with FNV hash for integrity verification.

Only persistent keys are transferred. Ephemeral (in-memory-only) data is not transferable.

17.7.2 Import

The import applies entries to the destination partition's persistence backend. The import is idempotent: applying the same page twice produces the same result.

17.7.3 Lock Transfer

Range locks are transferred separately. The `FilterAndClamp` method filters the source partition's lock state for locks that fall within the moving sub-interval. These locks are re-imported on the destination partition.

If leadership changes during the transfer, `EnsureLocksOnDestinationLeaderAsync` re-imports the locks on the new leader.

17.8 Prefix Operations and Splits

Prefix operations (`GetByBucket` and `ScanAllByPrefix`) scan all keys that share a prefix. When a key-range space splits, the prefix scan on a single partition may miss data that moved to the new partition.

`RangeRouting.IsPrefixOpSafe` checks whether a key-range space was split in a way that would make a prefix scan incomplete. If the check returns false, the operation must query multiple partitions or return an error.

17.9 Server Configuration

These command-line flags control partitioning behavior:

Flag	Default	Description
`--initial-cluster-partitions`	3	Number of partitions at cluster startup.
`--range-split-threshold`	1000	Key count that triggers an automatic split.
`--range-split-min-range-size`	10	Minimum keys per half after a split.
`--range-split-settle-window`	10s	Cooldown after a split before re-evaluation.
`--range-merge-min-size`	10	Key count below which adjacent ranges may merge.
`--range-collection-interval`	60s	Interval between count-based sampling passes.
`--range-split-load-threshold`	0 (disabled)	Operations per second for load-based split.
`--range-split-load-min-queue-depth`	8	WAL queue depth gate for load-based split.
`--range-split-load-window`	15s	Sustained load window before a split fires.
`--range-split-load-poll-interval`	5s	Poll frequency for load signals.

Load-based splitting is disabled by default (`--range-split-load-threshold` is 0). Enable it by setting a non-zero threshold.

17.10 Failure Scenarios

17.10.1 Stale Routing After a Split

1. Node A routes a write to partition P using generation 5.
2. A split completes. The range map updates. The new generation is 6.
3. The write arrives at partition P's leader. The generation fence check finds routedGeneration (5) does not match the live generation (6).
4. The leader returns `MustRetry`.
5. The client retries. The routing layer reads the updated range map and sends the request to the correct partition (P or P') with generation 6.

The generation fence prevents the write from landing on a partition that no longer owns the key.

17.10.2 Split During an Active Transaction

1. A transaction reads key K on partition P.

2. A split moves the range containing K to partition P'.
3. The transaction tries to write key K. The routing layer or the generation fence rejects the write with `MustRetry`.
4. If wrapped in `RetryableTransaction`, the transaction starts over with a fresh session.

The transaction is not corrupted. The split causes a transient failure, and the retry mechanism handles it.

17.10.3 State Transfer Failure Mid-Split

1. The bulk copy (step 5) or catch-up copy (step 7) fails due to a network error or node crash.
2. The split is aborted. The original range remains intact on partition P.
3. The `RangeSplitTrigger` cleans up the orphaned partition P' that was created in step 4.
4. On the next trigger cycle, the trigger re-evaluates and may start a new split attempt.

Because the atomic cutover (step 8) did not execute, the range map is unchanged. No data is lost.

17.10.4 Indivisible Range

1. The split trigger fires on a range where all writes target a single key.
2. The `RangeSplitPolicy` detects that the write-centroid imbalance is at the maximum (1.0).
3. The trigger skips the split. The range stays as is.

A range with a single hot key cannot be split further. The application must distribute writes across multiple keys to avoid this bottleneck.

17.11 Summary

Kahuna distributes keys across partitions using two routing modes. Hash-based routing is the default: a fixed pool of partitions, uniform distribution, no key locality. Key-range routing is opt-in: contiguous key intervals on the same partition, efficient range scans, but requires range management.

The `RangeMap` stores the mapping from key intervals to partitions. The `RangeMapStore` replicates it through the meta partition (partition 0). Every mutation is a single Raft proposal.

Range splits follow a multi-step protocol: bulk copy, quiesce, catch-up copy, atomic cutover. The quiesce mechanism uses two guards (an actor-local lock and a replicated descriptor deadline) to prevent writes during the transition. The generation fence rejects requests that use a stale routing generation.

Automatic triggers split ranges when they grow too large (count-based) or too hot (load-based). Automatic merges recombine small adjacent ranges. Both triggers use settle windows and guards to prevent oscillation.

The next chapter examines Raft consensus and replication: how Kahuna uses Kommander for leader election, log replication, and state transfer.

18 Consensus and Replication

Every write in Kahuna goes through Raft consensus. Every linearizable read confirms leadership through a quorum check. Raft is the foundation that makes Kahuna fault-tolerant: it ensures that all replicas agree on the same ordered sequence of operations, even when nodes crash or networks partition.

This chapter explains how Kahuna uses Raft. It covers the consensus problem, the Raft algorithm, per-partition Raft groups, log entry types, linearizable reads, Hybrid Logical Clocks, and the replication pipeline that applies committed entries to each subsystem.

18.1 The Consensus Problem

A distributed system that stores data on multiple nodes must answer a question: when two nodes disagree about the state of a key, which one is correct?

Without a consensus protocol, the answer is undefined. A network partition can leave two nodes with different values for the same key, and neither node knows which value is authoritative. This is the split-brain problem.

Raft solves split-brain by electing a single leader for each group of replicas. All writes go through the leader. The leader replicates each write to a majority of nodes before the write is committed. A write that is committed on a majority cannot be lost, even if a minority of nodes crash.

The guarantee is precise: Raft ensures that all nodes apply the same sequence of log entries in the same order. If entry N is committed, every node that is alive and reachable will eventually have entry N at position N in its log. This is the foundation of consistency in Kahuna.

18.2 Kommander: The Raft Library

Kahuna does not implement Raft itself. It uses Kommander, an external Raft library. Kommander provides the `IRaft` interface, which exposes:

- **Leader election.** Heartbeat-based election with randomized timeouts.
- **Log replication.** Append-only log with majority-based commit.
- **Write-ahead log (WAL).** Durable storage for log entries before replication. Three implementations: RocksDB (production), SQLite (alternative), and in-memory (tests).
- **Hybrid Logical Clocks (HLC).** Timestamps that combine physical time with a logical counter for causal ordering across nodes.
- **State transfer.** Snapshot-based catch-up for followers that fall behind log compaction.
- **Membership management.** SWIM protocol for failure detection, learner promotion, and graceful decommission.

Kahuna registers callbacks with Kommander to receive events: committed log entries, leadership changes, and restore notifications. Kommander handles the protocol. Kahuna handles the state.

18.3 Per-Partition Raft Groups

Chapter 16 described how Kahuna distributes keys across partitions. Each partition has its own independent Raft group. This means each partition has its own leader, its own log, and its own election cycle.

Per-partition groups provide two benefits:

1. **Independent leadership.** Different partitions can have different leaders. If node A leads partitions 1 and 2, and node B leads partition 3, a failure of node A triggers elections only for partitions 1 and 2. Partition 3 is unaffected.
2. **Independent throughput.** Each Raft group processes proposals independently. A slow proposal on partition 1 does not block proposals on partition 2.

Kommander creates a partition with `CreatePartitionAsync(partitionId, mode, hashRange, cancellationToken)`. The `mode` parameter specifies the routing mode (routed or unrouted). Unrouted partitions are created during range splits before the cutover (Chapter 16).

The default cluster starts with a configurable number of partitions (`--initial-cluster-partitions`, default 3). Partition 0 is the meta partition. Partitions 1 through N are data partitions. Range splits create additional partitions with higher IDs.

18.3.1 Shared Executor Pool

Each Raft group needs a thread to process proposals and heartbeats. With many partitions, a dedicated thread per partition wastes resources. Kommander provides a shared executor pool (`--raft-enable-shared-executor-pool`, default true) that multiplexes all partition executors onto a bounded thread pool. This is necessary for clusters with hundreds or thousands of partitions.

The pool size is controlled by `--raft-executor-pool-size` (default 0, which auto-sizes to the processor count). Raise this value above the core count only if operations queue behind busy partitions while the CPU is not saturated. As a diagnostic escape hatch, set the environment variable `KAHUNA_SHARED_POOL=0` to force the original one-thread-per-partition model even when the CLI flag is enabled.

18.3.2 Partition Quiescence

Idle partitions that receive no proposals and no client reads are quiesced (`--raft-enable-quiescence`, default true). A quiesced partition stops sending per-partition heartbeats and relies on the SWIM protocol for failure detection. When a new proposal or read arrives, the partition wakes up.

Quiescence reduces network traffic in clusters with many partitions where only a fraction are active at any time. The `QuiesceAfter` timeout (default 1500 ms) controls how long a partition must be idle before it quiesces.

18.4 Leader Election

Raft uses heartbeat-based leader election. The leader sends periodic heartbeat messages to all followers. If a follower does not receive a heartbeat within its election timeout, it becomes a candidate and starts an election.

18.4.1 Election Timing

Kommander uses randomized election timeouts to prevent split votes:

Parameter	Default	Description
`HeartbeatInterval`	500 ms	How often the leader sends heartbeats to followers.
`StartElectionTimeout`	2000 ms	Lower bound of the randomized election timeout.
`EndElectionTimeout`	4000 ms	Upper bound of the randomized election timeout.
`VotingTimeout`	1500 ms	How long a candidate waits for a quorum of votes.
`CheckLeaderInterval`	250 ms	How often the timer fires to check leader state.
`TimerInitialDelay`	2500 ms	Grace period after startup before timers fire.

Each follower picks a random timeout between `StartElectionTimeout` and `EndElectionTimeout`. The randomization ensures that followers do not all start elections at the same time after a leader failure. The follower with the shortest timeout becomes a candidate first and usually wins the election.

If an election fails (no candidate receives a majority), each candidate adds a random increment between `StartElectionTimeoutIncrement` (100 ms) and `EndElectionTimeoutIncrement` (200 ms) to its timeout and tries again. This further spreads out retries.

The critical constraint: `HeartbeatInterval` must be less than `StartElectionTimeout`. If heartbeats are slower than the election timeout, followers start unnecessary elections.

18.4.2 Pre-Vote Protocol

Kommander uses a pre-vote protocol. Before a follower starts a real election (which increments the term and forces other nodes to step down), it sends a pre-vote request to check whether it would win.

A pre-vote does not increment the term. If the pre-vote fails (the majority does not respond or already has a leader), the follower does not disrupt the cluster. This prevents a problem where a partitioned node repeatedly increments its term and forces the healthy majority to step down when the partition heals.

18.4.3 The Election Sequence

1. A follower's election timeout fires. No heartbeat arrived from the leader.
2. The follower sends pre-vote requests to all other voters.
3. If a majority responds positively, the follower becomes a candidate and increments its term.
4. The candidate sends `RequestVote` messages to all voters.
5. Each voter grants its vote to the first candidate it receives in the new term (at most one vote per term).
6. If the candidate receives votes from a majority, it becomes the leader.
7. The new leader sends a heartbeat immediately to establish its authority.

8. The new leader commits a barrier entry (a no-op) to confirm that its log is up to date. The `LeadershipBarrierTimeout` (default 10 seconds) is the maximum wait for this barrier to commit.

18.4.4 Check-Quorum

The pre-vote protocol prevents a partitioned follower from disrupting the cluster. Check-quorum handles the complementary case: a partitioned leader.

When check-quorum is enabled (`--raft-enable-check-quorum`), a leader tracks same-term heartbeat acknowledgements from voters. If the leader has not received an acknowledgement from a majority within `CheckQuorumIntervalMultiplier x HeartbeatInterval` (default 8 x 500 ms = 4 seconds), it steps down to follower. Tune the multiplier with `--raft-check-quorum-interval-multiplier`.

Without check-quorum, a partitioned leader continues to believe it is the leader. It rejects writes (they cannot commit without a quorum), but it may serve stale reads from its local state. Check-quorum forces the partitioned leader to step down so it stops serving any traffic.

Check-quorum is disabled by default. Enable it in production clusters where a partitioned leader serving stale reads is not acceptable.

18.4.5 Stale-Leader Fencing

Check-quorum detects a partitioned leader at the Raft level. Stale-leader fencing adds a second layer at the application level. When a node loses leadership, the `LockActor` and `KeyValueActor` clear their staged state: pending write intents, held locks, and replication intents that belong to the old term. This prevents an isolated leader from staging writes that cannot commit.

The fencing mechanism also applies term checks to Raft proposals. A proposal carries the leader term in which it was created. If the proposal reaches Raft after a term change, it is rejected. This closes the window between a leadership loss and the actor's cleanup.

18.4.6 Apply Fingerprints

Each partition maintains an apply fingerprint: a rolling digest of the log entries it applied. Nodes exchange fingerprints over inter-node gRPC. If two replicas of the same partition diverge in their fingerprint at the same log index, the cluster surfaces the divergence as a diagnostic metric. A range split also checks the source partition's fingerprint before it proceeds. If the source leader's apply state is incomplete, the split is refused until the leader catches up.

18.4.7 Leadership Change Notification

When a partition's leader changes, Kommander calls the `OnLeaderChanged` callback. Kahuna receives this through the `ReplicationService` and notifies all subsystem managers. This lets each manager update its local routing state and reject requests that target partitions where this node is no longer the leader.

18.5 Log Entry Types

Every durable mutation in Kahuna is serialized into a Raft log entry. The `ReplicationSerializer` encodes and decodes these entries. Each entry carries a type string that identifies its content.

Type String	Content	Partition
`"kv"`	Key-value mutations: set, delete, extend.	Data partition
`"lock"`	Lock mutations: lock, unlock, extend.	Data partition
`"rangemap"`	Range-descriptor map snapshot.	Meta partition (0)
`"snapshotfloor"`	Snapshot-floor hold registry (MVCC retention boundary).	Meta partition (0)
`"coorddecision"`	Durable coordinator decision record delta (2PC commit/abort).	Data partition
`"receipt"`	Completion receipts (used during split/merge handoff).	Data partition
`"txnrecord"`	Transaction record transitions (init, commit, abort).	Data partition
`"preparedintent"`	Prepared-intent transitions (prepare, resolve, remove).	Data partition

Key-value and lock mutations are the most common types. They represent the application-visible writes. The remaining types are internal coordination state for transactions, range management, and MVCC.

18.5.1 Proposing Entries

Kommander exposes three methods for submitting log entries:

- `ReplicateLogs(partitionId, type, data)`: submit a single entry.
- `ReplicateLogs(partitionId, type, IReadOnlyList<byte[]>)`: submit a homogeneous batch (all entries share the same type string).
- `ReplicateEntries(partitionId, IReadOnlyList<RaftProposalEntry>)`: submit a heterogeneous batch where each entry can have a different type.

Chapter 15 described how the `PartitionWriteAggregatorActor` batches key-value writes. It uses `ReplicateEntries` to submit multiple KV mutations as a single Raft proposal. Lock mutations use individual `ReplicateLogs` calls.

18.5.2 Two-Phase Commit in the Log

Some proposals use explicit two-phase commit within Raft. The `ReplicateLogs` method accepts an `autoCommit` parameter. When `autoCommit` is false, the entry is proposed but not committed until the caller explicitly calls `CommitLogs(partitionId, ticketId)` or `RollbackLogs(partitionId, ticketId)`. Kahuna uses this for transaction coordination.

18.6 The Replication Pipeline

When Raft commits a log entry, every node in the group must apply it to its local state. Kahuna uses a replicator/restorer pattern for this.

18.6.1 Replicators: Normal Operation

During normal operation, Kommander calls the `OnReplicationReceived` callback for each committed log entry. The `ReplicationService` forwards this to `KahunaManager`, which dispatches based on the entry type.

The `KeyValueReplicator` handles `"kv"` entries. For each committed key-value mutation (`TrySet`, `TryDelete`, `TryExtend`), it performs four steps in order:

1. **Register pending.** The durability tracker records that this log entry is pending, which prevents the durability floor from advancing past it until the background writer flushes it to disk.
2. **Record in the unflushed overlay.** The unflushed write index caches the committed value so reads can see it before the background writer flushes it.
3. **Enqueue for persistence.** The replicator sends a `QueueStoreKeyValue` message to the `BackgroundWriterActor` for asynchronous disk flush.
4. **Invalidate or apply on the actor.** The replicator sends an `InvalidateOrApply` message to the `KeyValueActor` through the consistent-hash router. On the leader, the actor invalidates its cached entry (it already has the value from processing the original request). On followers, the actor applies the new value.

After these steps, the replicator records a completion receipt. It also records the write in the write-frequency registry for key-range partitions, which the split trigger uses.

The `LockReplicator` follows the same pattern for `"lock"` entries: register pending, record in the unflushed lock overlay, enqueue for persistence, and send `InvalidateOrApply` to the `LockActor`.

18.6.2 Restorers: Startup and Recovery

When a node starts up or a new leader is elected, committed log entries must be replayed to rebuild in-memory state. Kommander calls the `OnLogRestored` callback for each entry in the write-ahead log.

The `KeyValueRestorer` handles `"kv"` entries during restore. It performs a simpler sequence than the replicator:

1. Register pending in the durability tracker.
2. Record in the unflushed overlay.
3. Enqueue for persistence.
4. Record the completion receipt.

The restorer does not send `InvalidateOrApply` to actors. During restore, actors are not yet active. They are rebuilt after the restore completes.

The `LockRestorer` follows the same simplified pattern for lock entries.

18.6.3 The Wiring Layer

The `ReplicationService` is an ASP.NET `BackgroundService` that wires Raft events to `IKahuna`:

```
raft.OnLogRestored      → kahuna.OnLogRestored
raft.OnReplicationReceived → kahuna.OnReplicationReceived
raft.OnReplicationError  → kahuna.OnReplicationError
raft.OnLeaderChanged    → kahuna.OnLeaderChanged
raft.OnMembershipChanged → OnMembershipChanged
```

On startup, the service joins the cluster (either as a new member or by joining an existing cluster with `--join-existing`). On shutdown, it unwires events and optionally performs a graceful leave (`--graceful-leave-on-shutdown`).

18.7 Linearizable Reads

Chapter 15 described the read path: the locator calls `ConfirmLeadershipForRead` before serving a read. This section explains the protocol in detail.

18.7.1 The Problem with Local Reads

A node that believes it is the leader can serve reads from its local state. But what if the node is no longer the leader? A network partition can isolate a node from the majority. The isolated node still believes it is the leader (no heartbeat timeout fired yet), but a new leader was elected on the majority side. The isolated node's state is stale.

If the isolated node serves reads from its local state, those reads return stale data as a successful response. There is no replication step (as with writes) to catch the stale belief. The client receives outdated values and does not know they are outdated.

18.7.2 The Read-Index Protocol

`ConfirmLeadershipAsync` implements the read-index protocol from section 6.4 of the Raft dissertation. The protocol works in three steps:

1. **Record the current commit index.** The leader notes the highest committed log index at the time the read starts.
2. **Confirm leadership with a quorum.** The leader sends a lightweight message to all followers. A majority must respond, confirming that they still recognize this node as their leader in the current term. If the majority does not respond within `LeadershipConfirmationTimeout` (default 2 seconds), the confirmation fails.
3. **Wait for local application.** The leader waits until its local state machine has applied all entries up to the recorded commit index. This ensures the read reflects all committed writes.

If all three steps succeed, the read is linearizable: it reflects every write that committed before the read started, and no write that committed after.

18.7.3 Coalescing

Multiple concurrent reads can share a single confirmation round. When several `ConfirmLeadershipAsync` calls overlap, they coalesce into one quorum check. A confirmation that completed within the last heartbeat interval is reused for subsequent reads. This means

that under steady load, the cost is approximately one quorum round-trip per heartbeat interval, regardless of read volume.

18.7.4 AmILeader vs ConfirmLeadershipAsync

Two leadership checks serve different purposes:

Method	Mechanism	Use Case
`AmILeaderQuick`	Local published state. No network call.	Write path: sufficient because a write proposal on a deposed leader fails at the Raft replication step.
`ConfirmLeadershipAsync`	Quorum confirmation. Network round-trip.	Read path: necessary because reads have no replication step to catch a stale leader.

Writes are self-validating. A write proposal submitted to a deposed leader is rejected by Raft because the deposed leader cannot replicate to a majority. The proposal fails, and the client retries.

Reads are not self-validating. A deposed leader can serve stale data from its local state without any rejection mechanism. The quorum confirmation is the mechanism that prevents this.

18.8 Hybrid Logical Clocks

Kahuna uses Hybrid Logical Clocks (HLC) for cross-node event ordering. An HLC timestamp combines three components:

Component	Type	Description
`L`	long	Physical timestamp in unix epoch milliseconds.
`C`	uint	Logical counter. Differentiates events with the same physical time.
`N`	int	Node ID. Breaks ties between events on different nodes.

18.8.1 Total Ordering

HLC timestamps form a total order. Comparison follows a priority chain: `L` first, then `C`, then `N`. Two events on different nodes with the same physical timestamp are ordered by their counter. Two events with the same timestamp and counter are ordered by node ID. No two events produce the same `(L, C, N)` tuple.

18.8.2 How HLC Advances

The clock advances differently for local events and received messages:

Local event (send or local). The clock computes `newL = max(currentL, physicalTime)`. If `newL` equals `currentL`, the counter increments. If `newL` is greater, the counter resets to zero. The

clock swaps the new value using a lock-free compare-and-swap (CAS) loop. Zero allocation per event.

Receive event. The clock computes `newL = max(currentL, messageL, physicalTime)`. If all three are equal, the counter is `max(currentC, messageC) + 1`. This is the standard HLC receive algorithm from Kulkarni et al.

18.8.3 Encoding

The HLC packs `L` and `C` into a single 64-bit long: the high 42 bits store `L` (unix milliseconds, valid to approximately year 2109), and the low 22 bits store `C` (approximately 4.19 million events per millisecond). If the counter overflows 22 bits, the clock rolls to `(L+1, 0)` to preserve causality.

18.8.4 Why HLC

Physical clocks alone are insufficient for ordering events across nodes, because wall clocks drift. Logical clocks (Lamport clocks) provide causal ordering but have no relationship to physical time. HLC combines both: it tracks causality like a logical clock and stays close to physical time like a wall clock. This makes HLC timestamps useful both for ordering (MVCC, conflict detection) and for human-readable timestamps (last-modified fields, expiration deadlines).

18.9 Voter and Learner Roles

Each node in a Raft cluster has a role that determines its participation in consensus:

Role	Votes?	Receives Replication?	Description
`Voter`	Yes	Yes	Full participant. Counts toward quorum in elections and commits.
`Learner`	No	Yes	Receives committed entries but does not vote. Cannot become leader.
`Leaving`	No	Yes	Committed during graceful decommission. Replicas are evacuated to other nodes.

18.9.1 Learner Promotion

A new node joins the cluster as a learner. It receives committed log entries and builds up its state without affecting the cluster's quorum size. When the learner's log catches up to within `LearnerPromotionLag` entries (default 10) of the leader and stays within that threshold for `LearnerPromotionStableWindow` (default 3 seconds), Kommander promotes it to voter automatically.

This two-phase join prevents a slow new node from reducing the cluster's availability. If a learner joined as a voter immediately, its slow replication would block commits (the quorum would include a node that is far behind).

18.9.2 Graceful Decommission

When a node is decommissioned with `--graceful-leave-on-shutdown`, it transitions to the `Leaving` role. The cluster evacuates its partition replicas onto surviving nodes. After evacuation completes, the node is removed from the cluster roster.

18.10 Replication Factor

The replication factor controls how many copies of each partition exist in the cluster. The default is 0, which means full replication: every voter hosts every partition. This is appropriate for small clusters (3 to 5 nodes).

For larger clusters, a fixed replication factor (for example, 3) limits each partition to a subset of nodes. This reduces storage and network overhead. The replication factor can be set per partition with `SetReplicationFactorAsync`.

18.11 State Transfer

When a follower falls far behind the leader's log (because it was offline or the WAL was compacted), it cannot catch up by replaying log entries. The entries it needs are gone.

Kommander handles this with state transfer. The leader sends a snapshot of the partition's state to the follower. Kahuna registers three state transfer handlers:

- `IRaftStateMachineTransfer`: transfers key-value data during range splits and merges (Chapter 16).
- `IRaftSystemStateTransfer`: transfers system state (meta partition) for node repair.
- `IRaftPartitionStateTransfer`: transfers user data partition state for follower catch-up.

After the snapshot is applied, the follower resumes normal log replication from the leader's current position.

18.12 WAL Compaction

The write-ahead log grows as entries accumulate. Periodically, Kommander compacts the WAL by discarding entries that all nodes have applied. The `CompactEveryOperations` setting (default 10,000) controls how often compaction runs.

Before compacting, the system checks retention holds. The `SetMinRetainIndex` and `AcquireRetentionHold` methods let subsystems prevent compaction of entries they still need (for example, entries needed for state transfer).

After compaction, entries before the compaction point are gone. A follower that needs those entries must use state transfer instead of log replay.

18.13 Failure Scenarios

18.13.1 Leader Failure

1. The leader crashes or becomes unreachable.

2. Followers stop receiving heartbeats. Each follower's election timeout is random (between 2000 and 4000 ms).
3. The follower with the shortest remaining timeout becomes a candidate first. It sends pre-vote requests.
4. If the pre-vote succeeds, the candidate increments its term, sends `RequestVote` messages, and wins the election.
5. The new leader sends a heartbeat and commits a barrier entry.
6. Normal operation resumes. The new leader serves reads and writes for the partition.

Committed entries are not lost. The new leader has all committed entries (Raft's election rule ensures the candidate with the most up-to-date log wins).

18.13.2 Network Partition

1. The cluster splits into a majority partition (for example, 2 of 3 nodes) and a minority partition (1 node).
2. The majority elects a new leader and continues to serve reads and writes. Commits succeed because the majority forms a quorum.
3. The minority cannot commit writes (no quorum). The node on the minority side may still believe it is the leader for some partitions, but its proposals fail.
4. Reads on the minority side fail if `ConfirmLeadershipAsync` is used (the quorum check fails). Without the quorum check, the minority node would serve stale data.
5. When the partition heals, the minority node receives the committed entries it missed and catches up.

18.13.3 Split Vote

1. Two followers start elections at the same time (their timeouts expire simultaneously).
2. Each receives some votes but neither receives a majority.
3. The election fails. Both candidates wait a random increment (100 to 200 ms) before trying again.
4. The randomized increment makes it unlikely that both candidates time out simultaneously again.
5. One candidate wins the next round.

The pre-vote protocol reduces the impact of split votes. A candidate that loses the pre-vote does not increment its term, so the cluster's term does not advance unnecessarily.

18.13.4 Log Divergence After Partition Heal

1. A network partition separates the old leader from the majority.
2. The majority elects a new leader and commits new entries.
3. The old leader may have uncommitted entries in its log (entries it proposed but could not replicate to a majority).
4. When the partition heals, the old leader discovers the new term and steps down.
5. Raft reconciles the logs: the old leader's uncommitted entries are overwritten by the new leader's committed entries. The log converges to the committed sequence.

Uncommitted entries are lost. This is correct: they were never acknowledged to the client (the proposal did not succeed), so no data loss occurs from the client's perspective.

18.13.5 Stale Reads from a Deposed Leader

1. Node A is the leader. A network partition isolates it from the majority.
2. Node B becomes the new leader on the majority side. New writes commit on node B.
3. Node A still believes it is the leader (its election timeout has not yet fired).
4. A client sends a read to node A.
5. Node A calls `ConfirmLeadershipAsync`. The quorum check fails because node A cannot reach a majority.
6. Node A returns `MustRetry`. The client retries on another node and reaches the current leader.

Without `ConfirmLeadershipAsync`, step 5 would not exist. Node A would serve stale data from its local state, and the client would receive outdated values without any indication.

18.14 Summary

Kahuna uses Raft consensus through the Kommander library. Each partition has its own Raft group with independent leadership, log, and election cycle. The shared executor pool and partition quiescence make this practical for clusters with many partitions.

Leader election uses randomized timeouts and a pre-vote protocol to prevent disruption. Committed entries are applied through the replicator/restorer pattern: replicators handle normal operation; restorers handle startup and recovery.

Linearizable reads use the read-index protocol: the leader confirms its authority with a quorum before serving a read. Concurrent confirmations coalesce for efficiency. This prevents a deposed leader from serving stale data.

Hybrid Logical Clocks provide total ordering across nodes by combining physical time with a logical counter. HLC timestamps are used for MVCC, conflict detection, and human-readable timestamps.

The replication pipeline transforms committed log entries into state changes across all subsystems. Eight log entry types cover key-value mutations, lock mutations, range map changes, transaction coordination, and MVCC retention.

The next chapter examines the persistence layer: how Kahuna stores data on disk, how the background writer flushes committed entries, and how the unflushed overlay bridges the gap between Raft commit and disk write.

19 The Actor Model and Concurrency

Traditional concurrent data structures use locks. A thread acquires a lock, reads or writes shared state, and releases the lock. This works, but it is error-prone: deadlocks, priority inversions, and forgotten unlock calls are common sources of bugs in concurrent systems.

Kahuna takes a different approach. It uses the actor model for concurrency. Each partition's state is owned by an actor that processes messages one at a time in a single-threaded loop. There are no locks on the in-memory state within an actor. Concurrency comes from having many actors, not from shared-memory parallelism within one.

This chapter explains how Kahuna uses the Nixie actor system, how messages are routed to actors, and how the handler pattern structures actor logic.

19.1 The Actor Model

The actor model has three rules:

1. **Isolated state.** Each actor owns its state. No other actor or thread can read or write that state directly.
2. **Message passing.** Actors communicate by sending messages. A message is a request object that the actor processes.
3. **Sequential processing.** Each actor processes one message at a time. The next message is not processed until the current one completes.

These three rules eliminate data races by construction. If only one thread ever touches the state, there is no race. If actors communicate only through messages, there is no shared mutable state.

The trade-off is that actors cannot share data structures. If two actors need the same data, one must send a message to the other. This adds latency compared to a direct memory read, but it removes the need for synchronization.

19.2 Nixie: The Actor Framework

Kahuna uses Nixie, an external actor framework. Nixie provides three building blocks: the actor interface, the actor reference, and the actor system.

19.2.1 The Actor Interface

Nixie defines two variants of the actor interface:

Fire-and-forget. The sender does not wait for a response.

```
public interface IActor<in TRequest> where TRequest : class
{
    public Task Receive(TRequest message);
}
```

Request/response. The sender awaits a response.

```
public interface IActor<in TRequest, TResponse>
    where TRequest : class
    where TResponse : class?
{
```

```
    public Task<TResponse?> Receive(TRequest message);
}
```

Both variants have a single method: `Receive`. The actor system calls this method for each message in the actor's inbox. The actor processes the message and returns a result (or completes the task for fire-and-forget actors).

19.2.2 The Actor Reference

An `IActorRef` is a handle for sending messages to an actor. It provides:

- `Send(message)`: fire-and-forget delivery. The message is enqueued and the caller continues.
- `Ask(message)`: request/response delivery. The caller awaits the result.
- `TrySend(message)`: returns false if the inbox is full (bounded inbox mode).
- `TryAsk(message)`: returns false and a null task if the inbox is full.

The caller never accesses the actor's internal state. All interaction goes through the reference.

19.2.3 The Actor System

The `ActorSystem` is the container that manages actor lifecycles. It provides:

- `Spawn<TActor, TRequest, TResponse>(name, args)`: create an actor with constructor arguments.
- `SpawnWithOptions<>()`: create an actor with inbox bounds and control message classifiers.
- `Get<>(name)`: find an actor by name.
- `StartPeriodicTimer`: send a message to an actor on a repeating interval.
- `ScheduleOnce`: send a message to an actor after a delay.
- `Shutdown<>` / `GracefulShutdownAll`: stop actors and drain their inboxes.

19.2.4 The Mailbox and Processing Loop

Each actor has an inbox implemented as a `ConcurrentQueue`. When a message arrives, the actor runner enqueues it and checks whether the processing loop is idle. If idle, it schedules the actor on the thread pool.

The processing loop drains messages one at a time:

1. Dequeue the next message from the inbox.
2. Call `await Actor.Receive(message)`.
3. If more messages are in the queue, continue to step 1.
4. If the queue is empty, mark the actor as idle and return the thread.

A compare-and-swap flag (`processing`) ensures that only one thread runs the loop at any time. This is the single-threaded guarantee: even though the actor runs on pool threads, it never runs on two threads simultaneously.

The actor runner uses `ThreadPool.UnsafeQueueUserWorkItem` instead of `Task.Run` to avoid `ExecutionContext` capture overhead. This is a performance optimization for high-throughput actors.

19.2.5 Bounded Inbox

An actor can be created with a maximum inbox size. When the inbox is full, `Send` and `Ask` throw an `ActorBusyException`. The message is never enqueued. The caller can catch this exception and return `MustRetry` to the client.

Some messages are classified as control messages. Control messages bypass the inbox bound and are delivered ahead of ordinary messages. Within each class (control vs ordinary), messages are delivered in FIFO order. Control messages overtake the ordinary backlog but maintain order among themselves.

In Kahuna, the following message types are classified as control messages for `KeyValueActor`:

- `CompleteProposal` and `ReleaseProposal` (Raft proposal lifecycle)
- `InvalidateOrApply` (follower replication)
- `ResumeRead` (deferred disk reads)
- `FlushAck` (durability floor advancement)
- `Collect` (periodic garbage collection)
- `GetSafeTimestamp` (MVCC timestamp queries)

These messages must not be blocked by a full inbox. A `CompleteProposal` message that is rejected would leave a Raft proposal permanently pending.

19.3 Consistent-Hash Routing

Each partition can have multiple actor instances. Messages are routed to a specific instance based on the key's hash. This allows parallelism within a partition for non-overlapping keys.

Nixie provides a `ConsistentHashActorStruct` router. The router creates N instances of the actor type. When a message arrives, the router computes the target instance:

```
instance = instances[(message.GetHash() & int.MaxValue) % instances.Count]
```

The message must implement `IConsistentHashable`, which exposes a `GetHash()` method. The hash of the key determines which actor instance processes the message.

This is a simple modulo routing, not a virtual-node ring. The same key always routes to the same actor instance. Different keys may route to different instances, which allows concurrent processing of messages for non-overlapping keys within the same partition.

The number of actor instances per partition is set by the `KeyValueWorkers` configuration. More workers means more parallelism within a partition, but also more memory (each actor holds its own B-tree cache).

19.4 Kahuna's Actor Hierarchy

Kahuna uses five types of actors, each with a distinct responsibility:

Actor	Interface	Role
`KeyValueActor`	`IActor<KeyValueRequest, KeyValueResponse>`	Holds key-value entries for a partition. Processes reads, writes, and transaction operations.
`LockActor`	`IActor<LockRequest, LockResponse>`	Holds lock state for a partition. Processes lock, unlock, and extend operations.
`SequenceActor`	`IActor<SequenceRequest, SequenceResponse>`	Holds sequence block allocation state. Serves next-value requests from a reserved block.
`PartitionWriteAggregatorActor`	`IActor<PartitionWriteMessage, PartitionWriteAck>`	Batches key-value write proposals for Raft replication.
`BackgroundWriterActor`	`IActor<BackgroundWriteRequest>`	Flushes committed Raft entries to the persistence backend. Fire-and-forget.

The first three actors hold partition state. The last two are infrastructure actors that handle the write pipeline.

19.5 KeyValueActor

The `KeyValueActor` is the most complex actor in Kahuna. It holds the in-memory state for key-value entries in a partition and dispatches operations to handler classes.

19.5.1 In-Memory State

The actor holds four data structures:

B-tree cache. A `BTree<string, KeyValueEntry>` with order 32. The B-tree stores key-value entries in sorted order by key. The sorted structure enables efficient prefix scans (`ScanByPrefix`) and range scans (`GetByRange`). The B-tree is a custom implementation in `Kahuna.Core/Utils/BTree.cs`.

Prefix write intents. A `Dictionary<string, KeyValueWriteIntent>` keyed by prefix. When a transaction operates on a bucket (prefix scan), it registers a write intent on the prefix. Other transactions that touch the same prefix detect the intent and handle conflicts.

Range locks. A `Dictionary<string, List<KeyValueRangeLock>>` keyed by prefix. Range locks protect sub-intervals within a key space during splits and transactions. Multiple non-overlapping range locks can coexist on the same prefix. Each range lock measures its lease

in elapsed (monotonic) time, not HLC timestamps. A wall-clock jump does not prematurely expire all leases. When a range lock lapses, a `LapsedRangeLockRegistry` records the holder so that the conflict probe can answer `Unlocked` for that transaction.

In-flight proposals. A `Dictionary<int, KeyValueProposal>` keyed by proposal ID. Each entry tracks a Raft proposal that is awaiting commit. When the replicator sends a `CompleteProposal` message, the actor resolves the proposal's `TaskCompletionSource`.

19.5.2 The Handler Pattern

The `KeyValueActor` constructor creates approximately 25 handler instances. All handlers share a `KeyValueContext` object that provides access to the actor's state, the persistence backend, and the Raft interface.

The `Receive` method dispatches messages to handlers based on the message type:

```
public async Task<KeyValueResponse> Receive(KeyValueRequest message)
{
    switch (message.Type)
    {
        case KeyValueRequestType.TrySet:
            return await trySetHandler.Execute(message);

        case KeyValueRequestType.TryGet:
            return await tryGetHandler.Execute(message);

        case KeyValueRequestType.TryDelete:
            return await tryDeleteHandler.Execute(message);

        // ... ~25 more cases
    }
}
```

Each handler is a class with an `Execute` method that takes a `KeyValueRequest` and returns a `KeyValueResponse`. The handler classes live in `Kahuna.Core/KeyValues/Handlers/`. There are 41 handler files covering all key-value operations:

- **CRUD handlers.** `TrySetHandler`, `TryGetHandler`, `TryDeleteHandler`, `TryExtendHandler`, `TryExistsHandler`.
- **Batch handlers.** `TryGetByBucketHandler`, `TryGetByRangeHandler`, `TryScanByPrefixHandler`.
- **Transaction handlers.** `TryPrepareMutationsHandler`, `TryCommitMutationsHandler`, `TryRollbackMutationsHandler`.
- **Lock handlers.** `TryAcquireExclusiveLockHandler`, `TryAcquireExclusivePrefixLockHandler`, `TryAcquireExclusiveRangeLockHandler`.
- **Lifecycle handlers.** `CompleteProposalHandler`, `InvalidateOrApplyHandler`, `ResumeReadHandler`, `EvictPartitionHandler`, `TryCollectHandler`.

The handler pattern keeps the `Receive` method short. Each handler encapsulates one operation's logic: input validation, state lookup, conflict detection, proposal creation, and response construction.

19.5.3 Backpressure

The actor implements two layers of backpressure:

1. **Inbox bound.** If the actor's inbox is full, the consistent-hash router catches the `ActorBusyException` and returns `MustRetry`. The message is never enqueued.
2. **Read backpressure.** If the persistence backend's read scheduler is saturated, the handler throws a `ReadBackpressureExceededException`. The actor catches it and returns `MustRetry`.

Both layers protect the system from overload. The client retries after a short delay.

19.5.4 Periodic Collection

Every 500 operations, the actor checks whether its memory budget is exceeded (`IsOverBudget()`). If so, it sends itself a `Collect` message. The collection handler evicts expired entries from the B-tree cache. Because `Collect` is a control message, it bypasses the inbox bound and is processed promptly.

19.6 Ephemeral vs Persistent Actors

The `KeyValueActorRouters` class creates two separate consistent-hash rings of `KeyValueActor` instances:

Persistent ring. For keys with `KeyValueDurability.Persistent`. These actors use the shared prepared-intent store and transaction-record store. Writes go through Raft replication and are flushed to disk.

Ephemeral ring. For keys with `KeyValueDurability.Ephemeral`. These actors get empty (no-op) prepared-intent and transaction-record stores. Writes are in-memory only: no Raft replication, no disk persistence. Data is lost on restart.

The locator selects the ring based on the durability of the operation. This separation prevents ephemeral operations from contaminating the transaction state of persistent operations.

Each ring has `KeyValueWorkers` actor instances. Both rings use bounded inboxes with the same control message classifier.

19.7 LockActor

The `LockActor` holds the lock state for a partition. Its state is simpler than the `KeyValueActor`:

- `Dictionary<string, LockEntry> locks`: lock entries keyed by resource name. Each entry records the owner, expiry, and fencing token.
- `Dictionary<int, LockProposal> proposals`: in-flight Raft proposals.
- `HashSet<string> keysToEvict`: eviction set for expired locks.

The `Receive` method dispatches to internal methods for `TryLock`, `TryUnlock`, `TryExtendLock`, `Get`, `CompleteProposal`, `ReleaseProposal`, `InvalidateOrApply`, and `EvictPartition`.

Lock proposals use a `LockProposalActor` through a balancing router (round-robin), not a consistent-hash router. Locks submit one Raft proposal per operation (Chapter 15), so there is no need for key-based routing of proposals.

The actor runs periodic collection every 500 operations to evict expired locks.

19.8 SequenceActor

The `SequenceActor` holds block allocation state for sequences:

- `Dictionary<string, SequenceBlock> blocks`: one block per sequence name. Each block is a reserved window of IDs (a low-water mark and a high-water mark).

When a `NextSequenceValue` request arrives, the actor checks the block for the named sequence. If the block has remaining IDs, the actor returns the next value from memory. No Raft proposal is needed.

If the block is exhausted, the actor allocates a new block by writing the updated high-water mark to the key-value subsystem (sequences store their durable state as key-value entries). This write goes through the full KV pipeline, including Raft replication.

A `useStamp` counter tracks LRU access for eviction of resident blocks when memory is constrained.

If a `CancellationToken` fires during block allocation (the KV write times out or is cancelled), the actor drops the block. It cannot know whether the write committed. On the next request, a fresh block allocation reads the durable high-water mark and starts from the correct position.

19.9 PartitionWriteAggregatorActor

The `PartitionWriteAggregatorActor` batches key-value write proposals for efficiency (Chapter 15 described the batching protocol in detail). It is a Nixie actor (`IActor<PartitionWriteMessage, PartitionWriteAck>`) with per-partition state in a dictionary.

The actor never awaits Raft on the mailbox thread. When a batch is ready, it dispatches the Raft replication as a detached operation. The Raft result arrives later as a `BatchComplete` control message, which the actor processes to resolve the individual proposals.

Before dispatch, each item is re-checked against the range map. If a key's range moved since the item was admitted (a split completed while the item was queued), the item is released with a retryable failure. This prevents stale writes from reaching Raft. A maximum of 1,024 releases per dispatch prevents starvation of the processing loop.

19.10 BackgroundWriterActor

The `BackgroundWriterActor` is a fire-and-forget actor (`IActor<BackgroundWriteRequest>`) that flushes committed Raft entries to the persistence backend. It receives `QueueStoreKeyValue` and `QueueStoreLock` messages from the replicators.

The actor batches writes: up to 1,024 items or 512 KB per flush. It retries failed writes up to 5 times with backoff. A revision cleanup queue (capped at 10,000 entries) removes old revisions after the MVCC retention window expires.

19.11 The Single-Threaded Invariant

The actor model provides safety through a contract: no shared mutable state across actors, and no concurrent access to an actor's state.

This contract holds as long as the actor does not leak references to its internal state across `await` points. Consider this violation:

```
// INCORRECT: reference to internal state escapes the actor
var entry = keyValuesStore.Get(key);

// This await yields the thread. Another message could modify
// keyValuesStore before the continuation runs.
await SomeAsyncOperation();

// entry may now be stale or invalid.
entry.Value = newValue;
```

Between the `await` and the continuation, the actor processes no other messages (Nixie's single-threaded guarantee holds). But if the reference escapes to a callback or a detached task, two threads could touch the state simultaneously.

Kahuna's handlers follow a discipline: read state, compute the result, create a proposal if needed, and return. Long-running operations (disk reads, Raft proposals) are handled by sending a message to the actor when the operation completes, not by holding a reference across the await.

19.12 Failure Scenarios

19.12.1 Actor State After Leader Change

Write intents, prefix locks, and range locks live in actor memory. They are not persisted and not replicated through Raft. When a leader change occurs:

1. The old leader's actors hold stale state (write intents from transactions that were in progress).
2. The new leader's actors start with empty state (no write intents, no prefix locks, no range locks).
3. In-flight transactions on the old leader receive `MustRetry` or `Aborted`.
4. The new leader accepts new transactions. The old write intents on the old leader expire naturally.

This is safe because write intents are optimistic: they record what a transaction intends to do. If the leader changes before the transaction commits, the transaction is aborted and retried on the new leader. The committed state (in Raft) is authoritative. The in-memory state (in actors) is transient.

19.12.2 Inbox Overflow

1. A burst of requests fills a `KeyValueActor`'s inbox to its maximum size.
2. New `Send` and `Ask` calls throw `ActorBusyException`.
3. The routing layer catches the exception and returns `MustRetry` to the client.
4. The client retries after a delay.
5. The actor drains its inbox and begins accepting messages again.

Control messages (proposal completions, replication updates) bypass the inbox bound. This ensures that the Raft pipeline is not blocked by client request backpressure.

19.13 Summary

Kahuna uses the Nixie actor system for concurrency. Each actor owns its state and processes messages one at a time. There are no locks on in-memory state within an actor. Data races are eliminated by construction.

Messages are routed to actor instances through consistent-hash routing. The key's hash determines which actor instance processes the message. Multiple instances per partition allow parallelism for non-overlapping keys.

The `KeyValueActor` is the most complex actor. It holds a B-tree cache, write intents, range locks, and in-flight proposals. It dispatches operations to 41 handler classes through a switch on the message type.

Two separate actor rings (persistent and ephemeral) prevent cross-contamination between durable and in-memory-only operations. Bounded inboxes with control message bypass provide backpressure without blocking the Raft pipeline.

The single-threaded invariant is the foundation of the actor model's safety. As long as actors do not leak references to their internal state across async boundaries, the invariant holds and the state is consistent.

The next chapter examines MVCC and snapshot isolation: how Kahuna tracks multiple versions of a key and how transactions read a consistent snapshot.

20 MVCC and Snapshot Isolation

When two transactions read and write the same key at the same time, one of them must wait or abort. In a system that stores only the current value of each key, every read competes with every write. Reads block writes, writes block reads, and throughput drops under concurrency.

Multi-Version Concurrency Control (MVCC) solves this problem. Instead of storing one value per key, the system stores multiple versions. Each write creates a new version. Reads select the version that was current at the time the read started. Reads never block writes, and writes never block reads.

This chapter explains how Kahuna implements MVCC: the version chain, write intents, snapshot reads, the snapshot floor, revision pruning, and the disk fallback for deep history.

20.1 The Version Chain

Each key in Kahuna can have multiple versions. The `KeyValueEntry` stores the current version and an archive of older versions.

20.1.1 KeyValueEntry

The `KeyValueEntry` is a sealed class that holds the complete state of a key:

Field	Type	Description
`Value`	`byte[]?`	The current value.
`Revision`	`long`	The current modification revision (incremented on each write).
`LastModified`	`HLCTimestamp`	The HLC timestamp of the most recent write.
`Expires`	`HLCTimestamp`	The TTL expiry timestamp. Zero means no expiry.
`State`	`KeyValueState`	Set, Deleted, or Undefined.
`WriteIntent`	`KeyValueWriteIntent?`	Uncommitted transaction write (null if no active transaction).
`Revisions`	`KeyValueRevisionHistory?`	Archive of older versions.
`MvccEntries`	`Dictionary<HLCTimestamp, KeyValueMvccEntry>?`	Per-transaction snapshots for read-your-writes.
`FloorBoundaryRevision`	`long`	The revision pinned by the snapshot floor (default −1).
`FloorBoundaryCoverageEnd`	`HLCTimestamp`	Upper bound where the floor-boundary revision is authoritative.
`FlushedRevision`	`long`	Highest revision confirmed on disk. The entry is dirty while `Revision > FlushedRevision`.
`LastUsed`	`HLCTimestamp`	Last access time (for LRU eviction).
`LruPrev`, `LruNext`	`KeyValueEntry?`	Intrusive doubly-linked list pointers for LRU cache management.

The entry acts as the head of a version chain. The current version is in the `Value` and `Revision` fields. Older versions are in the `Revisions` archive.

20.1.2 Revision History

The `KeyValueRevisionHistory` stores older versions of the key as a compact sorted array of `(long Key, KeyValueRevisionEntry Value)` pairs in ascending revision order. Each `KeyValueRevisionEntry` is a readonly record struct with four fields: `Value`, `LastModified`, `Expires`, and `State`.

The array is bounded. At most `RevisionRetention + 1` entries are stored in memory: the configured retention count (default 16) plus an optional floor-boundary entry. Initial capacity is 1. The array grows by doubling. It never shrinks.

Lookups use binary search. The `TryGetRevisionAtOrBefore(snapshot)` method finds the highest revision whose `LastModified` is at or before the given snapshot timestamp. This is the core operation for snapshot reads.

The sorted-array structure replaces a dictionary. For the small bound (at most 17 entries), a sorted array is more memory-efficient than a hash table.

20.2 Write Intents

A write intent is an uncommitted transaction write. When a transaction calls `SetKeyValue` within a session, the system does not immediately apply the write to the committed state. Instead, it attaches a `KeyValueWriteIntent` to the entry.

20.2.1 KeyValueWriteIntent

The `KeyValueWriteIntent` is a sealed class with these fields:

Field	Type	Description
`TransactionId`	`HLCTimestamp`	The ID of the transaction that created this intent.
`Expires`	`HLCTimestamp`	The lease deadline. Zero means session-owned (no timeout).
`CommitTimestamp`	`HLCTimestamp`	Set at prepare time. Zero before prepare.
`RecordAnchorKey`	`string?`	Set at prepare. Points to the transaction record.

A write intent is live if its lease has not expired. Session-owned intents (with zero expiry) are live as long as the session is active.

20.2.2 Write Intent Lifecycle

1. **Create.** A transaction writes to a key. The system creates a `KeyValueWriteIntent` and attaches it to the entry. The entry's committed state does not change.
2. **Visibility.** The intent is visible only to its own transaction. Other transactions that read the key see the committed state, not the intent. If another transaction encounters a live write intent from a different transaction, it may receive `WaitingForReplication` (the safe-time wait mechanism) or detect a conflict.
3. **Prepare.** At 2PC phase 1, the intent receives a `CommitTimestamp` and a `RecordAnchorKey`. The system creates a durable prepared intent in the `PreparedIntentStore`.
4. **Commit.** At 2PC phase 2, the intent's value becomes the new committed state. The revision increments. The old committed value moves to the revision archive. The intent is cleared.
5. **Rollback.** If the transaction aborts, the intent is removed. The committed state is unchanged.

20.2.3 Write Intent Conflicts

When a `TryGet` handler encounters a write intent from another transaction:

- If the intent has expired, the handler clears it and reads the committed state.
- If the intent is live and the reader has a snapshot timestamp, the handler checks whether the intent's `CommitTimestamp` falls at or before the snapshot. If so, the read returns

`WaitingForReplication` (the system must wait for the intent to resolve before it can serve a consistent read).
- If the intent is outside the snapshot window, the handler reads the committed state.

When a `TrySet` handler encounters a live write intent from another transaction, the write is rejected. Only one transaction can hold a write intent on a key at a time.

20.3 MVCC Entries: Read-Your-Writes

When a transaction reads a key for the first time, the system snapshots the committed state into an `MvccEntries` dictionary, keyed by transaction ID. The snapshot is a `KeyValueMvccEntry` with the value, revision, timestamps, and state at the time of the read.

On subsequent reads within the same transaction, the system returns the snapshot. This provides read-your-writes consistency: a transaction always sees its own writes and the state it read at the start.

If the committed revision advances past the snapshot (another transaction committed between the reads), the handler returns `Aborted`. The transaction's view is stale and must be retried.

This mechanism is the foundation of snapshot isolation. Each transaction reads from a consistent snapshot. Writes from other transactions that commit after the snapshot are invisible.

20.4 Snapshot Reads

A snapshot read retrieves the value of a key as of a specific HLC timestamp. The client specifies the timestamp through the `snapshotMs` parameter on `GetKeyValue`.

20.4.1 Resolution Logic

The `TryGetHandler` resolves a snapshot read in two steps:

1. **Check the current version.** If the entry's `LastModified` is at or before the snapshot timestamp, the current version is the answer.
2. **Search the archive.** If the current version is newer than the snapshot, the handler calls `entry.TryGetRevisionAtOrBefore(readTimestamp)`. This binary-searches the in-memory revision archive for the highest revision whose `LastModified` is at or before the snapshot.
3. **Disk fallback.** If the in-memory archive does not contain the version (the archive was pruned, or the snapshot is older than the oldest retained revision), the handler calls `PersistenceBackend.GetKeyValueRevisionAtOrBefore(key, maxRevision, readTimestamp)`. This queries the persistence backend (RocksDB or SQLite) for the version.

If no version exists at or before the snapshot timestamp, the read returns `DoesNotExist`.

20.4.2 Disk Fallback for Deep History

The in-memory archive holds at most `RevisionRetention` versions (default 16). For keys with high write frequency, older versions are pruned from memory. These versions may still exist on disk if persistent revision retention is enabled.

The `ResumeRead` mechanism handles asynchronous disk fallback for cache misses on the current value (non-snapshot reads):

1. The actor checks the B-tree cache. The key is not present.
2. The actor dispatches an off-actor disk read (to avoid blocking the actor's message loop).
3. When the disk read completes, the system sends a `ResumeRead` control message back to the actor.
4. The actor reconciles the disk result with its current state (another write may have arrived while the disk read was in progress).

`ResumeRead` is a control message, so it bypasses the inbox bound and is processed promptly.

20.5 The Snapshot Floor

The snapshot floor is the minimum HLC timestamp below which revisions may be pruned. It protects long-running snapshot reads and transactions from losing the versions they depend on.

20.5.1 SnapshotFloorStore

The `SnapshotFloorStore` is a replicated hold registry on the meta partition (partition 0). It tracks active snapshot holds and computes the effective floor.

A snapshot hold is a record with four fields:

Field	Type	Description
`HoldId`	identifier	Unique ID for this hold.
`HolderId`	identifier	The entity that acquired the hold (transaction, client session).
`Timestamp`	`HLCTimestamp`	The snapshot point to protect.
`LeaseExpiry`	`HLCTimestamp`	Deadline for the hold. The hold expires if not renewed.

20.5.2 Hold Operations

- `AcquireAsync(holderId, timestamp, leaseMs)`: creates a hold. Idempotent by `(holderId, timestamp)`: a duplicate acquire returns the existing hold. The operation replicates through Raft on the meta partition.
- `RenewAsync(holdId, leaseMs)`: extends the lease deadline. Returns `DoesNotExist` if the hold expired.
- `ReleaseAsync(holdId)`: removes the hold. The floor rises when the lowest hold is released.
- `PurgeExpiredHoldsAsync()`: periodic cleanup of holds whose lease has lapsed.

20.5.3 Effective Floor

`GetEffectiveFloor(currentTime)` computes the minimum timestamp among all live holds. This is the effective snapshot floor. A revision whose `LastModified` is below the floor is a candidate for pruning.

The method has an O(1) fast path when the cache is valid and an O(N) slow path when a hold may have expired. Prune operations and acquire operations are ordered through a `pruneCommitLock` to prevent races.

20.5.4 Metrics

Two metrics track the floor state:

- `kahuna.snapshot_floor.live_holds`: the number of active holds.
- `kahuna.snapshot_floor.effective_floor_ms`: the effective floor timestamp in milliseconds.

20.6 Revision Pruning

The revision archive grows with each write. Without pruning, memory consumption grows without bound. Kahuna prunes old revisions based on three criteria.

20.6.1 In-Memory Pruning

The `RemoveExpiredRevisions` method in `BaseHandler` runs after each write. It computes a cutoff revision:

```
cutoff = currentRevision - RevisionRetention + 1
```

Revisions below the cutoff are candidates for removal. The method removes all revisions below the cutoff with one exception: the floor-boundary revision.

20.6.2 The Floor-Boundary Revision

If snapshot holds exist, the pruning logic pins the highest below-cutoff revision whose `LastModified` is at or before the effective snapshot floor. This revision is the floor-boundary revision. It stays in the archive even though it is below the retention cutoff.

The floor-boundary revision is the safety net for snapshot reads. A snapshot read at a timestamp near the floor can use this revision as its answer. Without it, the read would need to fall back to disk.

When revisions between the floor-boundary revision and the retention window are trimmed, the entry records `FloorBoundaryCoverageEnd`: the smallest `LastModified` among the trimmed revisions. This value marks the point where the floor-boundary revision stops being authoritative.

When `TryGetRevisionAtOrBefore` finds the floor-boundary revision as the best match, it checks whether the snapshot timestamp is below `FloorBoundaryCoverageEnd`. If the snapshot is at or above that boundary, a gap exists: there may be a trimmed revision on disk that is a better match. The method returns a miss, and the handler falls back to disk.

20.6.3 Persistent Revision Retention

On disk, revisions are subject to separate retention controls:

Flag	Default	Description
`--persistent-revision-retention-count`	0 (keep forever)	Maximum persisted revisions per key.
`--persistent-revision-retention-age`	0 (disabled)	Maximum age in seconds for persisted revisions.
`--persistent-revision-cleanup-interval`	300s	Minimum interval between cleanup sweeps.
`--persistent-revision-cleanup-batch-size`	1000	Maximum revision records deleted per sweep.

When both in-memory and persistent retention are configured, the system retains the most recent `RevisionRetention` versions in memory and up to `persistent-revision-retention-count` versions on disk. Older versions are deleted during cleanup sweeps.

20.7 Safe Timestamp

The safe timestamp is the minimum prepared `CommitTimestamp` across all live write intents in a shard. Any snapshot timestamp strictly below this value avoids cutting across a currently prepared transaction.

The `GetSafeTimestampHandler` scans both the B-tree store and the prefix locks to find this minimum. The safe timestamp is used by the snapshot coordinator to choose a consistent point for snapshot reads.

If no write intents are active, there is no lower bound, and any timestamp is safe.

20.8 Prepared Intents and MVCC

The `PreparedIntentStore` is a partition-scoped authority for durable prepared intents. At most one live intent can exist per key. The store uses a `ConcurrentDictionary` for concurrent reads during visibility lookups, with mutations serialized by a lock.

When a `TryGetHandler` encounters a durable prepared intent from another transaction, it calls `DurableReadVisibility.Resolve` to determine how to proceed:

- **UseIntentValue**: the intent's transaction committed. Use the intent's value as the read result.
- **UseExisting**: the intent's transaction aborted or the intent is not relevant to this read. Use the committed state.
- **Retry**: the intent's outcome is not yet known. Return `MustRetry` and let the client retry after the intent resolves.

This mechanism prevents a reader from seeing an inconsistent state when a transaction is in the middle of the two-phase commit protocol.

20.9 Failure Scenarios

20.9.1 Snapshot Hold Expiry During a Long Transaction

1. A transaction acquires a snapshot hold at timestamp T1 with a 30-second lease.
2. The transaction takes 45 seconds to complete.
3. After 30 seconds, the hold expires. The effective floor rises above T1.
4. Revision pruning removes versions at T1.
5. The transaction tries to read a key at its snapshot timestamp. The version was pruned.
6. The handler falls back to disk. If the persistent revision is also pruned, the read returns `DoesNotExist` or stale data.

Prevention: renew the snapshot hold before it expires. Set the lease duration longer than the expected transaction duration.

20.9.2 Stale Read After Floor-Boundary Gap

1. A key has revisions at timestamps T1, T5, T10, T15, T20 (revision numbers 1 through 5).
2. Retention is 2. The cutoff is revision 4 (keep revisions 4 and 5).
3. A snapshot hold exists at T3. Revision 1 (T1) is pinned as the floor-boundary revision.
4. Revisions 2 (T5) and 3 (T10) are trimmed. `FloorBoundaryCoverageEnd` is set to T5.
5. A snapshot read at T7 (between T5 and T10) hits the archive. The best match is revision 1 (T1), which is the floor-boundary revision.
6. The snapshot (T7) is above `FloorBoundaryCoverageEnd` (T5). The handler returns a miss.
7. The handler falls back to disk and finds revision 2 (T5), which is the correct answer.

The `FloorBoundaryCoverageEnd` check prevents the in-memory archive from returning an incorrect answer when revisions were trimmed.

20.9.3 Write Intent From a Crashed Transaction

1. A transaction writes to key K and creates a write intent.
2. The client crashes before sending the commit or rollback.
3. The write intent remains on the entry with a lease deadline.
4. After the lease expires, the next read or write to key K detects the expired intent and clears it.
5. The committed state is restored. No data corruption occurs.

Write intents have lease deadlines. Expired intents are cleaned up on the next access. The committed state is always authoritative.

20.10 Summary

Kahuna uses MVCC to allow reads without blocking writes. Each key stores multiple versions: a current version and an archive of up to 16 older versions in memory.

Write intents represent uncommitted transaction writes. They are visible only to their own transaction. Other transactions see the committed state. The intent lifecycle follows the 2PC protocol: create, prepare (with commit timestamp), commit (becomes the new version), or rollback (removed).

Snapshot reads resolve the version that was current at a given HLC timestamp. The resolution searches the in-memory archive first and falls back to disk for deep history.

The snapshot floor protects long-running reads from revision pruning. Snapshot holds lease a timestamp. While a hold is active, revisions at or above that timestamp are retained. The floor-boundary revision is pinned below the retention cutoff as a safety net.

Revision pruning bounds memory consumption. In-memory pruning runs after each write and keeps the most recent `RevisionRetention` versions. Persistent retention is controlled by separate flags for count and age.

The next chapter examines transaction internals: how sessions, 2PC coordination, write-intent resolution, and conflict detection work at the implementation level.

21 Transaction Internals

Chapter 6 introduced transactions from the application's perspective: script transactions for server-side logic and interactive sessions for client-driven workflows. Chapter 19 explained how MVCC stores multiple versions and how write intents represent uncommitted writes.

This chapter moves inside the transaction subsystem. It describes the two-phase commit (2PC) protocol as Kahuna implements it, durable intents, deferred settlement, conflict detection, admission control, session reaping, and transaction recovery.

21.1 Transaction Sessions

Every interactive transaction starts with a session. The `TransactionCoordinator` manages sessions in a `ConcurrentDictionary<HLCTimestamp, TransactionContext>`, keyed by transaction ID.

21.1.1 Starting a Session

`StartTransaction` performs these steps:

1. Validate the options (locking mode, timeout, priority).
2. Clamp the timeout to the server's `MaxTransactionTimeout`.
3. Acquire an admission slot through the `TransactionPriorityOrderer`. If no slot is available and the admission wait expires, return `AdmissionRefused`.
4. Mint an HLC-based transaction ID.
5. Create a `TransactionContext` and add it to the sessions dictionary.

The transaction ID is an HLC timestamp. It provides a globally unique identifier that also carries a causal ordering.

21.1.2 Operation Tracking

Each operation within a session (a read, write, lock acquisition) is tracked through `BeginOperation` / `CompleteOperation` / `CancelOperation`. The coordinator folds each operation's effects into the session's working set: which keys were read, which were modified, and which locks were acquired.

This working set is the input to the commit protocol. It tells the coordinator which partitions are involved and what needs to be validated.

21.2 Admission Control

The `TransactionPriorityOrderer` is a per-node admission gate that limits the number of concurrent transactions. Two independent orderer instances exist: one for interactive sessions and one for script transactions.

When the transaction count is below the ceiling, admission is immediate. When the count is at the ceiling, the caller parks on an awaitable. Parked callers are served in priority order (four priority levels).

`AdmitAsync` returns an `AdmissionLease`. The lease is `IDisposable` with exactly-once release through `Interlocked.Exchange`. A leaked lease (the caller crashes without disposing) permanently shrinks capacity. The session reaper eventually cleans up the session and releases the lease.

The admission wait is clamped to `MaxAdmissionWaitMs`. If the wait exceeds this value, the caller receives `AdmissionRefused`.

21.3 The Two-Phase Commit Protocol

When the client calls `Commit`, the coordinator enters the 2PC protocol. The protocol has four phases: prepare, validate, decide, and resolve.

21.3.1 Phase Overview

```
Prepare          Install write intents and acquire locks on each partition.
                 ↓
Validate         Check the read set for conflicts (optimistic mode).
                 ↓
Decide           CAS transition on the canonical transaction record:
                 Undecided → Committed or Undecided → Aborted.
                 ↓
Resolve          Materialize committed values or discard aborted intents.
```

21.3.2 Commit Entry Point

`CommitTransaction` begins with contention control. The coordinator enters a finalize slot on the `TransactionContext` through `context.EnterFinalize()`. This slot serializes concurrent commit, rollback, and reaper operations against the same session. If another thread already holds the slot, the caller receives a `Mirror` response (wait for the owner's outcome) or `Rejected` (`MustRetry`).

After acquiring the slot, the coordinator calls `FreezeForFinalize`, which drains in-flight operations. No new operations are accepted after the freeze. Then it calls `TwoPhaseCommit`.

21.3.3 Four Transaction Paths

The `TwoPhaseCommit` method handles four cases based on what the transaction modified:

Read-only transaction. No modified keys. The coordinator validates the read set only (conflict probe plus revision check). If any read key was modified by another transaction since the read, the result is `Aborted`. No Raft proposals are needed.

All-ephemeral transaction. All modified keys are ephemeral (in-memory only). The coordinator validates the read set, installs write intents via `PrepareMutations`, checks for commit conflicts, and applies the writes via `CommitMutations`. If any step fails, the writes are rolled back. No durable transaction record is created.

All-persistent (durable) transaction. All modified keys are persistent. The coordinator delegates to the `DurableTransactionFinalizer`, which drives the full durable 2PC protocol with a canonical transaction record and prepared intents.

Mixed transaction. Some modified keys are persistent and some are ephemeral. The coordinator prepares ephemeral keys first (so an ephemeral failure aborts before the durable decision). Then it runs the durable protocol for persistent keys. The durable decision drives the ephemeral commit or rollback.

21.4 Durable Transaction Finalization

The `DurableTransactionFinalizer` drives one transaction through the durable 2PC model. It works with frozen, immutable inputs: the transaction ID, epoch, coordinator key, commit timestamp, decision deadline, a manifest of participant partitions, and per-partition prepared intent sets.

21.4.1 Step 1: Staged-Base Validation

Before anything durable, the finalizer checks each write intent's `BaseRevision` and `BaseState` against the current committed state on the target partition. If the base moved (another transaction committed to the same key), the transaction aborts with a conflict. This check catches conflicts early, before any Raft proposals.

21.4.2 Step 2: Initialize the Canonical Record

The finalizer creates a canonical transaction record on the anchor partition in the `Undecided` state. This is a Raft proposal. The record contains the transaction ID, epoch, and manifest hash.

The anchor partition is determined by the transaction's anchor key. The canonical record is the single source of truth for the transaction's outcome.

21.4.3 Step 3: Prepare Intents

The finalizer installs prepared intents on each participant partition through Raft. Each prepared intent records the transaction ID, the key, the proposed value, the commit timestamp, and a pointer to the canonical record (the `RecordAnchorKey`).

If a prepare is blocked by a foreign intent from a different transaction that already has a terminal decision (committed or aborted but not yet settled), the finalizer helps settle that intent before retrying. This is called "prepare-conflict helping": the `resolveDecidedBlockers` mechanism settles the blocking intent and retries the prepare. The finalizer retries up to `MaxPrepareRetries` (8 attempts).

The finalizer never touches undecided foreign records. An undecided record belongs to a live transaction. Interfering with it would violate that transaction's isolation.

21.4.4 Step 4: Validate Read Set

For optimistic transactions with `ReadValidation.TrackAndValidate`, the coordinator rereads the current committed revision for each key in the read set. If any revision advanced since the snapshot, the transaction aborts.

21.4.5 Step 5: CAS Decide

The finalizer performs a compare-and-swap transition on the canonical record:

- **Commit**: `Undecided → Committed`. The commit is allowed only if `AttemptHlc <= DecisionDeadline`. A late commit attempt is rejected. If the deadline passed, recovery presumes abort.
- **Abort**: `Undecided → Aborted` or `absent → Aborted`. Abort can create a tombstone from absence (presumed abort).

The `TransactionRecordStateMachine` enforces these transitions deterministically:

Current State	Command	Result
absent	Initialize	Undecided
Undecided	Commit (within deadline)	Committed
absent or Undecided	Abort	Aborted
Committed	Commit (same)	Idempotent (no change)
Aborted	Abort (same)	Idempotent (no change)
Committed	Abort	Rejected (terminal is final)
Aborted	Commit	Rejected (terminal is final)
absent	Commit	Rejected (needs Undecided proof)

Once the record reaches `Committed` or `Aborted`, it is terminal. No further transitions are possible.

21.4.6 Step 6: Resolve Intents

After the decision is durable, the finalizer resolves the prepared intents:

- **Committed**: materialize the intent's value as the new committed version. Bump the revision. Move the old committed value to the revision archive.
- **Aborted**: discard the intent. The committed state is unchanged.

Resolution can be synchronous or deferred (asynchronous), based on the `DurableDeferredSettlement` configuration.

When `DurableByReferenceMaterialization` is enabled, the resolve step writes a reference to the prepared intent instead of copying the full value through the Raft log. Each replica resolves the value from its local PreparedIntentStore during apply. This reduces Raft log bandwidth for large values.

21.4.7 One-Phase Fast Path

For single-partition transactions, the finalizer bundles the record initialization, anchor prepare, and commit decision into one atomic Raft proposal. This reduces three Raft round trips to one. If the prepare fails (a `bundledPrepareProbe` gate rejects it), the bundle is rejected and the transaction falls back to the multi-step path.

When the anchor partition's leader is on another node, the coordinator forwards the bundle as a single typed inter-node RPC. The remote leader submits the bundle atomically. If the remote leader does not support the typed operation, the coordinator falls back to the normal two-phase flow.

In a multi-process cluster, a transaction whose validated read set extends beyond its written keys is excluded from the one-phase path. The read-set validation requires a separate round trip that cannot be bundled atomically with the prepare.

In a single-process Raft group (`SingleProcessRaftGroup = true`), this exclusion is relaxed. Read-carrying transactions remain eligible for the one-phase path because the validation, prepare, and commit execute in the same process with no replication window between them.

When `OnePhaseApplyTimeValidation` is enabled, the multi-process exclusion is lifted for read-carrying transactions. The bundled commit carries the transaction's on-partition read dependencies. Every replica validates those dependencies at apply time against a partition-scoped committed-head ledger. If any dependency is stale, the replica rejects the apply. All nodes in the cluster must run a version that persists this ledger before the flag can be enabled.

21.4.8 Anchor Bundling

For multi-partition transactions, the finalizer bundles the record initialization and the anchor partition's prepare into one Raft proposal. This saves one pre-decision Raft round trip compared to separate proposals.

21.5 Deferred Settlement

Deferred settlement is an optimization that moves intent resolution off the commit critical path. The decision is durable (the canonical record is committed in Raft). The resolution (materializing values, settling intents) happens asynchronously through a `ResolutionScheduler`.

The benefit: the commit returns to the client as soon as the decision is durable. The client does not wait for resolution.

If the background resolution is lost (the node crashes before resolution completes), the recovery sweep finds the committed record and resolves the intents. Correctness does not depend on the background run completing.

Deferred settlement is enabled by default (`DurableDeferredSettlement = true`). In practice, the system runs resolution synchronously for cross-node transactions to ensure read-your-writes consistency: a read immediately after a commit must see the committed value, which requires the intent to be resolved before the commit returns.

21.5.1 Read Visibility Under Deferred Settlement

When deferred settlement is active, a committed intent may not yet be materialized into the key's version chain. Reads must still return the correct value. The `PreparedIntentVisibility` module decides what a reader sees.

Three outcomes are possible:

1. **UseExisting.** The intent's commit timestamp is strictly after the reader's snapshot. The reader sees the prior committed value. No decision lookup is needed.
2. **UseIntentValue.** The intent's transaction record is `Committed` and the commit timestamp falls within the reader's snapshot. The reader sees the intent's value as if it were already materialized.
3. **Retry.** The intent's transaction record is still `Undecided` and the commit timestamp falls within the reader's snapshot. The reader cannot determine the correct value. The read returns `MustRetry` and the caller retries after a short backoff.

The canonical transaction record is the authority during deferred settlement. A reader never guesses whether an unsettled intent committed. It always consults the record.

21.5.2 Scan Merge for Prepared Intents

Range scans and prefix scans must produce correct results even when unsettled prepared intents exist. The `PreparedIntentScanMerge` module merges prepared intents into a scan page's results.

The merge applies these rules:

- A committed intent overrides an existing key's value or injects a new key into the result set.
- A committed delete or an expired committed intent excludes the key from results.
- An aborted intent is invisible.
- An intent below the scan's snapshot is invisible.
- An undecided intent within the scan's snapshot causes the entire page to retry.

Pagination is owned by the merge. Injected keys count toward the page limit. If the scanning transaction holds its own intent on a key, a foreign intent on the same key is skipped (read-your-own-write).

21.5.3 Stale-Base Veto

The staged-base fence (Step 1 of finalization) runs on the partition leader. Followers do not run this check. A replica-side veto provides a second line of defense.

When a follower applies a replicated prepare through Raft, it checks the intent's validated base against its own fence memory. If the base is stale (another transaction committed to the same key after the base was captured), the follower drives a best-effort `Abort` at the transaction's anchor partition.

This veto catches the case where the leader's fence memory was incomplete (for example, after a leader election) and admitted a prepare with a stale base. The veto is best-effort: if the transaction already committed at the anchor, the abort request loses the CAS race and the commit stands.

21.5.4 Pre-Decision Replica Fence Confirmation

The stale-base veto runs after the prepare is replicated. A complementary mechanism runs before the decision. Before the coordinator executes the CAS decide step, it can query replicas for their staged-base fence verdicts on the transaction's written keys.

If any replica reports a stale base, the coordinator aborts the transaction before committing. This catches conflicts that the leader's fence memory missed after a leader election. The pre-decision confirmation is coordinator-driven, not follower-driven.

21.5.5 Stale-Hydration Refusal

When a cache-miss read hydrates a key's value from the persistent store, the loaded row may be stale. This can happen on a newly promoted leader whose local persistent state lags behind the committed state.

The stale-hydration refusal mechanism compares the hydrated row's revision against the staged-base fence's remembered committed head for that key. If the hydrated row is strictly below the remembered head, the hydration is refused. The read returns `MustRetry` and the system schedules a convergence repair (a re-read from the durable store).

This mechanism prevents a freshly promoted leader from using a stale local row as a key's base for validation.

21.5.6 Lock Grant Term Fencing

Every lock grant records the Raft term under which it was issued. At commit time, the conflict probe asks each partition whether it still leads under the same term. A term change means the partition lost and regained leadership, and all locks issued under the old term were dropped.

If the term changed, the probe answers `Unlocked` for that grant. The coordinator refuses the commit with an `Aborted` status and a `Lost lock` reason. This prevents a stale lock (one that a new leader never knew about) from allowing a commit that should have been rejected.

Script transactions that lose a lock this way receive `MustRetry` instead of `Aborted`. The SDK's script retry loop can re-execute the script automatically. Interactive sessions still receive `Aborted`, because the client reads under the lost lock cannot be replayed.

21.5.7 Snapshot Clock Fencing

A snapshot read whose timestamp leads the partition's clock by more than 5 seconds cannot be fenced. The partition refuses the read with `MustRetry` instead of serving an unfenced result. The client retries the operation, and the partition's clock usually catches up within one attempt.

21.5.8 Write Timestamp Ordering

Every write is stamped strictly after the head it replaces. This ordering prevents a committed durable intent from carrying a timestamp newer than its replacing head, which would cause snapshot anomalies. Extend operations that arrived after a later overwrite are served correctly because the head's timestamp is always the latest.

21.6 Conflict Detection

Kahuna uses two conflict detection mechanisms, one for each locking mode.

21.6.1 Optimistic: Read-Set Validation

In optimistic mode, reads do not acquire locks. Conflicts are detected at commit time.

`ValidateReadSet` checks two conditions:

1. **Read observation conflict.** If the same key was observed at two different base revisions within one transaction (the key changed between two reads in the same session), the transaction aborts.
2. **Concurrent modification.** For each read key that is not in the modified set, the coordinator re-reads the current committed revision. If the revision changed since the snapshot, another transaction committed to that key. This transaction aborts.

`CheckCommitConflicts` probes for concurrent write intents. If a foreign write intent appeared after this transaction's read, a write-skew may exist. The transaction aborts.

Transactional reads also pin the MVCC version at first read through the snapshot source. If a competing transaction commits a newer version of the same key before this transaction commits, the pin detects the conflict early. This reduces the work that commit-time validation must do.

21.6.2 Pessimistic: Lock Acquisition

In pessimistic mode, the transaction acquires exclusive locks on every key it reads or writes. Other transactions that touch the same keys wait (up to the transaction timeout) or abort.

Lock acquisition happens before the operation, not at commit time. Conflicts are detected immediately: if the lock is held by another transaction, the client retries the acquisition with exponential backoff (1 ms to 50 ms cap) up to `MaxLockWaitMs` (default 3000 ms). If the lock is still held after the wait budget, the operation returns `MustRetry`.

Pessimistic mode eliminates read-set validation overhead. There are no concurrent modifications to detect because locks prevent them. The trade-off is lower concurrency: transactions wait for each other instead of running in parallel and detecting conflicts at commit.

21.7 Script Transactions

The `ScriptTransactionExecutor` handles script-driven transactions. It composes the `TransactionCoordinator` for all 2PC and lock-release operations.

`TryExecuteTx` parses the script and decides whether a transaction is needed. A single standalone command (one `SET` or `GET` with no `BEGIN`/`COMMIT`) runs directly without the admission gate. Multi-statement scripts or explicit `BEGIN` blocks open a transaction through the coordinator.

Script transactions use a separate admission orderer from interactive sessions. This prevents a burst of script transactions from starving interactive sessions (or the reverse).

An extra locking delay of 10 ms (`ExtraLockingDelay`) is added to pessimistic lock TTLs in script transactions. This accounts for the parsing and execution overhead.

21.8 Transaction Record Store

The `TransactionRecordStore` is the partition-scoped authority for canonical transaction records. Records are keyed by `(TransactionId, Epoch)`.

Mutations go through `Apply(TransactionRecordCommand)`, which is serialized by a lock. The `TransactionRecordStateMachine` evaluates the command against the current state and produces the transition (or a rejection).

The store is replicated through Raft. `Restore` and `Replicate` methods apply write-ahead log entries. The store persists per-partition snapshots to disk (temp file plus rename). Snapshots are loaded on startup. A version counter avoids unnecessary snapshot rewrites when no records changed.

21.9 Completion Receipts

The `CompletionReceiptStore` is a node-local store of completion receipts. A receipt records that a participant partition durably committed the value for a specific transaction and key.

Receipts serve two purposes:

1. **Leader-change resilience.** After a leader change, the new leader's actors have empty in-memory state. If a re-delivered commit arrives for a transaction that already committed, the receipt lets the server answer `Committed` instead of ambiguous `MustRetry`.

2. **Range split/merge gating.** Before a range split or merge cutover, `SettleSuppliedIntentsAsync` must resolve all durable intents in the moving range. Completion receipts are transferred to the new partition so it can serve re-commits correctly after cutover.

Receipts are removed by coordinator acknowledgement (`Forget`) or by an age-based backstop (`CollectExpired`). They are not evicted by size.

21.10 Session Reaping

Abandoned sessions (the client crashed, disconnected, or forgot to commit/rollback) are cleaned up by the session reaper.

`ReapAbandonedSessions` runs periodically on each collection sweep. For each session, it computes a deadline:

```
deadline = transactionId + Timeout + ReapGraceMs (15 seconds)
```

If the session has pending operations, the deadline is extended by `MaxParticipantEffectTtlMs` (15 seconds) to allow in-flight operations to complete.

When a session passes its deadline:

1. The reaper claims the finalize slot through `TryEnterReap()`.
2. `ReapSession` sets the action to `Abort` and releases the working set.
3. Three outcomes are possible:
   - All mandatory releases acknowledged: the session is removed. Terminal outcome `RolledBack` is retained for duplicate handling.
   - An operation is still unresolved past the extended deadline: `Errored`. The session is removed.
   - A mandatory release failed (transient): `MustRetry`. The session stays in the `Reaping` state. The next sweep retries.

The reaper also prunes retained terminal outcomes by age on each sweep.

21.11 Transaction Recovery

The `DurableTransactionRecovery` handles orphaned prepared intents. A prepared intent can become orphaned when the coordinator crashes after preparing but before deciding, or when a leader change occurs between prepare and resolve.

21.11.1 The Recovery Sweep

`SweepAsync` runs periodically on each partition leader. It groups unresolved prepared intents by `(TransactionId, Epoch, RecordAnchorKey)`. For each group, it looks up the canonical transaction record on the anchor partition.

The sweep produces one of four outcomes:

Record State	Action
Committed	Materialize values and settle intents.
Aborted	Discard intents and settle.
Undecided, past deadline	Drive a presumed-abort: CAS `Undecided → Aborted` at the anchor. A concurrent commit can still win the CAS. Return whatever the record became.
Undecided, within deadline	Skip. The transaction is still live.
No record, within retention horizon	Orphan prepare. Drive a presumed-abort: CAS `absent → Aborted`.
No record, past retention horizon	Hold the intent. The record may have been reclaimed by GC after a successful commit. Presuming abort would discard a committed value. Recovery holds the intent until the record reappears or an operator intervenes.

21.11.2 Presumed Abort

Kahuna uses the presumed-abort rule: any transaction without a committed decision record is aborted. If the coordinator crashes before deciding, the recovery sweep finds no record (or an `Undecided` record past its deadline) and aborts the transaction.

This is safe because the commit decision requires a durable `Committed` record. If no such record exists, the transaction did not commit. The prepared intents are discarded.

21.11.3 Prepare-Conflict Helping

When a prepare is blocked by a foreign intent whose record is already terminal (committed or aborted but not yet settled), the recovery mechanism settles the blocking intent immediately. This is `TryResolveDecidedBlockersAsync`. It never touches undecided records (those belong to live transactions).

This mechanism prevents a resolved-but-unsettled intent from blocking new transactions indefinitely.

21.12 Outcome Contract

Every transaction terminates with one of these outcomes:

Outcome	Meaning	Action
`Committed`	All writes applied.	Success.
`Aborted`	Conflict or validation failure. All writes rolled back.	Retry with a new session.
`RolledBack`	Explicit rollback completed.	Application logic decides next step.
`MustRetry`	Transient failure (drain timeout, unresolved durable decision). Session may still be live.	Retry.
`Errored`	Permanent unknown outcome (session expired, never existed).	Log and alert.

The coordinator retains terminal outcomes in a window after session removal. If a duplicate commit or rollback arrives for a session that already terminated, the coordinator returns the retained outcome instead of `MustRetry`.

21.13 Failure Scenarios

21.13.1 Coordinator Crash During 2PC

1. The coordinator prepares intents on partitions P1 and P2.
2. The coordinator crashes before the CAS decide step.
3. The prepared intents are orphaned on P1 and P2.
4. Each partition's recovery sweep finds the intents. It looks up the canonical record.
5. No committed record exists (the coordinator never decided). The sweep drives a presumed abort.
6. The intents are discarded. No data is corrupted.

21.13.2 Participant Crash After Prepare

1. The coordinator prepares intents on P1 and P2. P1 acknowledges. P2 crashes.
2. The coordinator decides `Committed` (it already has enough prepares).
3. P2 restarts. The recovery sweep finds the prepared intent and looks up the canonical record.
4. The record is `Committed`. The sweep materializes the value on P2.

The decision is durable. Resolution eventually reaches all participants, even after crashes.

21.13.3 Leader Change Between Prepare and Commit

1. Intents are prepared on P1 (old leader).
2. A leader change occurs. P1 is no longer the leader.
3. The coordinator's commit attempt reaches the new leader, which does not have the prepared intents in memory.
4. The coordinator receives `MustRetry`.
5. If wrapped in `RetryableTransaction`, the entire transaction retries with a fresh session on the new leader.

21.13.4 Abandoned Session

1. The client opens a session, reads several keys, and disconnects without committing.
2. The session's deadline passes: `transactionId + Timeout + ReapGraceMs`.
3. The reaper claims the finalize slot and aborts the session.
4. Write intents are rolled back. Locks are released.
5. The terminal outcome `RolledBack` is retained briefly for duplicate handling.

21.13.5 Concurrent Transaction Conflict

1. Transaction A reads key K at revision 5.
2. Transaction B reads key K at revision 5, writes a new value, and commits. Key K is now at revision 6.
3. Transaction A tries to commit. The read-set validation re-reads key K and finds revision 6.
4. Revision 6 does not match the snapshot revision 5. Transaction A aborts.
5. `RetryableTransaction` retries Transaction A with a fresh session. The retry reads revision 6.

21.14 Summary

Kahuna's transaction subsystem orchestrates 2PC through the `TransactionCoordinator` (session management) and the `DurableTransactionFinalizer` (durable 2PC protocol).

The protocol has four phases: prepare (install intents on participant partitions), validate (check the read set for conflicts), decide (CAS transition on the canonical record), and resolve (materialize or discard intents). Single-partition transactions use a one-phase fast path that bundles all three durable steps into one Raft proposal.

Conflict detection uses read-set validation for optimistic transactions and lock acquisition for pessimistic transactions. The `TransactionRecordStateMachine` enforces deterministic CAS transitions on the canonical record.

Recovery uses the presumed-abort rule: any transaction without a committed decision record is aborted. The recovery sweep resolves orphaned intents and settles decided-but-unresolved blocking intents.

Session reaping cleans up abandoned sessions after a grace period. Admission control limits concurrent transactions and serves waiters in priority order. Completion receipts provide leader-change resilience and gate range split/merge cutover.

The next chapter examines lock and sequencer internals: how the lock subsystem tracks ownership and fencing tokens, and how the sequencer allocates ID blocks.

22 Lock and Sequencer Internals

Chapter 4 introduced distributed locks from the application perspective: acquire, extend, release, and the fencing token that protects against stale holders. Chapter 5 introduced the sequencer: a monotonic counter that reserves values in blocks for throughput. Both subsystems appeared again in Chapter 18, which described the `LockActor` and `SequenceActor` at the actor-model level.

This chapter moves inside both subsystems. It explains how fencing tokens stay monotonic across leader changes, how lock mutations flow through Raft, how the unflushed-writes overlay prevents read gaps, how the sequencer reserves blocks through compare-and-swap on a key-value entry, and how idempotent reserves work.

22.1 Lock Entry

Each lock resource is represented by a `LockEntry`, a sealed class with seven fields:

Field	Type	Description
`Owner`	`byte[]?`	The identity of the current holder. Null when unlocked.
`Expires`	`HLCTimestamp`	The lease deadline. The lock is free after this time.
`FencingToken`	`long`	Monotonically increasing token. Incremented on each acquisition.
`LastUsed`	`HLCTimestamp`	Last access time (for cache eviction).
`LastModified`	`HLCTimestamp`	HLC timestamp of the most recent mutation.
`State`	`LockState`	`Locked` or `Unlocked` (default is `Locked`).
`ReplicationIntent`	`LockReplicationIntent?`	In-flight Raft proposal metadata (proposal ID and expiry).

The `FencingToken` is the field that matters most. Every time a client acquires a lock, the actor sets the new token to `entry.FencingToken + 1`. The token only goes up. A client that holds token 5 can verify that no other client received token 6 before the first client's write reaches the downstream resource.

22.2 The Lock Pipeline

Lock operations follow the same Manager, Locator, Actor pattern as key-value operations. The pipeline has four stages.

22.2.1 LockManager

The `LockManager` is the entry point. It creates two consistent-hash actor rings:

- **Persistent ring.** For locks with `LockDurability.Persistent`. These locks survive restarts.
- **Ephemeral ring.** For locks with `LockDurability.Ephemeral`. These locks exist only in memory.

The manager also creates a `BalancingActor` router for `LockProposalActor` instances. This router distributes Raft proposals across proposal actors in round-robin order. Lock proposals are independent of each other, so there is no need for key-based routing.

The manager holds a `DataPartitionRouter` for partition resolution, a `LockLocator` for leader routing, a `LockRestorer` for log replay, a `LockReplicator` for follower applies, and an `UnflushedLockWritesIndex` for read consistency.

22.2.2 LockLocator

The `LockLocator` resolves which node owns a lock resource. It calls `dataPartitionRouter.Locate(resource)` to find the partition, then checks `AmILeaderIfHosted` to determine whether this node leads that partition.

If this node is the leader, the locator dispatches the request to the local `LockActor` ring. If another node is the leader, the locator forwards the request through inter-node communication. If no leader is available (an election is in progress), the locator returns `MustRetry`.

22.2.3 LockActor

The `LockActor` holds the in-memory lock table: a `Dictionary<string, LockEntry>` keyed by resource name. Chapter 18 described the actor's structure. This section focuses on the three lock operations.

TryLock. The actor processes an acquisition in these steps:

1. Look up the resource in the lock dictionary. If the entry is not cached and the durability is persistent, load from the persistence backend.
2. If the entry has an active `ReplicationIntent`, reject the request. A Raft proposal is already in flight for this resource.
3. If the entry is locked, check whether the lease expired. If the lease is still active and the owner is different, reject the request.
4. Compute the new fencing token: `entry.FencingToken + 1`.
5. For persistent locks: create a Raft proposal (described below). The response is deferred until the proposal commits.
6. For ephemeral locks: apply the new state directly. Return the fencing token to the caller.

TryExtendLock. The actor validates that the caller is the current owner and holds the same fencing token. If both checks pass, the actor updates the `Expires` field with the new deadline. Extend does not increment the fencing token.

TryUnlock. The actor validates the owner. If the owner matches, the actor sets `State` to `Unlocked` and clears the `Owner` to null.

22.2.4 Write Aggregation

Persistent lock mutations batch through the same per-partition `PartitionWriteAggregatorActor` that key-value writes use. Lock entries and key-value entries share a single Raft proposal. This reduces the number of Raft round trips for lock-heavy workloads.

When the `LockActor` creates a proposal for a persistent lock, it submits a `LockProposalRequest` to the write aggregator. The aggregator collects lock and key-value proposals for the same partition and flushes them together.

Two flags tune the dispatch strategy. `--kv-write-precise-wake` fires the aggregator's timer from a dedicated thread with sub-millisecond precision instead of `Task.Delay`. `--kv-write-fill-target-trigger` enables fill-or-time dispatch: the aggregator dispatches a held buffer as soon as it reaches a learned fill target (the median of recent timer-dispatched batch sizes), instead of waiting for the timer.

On success, the aggregator sends a `CompleteProposal` message back to the `LockActor`. On failure (Raft rejected the proposal), it sends a `ReleaseProposal` message. Both are control messages that bypass the actor's inbox bound.

22.2.5 The Proposal Lifecycle

When the `LockActor` creates a proposal for a persistent lock, it follows this sequence:

1. Set a `LockReplicationIntent` on the entry. The intent records the proposal ID and the expiry deadline.
2. Add the proposal to the `proposals` dictionary (keyed by proposal ID).
3. Send the proposal to the `BalancingActor` router, which dispatches it to a `LockProposalActor`.
4. Mark the response as deferred (`ByPassReply = true`). The client waits on the proposal's `TaskCompletionSource`.

When `CompleteProposal` arrives:

1. Look up the proposal by ID. If not found (timed out or evicted), discard.
2. Validate that the `ReplicationIntent` on the entry matches the proposal. If it does not match, another operation replaced the intent. Discard.
3. Apply the committed values to the entry: `FencingToken`, `Owner`, `Expires`, `LastUsed`, `LastModified`, `State`.
4. Clear the `ReplicationIntent`.
5. Complete the `TaskCompletionSource` with the committed fencing token.

When `ReleaseProposal` arrives, the actor clears the `ReplicationIntent` and completes the `TaskCompletionSource` with an error. The client can retry.

22.3 Fencing Token Monotonicity

The fencing token is the contract that makes distributed locks useful. A lock without a fencing token cannot prevent a slow client from writing stale data after its lease expired and another client acquired the lock. The fencing token gives the downstream resource a way to reject writes from outdated holders.

Monotonicity means that each acquisition produces a token strictly greater than the previous one. Kahuna enforces this at the actor level: `entry.FencingToken + 1` is computed from the current entry state. Because the actor processes messages sequentially, two concurrent acquisitions cannot read the same token value.

22.3.1 The InvalidateOrApply Advance Guard

The replicator and restorer send `InvalidateOrApply` messages to the `LockActor` to update its cache with committed mutations. The actor applies these updates through an advance guard:

```
Apply only if:
  incoming.FencingToken > entry.FencingToken
  OR (incoming.FencingToken == entry.FencingToken
      AND incoming.LastModified > entry.LastModified)
```

This guard prevents two problems:

1. **Stale deliveries.** A delayed replication message from a previous term must not overwrite a newer acquisition.
2. **Duplicate deliveries.** The same Raft entry may be replayed during a leader change. The guard makes the replay idempotent.

The second condition (same token, newer `LastModified`) handles extend and unlock operations. These operations reuse the existing fencing token but carry a later HLC timestamp.

22.4 Lock Replication

The `LockReplicator` processes committed Raft log entries on followers. When a lock mutation commits in the Raft log, the replicator deserializes the `LockMessage` and performs four steps:

1. **Register pending.** Call `durabilityTracker.RegisterPending(partitionId, logId, DurabilityChannel.Flush)`. This tells the durability tracker that this log entry is committed but not yet flushed to disk.
2. **Record unflushed.** Call `unflushedLockWrites.Record(...)`. This updates the in-memory overlay so that a read can find the committed state before the background writer flushes it.
3. **Queue persistence.** Send a `QueueStoreLock` message to the `BackgroundWriterActor`. The background writer will flush the mutation to disk asynchronously.
4. **Send InvalidateOrApply.** Send the committed state to the persistent `LockActor` ring. The actor applies the update through the advance guard.

The replicator handles `TryLock`, `TryUnlock`, and `TryExtendLock` with the same four-step pattern. For `TryUnlock`, the `InvalidateOrApply` message passes null as the owner. This clears the holder in the actor's cache, matching the behavior of `CompleteProposal` on the proposing leader.

22.5 Lock Restoration

The `LockRestorer` replays committed Raft log entries during a leader change. When a partition's leadership moves to a new node, the Raft layer replays all committed entries that the new leader received from the old leader's log.

The restorer follows the same four-step pattern as the replicator:

1. Register pending with the durability tracker.
2. Record in the unflushed writes overlay.
3. Queue for background persistence.
4. Send `InvalidateOrApply` to the actor ring.

The restorer ensures that the new leader's actors hold committed state that is at least as recent as the replicated log. Without restoration, a re-promoted leader could mint fencing tokens from an outdated entry.

22.6 The Unflushed Writes Overlay

The `UnflushedLockWritesIndex` solves a specific race condition. Consider this sequence:

1. A lock is acquired. The Raft entry commits. The replicator records the mutation.
2. A leader change occurs before the background writer flushes the mutation to disk.
3. The new leader's `LockActor` has an empty cache for this resource.
4. The actor loads the entry from the persistence backend. The backend returns the old state (flushed before the mutation).
5. The actor treats the lock as free and grants it to a second client.

The overlay prevents step 5. Before reading from the persistence backend, the `LockActor` checks the overlay. If the overlay holds a newer mutation for this resource, the actor uses the overlay value instead of the stale disk value.

The overlay is a `ConcurrentDictionary<string, UnflushedLockWrite>`. Each entry records the owner, fencing token, expiry, timestamps, and state. The `Record` method keeps only the newest head per resource, using the same ordering as the advance guard: fencing token first, `LastModified` as the tiebreak.

The `RemoveFlushed` method prunes the overlay after a confirmed flush. It removes the entry only if the flushed mutation is at least as new as the overlay head. If a newer mutation was queued after the flush started, the overlay entry stays until that newer mutation is also flushed.

22.7 Lock Collection

The `LockActor` runs periodic garbage collection. Every 500 operations (`CollectThreshold`), the actor checks its memory budget. If the budget is exceeded, it sends itself a `Collect` control message.

The collection handler evicts entries where `(currentTime - LastUsed)` exceeds the cache entry TTL. It removes at most `CacheEntriesToRemove` entries per sweep. The eviction does not affect the durable state on disk. An evicted entry is simply reloaded from the persistence backend on the next access.

The `ProposalWaitTimeout` (10 seconds) bounds how long a proposal can stay in flight. If a proposal does not complete within this window, the actor releases it and returns an error to the caller.

22.8 Sequence Internals

The sequencer allocates monotonic values. Each sequence is backed by a durable record stored as a key-value entry under the reserved prefix `__kahuna:sequences:{name}`. The record holds the high-water mark: the highest value that any node reserved. Values above the high-water mark have not been allocated.

22.8.1 SequenceState

The durable record is a `SequenceState` object with these fields:

Field	Type	Description
`Name`	`string`	The sequence name.
`CurrentValue`	`long`	The high-water mark (highest reserved value).
`InitialValue`	`long`	The starting value.
`Increment`	`long`	The step between values.
`MaxValue`	`long?`	Optional upper bound.
`CreatedAt`	`HLCTimestamp`	When the sequence was created.
`UpdatedAt`	`HLCTimestamp`	When the record was last modified.
`Idempotency`	`Dictionary<string, SequenceIdempotencyEntry>`	Replayable allocations keyed by `reserve:{key}`.

The record is serialized by `SequenceStateCodec` and stored as the value of a key-value entry. The sequencer reads and writes this entry through the standard key-value pipeline, including Raft replication. One Raft commit per block reservation (not per value) is the core throughput optimization.

22.8.2 The Sequencer Pipeline

The sequencer follows the same Manager, Locator, Actor pattern as locks:

SequencerManager. The entry point. It creates a consistent-hash ring of `SequenceActor` instances (at least one, controlled by `SequencerWorkers`). It validates inputs (name length, increment, idempotency key length), resolves the owning node through the `SequenceLocator`, and dispatches to the local actor ring.

SequenceLocator. Resolves which node owns a sequence. The locator routes the sequence's storage key (`__kahuna:sequences:{name}`) through `DataPartitionRouter.Locate` to find the partition. Read operations use `ConfirmLeadershipIfHosted` (quorum-confirmed, linearizable). Mutation operations use `AmILeaderIfHosted` (local belief), because the CAS write that follows will fail on a deposed leader.

The locator returns `MustRetry` when no leader is available. If another node is the leader, the locator forwards the request through inter-node communication.

SequenceActor. Holds the in-memory block allocation state. Described in detail below.

22.8.3 Block-Based Allocation

The `SequenceActor` holds a `Dictionary<string, SequenceBlock>` of active blocks. Each `SequenceBlock` tracks:

- `StorageKey`: the key-value key (`__kahuna:sequences:{name}`).
- `State`: a `SequenceState` object that holds the last snapshot of the durable record (`Name`, `CurrentValue`, `Increment`, `InitialValue`, `CreatedAt`, and the `Idempotency` dictionary).
- `Revision`: the key-value entry revision at which this block was reserved. The CAS write uses this revision as its precondition.
- `Current`: the next value to serve from this block.

- `Ceiling`: the upper bound of the reserved window.
- `LastUsed`: LRU access counter.
- `Verified`: a `Stopwatch` timestamp for lease validation.

When a `Reserve` request arrives, the actor follows this logic:

1. **Load.** If no block exists for this sequence, read the durable record from the key-value subsystem through `SystemGetKeyValue`. Create a block from the record's `CurrentValue` and set the ceiling to `CurrentValue + (blockSize * increment)`.
2. **Lease check.** If the block exists but the lease expired (`Stopwatch.GetElapsedTime` exceeds `SequencerBlockLease`, default 5 seconds), mark the block for refresh. The refresh re-reads the durable record to verify that another node did not advance the high-water mark.
3. **Idempotency check.** If the request carries an idempotency key, check the block's idempotency dictionary. If the key matches a recorded allocation, return the recorded result without touching storage.
4. **Plan from block.** If the block has room (`Current + count * increment <= Ceiling`), compute the allocation from memory. No Raft proposal is needed.
5. **Bump the ceiling.** If the block is exhausted or does not have room, compute a new ceiling and write it to the durable record through a CAS operation (`SetIfEqualToRevision`). If the CAS succeeds, extend the block. If the CAS fails (another node won the race), refresh the block and retry.
6. **Retry.** The actor retries failed CAS operations with decorrelated jitter backoff. The retry policy allows up to 16 attempts with a median first delay of 1 millisecond.

Step 4 is the fast path. When the block has remaining capacity, the actor returns the next value purely from memory. No disk access, no Raft proposal, no network round trip. This is why the sequencer achieves high throughput: a block of 1,000 values (the default `SequencerBlockSize`) costs one Raft commit, and the remaining 999 values are served from local state.

22.8.4 Idempotency

The sequencer supports idempotent reserves. A client can pass an idempotency key with each reserve request. If the actor already allocated values for that key, it returns the same allocation without consuming new values.

Idempotency entries are stored in the durable record's `Idempotency` dictionary, keyed by `reserve:{key}`. Each entry is a `SequenceIdempotencyEntry` with the `SequenceAllocation` and a `CreatedAt` timestamp.

The retention window is bounded by two parameters:

Parameter	Default	Description
`SequencerIdempotencyRetentionMax`	256	Maximum entries per sequence record.
`SequencerIdempotencyRetentionTtl`	600s	Maximum age for an entry.

When the actor persists a record, `SequenceStateCodec.Prune` removes entries that exceed either bound. Oldest entries are removed first. After pruning, a replayed idempotency key that was evicted is treated as a new allocation.

22.8.5 Block Lease Revalidation

A block is a reservation against the durable record. The reservation is valid as long as no other node advanced the high-water mark past this block's ceiling. The block lease (`SequencerBlockLease`, default 5 seconds) is the interval after which the actor must revalidate.

The lease is tracked by a `Stopwatch` timestamp (`Verified` field), not by an HLC clock. The `Stopwatch` is monotonic and local, which avoids clock-skew issues with remote timestamps.

When the lease expires:

1. The actor marks the block's state as `Refresh`.
2. The actor re-reads the durable record through `SystemGetKeyValue`.
3. The `Adopt` method compares the durable record against the local block. If the durable record's `CreatedAt`, `InitialValue`, or `Increment` changed, the sequence was deleted and recreated with different parameters. The actor voids the reserved window: all unissued values in the old block are abandoned.
4. If the incarnation matches, the actor updates the block's `Current` and `Ceiling` from the durable record.

22.8.6 Incarnation Detection

The `Adopt` method detects whether the durable record represents the same sequence the block was reserved from. If a sequence is deleted and recreated with different parameters, the new record has a different `CreatedAt` or `InitialValue` or `Increment`. The actor recognizes the change and discards the stale block.

Without incarnation detection, the actor could serve values from a block that was reserved against a deleted sequence. The values would be correct numerically but would belong to the wrong logical sequence.

22.8.7 LRU Eviction

When the number of active blocks exceeds `SequencerMaxSequencesPerActor` (default 10,000), the actor evicts the least recently used blocks. The `Admit` method checks the count before each load. If the count exceeds the limit, it evicts the oldest 10% by `LastUsed` timestamp.

Eviction discards the in-memory block. Unissued values in the evicted block become gaps in the sequence. The next request for that sequence reloads the durable record and reserves a fresh block.

22.8.8 Leadership Change

When a partition's leadership moves to a new node, the `SequencerManager.OnLeaderChanged` method invalidates all blocks tied to that partition. The method sends an `Invalidate` request to every `SequenceActor` instance.

The invalidation is scoped by partition. Blocks on partitions that this node still leads are not affected. This prevents an unrelated election from discarding blocks for healthy partitions.

Invalidated blocks become gaps. The abandoned values (those reserved but not yet served) are never issued. The next request for the sequence reads the durable high-water mark and reserves a new block from there.

22.9 Failure Scenarios

22.9.1 Fencing Token Rollback After Leader Change

1. A client acquires lock R on node A. The fencing token is 5. The Raft entry commits and replicates to node B.
2. Node A crashes. Node B becomes the new leader.
3. Node B's `LockActor` has no cached entry for R. It loads from the persistence backend.
4. If the background writer on node B did not flush the mutation from step 1, the disk state shows fencing token 4.
5. Another client acquires lock R on node B. The actor computes `4 + 1 = 5`.
6. Two clients hold fencing token 5. Monotonicity is broken.

Prevention. The unflushed-writes overlay (step 2 in the replicator and restorer) holds the committed mutation. When node B loads the entry from disk, it checks the overlay first. The overlay shows fencing token 5. The actor computes `5 + 1 = 6`. Monotonicity is preserved.

This failure mode was found by Jepsen testing. The fix (in Kommander 1.0.10) addressed a WAL drain race that could cause the overlay to miss a committed entry during a narrow window around leadership transitions.

22.9.2 Fencing Tokens Under Replica Placement Moves

1. A cluster has three nodes: A, B, C. Lock R has fencing token 10 on partition P.
2. Node D joins the cluster. Partition P moves from node A to node D.
3. Node D does not have the lock entry in its actor cache or persistence backend.
4. Node D receives a lock request for R.

The partition move includes a state transfer that copies key-value entries. Lock entries are also transferred through the Raft log replay (the restorer). If the replay completes before D accepts lock requests, the token is correct. If a gap exists, the token may regress.

This is an open finding from Jepsen testing. The risk window is the interval between the moment D begins accepting requests and the moment the full log replay completes.

22.9.3 Lock State Lost on Leader Change

1. A client acquires an ephemeral lock on node A. No Raft entry is created. The lock exists only in the actor's dictionary.
2. Node A crashes. Node B becomes the leader.
3. Node B's `LockActor` has no entry for the lock. The lock is gone.
4. Another client acquires the same lock on node B. The fencing token starts at 1.

Ephemeral locks are in-memory only. They are not replicated and cannot survive a leader change. Persistent locks survive because the Raft log records every mutation. Ephemeral locks are faster (no Raft proposal) but have weaker guarantees.

For persistent locks, the in-memory state (write intents, proposal metadata) is also lost on leader change. But the committed state in the Raft log is authoritative. Operations in progress at the time of the leader change receive `MustRetry`. The client retries on the new leader.

22.9.4 Locks Not Included in PITR Images

Point-in-time recovery (PITR) restores a cluster from a snapshot of the key-value state. Lock state is stored separately from the key-value state: locks have their own log entry type (`ReplicationTypes.Locks`) and their own persistence path.

A PITR restore does not include lock state. After a restore:

1. All locks are absent from the restored cluster.
2. Fencing tokens start from 0 (or from whatever the persistence backend holds, if locks were flushed before the snapshot).
3. Clients that held locks before the restore must re-acquire them. Their old fencing tokens are no longer valid.

Applications that depend on fencing token continuity must account for this. A PITR restore is a discontinuity in the fencing token sequence.

22.9.5 Sequence Block Lease Expiry

1. Node A reserves a block of 1,000 values (101 to 1,100) for sequence S.
2. Node A serves values 101 to 500 to clients.
3. Node A becomes partitioned from the cluster. The block lease (5 seconds) expires.
4. Node A cannot revalidate the block (the KV read fails or times out). The block is marked `Refresh`.
5. Node B becomes the new leader for the partition. It reads the durable record, which shows `CurrentValue = 1100` (the high-water mark from node A's reservation).
6. Node B reserves a new block: values 1,101 to 2,100.
7. Node A's partition heals. Its stale block is invalidated by `OnLeaderChanged`.
8. Values 501 to 1,100 were reserved but never served. They become gaps in the sequence.

Gaps do not violate the monotonicity guarantee. The sequencer guarantees that each value is served at most once and that values are monotonically increasing within a single actor. It does not guarantee that every value in the range is served. The block model trades density for throughput.

22.10 Summary

Locks and sequences are simpler than transactions, but both subsystems have important correctness invariants.

The lock subsystem guarantees fencing token monotonicity through three mechanisms: sequential actor processing (`FencingToken + 1`), the advance guard in `InvalidateOrApply` (which rejects stale or duplicate replication messages), and the unflushed-writes overlay (which bridges the gap between Raft commit and disk flush). Each lock mutation creates its own Raft log entry through the `LockProposalActor`.

The sequencer achieves high throughput through block-based allocation. A block of 1,000 values costs one Raft commit. The remaining values are served from actor memory with no disk or network access. Block leases (5 seconds) force periodic revalidation against the durable

record. Idempotency keys allow clients to replay reserve requests without consuming new values. The retention window bounds the idempotency dictionary to 256 entries or 10 minutes.

Both subsystems follow the Manager, Locator, Actor pipeline. Leadership changes invalidate in-memory state: the lock restorer replays committed entries to rebuild the actor cache, and the sequencer manager invalidates blocks for the affected partition. Ephemeral locks have no Raft log and cannot survive a leader change.

The next chapter examines the persistence layer: how Kahuna writes committed state to disk through the background writer, how the three storage backends (Memory, SQLite, RocksDB) differ, and how the IO scheduler manages read and write throughput.

23 Persistence and Storage Engines

Chapter 15 traced a write request from the client through the actor layer and into Raft. Chapter 17 explained how Raft replicates log entries across nodes. Neither chapter covered what happens after a log entry is committed: the committed state must reach durable storage so that a node can recover without replaying the full log. This chapter explains that final step.

Kahuna separates the Raft commit frontier from the disk-flush frontier. The Raft log is the source of truth for durability: once a majority of nodes confirm a log entry, the entry is committed. The background writer then moves committed state to a storage backend asynchronously. The gap between the two frontiers creates a window where a read might miss recently committed data. The unflushed overlay closes that gap by merging queued writes into every read path. This chapter covers the persistence interface, the three storage backends, the background writer, the IO scheduler, the cache eviction cycle, and the durability tracker.

23.1 The Persistence Interface

The `IPersistenceBackend` interface defines the contract that every storage backend must satisfy. It contains approximately 20 methods that fall into five categories.

Write methods store committed state:

- `StoreLocks` persists a batch of lock entries.
- `StoreKeyValues` persists a batch of key-value entries, including revision history rows and no-revision provenance markers.
- `StoreDurabilityFloors` persists per-partition watermarks that track the highest durable log index.

Point-read methods retrieve individual entries:

- `GetLock` returns a single lock by resource name.
- `GetKeyValue` returns the current version of a key.
- `GetKeyValues` returns multiple keys in a single batched call (MultiGet on RocksDB).
- `GetKeyValueRevision` returns a specific revision of a key.
- `GetKeyValueRevisionAtOrBefore` returns the newest revision at or before a given ceiling, used for MVCC snapshot reads.

Scan methods support prefix and range queries:

- `GetKeyValueByPrefix` returns all current-version entries whose key starts with a given prefix.
- `GetKeyValueByRange` returns current-version entries within a key range, with a limit.
- `ScanKeyValues` and `ScanLocks` perform cursor-based paged scans across the full store.

Maintenance methods manage storage lifecycle:

- `DeleteKeyValues` and `DeleteLocks` remove entries permanently.
- `PruneKeyValueRevisions` deletes old revision history rows that fall outside the retention window.
- `GetPrunedHistoryFloor` returns the oldest surviving revision timestamp after pruning.

Checkpoint methods produce consistent snapshots:

- `CreateCheckpoint` captures the current state for Raft state transfer.

- `CreateCheckpointAsOf` captures state as of a specific HLC timestamp for point-in-time recovery. The `SupportsExactAsOfCheckpoint` property indicates whether the backend can produce an exact as-of image.

Every backend implements this full interface. The choice of backend affects performance characteristics, memory usage, and recovery behavior, but the semantics remain the same.

23.2 The Memory Backend

`MemoryPersistenceBackend` stores all data in process memory. It is designed for tests and for ephemeral deployments where durability across restarts is not required.

The backend uses three data structures:

1. A `SortedList<string, KeyValueEntry>` for current key-value data. The sorted list is array-backed, so prefix and range lookups use binary search through a `LowerBound` method (standard binary search that returns the first index at or above the target).
2. A `ConcurrentDictionary<string, LockEntry>` for lock data.
3. A `ConcurrentDictionary<string, ConcurrentDictionary<long, KeyValueEntry>>` for revision history. Each revision stores an independent snapshot of the entry, not a reference to the current-version entry.

The memory backend tracks no-revision provenance in a separate dictionary that records the earliest and latest HLC timestamps of writes that carried no explicit revision. This information is necessary for as-of checkpoint safety: if a no-revision write in the boundary window was overwritten and cannot be reconstructed, `CreateCheckpointAsOf` fails closed with `ExactCheckpointUnavailableException`.

Checkpoints serialize the in-memory state to JSON files (`store.json` and `locks.json`) in a temporary directory, then atomically rename the directory into place. The `OpenCheckpoint` static factory deserializes the JSON back into a fresh backend instance.

The `PruneKeyValueRevisions` method is a no-op on the memory backend. It returns zero deleted revisions because the memory backend retains everything for the lifetime of the process.

23.3 The SQLite Backend

`SqlitePersistenceBackend` stores data in SQLite databases on disk. It provides durability without external dependencies and works well for single-node deployments or moderate workloads.

23.3.1 Shard Layout

The backend distributes data across up to 8 SQLite databases (the constant `MaxShards`). Keys are assigned to shards through `HashUtils.InversePrefixedHash`: the method extracts the prefix before the last `/` separator and hashes it, so all keys that share a prefix land in the same shard. Prefix scans use `HashUtils.ConsistentHash` at one level lower to ensure the scan targets a single shard.

Each shard has its own `SqliteConnection` and its own `ReaderWriterLock`. Write operations acquire the writer lock, begin a transaction, prepare a statement once, bind and execute per row, and commit. Read operations acquire the reader lock with a 5-second timeout.

23.3.2 Schema

Each shard database contains six tables:

- `locks`: resource as primary key, owner as BLOB, HLC triplets for expires, lastUsed, and lastModified, fencing token, and state.
- `keys`: key as primary key, revision, value as BLOB, HLC triplets, and state.
- `keys_revisions`: composite primary key of (key, revision), same columns as `keys`. Indexed by `(key, revision DESC)` and by `lastModifiedPhysical` for efficient pruning sweeps.
- `keys_norev`: key as primary key, earliest and latest HLC triplets for no-revision provenance tracking.
- `pitr_meta`: metadata for point-in-time recovery markers.
- `durability_floor`: partition as integer primary key, floor as integer. Always stored on shard 0.

All connections use WAL journal mode, `synchronous=NORMAL`, and `temp_store=MEMORY`. Connection pooling is disabled (`Pooling=False`) to prevent file descriptor leaks.

23.3.3 No-Revision Provenance

When a write carries no explicit revision, the backend must record provenance so that as-of checkpoints can detect boundary conflicts. The SQL upsert uses CASE expressions to compute the minimum of the earliest HLC and the maximum of the latest HLC against the existing row, all within a single `INSERT...ON CONFLICT` statement. The comparison follows HLC order: physical timestamp first, then counter, then node.

23.3.4 Pruned History Floor

The backend tracks the oldest surviving revision timestamp after pruning in the `pitr_meta` table under the key `pruned_history_floor`. If the floor becomes corrupt or unreadable, the backend sets a `FailClosedFloor` value of `(int.MaxValue, long.MaxValue, uint.MaxValue)`, which prevents any as-of checkpoint from succeeding. This fail-closed behavior ensures that a corrupted floor never allows an incomplete checkpoint.

The backend also maintains resumable sweep cursors (`sweepShardCursor` and `sweepKeyCursor`) so that revision pruning can progress across multiple invocations without restarting from the beginning.

23.4 The RocksDB Backend

`RocksDbPersistenceBackend` stores data in a RocksDB database. It is the recommended backend for production deployments because RocksDB provides efficient compaction, bloom filters, and a shared block cache.

23.4.1 Column Families and Tuning

The backend creates two column families:

1. `kv` for key-value data: 64 MB write buffer, 3 write buffers maximum, merge after 1 buffer fills.
2. `locks` for lock data: 8 MB write buffer, 2 write buffers maximum, merge after 1 buffer fills.

A static LRU block cache of 256 MB is the fallback when no shared resources are provided. Each column family uses 10-bit bloom filters with full-key filtering. Index and filter blocks are cached, and L0 filter and index blocks are pinned in memory.

Database-level options include: create-if-missing, `AbsoluteConsistency` WAL recovery mode, parallelism set to `max(2, ProcessorCount)`, maximum 2 flush threads, and maximum `max(2, ProcessorCount / 2)` compaction threads. Write operations use `SetSync(true)` for an explicit fsync after each WAL write. Maintenance scans use `SetFillCache(false)` to prevent sweep reads from polluting the block cache.

When the constructor receives a `RocksDbSharedResources` object, the backend uses a shared block cache and write-buffer manager. This allows the persistence backend and the Raft WAL to share one unified memory budget, which prevents either subsystem from starving the other.

23.4.2 Key Encoding

RocksDB stores all data for a logical key in a contiguous region of the sorted keyspace through suffix conventions:

- `{key}~CURRENT` stores the current version.
- `{key}~{revision}` stores a specific revision in the history.
- `{key}~NOREV` stores the no-revision provenance marker.

The ~ separator ensures that all versions of a key sort together. The constant `CurrentMarker` is defined as `"~CURRENT"`, with a pre-encoded UTF-8 variant `CurrentMarkerUtf8` for zero-allocation key construction.

23.4.3 Write Path

Write operations use a `WriteBatch` to group all mutations into a single atomic write. Key buffers come from `ArrayPool<byte>.Shared` when the key exceeds 256 bytes (`KeyStackThreshold`), or from `stackalloc` for shorter keys. Values are serialized through protobuf via `CodedOutputStream`.

No-revision provenance is merged batch-locally: a `Dictionary<string, (HLCTimestamp, HLCTimestamp)>` accumulates the earliest and latest timestamps across all items in the batch, then persists one row per key through `BuildNoRevKey` and `PackNoRev`. The provenance row is 48 bytes (`NoRevProvenanceSize`, which is 6 longs).

23.4.4 Read Path

Point reads construct the `~CURRENT` key in a `stackalloc` buffer and call `db.Get`. Batched point reads use `db.MultiGet` with a per-key `~CURRENT` suffix and column family array.

Revision reads (`GetKeyValueRevisionAtOrBefore`) seek a forward iterator to `{key}~` and scan all revision rows. The method keeps the best match: the highest revision at or below the requested ceiling whose LastModified does not exceed the read timestamp. Sibling keys are rejected through a full decimal parse of the suffix.

Prefix and range reads also use iterators. They filter for `~CURRENT` suffixed keys and cap results at the caller's limit.

23.4.5 Revision Pruning

The `PruneRevisionsForKey` method sorts revisions in descending order and applies two retention rules: a count threshold (retain the newest N) and an age threshold (retain revisions whose `lastModifiedPhysical` is at or above the cutoff). Two categories of revisions are always protected: the current revision and, when a pruned-history floor is active, the boundary revision (the highest revision whose LastModified does not exceed the floor timestamp) plus everything newer.

After deleting revisions, the method stages the pruned-history floor (the oldest surviving revision's HLC) in the same `WriteBatch`. This ensures that a crash cannot leave the floor trailing behind the actual deletes.

The backend-wide sweep iterates `~CURRENT` rows through a `sweepCursor`, bounded by a configurable batch size for both keys inspected and revisions deleted per pass. The cursor wraps around when it reaches the end of the keyspace.

23.4.6 As-Of Checkpoint

`CreateCheckpointAsOf` first takes a native RocksDB checkpoint, then trims the copy. The trim pass streams through every key and uses a `KeyTrimState` to decide which rows to keep. Write batches flush every 4096 operations (`TrimBatchFlushThreshold`). The lock column family is dropped entirely from as-of images. A final full compaction of both column families physically purges tombstoned data from the trimmed copy.

23.5 The Background Writer

`BackgroundWriterActor` is a fire-and-forget actor (it implements `IActor<BackgroundWriteRequest>`) that moves committed state from the in-memory actor layer to the persistence backend. Chapter 18 described its role as one of the five core actor types. This section covers its internal mechanics.

23.5.1 Batching and Flush Cycle

The writer maintains two separate queues: `dirtyLocks` and `dirtyKeyValues`, both of type `Queue<BackgroundWriteRequest>`. When actors commit state through Raft, they enqueue write requests into these queues.

Each flush cycle drains both queues in batches. The maximum batch size is 1024 items (`MaxBatchSize`) or 512 KB (`MaxPacketSize`, which is 524,288 bytes), whichever limit is reached first. The flush cycle runs within a time budget set by `DirtyObjectsWriterDelay` (default 5000 ms).

23.5.2 Retry and Failure Handling

Each batch is retried up to 5 times (`WriteRetries`) with a base delay of 1000 ms between attempts. If all retries fail, the batch is retained in `pendingLockItems` or `pendingKeyValuesItems` for the next cycle. This retention ensures that no committed data is silently dropped.

23.5.2.1 Storage Engine Recovery

RocksDB can latch a background error after a failed WAL append (for example, when the disk is full). Once latched, every later write returns the cached error without new I/O. The retained-batch retries can never succeed, even after an operator frees disk space.

The `BackgroundWriterActor` tracks consecutive flush cycles in which every store call failed. After `StoreFailureCyclesBeforeRecovery` consecutive failure cycles (default 3), the writer calls `TryRecoverFromStorageFailure()` on the persistence backend. The RocksDB backend implements this method by closing and reopening the database engine. After a successful reopen, retained batches can drain without a process restart.

A single successful store call in any cycle resets the failure counter. This prevents a transient write error from triggering an unnecessary engine reopen.

23.5.2.2 Disk Stall Detection

The background writer tracks in-flight store writes with a timestamp. When any store write takes longer than `PersistenceWriteStallWarnMs` (default 500 ms), the writer logs a stall warning. The `kahuna.persistence.oldest_inflight_write_age_ms` gauge exposes the age of the oldest in-flight write. The `kahuna.persistence.store_write_duration` histogram tracks the wall time of each store call.

When a node's persistence is stalled, the transaction coordinator routes session traffic away from that node. This prevents a single slow disk from turning committed transactions into unknown outcomes. The stalled node continues to receive Raft replication traffic but does not serve new writes until its persistence recovers.

After a successful key-value flush, the writer calls `flushNotificationSink.NotifyFlushed(key, revision)` for each item in the batch. This notification routes a `FlushAck` message back to the owning `KeyValueActor`, which advances the `FlushedRevision` field on the entry.

Pooled request objects are returned to `BackgroundWriteRequestPool` after processing.

23.5.3 Durability Tracking

The writer owns a `PartitionDurabilityTracker` that tracks the relationship between Raft log indexes and their persistence status. The tracker maintains per-partition state through four durability channels:

1. **Flush**: the primary write channel for key-value and lock data.
2. **Receipts**: transaction completion receipts.
3. **TransactionRecords**: durable transaction state records.
4. **PreparedIntents**: prepared write intents for two-phase commit.

Each channel has its own `HighestApplied` ceiling. The tracker computes a watermark per partition: the highest contiguous resolved index (computed as the first pending index minus one, or `HighestRegistered` if no entries are pending). The `AdvanceDurabilityFloors` method persists floor updates for every partition whose watermark advanced past the last persisted value.

The `Forget` method retires a partition by setting a `Removed` flag under a lock, then removing the exact object from the dictionary. This prevents racing registrations from mutating an orphan state object.

23.5.4 Revision Cleanup

The writer performs two forms of revision cleanup:

1. **Targeted cleanup** (`RunTargetedRevisionCleanup`): prunes revisions for recently written keys. The set of candidate keys is bounded at 10,000 entries (`MaxPendingCleanupKeys`). The method opens a prune-delete window through `BeginPrune`/`EndPrune` on the snapshot floor store and uses a `ConfirmFloorRegistryFreshness` gate to ensure safe pruning.
2. **Full sweep** (`RunFullRevisionSweep`): a backend-wide scan at the configured `PersistentRevisionCleanupInterval`. The sweep resumes immediately when a backlog remains. Both forms use the same floor-sampling window pattern.

23.5.5 Checkpoint Coordination

The `CheckpointPartitions` method runs at the end of each flush tick. It checkpoints partitions whose dirty-since time exceeds `CheckpointInterval`. The method skips partitions with unflushed writes and drops non-leader partitions. It captures ceilings from the durability tracker for three channels (Receipts, TransactionRecords, PreparedIntents), persists each store's snapshot, and calls `raft.ReplicateCheckpoint` to distribute the checkpoint to followers.

23.6 The Unflushed Overlay

`UnflushedOverlayPersistenceBackend` is a decorator that wraps any `IPersistenceBackend` and merges queued-but-not-yet-persisted writes into every read path. This ensures read-your-writes consistency between the commit frontier and the flush frontier.

The overlay maintains two indexes: `UnflushedKeyValueWritesIndex` for key-value writes and `UnflushedLockWritesIndex` for lock writes. When the background writer successfully flushes a batch, it calls `RemoveFlushed(key, revision, lastModified)` on the overlay to retire each item. If the flush fails, overlay entries remain so they continue to cover retried writes.

23.6.1 Read Merging

Every read method follows the same pattern:

1. Read from the inner (disk) backend.
2. Check the overlay for a matching entry.
3. Return the overlay entry if the inner result is null or not newer.

The freshness check (`IsInnerNewer`) compares revisions first, then LastModified timestamps at equal revisions: the inner entry wins only if `entry.Revision > queued.Revision || (entry.Revision == queued.Revision && entry.LastModified > queued.LastModified)`.

Batched reads (`GetKeyValues`) short-circuit the overlay scan when `unflushedWrites.IsEmpty`, which avoids overhead when the flush frontier has caught up.

23.6.2 Scan Merging

Prefix and range scans use a `MergeScan` algorithm. The method builds a dictionary of disk and overlay entries keyed by string. The overlay entry wins unless the disk entry has a higher revision (or the same revision with a later LastModified). The result is sorted by key order and capped at the caller's limit. Deleted overlay entries are kept (tombstone surfacing) so that the caller observes deletes that have not reached disk.

Cursor-based whole-family scans (`ScanKeyValues`, `ScanLocks`) pass through to the inner backend without overlay merging. These scans cannot window the unflushed set safely because the cursor state belongs to the backend. Callers that need consistency must drain the background writer before starting a full scan.

23.6.3 Pass-Through Operations

Write operations (`StoreKeyValues`, `StoreLocks`) delegate to the inner backend. On success, the overlay removes the flushed entries. Delete, prune, and checkpoint operations pass through directly because they operate on already-flushed data.

23.7 Flush Notification

`FlushNotificationSink` is a late-bound bridge between the background writer and the key-value layer. The sink holds a single callback (`Action<string, long>`) that is set when `KeyValuesManager` is constructed. The background writer calls `NotifyFlushed(key, revision)` after each successful batch write. The callback routes a `FlushAck` message to the owning `KeyValueActor`.

When the actor receives a `FlushAck`, it advances `FlushedRevision` on the matching entry. The update rule is: if the acked revision is at or below the entry's current revision and greater than the entry's current `FlushedRevision`, set `FlushedRevision` to the acked value. An entry is considered dirty (not evictable) when `Revision > FlushedRevision`. The default `FlushedRevision` is −1, so every entry starts dirty.

This mechanism creates a closed feedback loop:

1. The actor commits a write through Raft.
2. The actor enqueues the write to the background writer.
3. The background writer flushes the batch to the backend.
4. The writer calls `NotifyFlushed` on the sink.
5. The sink routes a `FlushAck` to the actor.
6. The actor advances `FlushedRevision` on the entry.
7. The entry becomes eligible for LRU eviction.

23.8 IO Scheduling and Backpressure

`FairReadScheduler` (in the Kommander library) provides bounded thread pools for backend IO operations. The background writer uses a dedicated write scheduler. Read operations from actors use a separate read scheduler. This separation prevents long-running writes from blocking reads.

Each scheduler enforces a maximum queue depth. When the queue is full, the scheduler throws `ReadBackpressureExceededException`, which includes the partition ID and the maximum queue depth. The exception propagates back to the client as a retriable error, signaling

that the node is under IO pressure. The `BackgroundWriterActor` catches this exception in `FlushLocks`, `FlushKeyValues`, and `AdvanceDurabilityFloors` to defer work to the next cycle rather than crashing.

23.9 Cache Eviction

Each `KeyValueActor` maintains an in-memory B-tree (`BTree<string, KeyValueEntry>` with branching factor 32) as its working cache. The B-tree entries are linked in an intrusive doubly-linked list that tracks access order from coldest (head) to hottest (tail). The actor checks whether eviction is necessary every 500 operations (`CollectThreshold`).

23.9.1 Budget

An actor is over budget when either condition is true:

- The store count exceeds `MaxEntriesPerActor`.
- The approximate store size in bytes (plus expiry heap overhead at 40 bytes per entry and tombstone queue overhead at 16 bytes per entry) exceeds `MaxBytesPerActor`.

23.9.2 The Eviction Cycle

The `TryCollectHandler` runs a five-step eviction cycle. Each step is bounded to prevent one collection pass from blocking the actor for too long. Steps use resumable cursors so that work continues across multiple cycles.

Step 1: Tombstone drain. The handler pops entries from the tombstone queue. It skips entries that are dirty (`IsDirty()`), have live write intents, or have replication intents. Skipped entries are deferred for a future cycle. The step processes at most `CollectBatchMax` entries.

Step 2: Expiry heap drain. The handler pops entries from the expiry priority queue (keyed by HLC timestamp). It validates each entry: the key must still exist in the store, the Expires timestamp must match the heap entry (an Extend operation changes the timestamp and makes the heap entry stale), and the entry must not be dirty or intent-held. Deferred entries are re-enqueued after the loop completes to prevent infinite re-popping. The step inspects at most `inspectionMax` entries.

Step 3: LRU eviction. The handler walks the intrusive linked list from coldest (head) to hottest (tail). It resumes from the `lruCursor` saved by the previous cycle. For each entry, it checks: is the entry clean (not dirty)? Does it have no live write intent? If both conditions are true, the entry is evictable. The handler projects the count and byte size after planned evictions and stops when the actor would be within budget. The budget check uses raw store bytes only, not heap overhead. The step is bounded by both `CollectBatchMax` evictions and `inspectionMax` inspections. If work remains, the handler saves its cursor position.

Step 4: Idle-TTL sweep. The handler walks the LRU list from coldest and evicts clean entries whose age (current time minus LastUsed) exceeds `CacheEntryTtl`. The sweep stops early when it reaches the first entry that is still within its TTL, because the list is ordered by access time. This step runs independently of budget pressure.

Step 5: Additional sweeps. Three targeted sweeps run at the end:

- `SweepExpiredPredicateLocks`: a bounded scan of prefix and range lock dictionaries through a resumable key cursor. Removes expired locks.

- `SweepExpiredReads`: expires registered in-flight backend reads that passed their deadline.
- `SweepDeadMvccSnapshots`: walks the LRU list and trims dead-session MVCC read snapshots that have zero expiry.

23.9.3 Backlog Handling

The handler detects a backlog when the LRU cursor is not null after the cycle (work remains) or when the expiry heap reached its inspection limit while still finding entries to evict. On backlog detection, the handler calls `ScheduleFollowUpCollect()` to trigger another cycle without waiting for the next 500-operation countdown.

23.9.4 Eviction Safety Invariants

Two invariants protect correctness:

1. **Never evict a dirty entry.** An entry is dirty when `Revision > FlushedRevision`. Evicting a dirty entry would lose committed data that the background writer has not yet persisted. The eviction cycle skips dirty entries in every step.
2. **Never evict an entry with a live write intent.** A write intent means a transaction has prepared but not committed. Evicting such an entry would break the transaction's isolation guarantees. The `HasLiveWriteIntent` check also clears expired intents: if the intent has a nonzero Expires timestamp and the deadline passed, the method clears the intent and returns false (the entry is then evictable).

23.10 Failure Scenarios

Backend IO stall. When the storage backend cannot keep pace with read requests, the `FairReadScheduler` queue fills up. The scheduler throws `ReadBackpressureExceededException`, which propagates to the client as a retriable error. The client can retry after a delay or route the request to a different node. The IO stall does not compromise data integrity because the Raft log remains the source of truth.

Background writer lag. If the background writer falls behind the commit rate, the unflushed overlay grows. Reads remain correct because the overlay merges queued writes into every read path. The lag increases memory usage and may delay checkpoint creation (because partitions with unflushed writes are skipped). The system self-corrects when the write load decreases, because the background writer processes its queue in batches on each tick.

Dirty entry eviction attempt. The eviction cycle encounters a dirty entry and skips it. This is the intended behavior: the entry stays in memory until the background writer flushes it and the `FlushAck` advances `FlushedRevision`. The skip prevents data loss. If the cache is under heavy pressure and most entries are dirty, the actor remains over budget until the background writer catches up. This situation is self-correcting.

Flush retry exhaustion. The background writer retries each batch up to 5 times. If all retries fail, the batch is retained in `pendingLockItems` or `pendingKeyValuesItems` for the next flush cycle. No committed data is lost. The next cycle attempts the same batch again. If the backend remains unavailable, the unflushed overlay continues to serve the queued data to readers.

Pruned history floor corruption. The SQLite backend tracks the oldest surviving revision timestamp. If the floor becomes unreadable, the backend sets `FailClosedFloor` (maximum possible HLC values), which blocks all as-of checkpoints. This prevents an incomplete check-

point from being produced. The operator must repair or recreate the database to restore as-of checkpoint capability.

24 Recovery and Fault Tolerance

A distributed system is only as reliable as its recovery path. Kahuna can replicate writes across a majority of nodes, route around failed leaders, and split overloaded ranges. None of these capabilities matter if a restarted node loses committed data, or if a crashed coordinator leaves orphaned intents that block future writes forever. This chapter explains how Kahuna recovers from node failures, leader changes, and data loss. It covers Raft log replay, state restoration through dedicated restorer classes, transaction recovery with the presumed-abort rule, whole-partition state transfer for follower catch-up, and point-in-time recovery from backups.

The chapter builds on three earlier chapters. Chapter 17 introduced Raft consensus and the replication pipeline. Chapter 20 explained the two-phase commit protocol and durable intents. Chapter 22 described the persistence backends and the background writer. Recovery ties these subsystems together: it is the code path that reconstructs consistent state from the durable artifacts they produce.

24.1 The Startup Sequence

When a Kahuna node starts, the `ReplicationService` (a hosted `BackgroundService`) orchestrates the full startup sequence. The sequence has two paths: a normal start and a point-in-time recovery (PITR) bootstrap.

Normal start. The service calls `JoinCluster` on the Kahuna manager. Kommander opens the write-ahead log (WAL), discovers which partitions this node hosts, and replays committed log entries through the `OnLogRestored` callback. After replay completes for all partitions, the node begins to accept new Raft proposals and client requests.

PITR bootstrap. If the operator passes both `--join-existing` and `--pitr-backup-dir` flags, the service runs a PITR restore before it joins the cluster. The restore resolves the backup chain through `BackupCatalog.ResolveAndValidateAsync`, flushes persistence, and calls `BootstrapHelper.BootstrapNodeAsync` with the chain artifacts, the target timestamp, the persistence backend, and a WAL adapter. The helper installs the base snapshot and replays incremental WAL segments up to the target HLC timestamp. Once the bootstrap completes, the node joins the cluster normally. The PITR path is an offline operation: the node does not serve traffic during the restore.

24.2 Raft Log Replay

On startup, Kommander replays every committed log entry from the WAL in log-index order. For each entry, it calls `OnLogRestored` on the `ReplicationService`, which forwards the entry to the Kahuna manager. The manager uses `ReplicationLogRouter.OwnerOf` to determine which subsystem owns the entry: key-value entries go to `KeyValueReplicationDispatcher`, and lock entries go to the lock subsystem.

`KeyValueReplicationDispatcher` dispatches each replayed entry by its replication type. Six types exist:

1. **KV entries.** Forwarded to `KeyValueRestorer.Restore`. The restorer deserializes the `KeyValueMessage`, decodes the entry state, registers the entry as pending with the durability tracker, records it in the unflushed-writes overlay, and queues a `QueueStoreKeyValue`

command to the background writer. If the entry carries a transaction ID, the restorer also rebuilds the completion receipt from the transaction ID, key, and record anchor key.
2. **RangeMap entries.** Restored into `RangeMapStore` and synchronized with `KeySpaceRegistry`. These entries define the partition layout (range boundaries, generation numbers).
3. **SnapshotFloor entries.** Restored into `SnapshotFloorStore`. These entries represent active MVCC snapshot holds that constrain revision pruning.
4. **TransactionRecord entries.** Restored into `TransactionRecordStore`. These entries are the canonical commit/abort decisions for transactions.
5. **PreparedIntent entries.** Restored into `PreparedIntentStore`. These entries represent the prepare phase of a two-phase commit.
6. **CompletionReceipt entries.** Restored into `CompletionReceiptStore`. These entries prove that a transaction has been fully settled on a given partition.

The restore path and the live replication path share the same dispatcher but call different methods. Restore uses `Restore` methods, while live replication uses `Replicate` methods. The distinction matters because live replication applies deduplication through the `DurableApplyResultLedger` for transaction records and prepared intents. The restore path does not deduplicate because it replays entries in strict log order and each entry appears exactly once.

24.3 The KeyValueRestorer

`KeyValueRestorer` is the workhorse of key-value recovery. Its `Restore` method performs five steps for each replayed entry:

1. **Deserialize.** Parse the raw bytes into a `KeyValueMessage`.
2. **Decode state.** Convert the serialized state enum back to the internal representation.
3. **Register pending.** Notify the `DurabilityTracker` that a write is in flight for this partition. The tracker uses this information to report accurate durability watermarks to Kommander.
4. **Record in overlay.** Insert the entry into the unflushed-writes overlay so that reads can see it immediately, even before the background writer flushes it to the storage backend.
5. **Queue for persistence.** Enqueue a `QueueStoreKeyValue` command to the background writer actor. The background writer batches these commands and flushes them to the storage backend in bulk.

The restorer also rebuilds completion receipts when the entry carries a transaction ID. A completion receipt proves that a transaction result has been applied to a specific key on a specific partition. Rebuilding receipts during replay ensures that the `CompletionReceiptStore` is accurate after restart.

24.4 The Lock Restorer

Lock entries follow the same pattern. The lock subsystem replays each lock entry from the WAL and restores the lock state into the lock actor. Persistent locks survive restarts because they are replicated through Raft. Ephemeral locks (locks with a lease duration) do not survive restarts. When a node restarts, ephemeral lock leases expire naturally because the holder can no longer renew them. The lock actor does not need special expiration logic during recovery: the standard lease-expiry check handles the cleanup.

24.5 Leader Change Recovery

A leader change is a common event in a Raft group. It happens when the current leader crashes, becomes partitioned, or steps down. The new leader must reconstruct the state that the old leader held in memory before it can serve reads and accept writes.

Kommander fires `OnLeaderChanged` when a partition's leadership changes. The Kahuna manager forwards this event to the sequencer subsystem first, then to the key-value subsystem. The key-value dispatcher's `OnLeaderChanged` handler is intentionally a no-op. The reason is architectural: cache coherency on the new leader is maintained not by replaying state during the leader change, but by `InvalidateOrApply` messages that flow through the normal replication path. When the new leader begins to process proposals, the replication pipeline ensures that its in-memory state converges with the committed log.

The sequencer subsystem handles leader changes differently. When a partition gains leadership, its `SequenceActor` must reserve a new block of sequence values from the Raft log before it can issue new sequence numbers. This ensures monotonicity: the new leader never reissues a value that the old leader might have issued but not yet replicated.

24.6 Transaction Recovery

Transactions are the most complex recovery target because a transaction can span multiple partitions, and a coordinator crash can leave intents stranded on any of them. Kahuna uses the presumed-abort rule: any transaction without a committed decision record is presumed aborted.

24.6.1 The Recovery Sweep

`DurableTransactionRecovery` runs periodic sweeps on every leader partition. Each sweep calls `PreparedIntentStore.DueForRecovery` to find prepared intents that have passed their recovery deadline. The sweep groups these intents by transaction ID, epoch, and record anchor key, then calls `DecideAsync` on each group.

`DecideAsync` follows a decision tree:

1. **Look up the canonical record.** Call `LookupRecordDelegate` to read the transaction record from the anchor partition.
2. **Record exists and is committed.** Return true (commit). The intents will be materialized into key-value entries.
3. **Record exists and is aborted.** Return false (abort). The intents will be discarded.
4. **Record exists but is undecided, and the deadline has not passed.** Return null. The sweep leaves the intent alone and will revisit it on the next cycle. This avoids aborting a transaction whose coordinator is still alive and working.
5. **Record is undecided and past the deadline, or no record exists (orphaned prepare).** Construct an `AbortTransactionCommand` with `TransactionAbortClass.PresumedAbort` and drive it at the anchor partition. The abort is not guaranteed to win: if the coordinator commits concurrently, the commit wins the CAS race on the transaction record. The sweep returns whatever decision actually prevails.

After a decision is made, `ResolveGroupAsync` settles the intents. For a commit, each intent is materialized into a key-value entry through `PreparedIntentMaterializer.ToKeyValueRecord`, replicated through Raft, and applied lo-

cally. For an abort, all intents are immediately marked as settleable. In both cases, the method issues a combined resolve-and-remove delta through `PreparedIntentStore.SerializeDelta` via the Raft replication path. This ensures that all replicas apply the resolution in the same log order.

24.6.2 Targeted Helping

Besides the periodic sweep, `DurableTransactionRecovery` offers targeted helping through `TryResolveDecidedBlockersAsync`. When a transaction's finalize phase discovers that another transaction's intents block its keys, it calls this method. The helper looks up the canonical record for each blocking intent. If the record is terminal (committed or aborted), the helper settles the blocking intents immediately. If the record is undecided, the helper does not apply presumed-abort. Only the periodic sweep aborts undecided transactions, and only after the deadline passes.

24.6.3 Pre-Cutover Barrier for Range Splits

`SettleSuppliedIntentsAsync` serves a different purpose: it ensures that no unresolved intents cross a range boundary during a split or merge. Before the cutover step of a range operation, the system calls this method with all intents in the affected range. The method settles intents whose canonical decision is terminal. An undecided intent past its recovery deadline goes through the presumed-abort protocol. An intent still within its deadline is left alone. The method returns the count of unsettled intents. A zero count means the range can proceed to the copy/cutover step.

24.7 Whole-Partition State Transfer

When a follower's WAL has been compacted below the entries it needs, the follower cannot catch up through normal log replication. Kommander triggers a whole-partition state transfer instead. Two transfer implementations exist: one for user data partitions and one for the meta partition.

24.7.1 User Data Partitions

`PartitionStateTransfer` implements `IRaftPartitionStateTransfer`. The export side drains the background writer first to ensure all committed state is on disk, then pages out all data belonging to the partition: key-value rows (via `PartitionDataEnumerator`), persistent locks, completion receipts, transaction records, and prepared intents. Each page carries an FNV-1a 64-bit checksum. The snapshot reflects at least the requested log index; newer state is allowed.

The import side uses a two-phase install process:

1. **Verify phase.** Read and checksum-verify the entire stream (header, key-value pages, lock pages, store sections) before changing any data. A corrupt or truncated snapshot is a clean no-op.
2. **Install phase.** Acquire the per-partition `installGate` semaphore (mutual exclusion with unhost-purge operations). Write a durable install marker. Drain persistence. Purge existing backend rows for the partition. Apply all key-value rows, lock rows, completion receipts, transaction records, and prepared intents. Persist per-partition store snapshots. Run resident-state invalidation hooks to drop cached lock leases and key-value entries. Clear the install marker only on full success.

A crash during the install phase leaves the durable marker file (`partition-install-{partitionId}_{storageRevision}.incomplete`) on disk. On the next startup, `IsInstallIncomplete` detects the marker, and the sender retries the entire transfer from scratch. The marker ensures that a partial install never produces a corrupt partition.

`PurgeUnhostedPartitionAsync` removes all data for a partition that this node no longer hosts. It deletes backend rows, store slices, the durability floor, and the install marker. The purge is serialized against imports through the same `installGate` semaphore. If the partition is re-gained during the purge, the operation aborts.

24.7.2 Meta Partition

`MetaSystemStateTransfer` implements `IRaftSystemStateTransfer`. The meta partition hosts two state machines: `RangeMapStore` (partition layout) and `SnapshotFloorStore` (MVCC hold registry). The export side serializes both stores into a single `MetaSystemStateMessage`. The import side deserializes and validates both sub-states before mutating either store. This atomicity guarantee prevents a state where one store is updated and the other is stale. Both stores are idempotent under log-tail replay.

24.8 Snapshot Holds and Revision Safety

The `SnapshotFloorStore` is a replicated, reference-counted, leased registry of MVCC snapshot holds. It lives on the meta partition. Each hold pins a minimum HLC timestamp: the revision pruning system will not delete any revision at or after the held timestamp.

Mutations replicate as keyed deltas on the meta partition through the `SnapshotFloor` replication type. The deltas are idempotent by hold ID. Three operations exist:

- `AcquireAsync` creates a new hold with a lease duration.
- `RenewAsync` extends the lease of an existing hold.
- `ReleaseAsync` removes a hold.

All three are leader-only operations, serialized locally by a `mutateGate` and globally by the meta Raft log.

Two safety mechanisms protect against stale holds:

1. **Lease expiry.** `PurgeExpiredHoldsAsync` runs periodically and removes holds whose lease has elapsed. This prevents a crashed holder from pinning MVCC history forever.
2. **Prune-delete window.** `BeginPrune` and `EndPrune` define a window during which an acquire fails closed with `MustRetry`. This prevents a new hold from being created in the gap between when the pruner reads the floor and when it deletes old revisions.

The store persists its full hold set to `snapshotfloor_{storageRevision}.snapshot` through atomic tmp-and-rename. On cold restart, `LoadFromDisk` reconstructs the hold set from this snapshot file.

24.9 PITR Backup and Restore

Chapter 27 covers backup operations in detail. This section explains only the recovery-relevant mechanics.

`BackupFacade` wraps `BackupService` with error mapping and node wiring. A node without a configured `BackupDir` receives `BackupFacade.Disabled()`, which answers `IsConfigured = false` to all callers.

The facade wires four barriers into the backup pipeline:

1. **Flush barrier.** The `flushPersistenceAsync` delegate forces the background writer to flush all queued writes before the checkpoint capture begins.
2. **Applied-HLC probe.** The backup waits until the background writer's watermark catches up with the committed-write watermark for each partition.
3. **MVCC snapshot hold.** The facade acquires a snapshot hold with a 600-second lease through `KeyValuesManager.AcquireSnapshotHold`. This hold pins revision history at the backup cut so that pruning cannot reclaim revisions during the checkpoint, hash, verify, and publish window. The hold is renewed at roughly one-third of the lease interval. A failed renewal aborts the backup.
4. **WAL retention hold.** The facade acquires a retention hold through `raft.AcquireRetentionHold` to prevent WAL compaction of incremental prefixes during the read.

The retention policy is configurable: `MaxChains` limits the number of retained backup chains, `MaxAge` limits how far back chains can reach, and `MaxTotalBytes` caps total storage. A periodic `BackupGcReaperActor` enforces the policy when `BackupGcInterval` is greater than zero.

Backup operations include `TakeFullAsync`, `TakeIncrementalAsync`, `TakeCoordinatedAsync`, `ListAsync`, `GetChainAsync`, and `RestoreToAsync`. All operations return typed `KahunaBackupOutcome` values. Failure outcomes include `NeedsFull`, `ParentMissing`, `TargetConflict`, `TargetOutsideCoverage`, `TopologyChanged`, `RetryableLeadershipLoss`, `CorruptArtifact`, `CorruptChain`, and `IoError`.

24.10 Durability Tracking

`KahunaDurabilityProvider` implements Kommander's `IApplicationDurabilityProvider` interface. It reports the highest log index whose state has reached durable storage for a given partition. Kommander uses this information to decide when it can compact the WAL: entries below the durability floor are safe to discard because the application can reconstruct them from the storage backend.

The provider returns the maximum of two values:

1. **Live watermark.** The `PartitionDurabilityTracker` tracks in-flight writes registered by the restorer and the replication path. When the background writer confirms a flush, the tracker advances the watermark.
2. **Persisted floor.** The `IPersistenceBackend.GetDurabilityFloor` method returns the floor stored on disk. This value is read once per partition at startup and cached.

`Forget(partitionId)` evicts the cached floor when a partition is un-hosted. This prevents a stale floor from vouching for data that was purged during a replica move.

24.11 Failure Scenarios

Node crash and restart. Kommander replays the WAL. Restorers rebuild in-memory state. The background writer flushes any entries that did not reach the storage backend before the

crash. The durability tracker reports accurate watermarks after the flush completes. The node is ready to serve traffic.

Leader change with in-flight transactions. The old leader's uncommitted proposals are lost. The new leader's `DurableTransactionRecovery` sweep finds orphaned intents on its partitions once their recovery deadline expires. The sweep looks up each transaction's canonical record. If no record exists, the sweep presumes abort and drives an abort at the anchor partition. Committed transactions are materialized normally.

Incomplete state transfer. If a node crashes during a partition state transfer, the durable install marker remains on disk. On restart, `IsInstallIncomplete` detects the marker. The sending node retries the entire transfer. The verify-then-install sequence ensures that no corrupt data contaminates the partition.

WAL corruption. If the WAL on a single node is corrupt, Raft recovers the node through state transfer from a healthy peer. The peer exports the full partition state with checksums. The recovering node verifies the checksums before installing the state.

Abandoned transactions. A coordinator process can crash after preparing intents but before writing a decision record. The intents become orphans. The recovery sweep on each participant partition detects the orphaned intents after the recovery deadline. Because no decision record exists, the sweep applies presumed-abort. The abort command is replicated through Raft so all replicas converge on the same decision.

Split with unresolved intents. Before a range split proceeds to the cutover step, the system calls `SettleSuppliedIntentsAsync` to resolve all intents in the affected range. The method settles terminal intents immediately. Undecided intents past their deadline are aborted through the presumed-abort protocol. Intents still within their deadline are left alone, and the split waits. Only when the unsettled count reaches zero does the split proceed.

25 Deployment and Cluster Sizing

A Kahuna cluster that is configured correctly can tolerate node failures, sustain high throughput, and recover without operator intervention. A cluster that is configured incorrectly (an even number of nodes, no HTTPS, a replication factor that exceeds the node count) will fail in ways that are difficult to diagnose under pressure. This chapter explains how to deploy Kahuna, how to choose the right number of nodes, and how to configure HTTPS, storage paths, and replication.

The chapter builds on several earlier chapters. Chapter 16 introduced partitions and the meta partition. Chapter 17 explained Raft consensus and the quorum rule. Chapter 22 described the persistence backends. Chapter 23 covered recovery. Deployment ties these concepts together into a set of practical decisions: how many nodes, how many replicas, what storage backend, and what network topology.

25.1 Standalone Mode

Standalone mode runs a single Kahuna node with no peers. It is suitable for local development, testing, and evaluation. To start a standalone node, install the .NET global tool and run it with no arguments:

```
dotnet tool install -g Kahuna.Server
kahuna-server
```

The server starts on HTTP port 2070 and binds cleartext gRPC on port 2072 by default. It creates three partitions by default (the `--initial-cluster-partitions` flag overrides this value). Data is stored in the default home directory:

- Linux and macOS: `~/.local/share/kahuna`
- Windows: `%LOCALAPPDATA%\kahuna`

The `KAHUNA_HOME` environment variable overrides the root directory. Within that root, Kahuna creates two subdirectories: `data/` for the storage backend and `wal/` for the write-ahead log. The `--storage-path` and `--wal-path` flags override these paths individually.

Standalone mode works by creating phantom witness nodes. The `EmbeddedKahunaNode` class generates phantom peers so that the single node satisfies the Raft majority-quorum requirement (two out of three) without real network traffic. This design means the node uses the same Raft code path as a multi-node cluster. There is no separate single-node protocol.

25.1.1 Storage Backend Selection

Standalone mode uses RocksDB by default. The `--storage` flag selects the backend:

- `rocksdb` (default): persistent, production-grade, column-family isolation.
- `sqlite`: persistent, 8-shard parallel writes, simpler operational footprint.
- `memory`: volatile, fastest for tests, data is lost on restart.

The `--wal-storage` flag selects the write-ahead log backend independently. Both flags accept the same three values.

25.2 Multi-Node Cluster

A multi-node cluster requires three things: an odd number of nodes, a list of peer addresses, and HTTPS certificates for inter-node communication. Each node must know the addresses of its peers at startup.

25.2.1 Static Discovery

Kahuna uses static discovery. Each node receives the addresses of its peers through the `--initial-cluster` flag. The flag accepts one or more peer URLs. A node does not list itself: it lists the other nodes.

For a three-node cluster on separate hosts:

```
# Node 1 (host1)
kahuna-server \
  --raft-nodename kahuna1 --raft-nodeid 1 \
  --raft-host 0.0.0.0 --raft-port 2070 \
  --https-ports 2071 --https-certificate /path/to/cert.pfx \
  --initial-cluster https://host2:2071 https://host3:2071

# Node 2 (host2)
kahuna-server \
  --raft-nodename kahuna2 --raft-nodeid 2 \
  --raft-host 0.0.0.0 --raft-port 2070 \
  --https-ports 2071 --https-certificate /path/to/cert.pfx \
  --initial-cluster https://host1:2071 https://host3:2071

# Node 3 (host3)
kahuna-server \
  --raft-nodename kahuna3 --raft-nodeid 3 \
  --raft-host 0.0.0.0 --raft-port 2070 \
  --https-ports 2071 --https-certificate /path/to/cert.pfx \
  --initial-cluster https://host1:2071 https://host2:2071
```

Each node receives a unique `--raft-nodename` and `--raft-nodeid`. The node name is a human-readable label. The node ID is a numeric identifier that Raft uses internally. Both must be unique across the cluster.

25.2.2 Joining an Existing Cluster

The `--join-existing` flag tells a node to join a running cluster through seed-based discovery. Instead of forming a new cluster, the node contacts the seed addresses listed in `--initial-cluster` and requests membership. This path is for adding nodes to a cluster that is already operational.

```
kahuna-server \
  --raft-nodename kahuna4 --raft-nodeid 4 \
  --raft-host 0.0.0.0 --raft-port 2070 \
  --https-ports 2071 --https-certificate /path/to/cert.pfx \
  --initial-cluster https://host1:2071 https://host2:2071 \
  --join-existing
```

The `ReplicationService` detects the `--join-existing` flag and calls `JoinCluster(seeds, stoppingToken)` on the Raft manager. The seed-based path sends a membership request to the existing cluster. The cluster's current leader processes the request and adds the new node as a member. Once accepted, the new node receives the partition map and begins to participate in replication.

25.2.3 Graceful Leave

The `--graceful-leave-on-shutdown` flag commits a `RemoveMember` operation when the node shuts down. This tells the cluster to remove the node from the membership roster. Do not enable this flag during rolling restarts: a rolling restart temporarily removes and re-adds each node, which causes unnecessary partition re-assignments.

25.3 Docker Compose Deployment

The Kahuna repository includes a Docker Compose file (`docker/local.yml`) that defines a three-node cluster. This section walks through its structure.

25.3.1 Image Structure

The Docker images use a two-stage build. The build stage compiles the source on the `mcr.microsoft.com/dotnet/sdk:10.0` image. The runtime stage copies the compiled output to the `mcr.microsoft.com/dotnet/aspnet:10.0` image. The runtime image is smaller because it does not include the SDK.

The standalone image (`Dockerfile.standalone`) is published to Docker Hub as `kahunakv/kahuna`. It exposes two ports: 8081 (HTTP) and 8082 (HTTPS). It creates a `/data` volume with `data/` and `wal/` subdirectories. The entrypoint script starts the server in standalone mode with no `--initial-cluster` flag.

The cluster image (`DockerfileLocal`) accepts build arguments for node identity, ports, and the peer list. The entrypoint command passes these values as command-line flags:

```
dotnet /app/Kahuna.Server.dll \
  --raft-nodename $KAHUNA_RAFT_NODENAME \
  --raft-nodeid $KAHUNA_RAFT_NODEID \
  --raft-host $KAHUNA_RAFT_HOST \
  --raft-port $KAHUNA_RAFT_PORT \
  --http-ports $KAHUNA_HTTP_PORTS \
  --https-ports $KAHUNA_HTTPS_PORTS \
  --https-certificate /app/certificate.pfx \
  --initial-cluster $KAHUNA_INITIAL_CLUSTER \
  --storage rocksdb \
  --storage-path /storage/data \
  --wal-storage rocksdb \
  --wal-path /storage/wal
```

25.3.2 The Compose File

The `local.yml` file defines three services (`kahuna1`, `kahuna2`, `kahuna3`) on a bridge network with static IP addresses (`172.30.0.2`, `172.30.0.3`, `172.30.0.4`). Each node maps two host ports (HTTP and HTTPS) to the container's internal ports 2070 and 2071:

Service	HTTP (host)	HTTPS (host)	Static IP
kahuna1	8081	8082	172.30.0.2
kahuna2	8083	8084	172.30.0.3
kahuna3	8085	8086	172.30.0.4

Each service uses a named volume (for example, `kahuna1-data`) mounted at `/storage`. This volume persists RocksDB data and WAL files across container restarts.

Each node's `--initial-cluster` flag lists the HTTPS addresses of the other two nodes, using their static IPs and internal port 2071. Node 1, for example, receives `https://172.30.0.3:2071 https://172.30.0.4:2071`.

To start the cluster:

```
docker compose -f docker/local.yml up -d
```

Log levels are configurable through environment variables: `KAHUNA_SERVER_DEFAULT_LOG_LEVEL` controls the root logger level, and `KAHUNA_SERVER_LOG_LEVEL` controls the Kahuna-specific logger level.

25.3.3 Health and Readiness

After starting the containers, verify that all nodes are ready. Kahuna exposes a health endpoint at `GET /v1/cluster/health`. The endpoint returns HTTP 200 when the node is ready to serve requests, or HTTP 503 when the node is still initializing.

```
curl -k https://localhost:8082/v1/cluster/health
```

The response body is a JSON object with four fields:

```
{
  "ready": true,
  "initialized": true,
  "localRole": "Voter",
  "hostedPartitions": 16
}
```

The `ready` field is true when all three readiness conditions below are met. The `localRole` field shows the node's current membership role. The `hostedPartitions` field reports the number of partitions assigned to this node.

A node can open its port and respond to HTTP requests roughly one second after launch. However, it refuses every key-value request until cluster initialization completes (partition map received and applied). This window can last tens of seconds after a restart.

The readiness logic checks three conditions:

1. The Raft engine is initialized (the cluster has formed and the partition map is received).
2. The node's local role is not `NotMember` (the node has not been evicted).
3. The node's local role is not `Leaving` (the node is not in the process of decommissioning).

For deeper readiness verification, the CI scripts use a two-phase probe. Phase one checks HTTP liveness with a simple connection test. Phase two sends a `try-lock` request and waits

for any response other than `MustRetry`. A non-retry response confirms that leader election is complete and the node can process writes.

25.3.4 Additional Cluster Endpoints

The cluster API provides several informational endpoints:

- `GET /v1/cluster/membership` returns the roster of cluster members, including their roles, versions, and connectivity state.
- `GET /v1/cluster/placement` returns the partition map: which partitions exist, which nodes host replicas of each partition, and the current replication factor.
- `POST /v1/cluster/replication-factor` adjusts the replication factor for individual partitions (see the Replication Factor section below).
- `POST /v1/cluster/leave` initiates a graceful decommission of the node that receives the request.

25.4 Kubernetes Deployment

The Kahuna Kubernetes Operator automates deployment and lifecycle management of Kahuna clusters on Kubernetes. The operator is an alpha-stage project, hosted in a separate repository (`kahunakv/kahuna-k8s-operator`). It provisions clusters, scales them, and manages their lifecycle through a custom resource definition (CRD).

The operator handles the concerns that Docker Compose leaves to the operator: stable network identities (through StatefulSets), persistent volume claims, rolling upgrades, and automated seed-list management. Because it is alpha software, consult the operator's repository for current installation instructions and supported features.

25.5 HTTPS Configuration

Kahuna uses HTTPS for two purposes: client-to-node communication and node-to-node (inter-cluster) communication. Three command-line flags control HTTPS:

- `--https-ports`: one or more ports to bind for HTTPS. Default: 2071 when a certificate is provided.
- `--https-certificate`: path to a PFX (PKCS#12) certificate file.
- `--https-certificate-password`: password for the PFX file, if encrypted.

When no certificate is provided, the node runs in HTTP-only mode. If `--https-ports` is specified without `--https-certificate`, the server exits with a validation error.

25.5.1 Inter-Node Communication Scheme

Two flags control the URI scheme for inter-node traffic:

- `--raft-http-scheme`: the scheme for REST-based inter-node calls. Default: `https://`.
- `--raft-grpc-scheme`: the scheme for gRPC-based inter-node calls. Default: `https://`.

In development and testing environments, the `--raft-allow-insecure-certificate-validation` flag disables certificate validation for all inter-node traffic. This flag allows self-signed certificates to work without configuring a certificate authority. Do not use this flag in production.

25.5.2 Certificate Validation

When a certificate is loaded, Kahuna extracts the certificate thumbprint and stores it as the `HttpsTrustedThumbprint` in the Raft configuration. This thumbprint can be used for certificate pinning in inter-node communication.

For production deployments, use certificates signed by a trusted certificate authority. If you use self-signed certificates, distribute the CA root certificate to all nodes and configure the operating system's trust store. The `--raft-allow-insecure-certificate-validation` flag is a development convenience, not a production solution.

25.5.3 Cleartext gRPC

The `--grpc-cleartext-ports` flag binds HTTP/2 listeners without TLS. These ports speak gRPC only (HTTP/2 cleartext, also called h2c). REST clients must use the HTTP or HTTPS ports.

In standalone mode, port 2072 is bound for cleartext gRPC by default. In a multi-node cluster, no cleartext gRPC port is bound unless the flag is set explicitly.

Cleartext gRPC is useful in two scenarios:

- The TLS termination runs on a load balancer or sidecar proxy in front of Kahuna. The proxy handles encryption. Kahuna receives cleartext traffic from the proxy.
- Local development, where TLS adds complexity without security benefit.

Do not expose cleartext gRPC ports to untrusted networks. The traffic carries no encryption and no authentication.

25.5.4 Mutual TLS

Mutual TLS (mTLS) adds certificate-based authentication to inter-node communication. When mTLS is enabled, each node presents a client certificate and validates the peer's certificate before it accepts inter-node RPCs. The `NodeTransportGate` enforces this policy on every peer-only RPC method: 2PC participant calls, inter-node batching streams, and Raft transport.

Configure mTLS with the `--raft-transport-security` JSON flag. The JSON object must include:

- `NodeAuthenticationMode`: set to `MutualTls`.
- `CertificatePath`: path to the node's TLS certificate (PEM or PFX).
- `KeyPath`: path to the private key.
- `TrustedCaPaths`: an array of CA certificate paths that the node trusts.

```
kahuna-server \
                                                    --raft-transport-
security   '{"NodeAuthenticationMode":"MutualTls","CertificatePath":"/certs/
node.pem","KeyPath":"/certs/node.key","TrustedCaPaths":["/certs/ca.pem"]}' \
  --https-ports 2071
```

The trust decision uses the same `RaftTransportAuthenticator.ValidatePeerCertificate` method that Raft uses. This ensures that Raft traffic and Kahuna's inter-node traffic agree on peer identity.

mTLS operates independently from HTTPS client certificates. A client connects over HTTPS with its own certificate (if required by the listener). The `NodeTransportGate` checks only inter-node callers.

25.6 Cluster Sizing

Cluster sizing determines how many node failures the system can tolerate while it continues to serve reads and writes. The sizing decision follows directly from the Raft quorum rule.

25.6.1 The Quorum Rule

A Raft group requires a majority of its voting members to agree before it commits a log entry. For a group with N voting members, the majority is `floor(N/2) + 1`. The group can tolerate `N - floor(N/2) - 1` failures.

Nodes	Quorum	Tolerated failures
1	1	0
3	2	1
5	3	2
7	4	3

25.6.2 Why Odd Node Counts

An even-number cluster wastes resources without improving fault tolerance. A cluster of 4 nodes requires a quorum of 3 and tolerates 1 failure, the same as a cluster of 3 nodes. The fourth node adds cost (CPU, memory, disk, network) without increasing availability.

Worse, an even-number cluster increases the risk of a tie during a network partition. If a 4-node cluster splits into two groups of 2, neither group has a majority. Both halves become unavailable. A 3-node cluster that loses one node still has a group of 2, which is a majority. The remaining two nodes continue to serve requests.

For these reasons, always deploy an odd number of nodes: 3 for most workloads, 5 for higher fault tolerance, 7 for environments that require tolerance of 3 simultaneous failures.

25.6.3 The SWIM Failure Detector

Kahuna uses the SWIM protocol (Scalable Weakly-consistent Infection-style Process Group Membership) for failure detection. SWIM runs as a protocol layer within Kommander. It detects node failures and removes failed nodes from the membership roster.

The failure detection pipeline has four stages:

1. **Ping.** Each node sends a direct ping to a randomly selected peer on every ping interval (`--raft-ping-interval`, default 1000ms). If the peer responds within the ping timeout (`--raft-ping-timeout`, default 500ms), it is marked healthy.
2. **Indirect ping.** If the direct ping fails, the node asks a small number of other peers (`--raft-indirect-ping-fanout`, default 2) to ping the suspect on its behalf. This step distinguishes a failed node from a network path failure.

3. **Suspicion.** If both direct and indirect pings fail, the node enters the suspect state. It remains suspect for the suspicion timeout (`--raft-suspicion-timeout`, default 5000ms). If it responds during this window, it returns to healthy.
4. **Eviction.** If the node remains unresponsive past the suspicion timeout, it is marked dead. After a grace period (`--raft-dead-member-eviction-grace`, default 30000ms), it is evicted from the membership roster.

The `--raft-enable-auto-rejoin` flag (default true) allows an evicted node to rejoin the cluster automatically when it comes back online.

25.6.4 Gossip

Membership state propagates through gossip. Each node periodically selects a random subset of peers and exchanges membership information. Two flags control gossip:

- `--raft-gossip-interval` (default 5000ms): how often a node initiates a gossip round.
- `--raft-gossip-fanout` (default 2): how many peers receive each gossip message.

Gossip ensures that membership changes (joins, leaves, evictions) reach all nodes within a bounded number of rounds, even in the absence of a centralized coordinator.

25.7 Replication Factor

The replication factor (RF) controls how many voter replicas exist for each partition range. Two modes of replication exist.

25.7.1 Full Replication

The default replication factor is 0, which means full replication. In this mode, every voter node hosts every partition range. A 3-node cluster with RF 0 stores three copies of every partition. This mode is the simplest to operate because every node can serve reads for every partition. Full replication is suitable for clusters with a small to moderate number of nodes (3 to 7).

25.7.2 Selective Replication

When the cluster grows beyond a handful of nodes, full replication becomes expensive. Setting `--raft-replication-factor` to a positive integer enables selective replication. Each partition range receives a replica set of exactly that size. Quorum is computed per range over its voter replicas only.

For example, in a 9-node cluster with RF 3, each partition range is stored on 3 of the 9 nodes. The quorum for each range is 2. A single node failure does not affect any range (each range still has 2 of 3 replicas). Two node failures affect only the ranges that had replicas on both failed nodes.

Prefer odd replication factors for the same reason as odd node counts: an even RF wastes a replica without improving fault tolerance.

25.7.3 Validation

Kahuna validates the replication factor at startup:

- A negative RF is rejected.
- An even RF produces a warning. The system starts but logs a recommendation to use an odd value.

- An RF that exceeds the number of seed nodes produces a warning. The system starts but cannot place the requested number of replicas until more nodes join.

25.7.4 Zone-Aware Placement

The `--raft-zone` flag assigns a zone label to a node (for example, `us-east-1a`). The replica placement algorithm uses zone labels to spread replicas across failure domains. If three nodes are in zone A and three are in zone B, the placement algorithm avoids putting all replicas of a partition in the same zone. Zone-aware placement requires at least as many distinct zones as the replication factor.

25.8 Storage Path Configuration

Production deployments should place the data directory and the WAL directory on separate storage devices when possible.

25.8.1 Path Flags

Three mechanisms control storage paths, in order of precedence:

1. `--storage-path` and `--wal-path` flags override individual directories.
2. `KAHUNA_HOME` environment variable overrides the root directory. Data goes to `$KAHUNA_HOME/data` and WAL goes to `$KAHUNA_HOME/wal`.
3. Default home directory (`~/.local/share/kahuna` on Linux and macOS, `%LOCALAPPDATA%\kahuna` on Windows).

25.8.2 Storage Revision

The `--storage-revision` flag (default `v1`) and `--wal-revision` flag (default `v3`) append a version suffix to the storage path. These flags allow a node to maintain separate storage directories for different schema versions. During an upgrade, a new revision can coexist with the old one until migration completes.

25.8.3 RocksDB Shared Memory

When both the storage backend and the WAL backend use RocksDB, the `--rocksdb-shared-memory` flag enables a shared memory pool. This pool unifies the block cache and write buffer manager across both RocksDB instances. Two flags control the pool size:

- `--rocksdb-shared-memory-budget-mb` (default 320 MiB): the total budget for the shared block cache.
- `--rocksdb-shared-memtable-budget-mb` (default 128 MiB): the budget for the shared write buffer manager.

Sharing memory prevents the two RocksDB instances from competing for the operating system's page cache. Without sharing, each instance manages its own buffer pool, and the combined memory usage can exceed the intended limit.

25.8.4 Thread Pool Tuning

Kahuna sets the .NET thread pool minimum to 256 worker threads and 128 I/O completion threads at startup. This prevents the thread pool from throttling under burst load. In the default .NET configuration, the thread pool starts with a small number of threads and ramps up slowly (one thread per 500ms). The high minimum ensures that the server can handle concurrent requests immediately.

25.9 Failure Scenarios

This section describes four failure scenarios and their effects on a correctly configured cluster.

25.9.1 Single-Node Failure in a Three-Node Cluster

When one node fails in a 3-node cluster, the remaining two nodes form a quorum (2 out of 3). All partitions that had the failed node as their leader elect a new leader from the surviving nodes. Client reads and writes continue without interruption, though latency may increase briefly during leader election. The SWIM failure detector marks the failed node as suspect, then dead, and finally evicts it after the grace period.

25.9.2 Two-Node Failure in a Three-Node Cluster

When two nodes fail, the single surviving node cannot form a quorum (1 out of 3). All Raft groups lose quorum. Reads of committed data may still succeed if the node serves stale reads, but writes become unavailable. The cluster remains in this state until at least one of the failed nodes recovers.

25.9.3 Even-Number Split Brain

A 4-node cluster that experiences a network partition into two groups of 2 loses quorum on both sides. Neither group can commit writes. A 3-node cluster partitioned into groups of 2 and 1 continues to operate on the side with 2 nodes (the majority). This scenario is the primary reason to avoid even node counts.

25.9.4 Under-Replicated Partition

An under-replicated partition has fewer live replicas than the configured replication factor. This happens when nodes fail and the cluster has not yet re-replicated the data to replacement nodes. The partition continues to serve requests as long as a quorum of its remaining replicas is intact. However, the partition is at higher risk: one more failure could cause quorum loss for that specific range.

The `GET /v1/cluster/placement` endpoint shows the current replica set for each partition. Monitor this endpoint to detect under-replicated partitions and take corrective action (restart failed nodes or add new nodes) before additional failures occur.

25.10 Production Checklist

Before deploying Kahuna to production, verify these items:

1. **Odd node count.** Deploy 3, 5, or 7 nodes. Never deploy an even number.
2. **HTTPS enabled.** Provide a valid PFX certificate to every node. Do not use `--raft-allow-insecure-certificate-validation` in production.
3. **Separate storage paths.** Place `--storage-path` and `--wal-path` on separate disks when possible. Use durable storage (SSD or NVMe), not ephemeral volumes.
4. **Replication factor.** Set an odd RF that does not exceed the node count. Use RF 0 (full replication) for small clusters, or RF 3 or 5 for larger clusters.
5. **Health monitoring.** Poll `GET /v1/cluster/health` on every node. Alert when any node returns 503 for more than 60 seconds.
6. **Placement monitoring.** Poll `GET /v1/cluster/placement` periodically. Alert on under-replicated partitions.

7. **Named volumes.** Use named Docker volumes or persistent volume claims in Kubernetes. Do not use ephemeral container storage for data or WAL directories.
8. **Graceful shutdown.** Do not enable `--graceful-leave-on-shutdown` during rolling restarts. Enable it only for permanent decommission.
9. **Dashboard.** Kahuna serves a browser-based operator dashboard at the HTTP root (`/`). Pass `--disable-dashboard` on hardened or headless nodes. Control the poll interval with `--dashboard-refresh-seconds` (default 5, range 1 to 300). Chapter 28 covers the dashboard panels in detail.

26 Cluster Membership and Scaling

A production cluster changes over time. Operators add nodes for capacity, remove nodes for maintenance, and replace nodes after hardware failures. Each of these operations changes the set of machines that participate in consensus. An incorrect membership change can cause quorum loss, data unavailability, or unnecessary state transfers. This chapter explains how Kahuna tracks cluster membership, how nodes join and leave, how the failure detector works, and how to perform rolling restarts safely.

The chapter builds on two earlier chapters. Chapter 17 introduced Raft consensus, leader election, and the quorum rule. Chapter 24 covered deployment modes and cluster sizing. Membership management is the runtime counterpart to those static decisions: it governs what happens when the cluster must grow, shrink, or heal while it serves traffic.

26.1 The Committed Roster

Kahuna stores cluster membership in a versioned roster on the system partition (partition zero). The roster is a Raft-committed data structure, so it inherits the linearizability guarantees of the system partition. The `ClusterMembership` class holds two fields: a `MembershipVersion` counter and a list of `ClusterMember` entries.

Every mutation (add, promote, remove, or role change) increments `MembershipVersion` by exactly one. Callers carry the version they read, so the `RaftSystemCoordinator` can detect stale writes and reject them. Gossip and discovery never mutate the roster directly. They inform the system about node liveness, but only committed Raft entries change who is in the cluster.

Each `ClusterMember` entry carries four fields:

1. **Endpoint.** The `host:port` address that matches the node's `RaftNode` identifier.
2. **NodeId.** A numeric identifier assigned at startup.
3. **Role.** One of `Learner`, `Voter`, `Leaving`, or `NotMember`.
4. **JoinedVersion.** The `MembershipVersion` at which this node first entered the roster as a Learner.

The `NotMember` role is never stored in a roster entry. It exists only as a return value from `RaftManager.LocalRole` when the local node does not appear in the committed roster at all.

26.2 Member Roles

A node moves through a defined set of roles during its lifetime in the cluster:

- **Learner.** The node receives replication traffic but does not count toward quorum. It cannot win elections or vote. A new node always enters the roster as a Learner.
- **Voter.** The node counts toward quorum. It can vote in elections and can become a leader. Promotion from Learner to Voter is automatic once the node catches up.
- **Leaving.** The node is being decommissioned. It no longer counts toward the roster-level quorum or placement candidacy, but it stays in the peer list. Heartbeats and replication keep flowing so that learner replicas catching up from it can still read its data. The transition is reversible: a drain that times out or is refused rolls the role back to Voter.
- **NotMember.** The node is absent from the committed roster. It was either evicted by the failure detector, removed by a graceful leave, or it never joined.

Liveness state (Alive, Suspect, Dead) lives in the gossip layer, not in the roster, so it never churns the Raft log.

26.3 Adding a Node

A new node joins an existing cluster by starting with the `--join-existing` flag and pointing `--initial-cluster` at one or more seed endpoints. The `ClusterJoinService` orchestrates the entire join sequence.

26.3.1 The Seed Join Flow

1. **Contact seeds.** The joining node sends a `JoinRequest` to each seed endpoint in turn. If the seed is not the system-partition leader, it returns a leader hint. The joining node follows the hint and retries.
2. **Leader admits Learner.** When the `JoinRequest` reaches the system-partition leader, the leader's `ReceiveJoin` method commits an `AddMember` entry to the system-partition Raft log. The new node enters the roster with role `Learner` and the current `MembershipVersion`.
3. **Start system partition.** The joining node starts its local system partition and marks itself as joined. From this point, the system-partition leader begins replicating log entries to the new node.
4. **Wait for initialization.** The joining node polls until the system coordinator receives and applies the partition map. This step completes when the node's data partitions are started.
5. **Wait for promotion.** The joining node polls until its local role changes from `Learner` to `Voter`. If the leader determines that promotion is permanently blocked (for example, the learner is below the WAL compaction floor and no snapshot transfer is registered), it signals the joining node through `SetJoinTerminalReason`. The joining node then fails fast with a descriptive error instead of waiting for the 60-second timeout.

The join flow uses a hard ceiling of 60 seconds. If the node does not reach Voter status within that window, it throws a `TimeoutException` with a diagnostic snapshot that includes the system-partition leader, the WAL frontier, the count of data partitions started, and the initialization flag.

26.3.2 Idempotent Admission

The `ReceiveJoin` handler is idempotent. If the joining node's endpoint is already in the roster (because a previous `AddMember` committed but the response was lost), the handler returns success with the current roster version. This prevents the joining node's retry loop from wasting the full 60-second timeout on a node that is already admitted.

26.4 Learner Promotion

The system-partition leader runs `CheckLearnerPromotionsAsync` on every `UpdateNodes` timer tick. This method measures the per-partition replication lag for each Learner in the roster.

26.4.1 The Lag Check

For each Learner, the method iterates over all partitions (system and data). For partitions that this node leads, the method reads the Learner's committed index directly from `lastCommitIndexes`. For partitions led by another node, the method queries that node via `GetRemoteFollowerLag`.

A Learner is considered "caught up" when its lag on every checked partition is within `LearnerPromotionLag` entries of the leader. The default value of `LearnerPromotionLag` is 10.

26.4.2 The Stable Window

A single lag check that passes is not enough. The Learner must remain within the lag threshold for a continuous period defined by `LearnerPromotionStableWindow`. The default value is 3 seconds.

The method tracks the first moment each Learner fell within the lag threshold in a dictionary keyed by endpoint. If the Learner stays caught up for the full stable window, the method sends a `PromoteMember` request to the system coordinator. If the Learner falls behind at any point, the stable window resets.

26.4.3 One at a Time

At most one membership change is in flight at a time. After promoting one Learner, the method returns immediately. The next timer tick handles any remaining Learners. This serialization prevents overlapping membership changes from conflicting on the system-partition log.

26.4.4 Terminal Blocks

If a Learner's committed index is below the WAL compaction floor on any partition, and no snapshot transfer is registered for that partition, the Learner can never catch up through log replay alone. The leader sets a terminal reason for the joining node so that the `JoinCluster` method on the joining side fails fast. The leader emits a warning log and adds the endpoint to a set so the terminal signal is sent only once.

26.5 Removing a Node

Kahuna supports two removal paths: the API-driven graceful leave and the shutdown-coupled leave.

26.5.1 Graceful Leave via the API

An operator calls `POST /v1/cluster/leave` on the node to decommission. The `ClusterLeave` facade wraps `RequestLeaveAsync` with error mapping and a deadline. The response includes the outcome, the roster version, whether the node's replicas were drained, and whether the request is retryable.

`RequestLeaveAsync` follows this sequence:

1. **Set the leave-requested latch.** This latch is sticky: once set, it never clears, even if the leave attempt fails. It blocks `AutoRejoinDriver` from re-admitting a node whose removal committed late.
2. **Check preconditions.** If the roster has not been committed yet, or if the node is not in the roster, the method returns immediately.
3. **Drain before removal.** If the committed partition map names this endpoint in any replica set and the placement rebalancer is enabled, the method first commits a role transition from `Voter` to `Leaving`. It then waits (up to `DecommissionDrainTimeout`, default 120 seconds) for the placement pass to evacuate every replica onto surviving nodes. When the committed partition map no longer names this endpoint, the drain is complete and the method proceeds to the final removal.

4. **Commit removal.** The method sends a `RemoveMember` request to the system-partition leader. If this node is the leader, it applies the removal locally. The request retries with leader-following and a 10-second deadline.
5. **Wait for propagation.** After the leader acknowledges the commit, the method polls until the local roster cache no longer contains the local endpoint. This confirms that the removal has propagated back to this node.

The method returns a `LeaveClusterResult` with one of these outcomes:

- **Committed.** The node is out of the roster.
- **NotAMember.** The node was not in the roster (already removed or never joined).
- **RefusedInsufficientVoters.** Removing this node would leave zero voters. Do not retry.
- **RefusedDrainInProgress.** Another node is already draining. Retry after that drain finishes.
- **DrainTimedOut.** The drain deadline expired before all replicas were evacuated. The role is rolled back to Voter so the node keeps serving. Replicas already moved stay moved. A retry resumes the drain.
- **Timeout.** The removal was not confirmed in time. It may still commit. Re-read the roster.

26.5.2 Drain Rollback

If the drain times out or is cancelled, `RollBackDrainAsync` commits a role transition from `Leaving` back to `Voter` on `CancellationToken.None` (the rollback must be attempted even after the caller's token fires). If the placement pass committed the final `RemoveMember` while the rollback was firing, the rollback observes `MemberNotFound` and reports the leave as completed. A node must never keep serving when it is out of the committed roster.

26.5.3 Shutdown-Coupled Leave

When a node shuts down with `--graceful-leave-on-shutdown` enabled, `LeaveCluster` runs a simpler path. It sets the `_leaving` latch immediately to suppress elections on all partitions, commits a `RemoveMember` entry if at least one other Voter exists, and then tears down. This path does not drain replicas. It is suitable for permanent decommissions where the operator plans to stop the process immediately after the leave commits.

The book plan warns: do not enable `--graceful-leave-on-shutdown` during rolling restarts. The next section explains why.

26.6 Rolling Restarts

A rolling restart replaces each node's binary one at a time without shrinking the cluster. The correct procedure is:

1. Stop the node. Do not call the leave endpoint. Do not enable `--graceful-leave-on-shutdown`.
2. Upgrade the binary or apply the configuration change.
3. Start the node with the same endpoint and data directory.
4. Wait for the node to rejoin the cluster and confirm that its role is Voter.
5. Repeat for the next node.

26.6.1 Why Not Use Graceful Leave

A graceful leave commits a `RemoveMember` entry. The surviving nodes treat the node as permanently gone. When the node restarts, it must rejoin as a Learner, catch up on all partitions, and wait for promotion. If the cluster uses placed replicas, the departure triggers a placement pass that evacuates the node's replicas onto survivors. When the node rejoins, another placement pass must redistribute replicas back. Each of these transfers costs network bandwidth and disk I/O.

By contrast, a restart without leave keeps the node in the roster. The SWIM failure detector marks it as Suspect, then Dead, but the `DeadMemberEvictionGrace` prevents eviction for 2 minutes by default. A node that restarts within that window resumes its position as a Voter without any state transfer.

26.6.2 Timing the Grace Period

The `--raft-dead-member-eviction-grace` flag controls how long a Dead node survives before the system-partition leader commits a `RemoveMember`. The default is 2 minutes. Set this value to exceed the longest expected restart time for any node. A cold start with a large RocksDB store can take 30 to 60 seconds, so the 2-minute default provides a comfortable margin.

If a restart takes longer than the grace period, the SWIM failure detector evicts the node. Auto-rejoin then kicks in (see the next section) and the node re-enters as a Learner.

26.7 The SWIM Failure Detector

Kahuna uses a SWIM-style protocol (Scalable Weakly-consistent Infection-style Membership) to detect failed nodes. The failure detector runs independently of Raft consensus and feeds liveness information to the system-partition leader.

26.7.1 Probe Rounds

Every `PingInterval` (default 1 second), each node picks one random peer and sends a direct ping. If the direct ping times out within `PingTimeout` (default 500 ms), the node sends indirect pings through `IndirectPingFanout` (default 2) randomly chosen intermediaries. The intermediaries forward the probe to the target and relay the response. Indirect probing reduces false positives caused by a single faulty network path.

26.7.2 State Transitions

A node that fails both direct and indirect probes transitions from Alive to **Suspect**. A Suspect node has `SuspicionTimeout` seconds (default 5) to refute the suspicion through a successful probe or gossip message. If it does not refute within that window, it transitions to **Dead**.

A Dead node enters the eviction grace period. After `DeadMemberEvictionGrace` (default 2 minutes), the system-partition leader commits a `RemoveMember` entry that removes the node from the roster. The eviction path re-probes the endpoint at commit time as a final safety check.

26.7.3 Tuning Parameters

Flag	Default	Purpose
`--raft-ping-interval`	1000 ms	Interval between SWIM probe rounds
`--raft-ping-timeout`	500 ms	Direct ping timeout
`--raft-suspicion-timeout`	5000 ms	Time a Suspect node has to refute
`--raft-dead-member-eviction-grace`	120000 ms	Grace period before a Dead node is evicted
`--raft-indirect-ping-fanout`	2	Number of intermediaries for indirect probes

Setting the suspicion timeout too short causes premature evictions on slow networks. Setting it too long delays detection of genuine failures. The default of 5 seconds balances these concerns for most datacenter deployments.

26.8 Auto-Rejoin

When a node discovers that it is no longer in the committed roster (typically because the eviction grace period expired during a restart), `AutoRejoinDriver` re-runs the join flow automatically. The driver uses exponential backoff (1 second to 30 seconds) and contacts both the remaining roster members and discovery peers.

Auto-rejoin is enabled by default (`--raft-enable-auto-rejoin true`). The driver is suppressed during a graceful leave (the removal is intentional) and before the node has ever been in a committed roster (first-time joins use their own admission loop).

The re-admitted node enters as a Learner and is promoted to Voter by the standard promotion machinery. A node that was evicted but stayed running is typically already caught up, so promotion is fast.

26.8.1 The Leave-Requested Latch

`RequestLeaveAsync` sets a `_leaveRequested` latch on its first call. This latch never clears. It blocks auto-rejoin from re-admitting a node whose operator-ordered decommission landed late. Without this latch, a sequence like "operator calls leave, leave times out, removal commits a moment later, auto-rejoin fires" would silently undo the decommission.

26.9 REST Endpoints

Kahuna exposes four cluster management endpoints:

GET /v1/cluster/membership. Returns the committed roster: each member's endpoint, node ID, role, and joined version. Also returns the local node's role and whether initialization is complete.

POST /v1/cluster/leave. Decommissions the local node. Returns 200 if the node left the roster, 409 if the removal was permanently refused (last voter), 503 if the node could not attempt the removal, and 504 if the attempt timed out.

GET /v1/cluster/health. Returns 200 when the node is ready to serve (initialized and in a serving role). Returns 503 during initialization, after eviction, or during decommission. Use this endpoint as a Kubernetes readiness probe.

GET /v1/cluster/placement. Returns the partition map: which nodes host each partition, in what replica role, and whether the answering node hosts each partition locally.

26.10 Configuration Flags

The following flags control membership behavior at the Kahuna server level:

Flag	Default	Purpose
`--join-existing`	false	Join a running cluster instead of static discovery
`--initial-cluster`	(none)	Seed endpoints for `--join-existing`
`--graceful-leave-on-shutdown`	false	Commit a removal on shutdown
`--raft-learner-promotion-lag`	10	Max log entries a Learner may trail the leader
`--raft-learner-promotion-stable-window`	3000 ms	How long a Learner must stay within the lag threshold
`--raft-decommission-drain-timeout`	120000 ms	How long a graceful leave waits for replica evacuation
`--raft-enable-auto-rejoin`	true	Re-join after dead-member eviction

26.11 Failure Scenarios

Premature eviction. If `--raft-suspicion-timeout` is set too short (for example, 1 second on a cloud network with occasional latency spikes), the failure detector declares healthy nodes Dead before they can refute the suspicion. This triggers unnecessary evictions, state transfers, and re-promotions. Increase the suspicion timeout until false positives stop.

Quorum loss from simultaneous removals. Removing two nodes from a three-node cluster at the same time leaves one node with no quorum peer. The remaining node cannot commit any Raft entry, including the `RemoveMember` for the second node. The cluster is unavailable until at least one node rejoins. Always remove one node at a time and wait for the roster to stabilize before removing the next.

Last-voter protection. `ReceiveLeave` refuses a `RemoveMember` when the removal would leave zero voters in the roster. The response includes `Terminal: true` so the leave loop does not

retry. To shut down a single-node cluster, stop the process directly instead of calling the leave endpoint.

Rolling restart with leave. Enabling `--graceful-leave-on-shutdown` during a rolling restart causes each restarted node to leave the roster and rejoin as a Learner. The placement rebalancer evacuates replicas on each departure and re-distributes them on each rejoin. For a five-node cluster, this produces ten full placement passes and potentially gigabytes of unnecessary data movement. Disable the flag during rolling restarts.

Drain timeout. If the placement rebalancer cannot evacuate all replicas within `DecommissionDrainTimeout`, the role rolls back to Voter and the node keeps serving. Replicas that were already moved stay moved. A retry resumes the drain from where it left off. Increase the timeout for clusters with large partitions.

Auto-rejoin after operator removal. If an operator removes a node through the REST API and the node stays running, auto-rejoin re-admits it as a Learner. To prevent this, either stop the node after the leave response, or start the node with `--raft-enable-auto-rejoin false`.

27 Range Management and Rebalancing

Chapter 16 explained how key-range routing maps contiguous intervals of keys to partitions. It described the split lifecycle: how one range becomes two through a nine-step sequence. This chapter goes further. It covers the operational side of range management: how to register key ranges, how to trigger splits and merges (both automatic and manual), how the leader balancer spreads Raft leadership across nodes, and how the placement controller moves replicas to maintain zone-aware fault tolerance.

The chapter builds on two earlier chapters. Chapter 16 introduced hash-based and key-range routing, the `RangeMap`, and the split lifecycle. Chapter 24 covered deployment modes and replication factor configuration. Range management is the runtime layer that keeps partitions balanced as data grows and access patterns shift.

27.1 Key-Range Registration

Before Kahuna can split or merge ranges in a key space, that key space must be registered for key-range routing. By default, every key space uses hash-based routing. Registration switches a key space to key-range mode and seeds the initial range descriptor.

27.1.1 The Registration API

The client exposes two methods:

```
KahunaRegisterKeyRangeResponse response =
    await client.RegisterKeyRange("orders");
```

`RegisterKeyRange` does two things in sequence:

1. **Flip the routing mode.** The `KeySpaceRegistry` on the contacted node changes the routing mode for the named key space from `Hash` to `KeyRange`. This flip is node-local and not replicated. Every node in the cluster must receive its own registration call.
2. **Seed the initial descriptor.** The method calls `EnsureKeyRangeSeededAsync` on the meta-partition leader. The leader creates a single `RangeDescriptor` that covers the entire key space: `StartKey=null, EndKey=null` (representing negative infinity to positive infinity). The descriptor's `PartitionId` is chosen by hashing the key-space name across the data partition pool `[1, InitialPartitions]`. The `Generation` starts at 1.

If the seed descriptor already exists, the method returns `AlreadySeeded` and does not create a duplicate. The seeding step is idempotent. The routing-mode flip is also idempotent: calling `RegisterKeyRange` twice on the same node has no additional effect.

The response includes a `Status` field with one of five values: `Seeded` (descriptor created), `AlreadySeeded` (descriptor existed), `Indeterminate` (commit status unknown), `InvalidInput` (empty key space), or `KeyRangeDisabled` (key-range routing is disabled on the server).

27.1.2 Removing a Key Range

```
KahunaRemoveKeyRangeResponse response =
    await client.RemoveKeyRange("orders");
```

`RemoveKeyRange` deletes all range descriptors for the named key space in a single atomic mutation on the meta partition. The method is idempotent. It refuses the call transiently if a

split is in progress and holding a quiesce window on any descriptor in that key space. It also refuses removal of internal schema-log key spaces (the `/meta` prefix).

After removal, keys in that key space fall back to hash-based routing. Data already stored in key-range partitions is not moved or deleted. To reclaim those partitions, the operator must remove them through the partition lifecycle API.

27.2 Monitoring Ranges

The `GetRanges` method returns the current range map for a key space:

```
KahunaRangeMapResponse ranges = await client.GetRanges("orders");

foreach (var descriptor in ranges.Descriptors)
{
    Console.WriteLine(
        $"[{descriptor.StartKey}, {descriptor.EndKey}) " +
        $"-> partition {descriptor.PartitionId} " +
        $"gen {descriptor.Generation}");
}
```

The response includes the `RoutingMode` (`KeyRange` or `Hash`), an `Initialized` flag that indicates whether the meta partition map has been applied, and an ordered list of `Descriptors`. Each descriptor carries `StartKey`, `EndKey`, `PartitionId`, and `Generation`.

Pass a `keySpace` parameter to filter by key space. Omit it to see all registered key spaces.

27.3 Automatic Splitting

Kahuna supports two automatic split triggers: count-based and load-based. Both run on the meta-partition leader.

27.3.1 Count-Based Splitting

The `RangeSplitTrigger` runs a sampling pass every `RangeCollectionInterval` seconds (default 60). For each key-range descriptor, the trigger samples keys through `GetByRange` in pages of 512, up to 4096 keys total. If the sampled count exceeds `RangeSplitThreshold` (default 1000), the trigger initiates a split.

The split key is chosen at the midpoint of the sampled keys. Both halves must contain at least `RangeSplitMinRangeSize` keys (default 10). If either half would fall below that minimum, the split is refused as indivisible.

After a split completes, the trigger applies a cooldown of `RangeSplitSettleWindow` seconds (default 10) to both new descriptors. This prevents cascading splits before the new ranges stabilize. An indivisible range receives a longer cooldown of `RangeSplitIndivisibleCooldown` (default 5 minutes) to avoid repeated sampling of a range that cannot split.

27.3.2 Load-Based Splitting

Load-based splitting reacts to sustained write pressure rather than data volume. A separate polling loop runs every `RangeSplitLoadPollInterval` seconds (default 5). The trigger evaluates three conditions as an AND-predicate:

1. **Write rate.** The partition's operations per second must reach or exceed `RangeSplitLoadThreshold`.
2. **WAL queue depth.** The partition's WAL queue depth must reach or exceed `RangeSplitLoadMinQueueDepth` (default 8).
3. **Commit wait (optional).** If `RangeSplitLoadMinCommitWaitMs` is set, the partition's commit wait must reach or exceed that value.

All three conditions must hold continuously for `RangeSplitLoadWindow` seconds (default 15). A single poll where any condition drops below its threshold resets the window.

A skew guard prevents splitting ranges where the key distribution is heavily one-sided. If the `LoadImbalanceMax` ratio (default 0.8) shows that most keys would land in one half, the split is refused.

Load-based splitting is disabled by default (`RangeSplitLoadThreshold` = 0). Set the threshold to a positive value to enable it. The threshold depends on the hardware: a value that causes WAL queue saturation on one machine may be well within capacity on another.

27.3.3 Split Serialization

A `splitLock` semaphore serializes all split operations: count-based, load-based, and manual. Only one split can run at a time across all triggers. The semaphore is released when the split completes or fails. Leadership loss clears all accumulated cooldown state.

27.4 Automatic Merging

The `RangeMergeTrigger` runs on the same timer as the count-based split trigger (every `RangeCollectionInterval` seconds). It scans all key-range key spaces for adjacent pairs where both ranges contain fewer keys than `RangeMergeMinSize` (default 10).

27.4.1 Merge Eligibility

Two ranges are eligible for merging when all of the following conditions hold:

1. Both ranges belong to the same key space.
2. They are adjacent: the left range's `EndKey` equals the right range's `StartKey`.
3. Both contain fewer keys than `RangeMergeMinSize`.
4. Neither range's partition has an operations-per-second rate at or above `RangeSplitLoadThreshold`. This load-warm guard prevents merge-then-split oscillation.
5. The meta-partition leader holds leadership on both the system partition and the meta partition.

27.4.2 The Merge Sequence

When the `RangeMerger` processes a merge candidate pair `[A,B)@P1` and `[B,C)@P2`, it follows this sequence:

1. **Validate adjacency.** Confirm that `left.EndKey == right.StartKey`.
2. **Check for concurrent moves.** Refuse if another range move already quiesces any part of the merged span `[A,C)`. The check covers both halves because the cutover destroys both descriptors.
3. **Acquire WriteFence range lock.** Take a WriteFence range lock on `[B,C)` at P2's data-partition leader with a TTL of 30 seconds. WriteFence (not Exclusive) blocks every writer

through the write-path check, tolerates a reader's Shared range lock, and plants no per-key write intents that would interfere with snapshot scans during the copy window.

4. **Publish the quiesce on the descriptor.** Write a `QuiescedUntil` deadline on the right descriptor through `RangeMapStore.QuiesceRangeAsync`. The descriptor quiesce rides the replicated range map, so it reaches every node and survives a leadership change on P2. The range lock and the descriptor quiesce are complementary guards: the lock is exactly ordered against in-flight writes on one actor, and the descriptor quiesce blocks writes on every node.
5. **Settle durable intents.** Under deferred settlement, a committed value can exist only as a decided-but-unsettled prepared intent. The copy captures base rows, not intents. The merge settles all decided intents in `[B,C)` so committed values materialize into base rows before the copy. An undecided intent still within its window refuses the merge. The trigger retries on the next tick after the coordinator decides.
6. **Copy `[B,C)` at a snapshot.** Page all key-value entries from `[B,C)` at an MVCC snapshot into partition P1 through the locator. Each page is replicated onto the survivor's Raft log.
7. **Transfer range locks.** Read live range locks from P2′s leader, clamp them to `[B,C)`, and import them into P1′s leader. This transfer is best-effort: correctness does not depend on it, but it prevents a foreign lock from going missing after cutover.
8. **Hand off transaction state.** Replicate the moving range's completion receipts, canonical transaction records, and prepared intents onto the survivor's Raft log. This handoff is not best-effort. The survivor becomes the sole route for `[B,C)` after cutover, and P2 is retired. A lost receipt, decision, or unresolved intent would be lost permanently. If the handoff fails, the merge aborts before cutover.
9. **Atomic cutover.** A single `MutateAsync` call replaces both descriptors with one new descriptor: `[A,C)@P1` with a generation incremented by one. The merged descriptor carries no quiesce, so the cutover ends the window atomically with the routing change.
10. **Release and retire.** The `finally` block releases the quiesce on the descriptor and releases the range lock on the retiring partition by partition ID (not by key location, because after cutover `[B,C)` routes to the survivor). The caller invokes `RemovePartitionAsync` to retire P2. If the removal call fails, the trigger retries it on the next tick.

The quiesce window closes on three independent terms. A successful cutover clears it atomically with the routing change. The `finally` block releases it on every other path. The deadline stamped on the descriptor lapses on its own, so a merge executor that dies mid-window leaves a range that reopens instead of one that refuses writes permanently.

The merge copies data in one direction only (right into left). Because a merge candidate is an under-min range, a single copy inside the window is enough and the window stays short. Orphan rows remain physically present on P2′s replicas but affect nothing: the partition is retired and no routing reaches it.

27.5 Manual Split and Merge

Operators can trigger splits and merges on demand without waiting for automatic thresholds.

27.5.1 Manual Split

```
KahunaSplitRangeResponse response =
    await client.SplitRange("orders", "orders/5000");
```

The call splits the range that covers `splitKey` at that exact key boundary. The meta-partition leader must handle the request. If the contacted node is not the leader, the response includes a `LeaderHint` for the client to retry.

The response `Status` field carries one of these values:

- `Succeeded`: the split completed.
- `NotLeader`: forward to the leader hint.
- `NoRange`: no range covers the split key.
- `InvalidSplitKey`: the split key falls outside the range or is empty.
- `BelowMinRangeSize`: one or both halves would be too small.
- `PartitionCreationFailed`: the new partition could not be created.
- `TransferFailed`: data transfer to the new partition failed.
- `QuiesceFailed`: the quiesce step failed.
- `CutoverFailed`: the atomic descriptor swap failed.
- `ConcurrentSplit`: another split is already in progress.
- `Indeterminate`: the outcome is unknown (check the range map).

On success, the response includes `NewPartitionId` (the partition created for the right half) and `NewGeneration` (the generation of the new descriptors).

27.5.2 Manual Merge

```
KahunaMergeRangesResponse response = await client.MergeRanges();
```

The call runs the same merge pass that the automatic trigger runs, but on demand. It evaluates all key-range key spaces for adjacent pairs below the merge threshold. The response includes a `Merges` count.

27.6 Leader Balancing

In a multi-node cluster, Raft leadership tends to concentrate on nodes that recover first after a restart or that win elections more often because of timing. This concentration creates hot spots: one node handles all reads and writes for many partitions while other nodes sit idle. The leader balancer redistributes leadership across nodes.

27.6.1 Enabling the Balancer

The leader balancer is disabled by default. Enable it with:

```
--raft-enable-leader-balancer true
```

The balancer runs on the system-partition leader (partition 0) every `LeaderBalancerInterval` milliseconds (default 30000, or 30 seconds).

27.6.2 The Three-Tier Strategy

The balancer uses a three-tier strategy. The tiers run in order. Each pass stops when its move cap is full.

Drain tier. When slow node avoidance is enabled (see below), the drain tier runs first. It evacuates every leadership from nodes that the `SlowNodeDetector` classified as degraded. For each slow node, the tier picks the hottest eligible partition and transfers it to the coolest healthy voter. A simulated-load table tracks destinations within one pass, so successive picks spread across healthy nodes instead of landing on the same one.

The drain tier owns its pass. When it emits any move, the count and load tiers are skipped. A drain deliberately unbalances the counts. If the count tier ran in the same pass, it would push leaderships back toward the node that is being emptied.

A node that holds many leaderships empties over several passes, not in one burst. The per-pass move cap and per-move filters still apply. A partition in cooldown or below `MinLeaderStabilityMs` stays where it is.

Count tier. The balancer computes the ideal leader count per node: total leaders divided by the number of live nodes. A node is over-loaded when its leader count exceeds `ceil(ideal) + CountDeadband - 1`. A node is under-loaded when its leader count falls below `floor(ideal)`. The balancer picks the hottest partition from the most over-loaded node and transfers its leadership to the most under-loaded node. Slow nodes are excluded as transfer destinations.

Load tier. When leader counts are balanced (no node exceeds the over-loaded threshold), the balancer checks load imbalance. If `(maxLoad - minLoad) / maxLoad` exceeds `LoadImbalanceThreshold` (default 0.25), the balancer emits count-neutral swaps: it moves one partition from a hot node to a cold node and one from the cold node to the hot node. This keeps the leader count unchanged while balancing actual workload. The balancer only emits the swap if the resulting load spread is strictly smaller than the current spread. Slow nodes are excluded from both ends of the swap.

27.6.3 Load Scoring

Each node reports its load through gossiped load reports at `LeaderBalancerReportInterval` (default 5 seconds). Reports older than `LeaderBalancerReportTTL` (default 20 seconds) are discarded.

The load score for a node is:

```
Load = OpsWeight * OpsPerSecond + QueueWeight * QueueDepth
```

`OpsWeight` defaults to 1.0. `QueueWeight` defaults to 0.5. Adjust these weights to emphasize throughput or queue pressure depending on the workload.

27.6.4 Move Filters

Before the balancer emits a move, it checks several filters:

1. The partition must be active in the partition map.
2. The current leader must have held leadership for at least `MinLeaderStabilityMs` (default 5000) milliseconds.
3. The target node must be an alive voter.
4. The partition must not be in a post-move cooldown (`MoveCooldown`, default 60 seconds).
5. The number of in-flight transfers must be below `MaxConcurrentTransfers` (default 2).
6. The total moves in this pass must be below `MaxMovesPerPass` (default 4).

27.6.5 Slow Node Avoidance

The `SlowNodeDetector` classifies nodes whose WAL commit-wait stands far above the cluster median. A classified node receives no new leadership and has its current leaderships drained by the drain tier.

Why a classifier and not a load-score term. Commit wait is a property of a node's storage device. The load score is a sum over partitions. A latency term folded into the score would be counted once per leadership, with an amplification that nobody chose. The load tier also emits count-neutral swaps, which would leave a degraded node with exactly as many write streams as before. Avoidance needs a gate and a drain. Those need a boolean, and a boolean needs hysteresis.

Relative, not absolute. The detector judges a node against the median of its peers, not against a fixed threshold. A legitimately heavy write load raises every node's wait together and leaves the ratio flat. An absolute threshold cannot tell a busy cluster from a degraded node. It would drain a healthy cluster at its busiest moment.

Classification algorithm. The detector collects WAL commit-wait samples from each live voter's gossiped health reports. It filters out nodes with too few samples or stale observations. If fewer than three judgeable nodes remain, the detector cannot compute a meaningful median and skips the pass.

For each pass the detector computes the median commit-wait and derives a threshold:

```
threshold = max(median * SlowNodeMultiplier, SlowNodeFloorMs)
```

`SlowNodeMultiplier` (default 3.0) is the ratio above the median. `SlowNodeFloorMs` (default 5.0 ms) prevents classification when the entire cluster runs at sub-millisecond commit-wait. A node whose commit-wait reaches or exceeds the threshold is a candidate.

Hysteresis. A candidate must exceed the threshold for `SlowNodeEnterPasses` consecutive passes (default 3) before it is classified as slow. A single clean pass resets the candidate's counter. Once classified, a node must show `SlowNodeExitPasses` consecutive clean passes (default 5) before the detector releases it.

Majority safeguard. If half or more of the live voters exceed the threshold, the detector treats the pass as a cluster-wide slowdown and classifies none. Draining a majority of nodes would be a self-inflicted outage.

When the system-partition leader loses leadership, the detector resets all state. A new leader re-derives classifications from the passes it observes itself.

27.6.6 The Transfer Mechanism

The system-partition leader sends a `TransferLeadershipSuggestionRequest` to the current leader of the target partition. The recipient validates the suggestion (checking that it still leads the partition and that the target is a viable candidate) and executes `TransferLeadershipAsync`. The suggestion has a `SuggestionTimeout` of 15 seconds. If the recipient does not confirm within that window, the move is abandoned and the partition enters cooldown.

27.7 Replica Placement

When a cluster runs with a replication factor greater than zero, the placement rebalancer manages which nodes hold replicas for each partition. The rebalancer runs on the system-partition leader every `PlacementPassInterval` milliseconds (default 5000).

27.7.1 The Placement Pass

Each pass works through three stages in order:

Stage 1: Complete decommission drains. For each cluster member with the `Leaving` role, check whether the committed partition map still names that member's endpoint. If no replicas remain, commit the `RemoveMember` entry to finish the decommission.

Stage 2: Drive in-flight transitions. The pass handles replicas that are mid-transition:

- Replicas being removed receive another `RemoveReplica` call.
- Learner replicas on a draining or evicted node are dropped immediately. This prevents a promote-then-evacuate loop where the system promotes a learner and then must move it off the same node.
- Learner replicas that have caught up are tracked. After the learner stays within the lag threshold for the full `LearnerPromotionStableWindow`, the pass promotes it to voter.
- A failing range does not abort the entire pass. Each range is isolated so that one stuck transition does not block progress on other ranges.

Stage 3: Plan rebalancing moves. This stage runs only when `EnablePlacementRebalancer` is true. The pass builds a `PlacementView` that includes all alive voters from the committed roster, their zone assignments, and the current partition map. It passes this view to the `PlacementPlanner`.

27.7.2 The Planner's Four Priorities

The `PlacementPlanner` evaluates ranges in four priority tiers, from highest to lowest:

1. **Repair under-replication.** If a range has fewer healthy voters than its replication factor, the planner adds a replica on the least-loaded alive node. This tier uses the repair budget: `MaxConcurrentReplicaRepairs` (default 3) minus the current count of transitional replicas.
2. **Trim over-replication.** If a range has more voters than its replication factor, the planner removes the excess. It prefers victims whose zone is already covered by another voter in the same range. Among equal candidates, it picks non-leaders first, then the most-loaded node.
3. **Repair zone-spread violations.** If a range has two voters in the same zone while another zone has a free node, the planner adds a replica in the uncovered zone. The over-replication trim in the next pass removes the duplicate. This two-pass approach avoids a simultaneous add-and-remove that could temporarily reduce the voter count below the replication factor.
4. **Balance replica-count skew.** If a node holds more replicas than `ceil(ideal) + ReplicaCountDeadband` (default 1), the planner moves one replica to the least-loaded node. This tier uses the transfer budget: `MaxConcurrentReplicaTransfers` (default 1) minus the current count of balance-move replicas.

27.7.3 Stability Guards

The planner applies several stability guards:

- A range with an existing transitional replica (a learner being promoted or a voter being removed) receives no new moves.
- At most `MaxReplicaMovesPerPass` (default 4) moves are emitted per pass.

- Each move creates a learner on the target node. The learner receives state through normal Raft replication or state transfer. After promotion, the original voter on the source node is removed.

27.7.4 Zone-Aware Spreading

Each node declares its zone through the `--raft-zone` flag. The zone string is gossiped through load reports. The planner uses zone information in two places:

1. **Add target selection.** When the planner adds a replica, it prefers nodes in zones not already covered by the range's existing voters.
2. **Trim victim selection.** When the planner removes a replica, it prefers nodes whose zone is duplicated within the range.

Zone information from remote nodes arrives through gossip after the initial partition map is committed. Initial placement may be zone-blind for nodes that have not yet gossiped. The rebalancer converges to a zone-spread layout within a few passes.

27.8 Per-Partition Replication Factor

By default, every partition uses the cluster-wide replication factor set by `--raft-replication-factor`. Kahuna allows overriding this value for individual partitions:

```
await raft.SetReplicationFactorAsync(partitionId: 5, replicationFactor: 5);
```

The call is system-leader only. It commits the new replication factor on the partition's range entry in the partition map, increments the range's `Generation` and the map's `MapVersion`, and replicates the change through Raft.

The placement rebalancer reads each range's `ReplicationFactor` field. If the field is greater than zero, it overrides the cluster-wide value. If the field is zero, the cluster-wide value applies.

A common pattern is to set the cluster-wide replication factor to zero (full replication) and override it for specific high-throughput partitions that do not need copies on every node. The per-partition override reduces replication traffic for those partitions while the remaining partitions stay fully replicated.

27.9 The Kahuna-Side Placement Coordinator

The `PartitionPlacementCoordinator` on each Kahuna node reacts to committed placement map changes. It maintains a `PartitionPlacementView` that tracks which partitions this node hosts.

When the hosted set changes:

- **Gained partitions.** The coordinator logs the gain and prepares the local actor and persistence for the new partition.
- **Lost partitions.** The coordinator evicts the durability floor for the partition, signals the background writer to stop flushing for it, and schedules `PurgeUnhostedPartitionSafelyAsync`. The purge removes all backend rows, store slices, the durability floor, and the install marker for the partition.

On first startup, when the initial partition map arrives, the coordinator runs a cleanup pass. Any partition data on disk that does not appear in the committed map is purged. This handles two cases: a crash during a previous purge, and replicas removed while the node was offline.

27.10 Configuration Reference

27.10.1 Split and Merge Flags

Flag	Default	Purpose
`--range-split-threshold`	1000	Key count for count-based auto-split (0 disables)
`--range-split-min-range-size`	10	Minimum keys each half must contain
`--range-split-settle-window`	10s	Post-split cooldown per descriptor
`--range-merge-min-size`	10	Key count below which adjacent ranges merge (0 disables)
`--range-collection-interval`	60s	Interval between sampling passes
`--range-split-load-threshold`	0	Ops/sec for load-based split (0 disables)
`--range-split-load-min-queue-depth`	8	Minimum WAL queue depth for load-based split
`--range-split-load-window`	15s	Sustained-load window before split triggers
`--range-split-load-poll-interval`	5s	Load polling interval
`--range-move-settle-timeout`	10s	Upper bound on the quiesce drain of unsettled intents before cutover (clamped to 15s)

27.10.2 Leader Balancer Flags

Flag	Default	Purpose
`--raft-enable-leader-balancer`	false	Enable leader redistribution
`--raft-leader-balancer-interval`	30000 ms	Planning pass cadence
`--raft-leader-balancer-report-interval`	5000 ms	Load report gossip cadence
`--raft-leader-balancer-report-ttl`	20000 ms	Stale report cutoff
`--raft-count-deadband`	1	Leader-count imbalance tolerance
`--raft-load-imbalance-threshold`	0.25	Load skew ratio that triggers load-tier balancing
`--raft-min-leader-stability-ms`	5000	Minimum leader tenure before a move
`--raft-move-cooldown`	60000 ms	Post-move cooldown per partition
`--raft-max-moves-per-pass`	4	Maximum moves per planning pass
`--raft-max-concurrent-transfers`	2	Maximum in-flight leadership transfers
`--raft-leader-balancer-ops-weight`	1.0	Weight of ops/sec in load score
`--raft-leader-balancer-queue-weight`	0.5	Weight of queue depth in load score
`--raft-suggestion-timeout`	15000 ms	Move confirmation timeout

27.10.3 Slow Node Avoidance Flags

Flag	Default	Purpose
`--raft-enable-slow-node-avoidance`	false	Enable slow-node classification and drain
`--raft-slow-node-multiplier`	3.0	Commit-wait ratio above median that marks a candidate
`--raft-slow-node-floor-ms`	5.0	Absolute floor below which no node is classified
`--raft-slow-node-enter-passes`	3	Consecutive passes above threshold before classification
`--raft-slow-node-exit-passes`	5	Consecutive clean passes before release
`--raft-slow-node-min-samples`	(varies)	Minimum WAL commit-wait samples before a node is judgeable
`--raft-slow-node-observation-ttl`	(varies)	Maximum age of a health sample before it is discarded

27.10.4 Placement Rebalancer Flags

Flag	Default	Purpose
`--raft-replication-factor`	0	Cluster-wide voter count per range (0 = full replication)
`--raft-enable-placement-rebalancer`	false	Enable replica rebalancing
`--raft-placement-pass-interval`	5000 ms	Controller pass cadence
`--raft-max-replica-moves-per-pass`	4	Maximum replica moves per pass
`--raft-max-concurrent-replica-transfers`	1	Maximum balance moves in flight
`--raft-max-concurrent-replica-repairs`	3	Maximum repair moves in flight
`--raft-replica-count-deadband`	1	Replica-count skew tolerance
`--raft-zone`	(none)	Locality hint for zone-aware spreading

27.11 Failure Scenarios

Hot partition from concentrated writes. If one key range receives most of the write traffic, load-based splitting can divide the range. Enable it by setting `--range-split-load-threshold` to a value below the point where WAL queue depth grows. The trigger requires sustained pressure for 15 seconds before it acts. If the key distribution is too skewed for the split to produce balanced halves (the `LoadImbalanceMax` guard), the application must redesign its key scheme.

Indivisible range. A range that covers a single key or whose sampled key count falls below `RangeSplitMinRangeSize` on either side of the midpoint cannot split. The trigger applies a 5-minute cooldown to avoid repeated sampling. The operator must intervene by redesigning the key scheme to distribute writes across more keys.

Split during active transaction. A range split quiesces the range before the cutover step. Transactions with prepared intents in the affected range are settled through `SettleSuppliedIntentsAsync` before the cutover proceeds. An undecided intent within its recovery deadline blocks the split until the deadline passes or the coordinator commits. The transaction's client receives a generation-fence error after the split completes and must retry with the new generation.

Merge-split oscillation. If the merge threshold and split threshold are set too close together, a range could merge below the merge threshold and then split above the split threshold in

alternating cycles. The load-warm guard prevents merging ranges with active write traffic. Set the merge threshold well below the split threshold to create a stable band.

Leader balancer thrashing. If `MoveCooldown` is too short or `CountDeadband` is zero, the balancer can shuffle leadership back and forth between two nodes on successive passes. Each transfer interrupts in-flight reads. Keep the deadband at 1 or higher and the cooldown at 60 seconds or longer.

Zone-blind initial placement. When a node first joins the cluster, its zone is not yet known to other nodes (the zone propagates through gossip). Initial replica placement may ignore zone constraints. The rebalancer corrects this within a few passes after gossip propagates. For clusters where zone spread is critical from the start, ensure all nodes are running and gossiping before registering key ranges.

28 Backup and Point-in-Time Recovery

A distributed system that replicates data across nodes can survive individual node failures. It cannot survive a bug that corrupts data on every replica, an operator who deletes the wrong key space, or a disk failure that takes out an entire availability zone. Backups provide the safety net for these scenarios. Point-in-time recovery (PITR) lets the operator restore data to any moment within the retention window, not just the latest state.

This chapter explains how Kahuna's backup system works: how to take full and incremental backups, how coordinated cluster snapshots capture a consistent point across all partitions, how PITR replays write-ahead log (WAL) segments to reach an exact timestamp, and how retention policies and garbage collection keep the backup store manageable.

The chapter builds on two earlier chapters. Chapter 22 described the persistence backends and the background writer. Chapter 23 introduced the startup sequence and the PITR bootstrap path. This chapter covers the backup machinery that produces the artifacts those paths consume.

28.1 Backup Types

Kahuna supports two backup types: full and incremental.

A **full backup** captures a complete checkpoint of the storage backend at a specific point in time. The checkpoint is a self-contained image: restoring from a full backup requires no other artifact. Full backups are larger and slower to produce, but they are the foundation of every backup chain.

An **incremental backup** captures only the WAL entries committed since its parent backup. It is smaller and faster to produce, but it depends on its parent. Restoring from an incremental backup requires the entire chain back to the root full backup.

The `BackupType` enum has two values: `Full` (0) and `Incremental` (1).

28.2 Backup Storage

Kahuna separates backup storage into two interfaces. This separation allows operators to replace the built-in local filesystem storage with a custom implementation (for example, an object store).

`IBackupStorageTarget` handles manifests. Manifests are small JSON objects that describe each backup (its ID, type, parent, timestamps, and artifact paths). The interface provides CRUD operations: put, get, delete, list, and list corrupt manifests.

`IBackupArtifactStore` handles artifacts. Artifacts are the checkpoint trees and WAL segments that contain the actual data. They can be large (multiple gigabytes). The interface provides byte-range reads, two-phase writes (write to a stream, then commit atomically), and checkpoint staging.

The `--pitr-backup-target` flag selects the storage provider. The built-in value `local` uses the filesystem under `--pitr-backup-dir`. Any other value requires the host application to register a `BackupStorageProvider` delegate on `KahunaConfiguration`. If no provider is registered for the configured target, the server refuses to start.

28.2.1 Publication Ordering

Kahuna publishes artifacts before manifests. A backup's data files are fully written and committed before the manifest that references them becomes visible. If the process crashes during a backup, the result is orphan artifacts with no manifest. The GC sweep reclaims these orphans. A manifest never points at absent data.

This ordering is the reason for the `IsIncomplete` flag on backup listings. An incomplete backup has artifacts but no manifest. It means the upload was interrupted before the manifest was published.

28.2.2 Scratch Directory

Some artifact stores (object stores) cannot write checkpoints directly. The persistence backend writes checkpoints to a local directory first, then the store uploads the files. The `--pitr-backup-scratch-dir` flag sets this local staging directory. Size it for one full backup.

If the artifact store reports `RequiresLocalScratch = true` and no scratch directory is configured, the server refuses to start. The built-in `local` store does not require scratch (it writes directly to the backup directory).

28.3 Taking a Full Backup

28.3.1 The Client API

```
KahunaBackupInfo info = await client.TakeFullBackupAsync(cancellationToken);

Console.WriteLine($"Backup {info.BackupId} created at {info.CreatedAtUtc}");
Console.WriteLine($"Covers {info.PartitionCount} partitions");
```

The response includes the `BackupId` (a GUID), the `CreatedAtUtc` timestamp, the backup `Type`, the `PartitionCount`, and the recoverable time window (`MinRecoverablePhysicalMs` to `MaxRecoverablePhysicalMs`).

28.3.2 The Internal Sequence

The `BackupDriver.RunFullAsync` method orchestrates the full backup through these steps:

1. **Snapshot the topology.** Compute an FNV-1a hash over the sorted partition IDs, their range generations, and the membership version. This hash is the `TopologyGeneration`. If the topology changes during the backup, the driver detects the mismatch and aborts.
2. **Record committed indices.** For each active, hosted partition, read the maximum committed WAL index before the flush. This establishes the coverage boundary.
3. **Wait for the applied-HLC barrier.** The background writer may have committed entries that are not yet flushed to the storage backend. The driver waits until the background writer's maximum enqueued commit HLC reaches or exceeds the target for each partition. The timeout is 30 seconds. If any partition does not converge, the backup fails closed.
4. **Flush persistence.** Call the flush barrier to force the background writer to write all queued entries to the storage backend.
5. **Compute the cut timestamp.** The cut is the snapshot HLC (for coordinated backups) or the maximum committed HLC across all partitions.

6. **Check the pruned-history floor.** If MVCC revision pruning already passed the cut timestamp, the backup cannot produce a consistent image. The driver refuses the backup.
7. **Acquire an MVCC snapshot hold.** The driver acquires a hold at the cut timestamp with a lease of 600 seconds. This hold prevents the revision pruning system from deleting history that the backup needs. A background loop renews the hold at roughly one-third of the lease interval (every 200 seconds). If the renewal fails, the driver cancels all remaining work.
8. **Create the checkpoint.** Call the persistence backend's `CreateCheckpointAsOf` method. The backend produces a directory of files that represent the state at the cut.
9. **Hash and measure artifacts.** Compute a SHA-256 hash of every checkpoint file and record its byte size. Set file permissions to owner-only (mode 0600 for files, 0700 for directories).
10. **Build the manifest.** Create a `BackupManifest` with all metadata: backup ID, type, creation time, format version, partition ranges (with WAL index and HLC coverage per partition), base cut, topology generation, cluster and covered partitions, checksums, and sizes.
11. **Verify artifacts.** Confirm that every file listed in the manifest exists, matches its checksum, and matches its recorded size.
12. **Sign the manifest.** If an HMAC key is configured, compute an HMAC-SHA-256 tag over the manifest's canonical payload and store it in the `Mac` field.
13. **Publish the manifest.** Write the manifest to the backup catalog.
14. **Release the snapshot hold.** Best-effort release. If the release fails, the hold expires naturally after the lease period.

28.4 Taking an Incremental Backup

28.4.1 The Client API

```
KahunaBackupInfo incremental = await client.TakeIncrementalBackupAsync(
    parentBackupId: info.BackupId,
    cancellationToken);
```

The caller specifies the parent backup ID. The incremental backup captures all WAL entries committed after the parent's coverage.

28.4.2 The Internal Sequence

The `BackupDriver.RunIncrementalAsync` method follows these steps:

1. **Resolve the parent manifest.** Load the parent from the catalog. Build high-water marks across the entire ancestor chain (not just the immediate parent). This transitive resolution ensures that the incremental covers exactly the gap since the last captured entry.
2. **For each active, hosted partition:**
   - Compute `fromIndex = parentHighWater.ToIndex + 1`. For partitions not covered by the parent (new partitions added after the full backup), use the WAL compaction floor.

- Check whether the WAL still contains entries at `fromIndex`. If WAL compaction already discarded those entries, the incremental cannot cover this partition. The driver returns `NeedsFullBackup`.
- Acquire a WAL retention hold at `fromIndex` to prevent compaction during the capture. Re-check the compaction floor after the hold to guard against a race.
- Stream WAL pages from `fromIndex` to the end (or to the snapshot timestamp for coordinated backups). Write the entries to a `partition_{id}.wal` segment file. Compute a SHA-256 hash during the write.

3. **Build, verify, sign, and publish** the manifest, following the same steps as a full backup.
4. **Release WAL retention holds.**

28.4.3 Automatic Fallback

If the incremental fails because WAL compaction passed the required index (`NeedsFullBackup`), the `BackupService` catches the failure and takes a full backup instead. The response marks `SubstitutionReason` to inform the caller that a full backup was taken in place of the requested incremental.

28.5 Coordinated Cluster Snapshots

A full or incremental backup on a single node captures only the partitions that node hosts. In a multi-node cluster with a replication factor less than the cluster size, no single node holds every partition. A coordinated backup captures a consistent point across all partitions in the cluster.

28.5.1 The Safe Timestamp

The `SnapshotCoordinator.ComputeSafeSnapshotTimeAsync` method computes the safe snapshot timestamp:

1. Query the cluster-wide minimum in-flight commit timestamp across all actor shards. This is the earliest HLC at which any currently executing transaction could commit.
2. If the minimum is greater than zero (transactions are in flight): set the snapshot timestamp to the HLC tick immediately before the minimum. Any in-flight transaction will commit at or after the minimum, so its effects will not appear in the snapshot. Any transaction that committed before the minimum is fully visible.
3. If the minimum is zero (the cluster is quiesced, no transactions in flight): set the snapshot timestamp to the maximum committed HLC across all partitions.

This calculation guarantees that the snapshot reflects a consistent cut: every committed transaction is either fully included or fully excluded. No partial transaction state crosses the cut boundary.

28.5.2 The Coordinated Backup API

```
KahunaBackupInfo coordinated =
    await client.TakeCoordinatedBackupAsync(cancellationToken);
```

The call must reach the meta-partition leader (the backup coordinator). If the contacted node is not the coordinator, the response returns `NotBackupCoordinator`.

The coordinator captures its term at the start. After producing the backup artifacts, it verifies that it still holds leadership and that the term has not changed. If leadership moved during the backup, the artifacts are discarded. This fencing prevents two coordinators from producing conflicting snapshots.

28.6 The Backup Manifest

Every backup produces a manifest that records everything needed to validate and restore from the backup. The manifest contains:

- **Identity.** `BackupId` (GUID), `Type` (Full or Incremental), `CreatedAtUtc`, `FormatVersion` (currently 1).
- **Lineage.** `ParentBackupId` (null for full backups). This link forms the backup chain.
- **Coverage.** `PartitionRanges` with per-partition WAL coverage: `FromIndex`, `ToIndex`, `FromHlc`, `ToHlc`, `ToTerm`.
- **Checkpoint.** `BaseCut` (the HLC at which the full checkpoint was cut). Present only on full backups.
- **Coordination.** `ClusterSnapshotTime` (the safe HLC for coordinated backups), `CoordinatorNode`, `CoordinatorTerm`.
- **Topology.** `ClusterId`, `StorageType`, `StorageRevision`, `TopologyGeneration`, `ClusterPartitions`, `CoveredPartitions`.
- **Integrity.** `Checksums` (SHA-256 hex per artifact file), `Sizes` (byte length per artifact file).
- **Authentication.** `Mac` (HMAC-SHA-256 hex tag, or null if no key is configured).

28.7 Manifest Authentication

Kahuna signs backup manifests with HMAC-SHA-256 to detect tampering. The `BackupManifestMac` class handles signing and verification.

28.7.1 The Canonical Payload

The MAC covers a deterministic serialization of the manifest fields: format version, backup ID, type, parent ID, creation time, cluster identity, coordinator node and term, storage type and revision, topology generation, base cut, cluster snapshot time, cluster and covered partition counts, partition ranges (sorted by partition ID), and files (sorted by key, with digest and size).

The sorted ordering ensures that the same manifest always produces the same payload, regardless of dictionary enumeration order.

28.7.2 Signing and Verification

`Sign(manifest, key)` computes the HMAC-SHA-256 tag and stores it in the manifest's `Mac` field. `Verify(manifest, key)` recomputes the tag and compares it with the stored value using `CryptographicOperations.FixedTimeEquals` (constant-time comparison to prevent timing attacks).

A missing MAC on a manifest fails verification closed: the manifest is rejected. If no MAC key is configured on the server, signing and verification are skipped entirely.

The key is loaded from a file specified by `--pitr-backup-mac-key-file`. An empty or missing file causes a startup error.

28.8 Backup Chains

A backup chain is a sequence that starts with a full backup and continues with zero or more incremental backups. Each incremental links to its parent through the `ParentBackupId` field. The chain forms a singly linked list from the newest incremental back to the root full backup.

28.8.1 Resolving a Chain

```
List<KahunaBackupInfo> chain =
    await client.GetBackupChainAsync(leafBackupId, cancellationToken);
```

The `BackupCatalog.ResolveAndValidateAsync` method traverses parent links from the leaf to the root. It returns the full ordered chain (root first, leaf last).

28.8.2 Chain Validation

The catalog validates five properties:

1. **Non-empty, starts with Full.** The chain must contain at least one entry, and the first entry must be a full backup.
2. **Subsequent entries are Incremental.** Every entry after the root must be an incremental backup.
3. **Parent links unbroken.** Each incremental's `ParentBackupId` must match the previous entry's `BackupId`.
4. **Identity consistency.** `ClusterId`, `StorageRevision`, and `TopologyGeneration` must match across all entries (for non-null values). A chain that spans a topology change is invalid.
5. **Per-partition index continuity.** For each partition, the incremental's `FromIndex` must equal the parent's `ToIndex + 1`. A gap means WAL entries were lost between backups. An overlap means entries would be replayed twice.

HLC ordering is also checked: `FromHlc` must be less than or equal to `ToHlc` within each partition range.

28.9 Listing Backups

```
List<KahunaBackupInfo> backups =
    await client.ListBackupsAsync(cancellationToken);

foreach (var b in backups)
{
    Console.WriteLine(
        $"{b.BackupId} {b.Type} {b.CreatedAtUtc} " +
        $"partitions={b.PartitionCount}");
}
```

The response includes metadata for every backup in the catalog. Each entry carries `IsInvalid` and `IsIncomplete` flags with an `InvalidReason` string for backups that failed validation.

28.10 Restoring to a Point in Time

PITR restores data to an exact HLC timestamp by replaying the full checkpoint and then applying incremental WAL segments up to the target time.

28.10.1 The Restore API

```
await client.RestoreToAsync(
    leafBackupId: chain.Last().BackupId,
    targetDir: "/data/kahuna-restored",
    targetTimeMs: 1719849600000,  // Unix milliseconds
    cancellationToken);
```

The restore is an offline operation. The target node does not serve traffic during the restore.

28.10.2 The Restore Sequence

The `BackupService.RestoreToAsync` method orchestrates the full restore:

1. **Resolve and validate the chain.** Traverse parent links and validate all five chain properties.
2. **Authenticate every manifest.** If a MAC key is configured, verify the HMAC-SHA-256 tag on every manifest in the chain. A failed verification aborts the restore.
3. **Verify every artifact.** For each file listed in each manifest, verify the SHA-256 checksum, the byte size, that the path contains no directory traversal attacks, and that the path is not a symbolic link.
4. **Validate partition coverage.** Refuse partial-cluster chains (chains that do not cover all cluster partitions).
5. **Validate the target timestamp.** Use `BackupChainCoverage.Resolve` to compute the exact recoverable window. Refuse a target outside that window.
6. **Destination safety checks.** Refuse if the target directory is a symbolic link, is non-empty, overlaps with the backup store or live storage, or falls outside the restore root (if configured).
7. **Copy the full checkpoint.** Copy all checkpoint files from the root full backup into a staging directory (a sibling of the final target). If `BackupRestoreThrottleMbps` is set, the copy respects the throughput budget.
8. **Verify the staged copy.** Re-check all checksums against the manifest.
9. **Open the persistence backend.** Initialize the storage backend at the staging directory.
10. **Replay incremental segments.** The `RestoreEngine.RestoreAsync` method processes each incremental manifest in chain order. For each partition in each incremental:
    - Copy the WAL segment file to a private staging directory. Hash the copy during transfer (SHA-256) and verify against the manifest.
    - Stream entries from the segment. For each entry, check the commit HLC (the entry's `LastModified` timestamp). Entries with a commit HLC after the target timestamp are skipped. Entries at or before the target are applied.
    - The engine does not break on the first entry past the target. Commit HLC is not monotonic across keys within a segment, so later entries may still fall within the target window.
    - Only key-value entries are replayed. Lock entries, range-map entries, and all other replication types are skipped.

- Entries are applied in batches of 256 through `StoreKeyValues` (an idempotent upsert by key and revision).

11. **Flush.** For memory backends, flush the merged result to disk.
12. **Atomic publish.** Move the staging directory to the final target path with `Directory.Move`. This is an atomic rename on the same filesystem.
13. **Cleanup on failure.** If any step fails, the staging directory is deleted or quarantined.

28.10.3 What PITR Does Not Restore

PITR images contain only key-value data. The `RestoreEngine` filters entries by replication type and skips everything except `KeyValues`. This means:

- **Locks are not restored.** Distributed locks (both persistent and ephemeral) do not survive a PITR restore. After a restore, the cluster starts with no active locks. Applications that depend on lock state must re-acquire their locks after recovery.
- **Range map and snapshot floor entries are skipped.** The range map is reconstructed from the partition configuration, not from backup data. Snapshot floor holds are transient and do not persist across restores.

28.11 PITR Bootstrap

A new node can join an existing cluster by bootstrapping from a backup chain instead of receiving state transfers from live peers. This path is useful when adding a node to a cluster where the WAL on existing nodes has been compacted past the required entries.

Start the node with both `--join-existing` and `--pitr-bootstrap-from` (the leaf backup GUID):

```
kahuna-server --join-existing \
  --initial-cluster node1:8081,node2:8082 \
  --pitr-backup-dir /backups/kahuna \
  --pitr-bootstrap-from a1b2c3d4-...
```

The `BootstrapHelper.BootstrapNodeAsync` method:

1. Validates partition coverage (the chain must cover all cluster partitions).
2. Validates the target timestamp against the chain's recoverable window.
3. Applies a guard rail: the target must fall within the PITR window (`now - pitrWindow - baseSnapshotInterval`).
4. Replays incremental segments through the same `RestoreEngine` used by the offline restore.
5. Writes synthetic `CommittedCheckpoint` WAL entries for each partition so that the Raft layer knows the node's committed index.
6. The node then joins the cluster normally. The leader sends only delta `AppendEntries` messages for entries committed after the bootstrap point.

28.12 Retention and Garbage Collection

Without retention policies, the backup store grows without bound. Kahuna provides chain-aware retention planning and periodic garbage collection.

28.12.1 Retention Policies

Three retention limits are available. All are optional. When multiple limits apply, the most restrictive one wins:

- **MaxChains.** The maximum number of retained backup chains. Chains are ranked newest-first. Excess chains (and all their artifacts) are deleted.
- **MaxAge.** The maximum age of a chain in seconds. Chains older than this limit are deleted.
- **MaxTotalBytes.** The maximum total byte size of all retained artifacts. When the limit is exceeded, the oldest chains are deleted first.

The newest chain is always kept, even if it alone exceeds `MaxTotalBytes`.

28.12.2 Chain-Aware Deletion

Retention operates on whole chains, not individual backups. A leaf backup (one that is not referenced as a parent by any other backup) identifies a chain. If the leaf is marked for deletion, the entire chain is deleted. Deletion order is descendants before ancestors: incremental backups are removed before their parent full backup. This prevents orphaned incrementals from remaining after the full backup is gone.

A kept leaf pins its entire transitive parent closure. An intermediate incremental that is shared between two chains is retained as long as either chain survives.

28.12.3 Orphan Sweep

Artifacts that exist on disk without a corresponding manifest are orphans. These arise from interrupted or failed backup operations. The garbage collector sweeps orphan artifacts but protects any backup that has a manifest file (including manifests with validation errors). This ensures that a corrupt but recoverable backup is never silently deleted.

28.12.4 The GC Reaper

The `BackupGcReaperActor` runs on a periodic timer:

- **First tick:** 15 seconds after node startup (the startup sweep).
- **Subsequent ticks:** every `BackupGcInterval` seconds (default 3600, or 1 hour).
- **Inline after backup:** the GC also runs after each successful backup operation.

GC is serialized with backup creation through a gate semaphore. This prevents a race where the GC deletes a manifest that a concurrent backup is about to reference as a parent.

28.13 REST Endpoints

Kahuna exposes backup operations through REST:

Method	Path	Purpose
POST	`/v1/backups/full`	Take a full backup
POST	`/v1/backups/incremental`	Take an incremental backup (body: `ParentBackupId`)
POST	`/v1/backups/coordinated`	Take a coordinated cluster backup
GET	`/v1/backups`	List all backups
GET	`/v1/backups/{id}/chain`	Resolve a chain from a leaf backup
POST	`/v1/backups/validate-chain`	Validate a chain (body: `LeafBackupId`)
POST	`/v1/restore`	Restore to a directory (body: `LeafBackupId`, `TargetDir`, `TargetTimeMs`)
POST	`/v1/backups/gc`	Run garbage collection (`?dryRun=true` for a dry-run inventory)

28.14 Backup Outcomes

Every backup operation returns a `KahunaBackupOutcome` that describes the result:

- **Ok.** The backup completed.
- **NotConfigured.** No backup directory is configured on this node.
- **NeedsFull.** The incremental cannot proceed because WAL compaction passed the required index. The service falls back to a full backup automatically.
- **ParentMissing.** The specified parent backup does not exist in the catalog.
- **CorruptChain.** The chain failed validation (broken links, index gaps, or identity mismatch).
- **CorruptArtifact.** An artifact's checksum or size does not match the manifest.
- **TargetConflict.** The target directory already exists or overlaps with another path.
- **TargetOutsideCoverage.** The requested PITR target falls outside the chain's recoverable window.
- **TopologyChanged.** The cluster topology changed during the backup.
- **RetryableLeadershipLoss.** The node lost leadership during the backup. The caller can retry.
- **NotBackupCoordinator.** The node is not the meta-partition leader (for coordinated backups).
- **ExactCheckpointUnavailable.** The persistence backend does not support exact-as-of checkpoints.
- **IoError.** A filesystem error occurred during the backup or restore.
- **Cancelled.** The operation was cancelled through the cancellation token.
- **InsecureRoot.** The restore root path failed safety checks.
- **RestrictedCoverage.** The chain does not cover all cluster partitions.

28.15 Backup Storage Abstraction

Kahuna separates backup storage into two interfaces. `IBackupStorageTarget` handles manifest CRUD (small-object operations). `IBackupArtifactStore` handles artifact bytes: checkpoint trees and WAL segments that can be multiple gigabytes each.

The built-in `local` target stores both manifests and artifacts under the directory specified by `--pitr-backup-dir`. For cloud or object storage, a host registers a custom `BackupStorageProvider` delegate on `KahunaConfiguration.BackupStorageProvider`. The delegate receives a `BackupStorageContext` (the target name, the backup directory, and the scratch directory) and returns a `BackupStoragePair` containing both stores.

The `--pitr-backup-target` flag selects the provider. The value `local` uses the built-in filesystem implementation. Any other value requires a registered provider. Startup validation refuses a non-local target that has no registered provider.

28.15.1 Capabilities

Each artifact store declares its capabilities through `BackupArtifactStoreCapabilities`:

Capability	Local	Object Store
`SupportsPosixHardening`	true (owner-only permissions, symlink rejection)	false (bucket policy handles security)
`SupportsCheapRangeReads`	true (local seek)	depends on store (HTTP range requests)
`RequiresLocalScratch`	false	true

When `RequiresLocalScratch` is true, every checkpoint first writes to a local staging directory and then uploads to the remote store. The `--pitr-backup-scratch-dir` flag provides that directory. Startup validation refuses a store that requires local scratch without a configured scratch directory.

28.15.2 Publication Ordering

Artifacts are always published before the manifest. A crash during a backup leaves orphan artifacts with no manifest. Orphan artifacts are safe: the garbage collector sweeps them. A manifest that points to absent artifacts can never occur, because the manifest is the last thing written. This ordering is enforced by the `IBackupCheckpointStagingArea` contract: the local store commits as a no-op (the checkpoint path is already the final path), while a remote store uploads the bytes on commit and discards scratch on disposal.

28.15.3 Incomplete Backups

The `ListBackupsAsync` response includes an `IsIncomplete` flag on entries where artifacts exist but no manifest does. An incomplete entry means the backup upload was interrupted before the manifest was published. Incomplete entries also carry `IsInvalid = true`. The distinction tells the operator whether a backup "never finished" (incomplete) or "finished but the manifest is corrupt" (invalid only).

28.16 Configuration Reference

Flag	Default	Purpose
`--pitr-window`	3600s (1h)	WAL retention window for PITR coverage
`--base-snapshot-interval`	1800s (30min)	Interval between automatic base checkpoints
`--pitr-backup-dir`	(none)	Root directory for backup artifacts and manifests
`--pitr-backup-target`	local	Storage target (local filesystem or registered provider)
`--pitr-backup-scratch-dir`	(none)	Local staging directory for non-local targets
`--pitr-backup-cluster-id`	(none)	Cluster identity string (gates chain resolution across clusters)
`--pitr-backup-mac-key-file`	(none)	Path to HMAC-SHA-256 key file for manifest authentication
`--backup-retention-max-chains`	0	Maximum retained chains (0 = unbounded)
`--backup-retention-max-age`	0	Maximum chain age in seconds (0 = unbounded)
`--backup-retention-max-bytes`	0	Maximum total retained bytes (0 = unbounded)
`--backup-gc-interval`	3600s	Periodic GC cadence (0 disables periodic GC)
`--backup-restore-throttle-mbps`	0	Restore copy throughput limit in MB/s (0 = unlimited)
`--pitr-restore-root`	(none)	Allowed root directory for remote restore operations
`--pitr-allow-unconfined-remote-restore`	false	Allow remote restores without a root directory
`--pitr-bootstrap-from`	(none)	Leaf backup GUID for PITR bootstrap on join

28.17 Failure Scenarios

Backup during high write load. A full backup acquires an MVCC snapshot hold and flushes the background writer. The flush blocks new writes from reaching the storage backend until the checkpoint completes. On a node with heavy write traffic, this pause can cause WAL queue buildup and increased commit latency. Schedule full backups during low-traffic windows.

Incremental backups are lighter because they stream WAL segments without pausing the writer.

Backup chain corruption. If an artifact file is corrupted on disk (bit rot, partial write, storage failure), the chain validation detects the mismatch through SHA-256 verification. The backup is marked as `CorruptArtifact`. The corrupted backup and all its descendants are unusable. Take a new full backup to start a fresh chain. The GC reaper does not delete corrupt backups automatically (the manifest still exists), so the operator must delete them manually or wait for retention policies to expire them.

Restore to a time before the pruned-history floor. MVCC revision pruning deletes old versions of key-value entries. If the target PITR timestamp falls before the pruned-history floor, the full backup checkpoint may already be missing the revisions needed for a consistent restore. The backup driver checks the floor before creating the checkpoint and refuses the backup if pruning already passed the cut. For the restore path, the chain coverage validator refuses a target outside the recoverable window.

WAL compaction breaks incremental chain. If the WAL is compacted past the high-water mark of the parent backup, the incremental cannot read the required entries. The driver returns `NeedsFullBackup`. The `BackupService` catches this outcome and takes a full backup as a fallback. To prevent this, set `--pitr-window` to a value longer than the expected interval between backups.

Topology change during backup. If a node joins, leaves, or if a range splits during the backup, the topology generation changes. The driver detects the mismatch and aborts. The caller can retry. For coordinated backups, the coordinator also verifies that its leadership term has not changed.

MAC key mismatch. If the HMAC key on the restoring node differs from the key used to sign the manifest, verification fails and the restore is refused. Keep the MAC key consistent across all nodes and across backup and restore operations. Store the key file outside the backup directory.

29 Observability and Performance Tuning

A Kahuna cluster that runs well in development may degrade under production load. A single hot partition, an undersized cache, or an aggressive WAL compaction policy can turn a responsive system into an unresponsive one. Operators need metrics to detect problems before users notice them, diagnostic tools to locate the bottleneck, and tuning parameters to resolve it.

This chapter covers three topics: the metrics Kahuna exposes, the benchmark tool for measuring baseline performance, and the tuning knobs that control the IO scheduler, write batching, caching, memory budgets, and transaction admission.

The chapter builds on three earlier chapters. Chapter 14 introduced the architecture. Chapter 22 described the persistence backends and the background writer. Chapter 24 covered deployment modes. Observability and tuning operate on the runtime behavior of those subsystems.

29.1 Metrics Architecture

Kahuna emits all metrics through `System.Diagnostics.Metrics` with a meter named `"Kahuna"` (version `"1.0"`). Instrument names follow the OpenTelemetry dot-separated lowercase convention: `kahuna.durable_tx.finalize_prepare_ms`, `kahuna.kv.write.batches`, and so on.

Kahuna does not bundle a Prometheus or OpenTelemetry exporter. The metrics are standard .NET instruments that any subscriber can consume:

- **dotnet-counters.** For local debugging: `dotnet-counters monitor --process-id <pid> Kahuna`.
- **OpenTelemetry SDK.** Wire an OTEL exporter (Prometheus, OTLP, or any other) in the hosting application's dependency injection container.
- **Embedded API.** When Kahuna runs as an `EmbeddedKahunaNode`, the host application owns the DI container and can register any exporter.

Prometheus exporters translate dots to underscores automatically, so `kahuna.kv.write.batches` becomes `kahuna_kv_write_batches` in Prometheus.

29.2 Status Dashboard

Kahuna ships a browser-based operator dashboard. The dashboard is on by default and serves at the HTTP root (`/`). To disable it, set `--disable-dashboard`.

The dashboard polls two read-only REST endpoints:

- `GET /v1/dashboard/summary` returns node identity, readiness, storage configuration, cluster mode, membership version, replication factor, uptime, heap bytes, and thread count.
- `GET /v1/dashboard/metrics` returns a curated snapshot of both Kahuna and Raft instruments, cumulative since process start. The browser computes rates between two successive readings.

The poll interval is `--dashboard-refresh-seconds` (default 5, clamped 1 to 300).

29.2.1 Dashboard Panels

The dashboard displays six panels:

1. **Node.** Endpoint, name, role, readiness, hosted partitions, cluster mode, member count, storage backend, backup configuration, version, uptime, heap bytes, thread count.
2. **Engine.** Curated metrics from both the `Kahuna` and `Kommander` meters. Counters appear as cumulative totals with computed rates. Histograms show count, sum, min, max, and last value.
3. **Roster.** Cluster membership table (from `GET /v1/cluster/members`).
4. **Placement.** Partition replica sets (from `GET /v1/cluster/placement`).
5. **Backups.** Backup catalog (from the backups endpoint).
6. **Key Ranges.** Per-key-space routing descriptors (from the ranges endpoint).

29.2.2 Metrics Collection

The `EngineMetricsCollector` subscribes to both the `"Kahuna"` and `"Kommander"` .NET Meters in-process. It accumulates measurements (counters sum, histograms track count, sum, min, max, and last value, gauges report instant value). The `BuildMetrics()` method filters through a curated allowlist and caps the response at 400 rows. The collector runs inside the Kahuna process. It does not export metrics to an external system. External export still requires a .NET OpenTelemetry exporter.

29.3 Key Metrics

Kahuna defines nine metrics classes. This section groups them by subsystem and highlights the instruments most useful for production monitoring.

29.3.1 Write Path Metrics

The `PartitionWriteAggregatorMetrics` class tracks the write batching pipeline:

Instrument	Type	Name
Counter	`kahuna.kv.write.admitted`	Direct writes admitted into the queue
Counter	`kahuna.kv.write.rejections`	Writes rejected, tagged by reason
Counter	`kahuna.kv.write.batches`	Raft batches dispatched
Counter	`kahuna.kv.write.entries`	Total log entries dispatched
Counter	`kahuna.kv.write.outcomes`	Batch outcomes, tagged by result (success, transient, permanent)
Histogram	`kahuna.kv.write.batch_items`	Entries per batch
Histogram	`kahuna.kv.write.batch_bytes`	Bytes per batch
Histogram	`kahuna.kv.write.queue_age`	Oldest item age in each dispatched batch (ms)
Histogram	`kahuna.kv.write.raft_duration`	Raft call duration (ms)
Gauge	`kahuna.kv.write.queued_items`	Items admitted but not yet completed
Gauge	`kahuna.kv.write.queued_bytes`	Bytes admitted but not yet completed
Gauge	`kahuna.kv.write.in_flight_partitions`	Partitions with an in-flight batch

What to watch. A rising `queued_items` gauge means writes are arriving faster than Raft can commit them. A high `queue_age` histogram means items wait a long time before dispatch. Rising `rejections` with reason `queue_full` means the node is shedding load.

Rejection reasons include `queue_full`, `oversized`, `inbox_full`, `stopping`, `fence_stale`, and `queue_expired`.

29.3.2 Transaction Metrics

The `DurableTransactionMetrics` class tracks the transaction lifecycle:

Instrument	Type	Name
Counter	`kahuna.durable_tx.one_phase_commits`	One-phase fast-path commits
Counter	`kahuna.durable_tx.one_phase_fallbacks`	One-phase eligible transactions that fell back to 2PC
Counter	`kahuna.durable_tx.admission_rejections`	Transactions refused at the outstanding cap
Counter	`kahuna.durable_tx.deadline_expiry_aborts`	Recovery aborts from deadline expiry
Counter	`kahuna.durable_tx.late_commit_rejections`	Commits rejected past the decision deadline
Counter	`kahuna.durable_tx.gc_records_reclaimed`	Transaction records removed by GC
Histogram	`kahuna.durable_tx.finalize_prepare_ms`	Prepare-stage wall time (ms)
Histogram	`kahuna.durable_tx.finalize_validate_ms`	Read-set validation wall time (ms)
Histogram	`kahuna.durable_tx.finalize_decision_ms`	Decision-stage wall time (ms)
Histogram	`kahuna.durable_tx.finalize_read_keys`	Read-set size per finalize
Histogram	`kahuna.durable_tx.decision_deadline_margin_ms`	Time remaining before deadline at decision (ms)
Gauge	`kahuna.durable_tx.outstanding`	Outstanding durable transactions
Gauge	`kahuna.durable_tx.resident_records`	Resident transaction records
Gauge	`kahuna.durable_tx.resident_prepared_intents`	Resident prepared intents
Gauge	`kahuna.durable_tx.resident_prepared_intent_bytes`	Resident prepared-intent value bytes
Gauge	`kahuna.durable_tx.resident_receipts`	Completion receipts resident on this node
Counter	`kahuna.durable_tx.gc_receipts_reclaimed`	Completion receipts released by the [illegible] GC [illegible]
Histogram	[illegible]	[illegible] transactions entered, [illegible] beyond_writes, [illegible] predicate [illegible] ion_read, [illegible] non-[illegible] tagged [illegible] read, [illegible]

What to watch. A high `outstanding` gauge indicates transactions that have not finished. Rising `admission_rejections` means the cap is too low for the workload. A shrinking `decision_deadline_margin_ms` means transactions are close to their deadline. If `deadline_expiry_aborts` is non-zero, some transactions are timing out before they can commit.

29.3.3 Transaction Admission Metrics

The `TransactionPriorityMetrics` class tracks the admission control gates. All gauges are tagged by `gate` (script or session) and `priority`:

Instrument	Type	Name
Gauge	`kahuna.tx_admission.in_flight`	Transactions holding an admission slot
Gauge	`kahuna.tx_admission.queued`	Transactions waiting for a slot
Gauge	`kahuna.tx_admission.max_queued`	High-water mark of waiters
Gauge	`kahuna.tx_admission.admitted`	Total transactions admitted since startup
Gauge	`kahuna.tx_admission.aged_promotions`	Waiters promoted by anti-starvation aging
Gauge	`kahuna.tx_admission.abandoned_while_waiting`	Waiters that timed out before admission
Gauge	`kahuna.tx_admission.rejected_queue_full`	Requests refused because the queue was full

What to watch. A non-zero `queued` gauge means the admission gate is deferring work. A rising `rejected_queue_full` means the node is shedding transactions. A high `aged_promotions` count means low-priority work is aging into higher priority slots.

29.3.4 Range and Split Metrics

The `RangeSplitMetrics` class tracks automatic range operations:

Instrument	Type	Name
Counter	`kahuna.range.splits`	Total splits committed (count-based and load-based)
Counter	`kahuna.range.split.indivisible_refusals`	Splits refused because no good split key exists
Counter	`kahuna.range.split.settle_skips`	Descriptors skipped because they are in a post-split settle window
Counter	`kahuna.range.split.no_relief_skips`	Load splits skipped because no peer node can host the child
Counter	`kahuna.range.merge.warm_skips`	Merge candidates skipped because the partition is warm
Counter	`kahuna.range.split.drain_refusals`	Moves refused because the range still held unsettled durable intents

What to watch. Rising `indivisible_refusals` means a range cannot split because writes concentrate on too few keys. Rising `no_relief_skips` means the cluster has no capacity to absorb new partitions after a split.

29.3.5 Snapshot Floor Metrics

The `SnapshotFloorMetrics` class tracks MVCC snapshot holds:

Instrument	Type	Name
Counter	`kahuna.snapshot_floor.missing_protected_version_total`	Floor-protected revisions scheduled for trimming (must stay 0)
Counter	`kahuna.snapshot_floor.prune_skipped_unconfirmed_total`	Prune cycles skipped because meta-partition catch-up is unconfirmed
Gauge	`kahuna.snapshot_floor.live_holds`	Live MVCC snapshot holds
Gauge	`kahuna.snapshot_floor.effective_floor_ms`	Effective floor timestamp (ms)

What to watch. `missing_protected_version_total` must stay at zero. A non-zero value means the pruning system attempted to delete a revision that was still protected by a snapshot hold. `live_holds` should stay low; a growing count means snapshot holds are not being released.

29.3.6 Cache Eviction Metrics

The `CollectMetrics` class tracks the per-partition key-value cache:

Instrument	Type	Name
Counter	`kahuna.collect.cycles`	Eviction cycles executed
Histogram	`kahuna.collect.cycle.duration`	Duration of each cycle (ms)
Counter	`kahuna.collect.evicted`	Entries evicted, tagged by reason
Counter	`kahuna.collect.inspected`	Entries inspected during scan
Counter	`kahuna.collect.backlogged`	Cycles that carried work past the eviction budget

Eviction reasons: `tombstone` (deleted entries drained), `expiry` (TTL elapsed), `lru` (evicted under budget pressure), `idle` (untouched past the idle TTL).

What to watch. A sustained high rate of `lru` evictions means the working set exceeds the cache budget. The partition re-reads evicted entries from the storage backend, which increases read latency and IO load.

29.3.7 Placement Metrics

The `PlacementMetrics` class tracks replica placement changes:

Instrument	Type	Name
Counter	`kahuna.placement.replicas_granted`	Partitions this node started hosting
Counter	`kahuna.placement.replicas_lost`	Partitions this node stopped hosting
Counter	`kahuna.placement.forwards_resolved`	Non-hosted operations that found a forward target
Counter	`kahuna.placement.forwards_unresolved`	Non-hosted operations with no forward target
Counter	`kahuna.placement.leader_hint_hits`	Forward targets answered by the gossiped leader hint
Counter	`kahuna.placement.leader_hint_misses`	Forward resolutions the leader hint could not answer

29.3.8 Backup Metrics

The `BackupIoMetrics` and `BackupGcMetrics` classes track backup operations:

Instrument	Type	Name
Counter	`kahuna.backup.operations`	Backups completed
Counter	`kahuna.backup.failures`	Backup attempts that failed
Counter	`kahuna.backup.bytes`	Artifact bytes written
Histogram	`kahuna.backup.duration_ms`	Backup duration (ms)
Counter	`kahuna.restore.operations`	Restores completed
Counter	`kahuna.restore.entries_applied`	WAL entries applied during restore
Counter	`kahuna.restore.failures`	Restore attempts that failed
Counter	`kahuna.restore.bytes`	Artifact bytes read by restores
Histogram	`kahuna.restore.duration_ms`	Restore operation duration (ms)
Counter	`kahuna.backup.gc.orphans_reclaimed`	Orphaned artifacts reclaimed by GC
Counter	`kahuna.backup.gc.retention_deletions`	Backups deleted by retention policy
Counter	`kahuna.backup.gc.runs`	Backup GC passes completed
Counter	`kahuna.backup.gc.bytes_reclaimed`	Artifact bytes freed by retention deletions

29.3.9 Persistence Metrics

The background writer and backlog monitor expose disk health:

Instrument	Type	Name
Gauge	`kahuna.persistence.oldest_inflight_write_age_ms`	Age of the oldest in-flight store write (ms)
Histogram	`kahuna.persistence.store_write_duration`	Wall time of each store write call
Gauge	`kahuna.durable_tx.retention_over_budget`	1 while retention budget is exceeded and early reclamation is active; 0 otherwise

The `UnflushedBacklogAlertPolicy` warns at 75% of the write-aggregator budget and when backpressure engages. Log noise is reduced to one warning per streak with 10-minute reminders through `RetentionBudgetLogGate`.

29.3.10 Scan Metrics

The `KeyValueScanMetrics` class tracks range and prefix scan operations:

Instrument	Type	Name
Counter	`kahuna.scan.snapshot_prefix_rows_examined_total`	Physical rows examined by snapshot prefix scans
Counter	`kahuna.scan.snapshot_prefix_entries_returned_total`	Entries returned by snapshot prefix scans
Counter	`kahuna.scan.abandoned_cancelled_total`	Backend scans skipped or stopped because their continuation expired

What to watch. A high ratio of `rows_examined` to `entries_returned` means scans read many rows to produce few results. This happens when tombstones or expired entries dominate the key range. A rising `abandoned_cancelled_total` means scan continuations time out before the client fetches the next page.

29.3.11 Raft Metrics

The Kommander library emits its own metrics under the `"Kommander"` meter. The dashboard and the `EngineMetricsCollector` observe these instruments. They are separate from the `"Kahuna"` meter.

Instrument	Type	Name
Counter	`raft.executor.operations_total`	Operations executed by the shared executor pool
Counter	`raft.executor.rejections_total`	Operations rejected by the executor (queue full)
Histogram	`raft.executor.operation_duration_ms`	Per-operation execution time (ms)
Gauge	`raft.executor.client_queue_depth`	Client operations queued in the executor
Counter	`raft.wal.operations_total`	WAL write operations completed
Counter	`raft.wal.batches_total`	WAL write batches committed
Histogram	`raft.wal.batch_size`	Entries per WAL batch
Gauge	`raft.wal.queue_depth`	WAL write operations queued
Gauge	`raft.wal.durability_floor_lag`	Entries between the committed index and the durable floor
Histogram	`raft.heartbeat_delay_ms`	Heartbeat round-trip delay (ms)
Counter	`raft.heartbeats_sent_total`	Heartbeats sent by leaders
Counter	`raft.elections_started_total`	Elections started (leader loss or startup)
Histogram	`raft.election_delay_ms`	Time between election start and leadership confirmation (ms)
Counter	`raft.stale_completions_total`	Completions for operations whose term is stale
Counter	`raft.backfill.no_progress_episodes_total`	Backfill attempts with no follower progress
Counter	`raft.snapshot.transfer_failures_total`	Snapshot transfers that failed

What to watch. A rising `raft.wal.queue_depth` means WAL writes queue faster than the backend can flush. A rising `raft.executor.client_queue_depth` means client proposals queue in the executor. High `raft.heartbeat_delay_ms` values indicate network latency or CPU saturation on the leader. Non-zero `raft.elections_started_total` during stable operation indicates unexpected leader changes.

29.3.12 Transaction Diagnostic Metrics

The `DurableTransactionMetrics` class also defines approximately 20 diagnostic counters for internal transaction coherence. These instruments track rare conditions such as staged-base fence wedges, materialization repairs, coherence reconciles, and recordless intent holds. They are most useful when debugging transaction correctness issues. They are visible through `dotnet-counters`, the dashboard, and any OTEL exporter.

The KV subsystem defines additional diagnostic counters: `kahuna.kv.retry_waits` (back-off sleeps by site), `kahuna.kv.scan_page_retry_budget_exhausted` (scans that exhausted their retry budget), `kahuna.kv.session_owned_intent_ceiling_expiries` (orphaned intent cleanup), and `kahuna.kv.abort_fenced_commit_applies` (commit applies refused because the record is terminal Abort).

29.4 The Benchmark Tool

Kahuna ships a benchmark tool (`kahuna-bench`) for measuring baseline performance and validating tuning changes. The tool supports multiple workloads, two execution modes, and correct latency measurement.

29.4.1 Workloads

The tool supports ten workload types:

Workload	Description
`set`	Write-only: set keys with random values
`get`	Read-only: get keys (after seeding)
`mixed`	Read-write: mix of get and set at a configurable ratio
`delete`	Delete-only: delete keys (after seeding)
`set-many`	Batch writes: set keys in batches
`delete-many`	Batch deletes: delete keys in batches
`txn`	Interactive transactions: read and write multiple keys in a transaction
`lock`	Distributed locks: acquire and release locks
`sequence`	Distributed sequences: reserve sequence values
`script`	Script execution: run a user-supplied `.4gl` script

29.4.2 Execution Modes

Closed-loop (default, `--rate 0`). A fixed number of concurrent workers (`--concurrency`, default 64) fire requests as fast as possible. Each worker waits for a response before sending the next request. This mode measures maximum throughput but underreports tail latency because slow requests naturally reduce the request rate.

Open-loop (`--rate N`). A producer emits N tickets per second through a bounded channel. Workers consume tickets and measure latency from the *intended* start timestamp, not the actual start. When the server is slow, tickets queue up in the channel. Each ticket's measured latency includes the queuing delay. This correctly accounts for coordinated omission: a slow response at time T delays all subsequent requests, and the benchmark counts that delay in every affected measurement.

The tool uses HdrHistogram with 3 significant digits for latency recording. Pacing uses a hybrid approach: `Task.Delay` for the coarse portion and `Thread.SpinWait` for sub-millisecond precision.

29.4.3 Benchmark Phases

Each benchmark run has three phases:

1. **Seed.** Populate the key space with initial data (for workloads that read or delete existing keys).
2. **Warmup.** Run the workload for `--warmup` seconds (default 5). Samples from this phase are discarded. This allows JIT compilation, connection pooling, and cache warming to stabilize.
3. **Measurement.** Run the workload for `--duration` seconds (default 30). Only samples from this phase appear in the results.

29.4.4 Output

The default console output shows a table with columns: Operation, Count, req/s, p50, p90, p95, p99, p99.9, max, mean, and error/miss counts. Alternative output formats are `json` and `csv` (`--format`).

29.4.5 Key Options

Option	Default	Purpose
`--workload`	mixed	Workload type
`--duration`	30	Measurement window (seconds)
`--warmup`	5	Warmup seconds (discarded)
`--concurrency`	64	Concurrent workers
`--rate`	0	Target requests per second (0 = closed-loop)
`--key-space`	10000	Number of distinct keys
`--key-prefix`	"bench:"	Key prefix
`--value-size`	128	Value payload bytes
`--read-pct`	50	Read percentage for the mixed workload
`--batch-size`	100	Keys per set-many or delete-many batch
`--keys-per-txn`	4	Keys per interactive transaction
`--txn-locking`	pessimistic	Transaction locking mode (pessimistic or optimistic)
`--durability`	persistent	Durability level (persistent or ephemeral)
`--script`	(none)	Path to a `.4gl` script for the script workload
`--timeout`	10	Per-request timeout (seconds)
`--format`	console	Output format (console, json, or csv)
`--seed`	0	RNG seed (0 = time-based)

29.4.6 Example: Mixed Workload Benchmark

```
kahuna-bench \
  --workload mixed \
  --read-pct 80 \
  --concurrency 128 \
  --duration 60 \
  --key-space 100000 \
  --value-size 256 \
  --format console
```

This runs a read-heavy mixed workload (80% reads, 20% writes) with 128 concurrent workers for 60 seconds across 100,000 keys.

29.4.7 Example: Open-Loop Latency Test

```
kahuna-bench \
  --workload get \
  --rate 5000 \
  --concurrency 128 \
  --duration 30 \
  --key-space 50000
```

This measures read latency at a fixed rate of 5,000 requests per second. The open-loop mode reports accurate tail latency even when the server cannot sustain the target rate.

29.5 IO Scheduler Tuning

The `FairReadScheduler` manages read operations across partitions. It provides fair scheduling so that a read-heavy partition cannot starve reads to other partitions.

29.5.1 How It Works

The scheduler maintains a per-partition FIFO queue. Worker threads drain requests across partitions in a fair rotation. Each scheduling cycle drains up to 64 operations (the `MaxBatchSize`). The scheduler supports read coalescing: multiple reads to the same partition can be batched into a single backend call (for example, RocksDB's `MultiGet`).

When a partition's queue reaches the configured depth limit, new read requests receive a `ReadBackpressureExceededException`. This exception signals the caller to retry after a backoff.

29.5.2 Configuration

Flag	Default	Purpose
`--backend-read-io-threads`	8	Number of read worker threads
`--backend-write-io-threads`	1	Number of write worker threads
`--backend-read-queue-depth`	4096	Maximum queued reads per partition

When to adjust. Increase `--backend-read-io-threads` if read latency is high and the CPU is not saturated. The default of 8 is suitable for most deployments. Increase `--backend-read-queue-depth` if you see `ReadBackpressureExceededException` errors during legitimate traffic spikes (not during an overload that should be shed).

29.6 Write Coalescing Tuning

The `PartitionWriteAggregator` batches writes to reduce the number of Raft proposals. It uses 8 single-threaded lanes. Writes to the same partition share a lane and accumulate into batches.

29.6.1 The Linger Delay

When a write arrives and no batch is in flight for that partition, the aggregator waits `LingerMs` milliseconds (default 1) before dispatching the batch. This short delay allows more writes to accumulate, which produces larger batches and fewer Raft round-trips. A linger of 0 dispatches immediately (lowest latency, smallest batches). A linger of 2 to 5 ms produces larger batches under load but adds latency to every write.

29.6.2 Batch Limits

Each batch is capped by item count (`MaxBatchItems`, default 512) and byte size (`MaxBatchBytes`, default 4 MiB). When either limit is reached, the batch dispatches immediately without waiting for the linger delay.

29.6.3 Queue Limits

Per-partition queue limits prevent a single partition from consuming all node memory:

Limit	Default	Purpose
`MaxQueuedItemsPerPartition`	8192	Maximum queued items per partition
`MaxQueuedBytesPerPartition`	32 MiB	Maximum queued bytes per partition
`MaxQueuedItemsGlobal`	131,072	Maximum queued items across all partitions
`MaxQueuedBytesGlobal`	512 MiB	Maximum queued bytes across all partitions

A `TerminalReserveItemsPerPartition` of 256 items is reserved for transaction settlement. This ensures that a saturated partition can still admit commit and settle operations.

Items that sit in the queue longer than `MaxQueueDelayMs` (default 1000 ms) are released with a `MustRetry` status. The Raft round-trip deadline is `BatchExecutionTimeoutMs` (default 30,000 ms).

29.6.4 Configuration

Flag	Default	Purpose
`--kv-write-linger-ms`	1	Linger delay before batch dispatch (ms)
`--kv-write-max-batch-items`	512	Maximum entries per batch
`--kv-write-max-batch-bytes`	4194304	Maximum bytes per batch (4 MiB)

29.7 Cache Configuration

Each partition maintains an in-memory cache of key-value entries. The cache serves reads without touching the storage backend. Cache hits avoid disk IO entirely.

29.7.1 Limits

Parameter	Default	Purpose
`MaxEntriesPerActor`	50,000	Maximum cached entries per partition
`MaxBytesPerActor`	256 MiB	Maximum cached bytes per partition
`CacheEntryTtl`	1800s (30 min)	Idle TTL before eviction

29.7.2 Eviction

The cache evicts entries for four reasons:

1. **Tombstone.** Deleted or undefined entries are drained from a tombstone queue.
2. **Expiry.** The entry's TTL (set by the application) elapsed.
3. **LRU.** The entry is the least recently used and the cache exceeds its entry or byte budget.
4. **Idle.** The entry was not accessed within `CacheEntryTtl` seconds.

When to adjust. If the `kahuna.collect.evicted` counter with reason `lru` is high, the working set exceeds the cache budget. Increase `MaxEntriesPerActor` or `MaxBytesPerActor`. If the node has limited memory, consider reducing the cache budget and accepting higher read latency, or add more nodes to distribute the working set.

29.8 RocksDB Shared Memory

When Kahuna uses RocksDB as its storage backend, each partition opens its own RocksDB instance. Without coordination, each instance allocates its own block cache and memtable buffers. On a node with many partitions, the total memory usage can exceed the available RAM.

The shared memory feature pools block cache and memtable memory across all RocksDB instances on the node.

29.8.1 How It Works

`RocksDbSharedResources.CreateWithUnifiedBudget` creates one LRU block cache and one `WriteBufferManager`. The memtable sub-budget is cost-charged to the block cache, so both share a single memory bound. The cache uses soft (non-strict) mode to prevent write errors when memtables flush. The `WriteBufferManager` runs with `allow_stall = false` to prevent cross-database flush coupling.

29.8.2 Configuration

Flag	Default	Purpose
`--rocksdb-shared-memory`	false	Enable shared memory budget
`--rocksdb-shared-memory-budget-mb`	320	Total shared block cache (MB)
`--rocksdb-shared-memtable-budget-mb`	128	Memtable sub-budget within the cache (MB)

When to enable. Enable shared memory on any node with more than a few partitions using RocksDB. Without it, each instance allocates independently, and total memory use grows with partition count. A 320 MB shared budget with a 128 MB memtable sub-budget is a reasonable starting point for nodes with 8 to 32 partitions. Increase the budget for larger partition counts or for nodes with more available RAM.

The `MemtableMemoryUsage` property on the shared resources object reports live memtable memory for monitoring.

29.9 Transaction Admission Control

Under high concurrency, too many simultaneous transactions can exhaust resources: each transaction holds read-set entries, write intents, and an admission slot. Admission control limits the number of concurrent transactions and prioritizes important work.

29.9.1 Priority Levels

Kahuna defines five transaction priority levels:

Level	Value	Purpose
`Background`	0	Bulk or deferrable work
`Low`	1	Lower than ordinary but latency-relevant
`Normal`	2	Default for all transactions
`High`	3	Latency-critical application work
`Critical`	4	Must not be starved; never demoted

Unknown priority values are normalized to `Normal`.

29.9.2 Two Independent Gates

Kahuna runs two independent admission gates: one for script transactions and one for interactive sessions. The gates are separate because interactive sessions hold admission slots much longer than scripts (the session stays open for multiple round-trips).

29.9.3 Admission Mechanism

When the transaction count is below the configured ceiling, admission is transparent: requests pass through with no delay. When the count reaches the ceiling, new requests enter a priority queue. The gate dispatches waiting requests in priority order (highest first, FIFO within the same priority).

29.9.4 Reserved Slots

A configurable number of slots from the concurrency ceiling are reserved for `High` and `Critical` transactions. Lower-priority transactions cannot use reserved slots, even when the total count is below the ceiling. At least one slot always remains available for lower priorities.

29.9.5 Anti-Starvation Aging

A low-priority transaction that waits longer than `AgingThresholdMs` (default 1000 ms) gains one effective priority level. This prevents indefinite starvation under sustained high-priority

load. A `Background` transaction that waits 1 second becomes effectively `Low`. After another second, it becomes effectively `Normal`. `Critical` transactions are never demoted.

29.9.6 Queue Cap

The `maxQueued` parameter limits the total number of waiting transactions. Beyond this limit, new callers receive a null result (retryable load shedding). The `rejected_queue_full` metric tracks these rejections.

29.9.7 Configuration

Flag	Default	Purpose
`--max-concurrent-transactions`	0	Script transaction concurrency ceiling (0 = no limit)
`--max-concurrent-sessions`	0	Interactive session concurrency ceiling (0 = no limit)
`--transaction-priority-reserved-slots`	0	Slots reserved for High and Critical
`--transaction-priority-aging-threshold`	1000	Wait time before aging one priority level (ms)

The durable finalizer has its own cap: `DurableDecisionOutstandingMax` (default 100,000), which limits the number of outstanding durable finalize operations.

29.10 Tuning Recipes

29.10.1 Recipe: Diagnose High Write Latency

1. Check `kahuna.kv.write.raft_duration` histogram. If Raft call duration is high, the bottleneck is consensus.
2. Check `kahuna.kv.write.queue_age` histogram. If queue age is high but Raft duration is normal, writes are queuing before dispatch. Increase `--kv-write-max-batch-items` or reduce `--kv-write-linger-ms`.
3. Check `kahuna.kv.write.queued_items` gauge. If the queue is growing, writes arrive faster than Raft can commit. Add more nodes, split hot ranges, or reduce write volume.

29.10.2 Recipe: Diagnose High Read Latency

1. Check `kahuna.collect.evicted` counter with reason `lru`. If LRU evictions are frequent, the working set exceeds the cache. Increase `MaxEntriesPerActor` or `MaxBytesPerActor`.
2. Check for `ReadBackpressureExceededException` errors. If present, the IO scheduler's queue is full. Increase `--backend-read-queue-depth` or `--backend-read-io-threads`.
3. If using RocksDB, check whether shared memory is enabled. Without it, each partition's block cache competes with other partitions for system memory.

29.10.3 Recipe: Diagnose Transaction Timeouts

1. Check `kahuna.durable_tx.decision_deadline_margin_ms` histogram. A shrinking margin means transactions are close to their deadline.

2. Check `kahuna.durable_tx.finalize_prepare_ms` and `kahuna.durable_tx.finalize_validate_ms` histograms. A slow prepare or validate stage points to write contention or a large read set.
3. Check `kahuna.tx_admission.queued` gauge. Non-zero values mean transactions are waiting for admission. Increase `--max-concurrent-transactions` or `--max-concurrent-sessions`, or add priority configuration to ensure critical work gets through.

29.10.4 Recipe: Tune for Throughput

1. Increase `--kv-write-linger-ms` to 2 to 5 ms. Larger batches reduce Raft round-trips.
2. Increase `--kv-write-max-batch-items` to 1024 or higher if the workload produces large bursts.
3. Enable RocksDB shared memory and increase the budget to match available RAM.
4. Increase `--backend-read-io-threads` to match the number of storage devices.
5. Enable the leader balancer (`--raft-enable-leader-balancer true`) to spread leadership across nodes.

29.10.5 Recipe: Tune for Latency

1. Set `--kv-write-linger-ms` to 0. Batches dispatch immediately.
2. Keep `--kv-write-max-batch-items` at the default or lower. Smaller batches commit faster.
3. Increase cache budgets (`MaxEntriesPerActor`, `MaxBytesPerActor`) to reduce cache misses.
4. Set `--backend-read-io-threads` high enough that reads never queue.

29.11 Failure Scenarios

Read backpressure. When the per-partition read queue exceeds `--backend-read-queue-depth`, the IO scheduler throws `ReadBackpressureExceededException`. The client receives a retryable error. This prevents unbounded memory growth from queued reads. If this error appears during legitimate traffic, increase the queue depth or add read threads. If the error appears during a traffic spike that exceeds the node's capacity, the backpressure is working as designed: it sheds load to protect the node.

Cache eviction storms. When the working set exceeds the cache budget, the eviction cycle runs frequently and evicts entries that are needed again shortly. Each eviction forces a backend read. Under sustained pressure, the cache becomes ineffective and read latency approaches raw backend latency. Increase the cache budget, reduce the working set (by adding partitions or nodes), or accept the latency cost.

Admission refused under load. When `--max-concurrent-transactions` is set too low for the workload, the admission gate rejects excess transactions. The `rejected_queue_full` metric rises. Applications receive retryable errors. Set the ceiling based on the node's memory and CPU capacity. Use priority levels and reserved slots to ensure critical transactions get through even when the gate is saturated.

Hot partition. All writes concentrate on one range, and one node handles all the write traffic for that range. The write queue on that partition grows while other partitions sit idle. The `queued_items` gauge shows a spike on one partition. Solutions: enable load-based range splitting (`--range-split-load-threshold`), enable the leader balancer to spread leadership, or redesign the key scheme to distribute writes.

30 Testing Distributed Systems

A unit test verifies that a function returns the correct value. An integration test verifies that two components work together. Neither verifies that a distributed system keeps its safety properties when a network partition splits the cluster, a process crashes mid-replication, or a clock drifts between nodes. Kahuna uses three layers of testing to cover this gap: embedded cluster tests that run in-process, end-to-end tests that exercise the REST API, and Jepsen tests that inject real faults and check formal correctness properties against the resulting histories.

This chapter builds on Chapter 17, which introduced Raft consensus, leader election, and the quorum rule. The findings described here are drawn from the kahuna-jepsen repository and its FINDINGS.md document.

30.1 Embedded Cluster Tests

Kahuna's test suite includes an embedded cluster that runs three Raft nodes inside a single process. No Docker containers, no network sockets, no disk writes. The `BaseCluster` abstract class in `Kahuna.Server.Tests` provides the scaffolding.

30.1.1 How BaseCluster Works

`BaseCluster` assembles a three-node cluster using two in-memory communication layers. `InMemoryCommunication` carries Raft messages (votes, appends, heartbeats) between the nodes. `MemoryInterNodeCommunication` carries inter-node requests (leader forwarding, remote lag queries). Both layers route messages by endpoint string and deliver them as direct method calls inside the same process.

Each node gets a distinct election timeout seed. The base seed is 91000 milliseconds, and each node adds its `NodeId` (0, 1, or 2) to produce seeds of 91000, 91001, and 91002. This determinism ensures that node 0 wins the first election in every test run. A `TimingScale` multiplier (read from the `KAHUNA_TEST_TIMING_SCALE` environment variable) stretches all timeouts for slow CI runners.

Storage is entirely in memory. Each node uses a memory-backed write-ahead log and a memory-backed state store. The default configuration creates three data partitions plus one system partition. No RocksDB instance is opened and no file touches disk.

30.1.2 Assembly Methods

`BaseCluster` exposes several factory methods for different test shapes:

1. **AssembleThreeNodeCluster.** The standard three-node setup. Creates three `EmbeddedKahunaNode` instances that share an `InMemoryCommunication` bus and a `MemoryInterNodeCommunication` bus.
2. **AssembleCluster.** A parameterized version that takes a node count. Each node receives the same shared communication layers.
3. **AssembleSwimCluster.** Creates a cluster with the SWIM failure detector enabled. The test can then verify that SWIM correctly marks dead nodes and triggers eviction.
4. **AssembleLeaderBalancerCluster.** Creates a cluster with the leader balancer enabled. The test can verify that leadership distributes evenly across nodes.
5. **BuildNode.** Creates a single node with a configurable replication factor and placement rebalancer flag. The test can mix nodes with different configurations in one cluster.

6. **BuildNodeWithExternalWal.** Creates a node with an external write-ahead log provider, used by the PITR bootstrap tests in Chapter 27.

30.1.3 Test Helpers

`BaseCluster` provides four helper methods that handle the timing and retry challenges of testing a consensus system:

- **RetryOnMustRetry.** Wraps a client call in a retry loop. When Kahuna returns `MustRetry` (because leadership moved or the node is not initialized), the helper retries after a short delay. This is the correct way to handle transient unavailability in a test.
- **RunUnderStableLeadership.** Waits until every partition has a stable leader, then runs the test body. If leadership moves during the body, the helper retries the entire sequence. This method is appropriate for tests that assume a fixed leader.
- **RetryAsync.** A general-purpose retry with configurable delay and count.
- **WaitUntilAsync.** Polls a condition at a configurable interval until it returns true or a timeout expires.

30.1.4 EmbeddedKahunaNode

Each node in the cluster is an instance of `EmbeddedKahunaNode`. This class implements `IAsyncDisposable` and provides the full Kahuna server stack in a single object.

`EmbeddedKahunaNode` has two constructors. The standalone constructor creates a single-node cluster with phantom witnesses that simulate quorum for a one-node deployment. The cluster constructor takes external `ICommunication`, `IInterNodeCommunication`, and `IDiscovery` implementations. `BaseCluster` uses the cluster constructor and injects the in-memory layers.

The node exposes two properties: `Kahuna` (the `IKahuna` interface for client operations) and `Raft` (the `IRaft` interface for Raft-level queries such as leader identity and committed indexes).

`StartAsync` joins the cluster and waits until every hosted partition has a leader. `FlushAsync` forces the background writer to drain its queue. `DisposeAsync` follows a careful teardown order: drain pending writes, dispose the Raft layer, shut down actors gracefully, and dispose shared RocksDB resources last.

30.1.5 What Embedded Tests Cover

Embedded tests cover the fast, deterministic cases: key-value read and write, lock acquire and release, sequencer allocation, transaction commit and abort, and the interactions between these operations under stable leadership. They do not inject network faults, do not simulate process crashes, and do not test clock skew. For those properties, Kahuna uses Jepsen.

30.2 Jepsen Testing

Jepsen is a fault-injection framework for distributed systems, written in Clojure by Kyle Kingsbury. It runs a cluster of real nodes (typically five), generates client operations, injects faults (called nemeses), records every operation and its result into a history, and then checks that history against a formal correctness model.

Kahuna maintains a dedicated Jepsen test suite in the kahuna-jepsen repository. The suite runs five Kahuna nodes in Docker containers, each built from the same .NET binary that ships

in production. The test infrastructure requires Docker with at least 2 CPUs, the .NET 10 SDK, and Leiningen with a JDK for the Clojure harness.

30.2.1 Running the Suite

A test run follows three steps:

1. Build a Kahuna tarball with `scripts/build-tarball.sh`.
2. Start the Docker cluster with `docker/up.sh`.
3. Run a workload with `lein run test`, passing the workload name, fault types, and timing parameters.

A typical invocation:

```
lein run test --workload register --faults partition \
  --nodes n1,n2,n3,n4,n5 --time-limit 180 --concurrency 9 --rate 10
```

30.2.2 Workloads

The suite defines five workloads. Each tests a different Kahuna primitive against a different correctness property.

Register. Tests the key-value store as a linearizable CAS (compare-and-swap) register. Clients perform reads, writes, and compare-and-swap operations on a small set of keys. The Knossos checker verifies linearizability: every operation must appear to take effect at a single instant between its invocation and its completion, and the resulting sequence must be consistent with a single serial execution.

Lock. Tests the distributed lock with two properties. Mutual exclusion: no two processes may hold the same lock at the same time. Fencing-token monotonicity: every successive lock acquisition must receive a strictly higher fencing token than the previous one. The checker is lease-aware, meaning it accounts for the lock's configured expiry when computing hold windows.

Append. Tests interactive transactions using Elle's list-append model. Each transaction reads one or more lists and appends a unique value to one or more lists. Elle constructs a dependency graph from the committed history and checks for cycles that would violate serializability. This workload found four real bugs in Kahuna's transaction path.

Sequencer. Tests the distributed sequencer with three properties. Uniqueness: no identifier is handed out twice. Range integrity: every identifier falls within the allocated range. Idempotent replay: a retried allocation returns the same result.

Snapshot. Tests pinned MVCC snapshots. A client opens a snapshot hold at a given timestamp, reads several keys, and verifies that the snapshot never changes its answer. The checker compares each read against the snapshot's pinned revision and fails if any read returns a different value.

30.2.3 Nemesis Types

The suite injects seven types of faults:

1. **Partition.** Isolates one or more nodes from the rest of the cluster using iptables rules. Tests whether the system handles a network split correctly.

2. **Kill.** Sends SIGKILL to one or more Kahuna processes. Tests crash recovery and data durability.
3. **Pause.** Sends SIGSTOP to freeze a process in place, then SIGCONT to resume it. Unlike kill, a paused node keeps its connections and leases open but stops responding.
4. **Clock.** Skews the system clock on one or more nodes. Tests whether the system depends on synchronized clocks for safety.
5. **Membership.** Removes a node from the cluster roster with a graceful leave, then rejoins it. Tests the membership change machinery from Chapter 25.
6. **Placement.** Moves replicas between nodes by adjusting per-partition replication factor overrides. Tests the placement planner from Chapter 26.
7. **Range.** Forces key-range splits and merges under load. Tests the range management machinery from Chapter 26.

30.2.4 Test Profiles

Beyond the default five-node, single-replica configuration, the suite supports two specialized profiles:

Replication-factor profile. Uses six nodes (to allow decommission of one without losing quorum) and sets a replication factor of 3. The placement nemesis moves replicas between nodes while the workload runs. This profile found that the placement rebalancer planned no moves at all (fixed in kahuna `09c99a1`).

Key-range profile. Registers the key space for ordered routing so that keys are served by their position in the key order rather than by hash. The range nemesis forces splits and merges. This profile found a read-skew anomaly caused by unsettled intents during a range split's data copy.

30.2.5 Correctness Checkers

Each workload uses a checker matched to the property it tests:

- **Knossos.** Used by the register workload. Knossos is a linearizability checker that searches for a linearization of the history. If no linearization exists, it produces a minimal counterexample.
- **Elle.** Used by the append workload. Elle constructs a transaction dependency graph and searches for cycles that violate the claimed isolation level (serializability in Kahuna's case).
- **Lock checker.** A custom checker that reconstructs the sequence of lock holds from the history, verifies mutual exclusion, and verifies fencing-token monotonicity. The checker is lease-aware: it accounts for expiry times when computing definite hold windows.
- **Sequencer checker.** A custom checker that verifies uniqueness, range integrity, and idempotent replay.
- **Snapshot checker.** A custom checker that verifies pinned snapshot reads return stable values.
- **Range checker.** A custom checker that fails a run if two acknowledged splits name the same destination partition or if the sampled range map contains gaps or overlaps. It also refuses to pass a run in which zero splits occurred, because such a run provides no evidence.
- **Placement checker.** A custom checker that tracks replica additions, promotions, and removals. It refuses to pass a run in which zero replicas moved, because such a run provides no evidence that the rebalancer works.

The last two checkers illustrate an important principle: a vacuity gate. A test that exercises no faults and observes no movement cannot claim that the system handles faults and movement correctly. These checkers return `:unknown` rather than `:valid? true` when their preconditions are not met.

30.3 What Jepsen Found

The kahuna-jepsen suite found real bugs in Kahuna and in its consensus library, Kommander. This section describes the most significant findings, what they taught about distributed systems, and how they were fixed.

30.3.1 Case Study 1: Stale Reads from a Minority-Partitioned Node

Workload: register, with network partitions.

The bug. A node isolated from the majority kept answering reads with its last-known value. The node knew it could not replicate (writes correctly returned `MustRetry`), but reads were served from local state without a quorum check.

The asymmetry was the diagnostic clue. On the same isolated node, writes refused with `MustRetry` because the node could not reach a quorum. Reads served from local state because `KeyValueLocator.LocateAndTryGetValue` checked `raft.AmILeader()`, a local belief that remained true until the node stepped down.

A concrete trace from a failing run:

```
20:00:04.47  proc 244 (n2)  :info  :write [19 0]  :must-retry
20:00:06.44  proc  81 (n1)  :ok    :write [19 1]
20:00:06.51  proc 253 (n2)  :ok    :read  [19 0]  ← stale
20:00:09.46  proc 253 (n2)  :ok    :read  [19 0]  ← still stale, 11 s in
```

Node n2′s value froze at the value committed just before the partition. The rest of the cluster moved on. This reproduced in 1 of 3 runs with different partition shapes and keys.

The fix. `ConfirmLeadershipForRead` replaced the local `AmILeader` check with a quorum-confirmed Raft read-index. When the node cannot confirm its leadership with a majority, it returns `MustRetry`. This fix was applied across the key-value, lock, and sequencer locators.

Verified: 8 of 8 clean runs against a prior 1-in-3 reproduction rate.

The lesson. A local leadership belief is not the same as confirmed leadership. In Raft, a leader that loses contact with the majority does not know it has lost contact. It continues to believe it is the leader until its election timer expires. Any read served on that belief alone is potentially stale.

30.3.2 Case Study 2: Fencing Token Rollback

Workload: lock, with network partitions.

The bug. The lock partition's fencing counter rolled back 36 grants and replayed them with different owners:

```
… 113(n3) 114(n3) 115(n5)   ← monotonic to 115
   79(n5)  80(n5)  81(n4)   ← restarts at 79
   82(n5)  83(n3)  84(n5) … ← monotonic again
```

Each replayed token went to a different owner than the first time. This happened five seconds after a single node was isolated, on the majority side.

The root cause was in Kommander, not Kahuna. When a new leader was promoted, its WAL drain read the log while enqueued writes still sat in the write scheduler's queue. The drain missed those writes. A fencing token is minted from `entry.FencingToken + 1`, where the entry comes from locally applied state. The newly promoted leader's state was incomplete, so it minted tokens from a stale base.

The first fix attempt (Kommander 1.0.9) made the problem worse. It introduced gap-detection logic that misclassified the queued writes as holes and orphaned everything above them on ordinary commits. A second attempt produced `:token-reused-by-other-owner`, a worse violation than monotonicity: two different owners held the same fencing token.

The fix. Kommander 1.0.10 corrected the WAL drain to account for writes still in the scheduler's queue.

Verified: Two independent sets of 8 runs each, all clean (16 of 16). At the 25% prior failure rate, 16 consecutive clean runs is a roughly 1% outcome by chance.

The lesson. A state machine that promotes to leader must finish applying all committed entries before it serves any request. If the promotion drain races with a write queue, the promoted leader serves from an incomplete projection of committed state.

30.3.3 Case Study 3: Transaction Anomalies

Workload: append, with partitions and kills.

The append workload found two distinct anomalies in the transaction path.

Lost update (incompatible-order). Two transactions read the same base list `[17, 18, 19]`. One appended 22 and the other appended 10. Both committed. The result was two incompatible lists: `[17, 18, 19, 22]` and `[17, 18, 19, 10]`. Elle ruled out every isolation model down to and including read-committed.

The root cause was three stacked defects. The one that this workload's shape exposes: a read-modify-write's base was never validated. The intent's `BaseRevision` was nominal and folded only into the dedup digest. Once the in-memory write-intent lease expired, a read-modify-write committed blind over a moved base in every locking and validation mode.

The fix. Each read-then-written key's pre-write observation is now frozen into its prepared intent as the validated base. A staged-base compare-and-set runs before anything durable is proposed.

Aborted read (G1a). A transaction whose `commit-tx-session` returned `Aborted` had its append read by a later committed transaction. The server told the client its transaction was aborted, but the writes were visible to other transactions.

The root cause went through two rounds. The first fix addressed exception-handling paths that reported a definite abort without first installing a durable Abort record. The fix made `FinalizeAdmission.Rejected` return `MustRetry`, and made the rollback and session reaper install a durable Abort through a record CAS before claiming `RolledBack`.

The anomaly recurred. The second root cause: the one-phase fast path's pre-propose validation set a conflict `Aborted` in `context.Result` when its write-skew probe found a concurrent intent, then fell back to the standard flow. The standard flow's re-validation passed and committed the transaction, but the client received the stale `Aborted` from the first attempt.

Verified: 22 runs at the load configuration that reproduces the anomaly (concurrency 10, rate 15), 21 informative, 6713 committed transactions, zero anomalies.

The lesson. "Fixed" needs a verification protocol. The first verification of the aborted-read fix used 11 clean runs at a load level that could not reproduce the bug. At 7%, eleven clean runs happen roughly 44% of the time. The second verification used the correct load level and accumulated enough runs to distinguish a 7% defect from zero.

30.3.4 Case Study 4: Snapshot Rewind to Floor Boundary

Workload: snapshot, with no faults.

The bug. With two or more concurrent snapshot holds, every hold except the oldest rewound to the oldest hold's revision. A snapshot that should have read a recent value instead read the value as of an earlier snapshot's pinned point. This produced 64 violating reads in a single local run, and it required no fault injection at all.

The root cause: when multiple holds were active, the snapshot floor collapsed onto the oldest hold's revision. All subsequent holds inherited that floor instead of their own pinned point.

The fix. Commit `65fcc70` corrected the floor calculation to track each hold's revision independently.

The lesson. Safety properties must hold under normal operation, not only under faults. A bug that requires no nemesis to trigger is more serious than one that requires a specific fault combination, because every production deployment exercises the no-fault case continuously.

30.3.5 Case Study 5: Deposed Leader Keeps Committing

Workload: register, with majorities-ring partitions.

The bug. Under a majorities-ring partition (where both sides hold a majority by sharing an overlapping node), a follower that grants a higher-term vote does not adopt the term in memory. The deposed leader's appends pass the term fence on that follower and continue to commit. Acknowledged writes are lost when the new leader reuses the same log indexes.

A concrete trace from the failing run:

```
n5/p3  Sending vote to n1:8082 on Term=2
n1/p3  proclaimed leader Term=2 Votes=3
n2/p3  Proposed logs Logs=98       <- Term=1, still committing
n5/p3  Received logs from leader n2 with Term=1  <- ACCEPTED
n2/p3  Committed proposal Logs=98  <- client gets :ok
n1/p3  Proposed logs Logs=98       <- index 98 reused, different entry
```

Node n5 had already voted for n1 in Term 2, but its in-memory `currentTerm` stayed at 1. The append fence (`currentTerm > leaderTerm`) evaluated to `1 > 1`, which is false, so Term-1 appends from the deposed leader n2 sailed through. The root cause was a conditional term

adoption gated on `nodeState != Follower`. Raft requires a node to adopt term T when it grants a vote in term T, regardless of its state.

The fix. Kommander now assigns `currentTerm = voteTerm` unconditionally when granting a higher-term vote, outside the step-down guard. A second fix makes a leader step down and persist the newer term when an append acknowledgment returns a higher term.

The lesson. Persisted state and in-memory state can diverge. The WAL recorded term 2 (from the vote persistence), but the running node still operated at term 1. A restart would have fenced correctly; the live node did not. Any safety check that reads only hard state would miss this window.

30.3.6 Read Skew Under Range Splits

Workload: append, with range splits (no other faults).

The bug. A range split's data copy captured base rows but missed committed-but-unsettled prepared intents. Reads served through the intent overlay on the original partition returned the correct value. After the split, reads on the new partition's leader returned the pre-commit base.

The fix. A settle-before-cutover barrier now resolves all decided intents before the split's copy step. An export-truncation fix and straggler re-routing completed the repair.

30.3.7 Fencing Tokens Under Replica Placement

Workload: lock, with placement moves at replication factor 3.

The bug. Fencing tokens went backwards and were reused by different owners when the placement nemesis moved replicas during live lock traffic. The prior fencing fixes (verified at 16 of 16) were never exercised against replica movement because the placement controller moved nothing until the rebalancer was fixed.

The fix. Corrections to the WAL drain and state transfer paths ensured that a promoted leader on a moved replica starts from a complete projection of committed lock state.

30.3.8 Write Skew Under Key-Range Routing

Workload: append, with key-range routing.

The bug. Under key-range routing, all keys in a key space collapse onto a single partition (the whole-space descriptor), so multi-key transactions become single-partition. A G2-item write skew appeared: two transactions each read the other's key as empty and committed conflicting appends. The anomaly preceded any range split by 93 seconds, so the split was excluded as a cause.

The fix. The root cause was traced to unsettled intents during a snapshot install triggered by a decommission drain. Dropping node-local cached entries on snapshot install, combined with the settle-before-cutover barrier, closed the anomaly.

All findings described in this chapter have been fixed and verified by the passing Jepsen CI suite.

30.4 Interpreting Jepsen Results

A Jepsen run produces a directory of artifacts: the full operation history, the checker's verdict, timeline plots, and (for Elle) the cycle that constitutes the violation. Reading these results correctly requires attention to several subtleties.

30.4.1 A Clean Run Is Not Proof

A property that fails at a 7% rate produces 11 consecutive clean runs roughly 44% of the time. The kahuna-jepsen suite was burned by this twice: once when 6 clean runs closed a write-skew finding that recurred, and once when 11 clean runs at a low load level closed an aborted-read finding that reproduced at a higher load level. The lesson: match the verification run count and configuration to the reproduction rate and conditions.

30.4.2 Discard Uninformative Runs

A run that commits zero transactions is not a clean result. It is an uninformative result. The checker reports `:valid? true` because there is nothing to violate, but the run provided no evidence. Roughly 1 in 10 runs in the kahuna-jepsen suite committed zero transactions, typically because of repeated kills that prevented cluster initialization. Always check commit counts before counting a run as evidence.

30.4.3 Vacuity Gates

The range and placement checkers refuse to call a run valid when their preconditions are not met. A placement run in which zero replicas moved reports `:valid? :unknown, :cause :vacuous` rather than `:valid? true`. This is intentional. Without the gate, ten jobs with a broken rebalancer would all report green, and "placement validated" would be the conclusion.

30.4.4 Load Level Matters

The aborted-read anomaly reproduced at concurrency 10 and rate 15 but never at concurrency 5 and rate 5. The original verification used the low-load configuration, and 20 clean runs at that configuration proved nothing about the high-load case. When a defect depends on timing, the verification must use the timing conditions that reproduce it.

30.4.5 Node Mapping

In the Jepsen history, each operation records a process number. To determine which node served the operation, apply these formulas: `thread = process mod concurrency`, `node = thread mod 5 + 1`. This mapping is necessary to correlate client-side observations with server-side logs when diagnosing a violation.

30.5 Why Conventional Tests Are Not Enough

Embedded cluster tests run in a single process with in-memory communication. They verify that the code does the correct thing when messages arrive in order, when no messages are lost, and when no processes crash. These are necessary tests, but they exercise none of the failure modes that define distributed system correctness.

A network partition creates two groups of nodes that can each communicate internally but not with each other. If both groups contain a majority (a majorities-ring partition), both can

elect leaders. If both leaders serve writes, committed data is lost when the partition heals. No in-memory communication layer simulates this.

A process crash at the wrong moment can lose data that the leader reported as committed. If the write-ahead log has not been flushed, or if a checkpoint has advanced past unflushed state, a restarted node serves from a stale projection. No in-memory storage layer simulates this.

Clock skew can cause a node to believe its lease is still valid when it has already expired from the perspective of every other node. No shared-process clock simulates this.

Jepsen fills this gap by running real processes on separate machines (or containers), injecting real faults through real system calls (iptables, SIGKILL, SIGSTOP, clock manipulation), and checking the resulting history against a formal model. The bugs it found in Kahuna were real bugs that affected real safety properties. None of them would have been caught by the embedded cluster tests.

30.6 Configuration Reference

The following table lists the flags relevant to the Jepsen test suite. These are Jepsen harness flags, not Kahuna server flags.

Flag	Default	Purpose
`--workload`	(required)	Which workload to run: register, lock, append, sequencer, or snapshot
`--faults`	partition	Comma-separated fault types: partition, kill, pause, clock, membership, placement, range, or none
`--nodes`	n1,n2,n3,n4,n5	Comma-separated node names
`--time-limit`	60	Test duration in seconds
`--concurrency`	5	Number of concurrent client threads
`--rate`	10	Target operations per second
`--nemesis-interval`	15	Seconds between nemesis operations
`--replication-factor`	0	Per-partition replication factor (0 means default single-replica)
`--key-range`	false	Enable key-order routing instead of hash routing
`--health-interval`	2.0	Seconds between readiness health-check samples

30.7 Failure Scenarios

False verification. A verification run set that uses the wrong load level, the wrong fault combination, or too few runs can produce a false "fixed" verdict. The kahuna-jepsen suite was burned by this pattern multiple times. Always match verification conditions to the conditions that reproduce the bug. Always compute the probability that the observed clean run count could occur by chance at the known reproduction rate.

Uninformative washouts. Runs that commit zero transactions produce a clean checker verdict but provide no evidence. This happened roughly 1 in 10 runs. The cause was nodes that opened their HTTP port before cluster initialization completed, returning `MustRetry` for every request. Discard these runs from the evidence count.

Checker bugs. A checker is itself code, and checker code can have bugs. The lock checker produced false mutual-exclusion violations because it overwrote the release timestamp on retry, stretching the computed hold window past the actual release. The fix preserved the earliest release attempt's timestamp. A false-positive checker is worse than a false-negative one: it erodes trust in real findings.

Sampling artifacts. Range maps and health checks are sampled at discrete intervals. A 2-second health-check interval reported a 50/50 split between initialization time and consensus recovery time that was pure quantization error. At 0.5 seconds, initialization turned out to be two orders of magnitude smaller. Lower the sampling interval before quoting a number.

31 Appendix A: Script Language Reference

This appendix is a compact syntax reference for the Kahuna script language. Chapter 7 explains each construct in detail with examples. Use this appendix as a quick-lookup card.

All keywords are case-insensitive. `SET`, `set`, and `Set` are the same command. String literals use single quotes (`'hello'`) or double quotes (`"hello"`). Identifiers that collide with reserved words can be escaped with backticks (`` `my/key` ``). Placeholders use the `@` prefix (`@param1`).

31.1 Data Types

Type	Literal syntax	Runtime type
Integer	`42`, `-7`, `0`, `0xFF`	64-bit signed integer
Float	`3.14`, `-0.5`	64-bit double-precision
String	`'hello'`, `"hello"`	UTF-8 string
Boolean	`true`, `false`	Boolean
Null	`null`	Null
Array	(no literal; created by `..` range or multi-row commands)	List of values

31.2 Variables

```
LET x = 42
LET name = "alice"
LET val = GET mykey
```

`LET` binds a name to an expression result. Variables are scoped to the script execution.

31.3 Key-Value Commands

31.3.1 Persistent Commands

Command	Syntax	Description
`SET`	`SET key value [flags]`	Write a value
`GET`	`GET key`	Read the current value
`GET` (revision)	`GET key @ revision`	Read a specific revision
`GET` (as-of)	`GET key AS OF timestamp`	Read the value as of an HLC timestamp
`EXISTS`	`EXISTS key`	Check whether the key exists
`EXISTS` (revision)	`EXISTS key @ revision`	Check existence at a specific revision
`EXISTS` (as-of)	`EXISTS key AS OF timestamp`	Check existence as of a timestamp
`DELETE`	`DELETE key`	Delete the key
`EXTEND`	`EXTEND key milliseconds`	Extend the key's TTL
`GET BY BUCKET`	`GET BY BUCKET prefix`	Read all keys under a prefix (returns array)
`GET BY BUCKET` (as-of)	`GET BY BUCKET prefix AS OF timestamp`	Read all keys under a prefix as of a timestamp
`SCAN BY PREFIX`	`SCAN BY PREFIX prefix`	Scan keys by prefix (returns array)
`SCAN BY PREFIX` (as-of)	`SCAN BY PREFIX prefix AS OF timestamp`	Scan by prefix as of a timestamp

All persistent commands accept an optional `LET` binding:

```
LET x = GET mykey
LET items = GET BY BUCKET 'services'
LET found = EXISTS mykey
```

31.3.2 Ephemeral Commands

Ephemeral commands operate on ephemeral (non-replicated) storage. Each persistent command has an ephemeral counterpart with an `E` prefix:

Persistent	Ephemeral
SET	ESET
GET	EGET
EXISTS	EEXISTS
DELETE	EDELETE
EXTEND	EEXTEND
GET BY BUCKET	EGET BY BUCKET
SCAN BY PREFIX	ESCAN BY PREFIX

Ephemeral commands support the same flags, revision reads, as-of reads, and LET bindings as their persistent counterparts.

31.4 SET Flags

Flags follow the value expression in a SET or ESET command. Multiple flags can be combined.

Flag	Syntax	Description
NX	SET key value NX	Set only if the key does not exist
XX	SET key value XX	Set only if the key already exists
EX	SET key value EX milliseconds	Set with a TTL (time to live) in milliseconds
CMP	SET key value CMP expected	Set only if the current value equals expected (compare-and-swap)
CMPREV	SET key value CMPREV revision	Set only if the current revision equals revision
NOREV	SET key value NOREV	Do not increment the revision counter

Flags combine freely:

```
SET mykey "hello" NX EX 30000
SET counter 10 CMP 9 EX 60000
SET config "v2" CMPREV 5
```

31.5 Operators

31.5.1 Arithmetic

Operator	Symbol	Example
Addition	+	x + 1
Subtraction	-	x - 1
Multiplication	*	x * 2
Division	/	x / 2

31.5.2 Comparison

Operator	Symbols	Example
Equal	`=`, `==`	`x = 10`, `x == 10`
Not equal	`<>`, `!=`	`x <> 10`, `x != 10`
Less than	`<`	`x < 10`
Greater than	`>`	`x > 10`
Less than or equal	`<=`	`x <= 10`
Greater than or equal	`>=`	`x >= 10`

31.5.3 Logical

Operator	Symbols	Example
And	`&&`	`x > 0 && x < 100`
Or	`||`	`x = 0 || x = 1`
Not	`!`, `NOT`	`!found`, `NOT found`

31.5.4 Special

Operator	Symbol	Example	Description
Range	`..`	`1..10`	Create an integer array from start to end
Array index	`[]`	`items[0]`	Access an element by index
Not set	`NOT SET`	`IF x = NOT SET THEN`	True when a `GET` found no value
Not found	`NOT FOUND`	`IF x = NOT FOUND THEN`	True when a key does not exist

31.6 Control Flow

31.6.1 IF / THEN / ELSE / END

```
IF condition THEN
  statements
END

IF condition THEN
  statements
ELSE
  statements
END
```

31.6.2 FOR / IN / DO / END

```
FOR item IN collection DO
  statements
END
```

The collection is typically a range (`1..10`), an array returned by `GET BY BUCKET`, or an array returned by `SCAN BY PREFIX`.

31.7 Transaction Blocks

31.7.1 BEGIN / COMMIT / ROLLBACK / END

```
BEGIN
  statements
  COMMIT
END

BEGIN (locking=pessimistic, timeout=5000)
  statements
  COMMIT
END
```

A `BEGIN` block opens a transaction. `COMMIT` commits the transaction. `ROLLBACK` aborts it. `END` closes the block. If the script ends the block without `COMMIT` or `ROLLBACK`, the transaction is auto-committed.

Scripts without a `BEGIN` block run in auto-commit mode: each statement is an independent operation.

31.7.2 Transaction Options

Options are specified in parentheses after `BEGIN`:

```
BEGIN (option=value, option=value)
```

Option	Values	Default	Description
`locking`	`pessimistic`, `optimistic`	`pessimistic`	Locking mode for the transaction
`autoCommit`	`true`, `false`, `yes`, `no`	`false`	Auto-commit after the last statement
`asyncRelease`	`true`, `false`, `yes`, `no`	`false`	Release locks asynchronously after commit
`timeout`	integer (milliseconds)	server default	Transaction execution timeout
`admissionWait`	integer (milliseconds)	server default	Maximum time to wait for an admission slot
`snapshot`	integer (Unix epoch ms)	(none)	Pin reads to a specific MVCC snapshot
`priority`	`background`, `low`, `normal`, `high`, `critical`	`normal`	Admission priority

31.8 Flow Control Statements

Statement	Syntax	Description
`RETURN`	`RETURN expression` or `RETURN`	Return a value from the script
`SLEEP`	`SLEEP milliseconds`	Pause execution (integer argument)
`THROW`	`THROW expression`	Raise a script error

31.9 Built-in Functions

31.9.1 Math Functions

Function	Arguments	Returns	Description
`abs(x)`	1 numeric	numeric	Absolute value
`pow(base, exp)`	2 numeric	numeric	Exponentiation
`round(x)`	1 numeric	numeric	Round to nearest integer
`ceil(x)`	1 numeric	numeric	Round up
`floor(x)`	1 numeric	numeric	Round down
`min(a, b)`	2 numeric	numeric	Smaller of two values
`max(a, b)`	2 numeric	numeric	Larger of two values

31.9.2 Type Conversion Functions

Function	Aliases	Arguments	Returns	Description
`to_int(x)`	`to_integer`, `to_long`, `to_number`	1	integer	Convert to 64-bit integer
`to_float(x)`	`to_double`	1	float	Convert to 64-bit float
`to_string(x)`	`to_str`	1	string	Convert to string
`to_bool(x)`	`to_boolean`	1	boolean	Convert to boolean
`to_json(x)`	(none)	1	string	Serialize to JSON string

31.9.3 Type Check Functions

Function	Aliases	Arguments	Returns	Description
`is_int(x)`	`is_integer`, `is_long`	1	boolean	True if integer
`is_float(x)`	`is_double`	1	boolean	True if float
`is_string(x)`	`is_str`	1	boolean	True if string
`is_bool(x)`	`is_boolean`	1	boolean	True if boolean
`is_null(x)`	(none)	1	boolean	True if null
`is_array(x)`	(none)	1	boolean	True if array

31.9.4 String Functions

Function	Arguments	Returns	Description
`concat(a, b)`	2 strings	string	Concatenate two strings
`upper(x)`	1 string	string	Convert to uppercase
`lower(x)`	1 string	string	Convert to lowercase
`length(x)`	1 string	integer	String length in characters
`len(x)`	1 string	integer	Alias for `length`

31.9.5 Metadata Functions

Function	Arguments	Returns	Description
`revision(x)`	1 (a GET result)	integer	Revision number of the value
`rev(x)`	1 (a GET result)	integer	Alias for `revision`
`expires(x)`	1 (a GET result)	integer	Expiry timestamp of the value (0 if none)
`current_time()`	0	integer	Current UTC time as Unix epoch milliseconds

31.9.6 Collection Functions

Function	Arguments	Returns	Description
`count(x)`	1 array	integer	Number of elements in the array

31.10 Reserved Words

The following words are reserved and cannot be used as bare identifiers. Escape them with backticks if they appear in key names.

`AND`, `AS`, `AT`, `BEGIN`, `BUCKET`, `BY`, `COMMIT`, `CMP`, `CMPREV`, `DEL`, `DELETE`, `DO`, `EDEL`, `EDELETE`, `EEXISTS`, `EEXTEND`, `EGET`, `ELSE`, `END`, `ESCAN`, `ESET`, `EX`, `EXISTS`, `EXTEND`, `FALSE`, `FOR`, `FOUND`, `GET`, `IF`, `IN`, `LET`, `NOREV`, `NOT`, `NULL`, `NX`, `OF`, `OR`, `PREFIX`, `RETURN`, `ROLLBACK`, `SCAN`, `SET`, `SLEEP`, `THEN`, `THROW`, `TRUE`, `XX`

31.11 Operator Precedence

From lowest to highest:

1. `..` (range)
2. `||` (or)
3. `&&` (and)
4. `=`, `==`, `<>`, `!=` (equality)
5. `<`, `>`, `<=`, `>=` (comparison)
6. `+`, `-` (additive)
7. `*`, `/` (multiplicative)
8. `!`, `NOT` (unary not, right-associative)
9. `[]` (array index)

Parentheses override precedence: `(x + 1) * 2`.

31.12 Placeholders

Placeholders let the caller pass parameters into a script at execution time. A placeholder is an `@` followed by an alphanumeric name:

```
SET @key @value NX EX @ttl
```

The caller binds values to placeholders through the `KeyValueParameter` list in the API call. Placeholders can appear anywhere a key name or expression is expected.

31.13 Complete Example

```
BEGIN (locking=pessimistic, timeout=5000)
  LET balance = GET `accounts/checking`
  IF balance = NOT SET THEN
    THROW "Account not found"
  END

  LET amount = to_int(balance)
  IF amount < 100 THEN
    ROLLBACK
  END

  SET `accounts/checking` (amount - 100)
  SET `accounts/savings` (to_int(GET `accounts/savings`) + 100)
  COMMIT
  RETURN amount - 100
END
```

32 Appendix B: Configuration Reference

This appendix lists every configuration parameter that the Kahuna server accepts. Each parameter is a command-line flag. The tables show the flag name, its default value, and a short description.

Parameters are grouped by subsystem. When a parameter controls a limit and the default is 0, the feature is disabled until an operator sets a positive value. When a flag description says "0 = no limit," Kahuna applies no bound on that dimension.

32.1 Server and Networking

Flag	Default	Description
`--host`	*	Host address to bind for incoming connections
`--http-ports`	(none)	Ports for incoming HTTP connections
`--https-ports`	(none)	Ports for incoming HTTPS connections
`--https-certificate`	(empty)	Path to the HTTPS certificate file
`--https-certificate-password`	(empty)	Password of the HTTPS certificate
`--grpc-cleartext-ports`	(none)	Ports for cleartext HTTP/2 (h2c) gRPC connections without TLS; a standalone node defaults to 2072
`--disable-dashboard`	(off)	Do not serve the browser operator dashboard at the HTTP root
`--dashboard-refresh-seconds`	5	Dashboard poll interval in seconds (1 to 300)

32.2 Storage and WAL

These parameters control where Kahuna stores its data and its write-ahead log (WAL).

Flag	Default	Description
`--storage`	`rocksdb`	Storage backend: `rocksdb`, `sqlite`, or `memory`
`--storage-path`	(empty)	Filesystem path for the data store
`--storage-revision`	(empty)	Storage revision tag
`--wal-storage`	`rocksdb`	WAL backend: `rocksdb`, `sqlite`, or `memory`
`--wal-path`	(empty)	Filesystem path for the WAL store
`--wal-revision`	`v1`	WAL revision tag
`--wal-sync-writes`	(off)	Enable synchronous (durable) WAL writes
`--disable-wal-sync-writes`	(off)	Disable synchronous WAL writes for faster local or test runs

32.3 RocksDB

These parameters apply only when the `rocksdb` storage backend is selected.

Flag	Default	Description
`--rocksdb-shared-memory`	disabled	Share one block cache and write-buffer manager between the KV/locks backend and the WAL
`--rocksdb-shared-memory-budget-mb`	`320`	Total shared block-cache budget in MiB (the memtable sub-budget is drawn from this pool)
`--rocksdb-shared-memtable-budget-mb`	`128`	Memtable sub-budget in MiB, cost-charged into the shared cache; must be less than or equal to the total budget
`--disable-rocksdb-direct-reads`	(off)	Use buffered reads backed by the OS page cache instead of direct I/O (direct I/O is on by default)
`--rocksdb-statistics`	disabled	Collect RocksDB statistics and dump them to the LOG file periodically; adds per-operation overhead

32.4 Cluster and Node Identity

Flag	Default	Description
`--initial-cluster`	(none)	Seed list of node endpoints for static discovery
`--join-existing`	`false`	Join a running cluster as a new node; the seed list serves as the join target
`--graceful-leave-on-shutdown`	`false`	On planned shutdown, commit a RemoveMember so the roster shrinks immediately; do not enable on rolling restarts
`--initial-cluster-partitions`	`3`	Number of Raft partitions created at cluster bootstrap
`--raft-nodename`	(empty)	Human-readable node name
`--raft-nodeid`	`0`	Numeric node identifier
`--raft-host`	`localhost`	Host address for Raft consensus and replication traffic
`--raft-port`	`2070`	Port for Raft consensus and replication traffic
`--raft-zone`	(none)	Locality hint (zone or rack); the placement planner spreads replicas across distinct zones when set

32.5 Actor Workers

These parameters set the number of actor workers that serve each primitive. Each worker is a Nixie actor with its own inbox.

Flag	Default	Description
`--locks-workers`	`0`	Lock actors (ephemeral and consistent); 0 = auto-size to max(32, CPU cores x 4)
`--keyvalue-workers`	`0`	Key-value actors (ephemeral and consistent); 0 = auto-size to max(32, CPU cores x 4)
`--background-writer-workers`	`1`	Background persistence writers
`--sequencer-workers`	`0`	Sequence actors; 0 = auto-size to max(8, CPU cores)

32.6 IO Scheduler

The IO scheduler runs dedicated thread pools for backend reads and writes, separate from the Raft WAL read pool.

Flag	Default	Description
`--read-io-threads`	`4`	Kommander WAL read threads
`--write-io-threads`	`16`	Kommander WAL write threads
`--backend-read-io-threads`	`8`	Kahuna backend read threads (point gets, scans), separate from the WAL pool
`--backend-write-io-threads`	`1`	Kahuna background-writer threads; the writer serializes on a single queue, so values above 1 create idle threads
`--backend-read-queue-depth`	`4096`	Per-partition pending-queue depth for backend reads; new reads are rejected with backpressure when full

32.7 Raft Executor

The shared executor multiplexes all Raft partitions onto a bounded thread pool. Each wake cycle drains operations in priority order: control, replication, client, maintenance.

Flag	Default	Description
`--raft-enable-shared-executor-pool`	`true`	Share a bounded thread pool across all partitions; required for thousands of partitions (e.g. after range splits)
`--raft-executor-pool-size`	`0` (auto)	Worker threads in the pool; 0 auto-sizes to the processor count
`--raft-max-drain-quantum-control`	`8`	Control-plane operations drained per wake cycle
`--raft-max-drain-quantum-replication`	`4`	Replication operations drained per wake cycle
`--raft-max-drain-quantum-client`	`2`	Client operations drained per wake cycle
`--raft-max-drain-quantum-maintenance`	`1`	Maintenance operations drained per wake cycle

32.8 Raft Consensus Timers

These parameters tune Raft leader election, heartbeat, and quorum-check behavior. The relationship between the heartbeat interval and the election timeout determines how quickly the cluster detects a failed leader.

Flag	Default	Description
`--raft-heartbeat-interval`	`500` ms	Leader heartbeat interval
`--raft-recent-heartbeat`	`100` ms	Window within which a heartbeat is recent
`--raft-voting-timeout`	`1500` ms	Vote-request wait timeout
`--raft-start-election-timeout`	`2000` ms	Minimum election timeout
`--raft-end-election-timeout`	`4000` ms	Maximum election timeout
`--raft-start-election-timeout-increment`	`100` ms	Minimum randomization increment added to the election timeout
`--raft-end-election-timeout-increment`	`200` ms	Maximum randomization increment added to the election timeout
`--raft-election-timeout-seed`	`0`	Seed for deterministic election timeouts; 0 = random, non-zero = deterministic (testing only)
`--raft-leadership-barrier-timeout`	`10000` ms	How long a new leader waits for its promotion barrier entry to commit before it reverts to follower
`--raft-leadership-confirmation-timeout`	`2000` ms	Maximum time a read-index confirmation waits for a quorum ack
`--raft-enable-check-quorum`	`false`	A leader that loses a majority of same-term acks for the check-quorum window steps down
`--raft-check-quorum-interval-multiplier`	`8`	Heartbeat intervals without a majority ack before the leader steps down
`--raft-check-leader-interval`	`250` ms	Leader liveness check interval
`--raft-timer-initial-delay`	`2500` ms	Delay before Raft timers start after node boot
`--raft-slow-state-machine-log`	`500` ms	Threshold for logging slow state-machine apply operations

32.9 Raft WAL and Compaction

Flag	Default	Description
`--raft-compact-every-operations`	`10000`	Committed operations between automatic WAL compactions
`--raft-compact-number-entries`	`100`	WAL entries removed per compaction batch
`--raft-max-entries-per-compaction`	`5000`	Maximum WAL entries processed per compaction run
`--raft-max-queued-client-proposals`	`2048`	Client proposals queued per partition before backpressure kicks in
`--raft-max-wal-queue-depth-per-partition`	`4096`	Per-partition WAL write queue depth limit
`--raft-max-global-wal-queue-depth`	`0`	Global WAL write queue depth across all partitions; 0 = no limit
`--raft-max-wal-batch-size`	`256`	Maximum WAL writes batched per storage flush
`--raft-max-wal-group-batch-partitions`	`64`	Maximum partitions coalesced into one WAL group-commit batch
`--raft-wal-group-commit-linger-ms`	`0`	Group-commit linger window in milliseconds; 0 = disabled
`--raft-wal-single-fsync-commit`	`true`	Ack on propose-quorum-durable and demote the commit marker to a lazy write (single-fsync fast path)
`--raft-sqlite-wal-shard-count`	`0` (auto)	SQLite shard databases for WAL; 0 = auto-size to processor count

32.10 Raft gRPC Transport

Flag	Default	Description
`--raft-grpc-scheme`	`https://`	URL scheme for gRPC peer channels
`--raft-grpc-channels-per-node`	`4`	Pooled gRPC channels per peer node (clamped to 1 to 64)
`--raft-grpc-enable-multiple-http2-connections`	`false`	Allow each channel to open multiple HTTP/2 connections
`--raft-grpc-enable-snapshot-compression`	`false`	Compress snapshot transfers over gRPC
`--raft-grpc-enable-append-logs-coalescing`	`false`	Coalesce multiple AppendLogs calls into one gRPC frame per write cycle
`--raft-grpc-append-logs-max-coalesce-batch`	`256`	Maximum AppendLogs items per coalesced gRPC frame
`--raft-http-scheme`	`https://`	HTTP scheme for REST-based Raft communication
`--raft-http-auth-bearer-token`	(empty)	Bearer token for REST-based Raft communication
`--raft-http-timeout`	`5 s`	Request timeout for REST-based Raft communication
`--raft-http-version`	`2.0`	HTTP version for REST-based Raft communication
`--raft-max-pre-auth-request-body-bytes`	`32 MiB`	Ceiling on a Raft REST request body before authentication
`--raft-transport-security`	(empty)	Transport security and node authentication settings (JSON)
`--raft-allow-insecure-certificate-validation`	(off)	Skip TLS certificate validation for inter-node gRPC; use only in development
`--raft-max-outbound-queue-bytes-per-peer`	`64 MiB`	Cap on buffered outbound bytes per peer; excess AppendLogs entries are dropped rather than accumulated; 0 disables the cap
`--raft-wal-retention-window`	`1 h`	How long a sealed WAL segment is kept after it is superseded by a snapshot; 0 disables

32.11 Raft WAL Shard Tuning

Flag	Default	Description
`--raft-wal-shard-write-buffer-size-mb`	`0` (64 MiB)	Memtable size per shard column family; 0 uses the Kommander default
`--raft-wal-shard-min-write-buffer-number-to-merge`	`0` (2)	Immutable memtables merged into one flush; 0 uses default
`--raft-wal-shard-max-write-buffer-number`	`0` (4)	Maximum memtables per shard column family; must exceed the merge count; 0 uses default
`--raft-wal-shard-level0-file-num-compaction-trigger`	`0` (8)	L0 file count that triggers compaction into the base level; 0 uses default
`--raft-wal-shard-level0-slowdown-writes-trigger`	`0` (28)	L0 file count at which WAL writers are slowed; 0 uses default
`--raft-wal-shard-level0-stop-writes-trigger`	`0` (44)	L0 file count at which WAL writers are stopped; 0 uses default
`--raft-wal-shard-max-bytes-for-level-base-mb`	`0` (256 MiB)	`max_bytes_for_level_base` for shard column families; size above the retained log keeps the live log in the base level
`--raft-wal-shard-universal-compaction`	`false`	Use universal instead of leveled compaction for the shard column families

32.12 Raft Snapshot Transfer

Flag	Default	Description
`--raft-snapshot-receive-session-ttl`	`30000` ms	Idle time before a snapshot-receive session is discarded
`--raft-snapshot-max-pending-sessions`	`8`	Maximum concurrent snapshot-receive sessions across all partitions
`--raft-snapshot-max-pending-bytes`	`512` MiB	Total buffered bytes across all in-progress snapshot-receive sessions
`--raft-allow-legacy-snapshot-senders`	`false`	Accept chunks from senders that predate session-metadata fields; for mixed-version clusters
`--raft-snapshot-transfer-step-timeout`	`120000` ms	Upper bound on one step of an outbound snapshot transfer; a step that makes progress resets the clock
`--raft-snapshot-rescue-max-consecutive-cycles`	`3`	Consecutive rescue cycles that return a still-below-floor follower before the convergence breaker trips; values <= 0 disable the breaker
`--raft-snapshot-rescue-probe-interval`	`300000` ms	While the convergence breaker is tripped, one rescue attempt per this interval; values <= 0 disable probing
`--raft-snapshot-export-retry-cache-max-bytes`	`64` MiB	Leader-side cache for one produced snapshot export; at or under this size the export is cached and re-sent on retry; values <= 0 disable the cache
`--raft-compaction-live-replica-lag-budget`	`100000`	Entry-count budget within which a follower still converging after snapshot rescue is held off the WAL compaction floor; values <= 0 disable the hold
`--raft-snapshot-staging-directory`	(auto for persistent backends)	Directory for spilling received snapshot chunks to disk instead of the large-object heap
`--raft-snapshot-staging-memory-bytes`	`67108864` (64 MiB)	Memory budget for staged snapshot bytes before spilling to disk

32.13 Raft Backfill

Backfill is the mechanism that ships missed entries to lagging followers. When a follower falls too far behind, the leader sends a full snapshot instead.

Flag	Default	Description
`--raft-backfill-enabled`	`true`	Ship catch-up batches to lagging followers; set false only when the deployment owns its catch-up story
`--raft-backfill-threshold`	`10`	Entries a follower may trail before active backfill starts
`--raft-max-backfill-entries-per-round`	`128`	Maximum entries shipped per heartbeat interval
`--raft-follower-saturation-backoff`	`1000` ms	Pause after a follower reports its WAL queue is saturated
`--raft-backfill-no-progress-pause-cap`	`30000` ms	Cap on exponential pause between batches to a follower whose commit frontier is not advancing
`--raft-backfill-no-progress-anchor-fallback-ships`	`2`	Consecutive fruitless backfill ships before the leader re-anchors at the follower's commit frontier; values <= 0 disable the fallback
`--raft-max-backfill-bytes-per-round`	`4` MiB	Byte cap on a single backfill round; keeps large-payload workloads under the gRPC message cap
`--raft-backfill-seeded-catch-up-grace-cap`	`180000` ms	Upper bound on how long the leader defers escalating a follower that a snapshot install just seeded
`--raft-learner-promotion-stall-warning-after`	`30000` ms	Diagnostic warning when a placement-added learner stays non-promotable longer than this threshold

32.14 Membership: Learner Promotion, Gossip, and SWIM

Learner promotion controls how a new node (learner) becomes a voting member. Gossip disseminates cluster state. SWIM is the failure detector that identifies dead nodes.

Flag	Default	Description
`--raft-learner-promotion-lag`	`10`	Maximum entries a learner may trail and still be eligible for promotion
`--raft-learner-promotion-stable-window`	`3000` ms	Duration a learner must stay within promotion lag before it becomes a voter
`--raft-gossip-interval`	`5000` ms	Interval between gossip anti-entropy rounds
`--raft-gossip-fanout`	`2`	Random peers contacted per gossip round; 0 disables gossip
`--raft-ping-timeout`	`500` ms	SWIM ping timeout
`--raft-indirect-ping-fanout`	`2`	Intermediary nodes for indirect SWIM probes
`--raft-suspicion-timeout`	`5000` ms	Duration a node stays Suspect before it is declared Dead
`--raft-dead-member-eviction-grace`	`30000` ms	Grace period before a Dead node is removed from the roster
`--raft-ping-interval`	`1000` ms	Interval between SWIM ping rounds; 0 disables the failure detector
`--raft-enable-auto-rejoin`	`true`	A node that finds itself removed from the roster re-runs the join flow instead of staying NotMember

32.15 Quiescence

Quiescence stops per-partition heartbeats on idle partitions, which eliminates O(N x M) heartbeat traffic across many partitions. The SWIM failure detector provides node-level liveness instead.

Flag	Default	Description
`--raft-enable-quiescence`	`true`	Quiesce idle partitions; requires SWIM (`--raft-ping-interval > 0` and less than `--raft-start-election-timeout`)
`--raft-quiesce-after`	`1500` ms	Idle time before a partition's leader quiesces it

32.16 Leader Balancer

The leader balancer runs on the P0 (partition-zero) leader and redistributes partition leadership across nodes. It uses a two-tier strategy: first it balances the leader count, then it balances load.

Flag	Default	Description
`--raft-enable-leader-balancer`	`false`	Master switch; enable only after all nodes support the feature
`--raft-leader-balancer-report-interval`	`5000` ms	Interval at which each node emits a load report on the gossip path
`--raft-leader-balancer-interval`	`30000` ms	Interval at which the P0 leader runs a planning pass
`--raft-leader-balancer-report-ttl`	`20000` ms	Maximum age of a load report before it is excluded from planning; must be greater than the report interval
`--raft-count-deadband`	`1`	Minimum leader-count imbalance above the ideal before the balancer emits moves
`--raft-load-imbalance-threshold`	`0.25`	Fractional load skew (max minus min over max) that triggers load-tier swaps when counts are balanced
`--raft-min-leader-stability-ms`	`5000` ms	Duration a partition must stay on one leader before it may be moved
`--raft-move-cooldown`	`60000` ms	Duration a partition is excluded from further moves after a transfer suggestion
`--raft-max-moves-per-pass`	`4`	Maximum move suggestions per planning pass
`--raft-max-concurrent-transfers`	`2`	Maximum in-flight transfer suggestions tracked simultaneously
`--raft-leader-balancer-ops-weight`	`1.0`	Weight of the ops/sec term in the composite load score
`--raft-leader-balancer-queue-weight`	`0.5`	Weight of the queue-depth term in the composite load score
`--raft-suggestion-timeout`	`15000` ms	Time the P0 leader waits for a move to be confirmed before it declares the move dropped
`--raft-enable-load-reports`	`false`	Gossip per-partition load reports even when nothing else consumes them

32.17 Slow Node Avoidance

The slow node avoidance subsystem extends the leader balancer. It classifies nodes whose WAL commit wait is far above the cluster median as degraded. The balancer refuses degraded nodes as transfer targets and drains their leaderships. This subsystem requires the leader balancer to be enabled.

Flag	Default	Description
`--raft-enable-slow-node-avoidance`	`false`	Master switch; the balancer classifies nodes whose WAL commit wait is far above the cluster median as slow, refuses them as transfer targets, and drains their leaderships
`--raft-slow-node-multiplier`	`3.0`	Factor above the cluster median commit wait before a node is a slow candidate; must be greater than 1.0
`--raft-slow-node-floor-ms`	`10.0`	Absolute floor in milliseconds below which a node is never a slow candidate
`--raft-slow-node-min-samples`	`20`	WAL group batches a node must have observed before its commit wait is judged; below this the node is unknown and never drained
`--raft-slow-node-observation-ttl`	`30000` ms	Maximum age of a node's last commit-wait observation before the node counts as unknown
`--raft-slow-node-enter-passes`	`3`	Consecutive balancer passes a node must look slow before classification
`--raft-slow-node-exit-passes`	`6`	Consecutive clean passes before a slow node is released; deliberately longer than enter-passes to prevent flapping

32.18 Replica Placement

The placement rebalancer runs on the P0 leader. It maintains the desired replication factor by adding or removing replicas, and it spreads replicas evenly across nodes and zones.

Flag	Default	Description
`--raft-replication-factor`	`0`	Desired voter replicas per partition range; 0 = full replication (every voter hosts every range)
`--raft-enable-placement-rebalancer`	`false`	Master switch for continual replica-placement rebalancing; initial placement at the configured replication factor is applied regardless
`--raft-placement-pass-interval`	`5000` ms	Interval between placement-controller passes
`--raft-max-replica-moves-per-pass`	`4`	Maximum new replica moves per pass, across all priorities
`--raft-max-concurrent-replica-transfers`	`1`	In-flight balance moves (cosmetic skew spreading) at any time
`--raft-max-concurrent-replica-repairs`	`3`	In-flight repair moves (re-replicating under-replicated ranges) at any time; budgeted separately from balance moves
`--raft-decommission-drain-timeout`	`120000` ms	Duration a graceful leave waits for replicas to be evacuated; on expiry the node is restored to voter
`--raft-replica-count-deadband`	`1`	Minimum per-node replica-count imbalance before balancing moves are emitted

32.19 Cache

Flag	Default	Description
`--revisions-to-cache`	`4`	Number of key revisions kept in the per-actor memory cache
`--cache-entry-ttl`	`1800` s	Maximum age of a cache entry before eviction (30 minutes)
`--cache-entries-to-remove`	`100`	Maximum entries removed per eviction sweep
`--script-cache-expiration`	`600` s	Compiled-script cache TTL (10 minutes)
`--extension-assemblies`	(none)	Comma-separated paths to extension assemblies that register user-defined script functions
`--function-slow-warn-ms`	`100` ms	Threshold for logging user-defined function calls as slow

Internal cache parameters (set in `KahunaConfiguration`, not exposed as CLI flags):

Parameter	Default	Description
`MaxEntriesPerActor`	`50,000`	Maximum cache entries per key-value actor
`MaxBytesPerActor`	`256` MiB	Maximum cache bytes per key-value actor
`CollectBatchMax`	`1,000`	Maximum entries evicted per collection sweep
`CollectionInterval`	`60` s	Interval between collection sweeps
`ScriptCacheMaxEntries`	`1,000`	Maximum compiled scripts held in the cache
`MaxKeyValueActorInboxSize`	`16,384`	Per-actor inbox bound for ordinary messages; control messages are exempt

32.20 Write Batching

The partition write aggregator coalesces direct key-value writes (SET, DELETE, EXTEND) into Raft proposals. This reduces the number of fsyncs.

Flag	Default	Description
`--kv-write-linger-ms`	`1`	Delay from the oldest queued write before its partition batch is proposed; 0 = immediate
`--kv-write-max-batch-items`	`512`	Maximum log entries per aggregator Raft call
`--kv-write-max-batch-bytes`	`4` MiB	Target serialized bytes per aggregator Raft call
`--kv-write-max-queued-items`	`8192`	Maximum admitted direct writes per partition (including in-flight)
`--kv-write-max-queued-bytes`	`32` MiB	Maximum admitted serialized bytes per partition (including in-flight)
`--kv-write-max-queue-delay-ms`	`1000`	Maximum time a write waits before dispatch; expired writes are released as MustRetry
`--kv-write-aggregator-inbox-size`	`16384`	Ordinary-submission inbox bound per aggregator lane; control messages are exempt
`--kv-write-max-in-flight-batches`	`1`	Maximum concurrent Raft proposals per partition; values above 1 enable pipelined batching (FIFO order preserved)
`--kv-write-post-completion-hold-ms`	`0`	Delay after a batch completes before the next sub-threshold batch is dispatched; full batches flush immediately; 0 = no hold
`--kv-write-precise-wake`	`false`	Fire the aggregator's dispatch timer from a dedicated thread with sub-millisecond precision instead of `Task.Delay`
`--kv-write-fill-target-trigger`	`false`	Dispatch a held buffer when it reaches the learned fill target (median of recent timer-dispatched batch sizes) instead of waiting for the timer

Internal write-batching parameters (set in `KahunaConfiguration`, not exposed as CLI flags):

Parameter	Default	Description
`KeyValueWriteTerminalReserveItemsPerPartition`	256	Extra item headroom per partition reserved for transaction settlement
`KeyValueWriteTerminalReserveBytesPerPartition`	4 MiB	Extra byte headroom per partition reserved for transaction settlement
`KeyValueWriteMaxQueuedItemsGlobal`	131,072	Node-global item cap for ordinary writes across all partitions
`KeyValueWriteMaxQueuedBytesGlobal`	512 MiB	Node-global byte cap for ordinary writes across all partitions
`KeyValueWriteTerminalReserveItemsGlobal`	8,192	Extra node-global item headroom reserved for transaction settlement
`KeyValueWriteTerminalReserveBytesGlobal`	64 MiB	Extra node-global byte headroom reserved for transaction settlement
`KeyValueWriteMaxOperationBytes`	64 MiB	Hard ceiling on a single write's serialized bytes
`KeyValueWriteBatchExecutionTimeoutMs`	30,000 ms	Timeout for a dispatched batch's Raft round trip
`KeyValueWriteMaxQueueDelayMs`	1,000 ms	Maximum residence before a write is released as MustRetry
`ReadContinuationTimeout`	30,000 ms	Timeout for a backend read in flight; on expiry, waiters receive MustRetry

32.21 Transactions and Admission Control

Transaction admission control gates how many script transactions and interactive sessions may execute concurrently. Chapter 28 explains the five priority levels, reserved slots, and anti-starvation aging in detail.

Flag	Default	Description
`--default-transaction-timeout`	`5000` ms	Default transaction lifetime timeout
`--default-admission-wait`	`5000` ms	Default time a caller queues for an admission slot; this is the wait to start, not the transaction lifetime
`--max-admission-wait`	`30000` ms	Hard upper bound on any admission wait
`--max-concurrent-transactions`	`0`	Script transactions that may execute concurrently; 0 = no limit (admission control disabled)
`--max-concurrent-sessions`	`0`	Interactive sessions that may be open concurrently; 0 = no limit
`--transaction-priority-reserved-slots`	`0`	Slots reserved for High and Critical priority only; 0 = no class distinction
`--transaction-priority-aging-threshold`	`1000` ms	Wait time for a queued transaction to gain one effective priority level; 0 = no aging
`--transaction-priority-max-queued`	`4096`	Maximum callers waiting per gate before further callers are refused; 0 = unbounded

Internal transaction parameters (set in `KahunaConfiguration`, not exposed as CLI flags):

Parameter	Default	Description
`MaxTransactionTimeout`	`300,000` ms	Hard upper bound on any transaction session lifetime (5 minutes)
`Phase2CommitTimeout`	`5,000` ms	Timeout for a single two-phase-commit Raft wait
`TransactionOutcomeRetentionMax`	`10,000`	Finalized outcomes retained for best-effort idempotency
`TransactionOutcomeRetentionTtl`	`5` min	Age before a retained outcome is pruned
`DurableDecisionOutstandingMax`	`100,000`	Maximum outstanding durable coordinator decision records
`DurableRecordRetentionMax`	`200,000`	Maximum terminal transaction records retained in memory; oldest past the floor are reclaimed first
`DurableRecordRetentionMaxBytes`	`256` MiB	Byte budget for terminal transaction records; triggers early reclamation when exceeded
`DurableRecordRetentionHeapPressure`	`0.85`	GC heap fraction at which the retention budget is tightened; never reclaims records younger than the floor
`DurableRecordRetentionFloor`	`90` s	Minimum age before a terminal record becomes eligible for budget reclamation
`DurableMaintenanceInterval`	`5` s	Tick interval for the durable maintenance sweep
`DurableRecordGcMaxPerPass`	`4,096`	Terminal records the GC sweep considers per pass
`CompletionReceiptRetentionTtl`	`10` min	Age after which an orphaned participant receipt is dropped
`StagedWriteIntentLeaseMs`	`15,000` ms	How long a transactional staged write intent stays live before competitors may write past it
`ScanPageRetryBudgetMs`	`5,000` ms	Per-page retry budget for range scans before the scan fails with the range and cursor named
`SessionOwnedIntentCeilingMs`	`0` (derived)	Maximum age of a session-owned intent; 0 = derived from MaxTransactionTimeout plus the reaper grace and participant-ef-
[illegible]	[illegible]	[illegible] key's last commit [illegible]; must exceed the longest transaction lifetime

32.22 Sequences

Flag	Default	Description
`--sequencer-block-size`	`1000`	Values reserved per compare-and-swap; one Raft commit serves this many values. Set to 1 for gap-free allocation
`--sequencer-idempotency-retention-max`	`256`	Maximum idempotency entries per sequence record; 0 = no cap
`--sequencer-idempotency-retention-ttl`	`600` s	Window within which a retried keyed reserve replays the same allocation; 0 = no age pruning
`--sequencer-max-sequences-per-actor`	`10000`	Maximum sequences resident per actor before LRU eviction; 0 = unbounded
`--sequencer-block-lease`	`5` s	Duration a reserved block is served from memory before revalidation against the durable record; 0 = never revalidate

32.23 Persistence

Flag	Default	Description
`--checkpoint-interval`	`30` s	Period at which a dirty partition checkpoints, advancing its WAL retention floor
`--dirty-objects-writer-delay`	`200` ms	Flush interval of the dirty-object writer
`--persistence-write-stall-warn-ms`	`500` ms	Threshold for logging a store write as a disk stall; stalled nodes are routed around for transaction coordination
`--persistent-revision-retention-count`	`0`	Maximum persisted key revisions to keep per key; 0 = keep forever
`--persistent-revision-retention-age`	`0` s	Maximum age of persisted revisions; 0 = no age limit
`--snapshot-hold-startup-grace-window`	`300` s	After restart, lapsed-but-registered snapshot holds continue to pin history for this long; gives clients time to renew or release holds
`--persistent-revision-cleanup-interval`	`300` s	Minimum interval between full revision cleanup sweeps
`--persistent-revision-cleanup-batch-size`	`10000`	Maximum revision records deleted per cleanup pass
`--persistent-revision-cleanup-on-write`	enabled	Run targeted cleanup after key-value writes
`--disable-persistent-revision-cleanup-on-write`	(off)	Disable targeted cleanup after writes

Internal persistence parameter (set in `KahunaConfiguration`, not exposed as a CLI flag):

Parameter	Default	Description
`RevisionRetention`	`16`	In-memory revision slots retained per key

32.24 Backup and Point-in-Time Recovery

Flag	Default	Description
`--pitr-window`	`3600` s	PITR recovery window; WAL entries older than now minus this value may be compacted (1 hour)
`--base-snapshot-interval`	`1800` s	Interval between base checkpoints per shard; must not exceed the PITR window (30 minutes)
`--pitr-backup-dir`	(empty)	Root directory for backup artifacts and manifests; empty = backups disabled
`--pitr-backup-target`	`local`	Storage target: `local` keeps backups on disk, other values require a registered storage provider
`--pitr-backup-scratch-dir`	(empty)	Local staging directory for remote-target backups; size it for one full backup
`--pitr-backup-cluster-id`	(empty)	Cluster identity stamped into manifests; set the same value on every node
`--pitr-backup-mac-key-file`	(empty)	Path to the HMAC-SHA-256 key file for manifest authentication; keep it outside the backup directory
`--pitr-restore-root`	(empty)	Server-owned root that confines restore destinations; setting this enables remote restore
`--pitr-allow-unconfined-remote-restore`	`false`	Allow remote restore without a configured restore root; insecure
`--pitr-bootstrap-from`	(none)	Backup ID (GUID) to restore from before the node joins the cluster; requires `--join-existing` and `--pitr-backup-dir`
`--pitr-target-time-ms`	`0`	HLC target time for PITR restore in milliseconds since Unix epoch; 0 = restore to the chain's natural maximum
`--backup-retention-max-chains`	`0`	Keep at most this many recent backup chains; 0 = no limit
`--backup-retention-max-age`	`0` s	Delete chains whose newest backup exceeds this age; 0 = no limit
`--backup-retention-max-bytes`	`0`	Keep recent chains within this byte budget (the newest chain is always kept); 0 = no limit
`--backup-gc-interval`	`3600` s	Interval between periodic backup GC passes; a pass also runs at startup (1 hour)
`--backup-restore-throttle-mbps`	`0`	Restore copy throughput budget in MB/s; 0 = unlimited

32.25 Key-Range Split and Merge

32.25.1 Count-based Split

Flag	Default	Description
`--range-split-threshold`	`1000`	Key count above which a range splits; 0 disables count-based auto-split
`--range-split-min-range-size`	`10`	Minimum keys each half must hold after a split
`--range-split-settle-window`	`10` s	Settle time after a split before the range may split again
`--range-collection-interval`	`60` s	Interval between split/merge sampling passes
`--range-merge-min-size`	`10`	Key count below which two adjacent ranges merge; 0 disables auto-merge
`--range-move-settle-timeout`	`10` s	Maximum seconds a split or merge holds its quiesce while draining in-flight transactions; writes are refused retryably for at most this long; 0 disables the wait; clamped to 15

32.25.2 Load-based Split

Load-based splitting fires when a partition sustains high throughput and its WAL queue is saturated. All thresholds must be met simultaneously.

Flag	Default	Description
`--range-split-load-threshold`	`0`	Log ops/sec above which a partition becomes a load-split candidate; 0 disables load-based auto-split
`--range-split-load-min-queue-depth`	`8`	Minimum WAL queue depth that must accompany the ops/sec rate
`--range-split-load-window`	`15` s	Duration the load predicate must hold before a split fires
`--range-split-load-poll-interval`	`5` s	Interval between load-signal polls

Internal load-split parameters (set in `KahunaConfiguration`, not exposed as CLI flags):

Parameter	Default	Description
`RangeSplitLoadMinCommitWaitMs`	`0`	Optional secondary gate on commit-wait latency; 0 = disabled
`RangeSplitLoadImbalanceMax`	`0.8`	Fraction above which a range is refused as indivisible (all writes hit one key)
`RangeSplitIndivisibleCooldown`	5 min	Suppression window after an indivisibility refusal

32.26 Internal-Only Parameters

The following parameters are set in `KahunaConfiguration` and are not exposed as CLI flags. They exist for embedded deployments that construct the configuration object directly in code.

Parameter	Default	Description
`SingleProcessRaftGroup`	`false`	Declares that all Raft voters live in one process; unlocks the one-phase durable-commit fast path for read-carrying transactions. Set automatically by the embedded standalone node. Do not set when a remote replica can join.
`BackupStorageProvider`	`null`	Host-supplied factory for backup storage; set in code by the DI wiring, not from the command line
`RestoreRoot`	(empty)	Server-owned root for restore path confinement (the CLI equivalent is `--pitr-restore-root`)
`AllowUnconfinedRemoteRestore`	`false`	Escape hatch for remote restore without confinement (the CLI equivalent is `--pitr-allow-unconfined-remote-restore`)

33 Appendix C: CLI Command Reference

This appendix lists every command available in `kahuna-cli`, the Kahuna command-line tool. Commands fall into two categories: flags passed on the command line for single-shot execution, and commands typed in the interactive shell. Both categories are documented here.

When no command flag is passed, `kahuna-cli` enters the interactive shell. The shell provides syntax highlighting, multi-line editing, and command history.

33.1 Global Options

These options apply to all commands:

Option	Short	Default	Description
`--connection-source`	`-c`	`http://127.0.0.1:8083`	Comma-separated list of node endpoints
`--default-timeout`	`-t`	`10`	Client-side timeout in seconds
`--format`		`console`	Output format: `console` or `json`
`--insecure`		`false`	Skip TLS certificate validation (for self-signed certificates)
`--node`		(none)	Target a specific node endpoint for cluster commands
`--force-rich`		`false`	Force rich terminal rendering (or set `KAHUNA_FORCE_RICH=1`)

33.2 Key-Value Commands

33.2.1 set

Write a value to a key.

CLI flag: `--set <key> --value <value> [--expires <ms>]`

Interactive: `SET key value [NX | XX] [EX milliseconds] [CMP expected] [CMPREV revision]`

The interactive form supports all SET flags described in Appendix A.

33.2.2 get

Read the current value of a key.

CLI flag: `--get <key>`

Interactive: `GET key`

33.2.3 get-by-prefix

Read all keys under a prefix (bucket scan).

CLI flag: `--get-by-prefix <prefix>`

Interactive: `GET BY BUCKET prefix`

33.2.4 scan-by-prefix

Scan keys by prefix and return their names.

CLI flag: `--scan-by-prefix <prefix>`

Interactive: `SCAN BY PREFIX prefix`

33.2.5 delete

Delete a key.

Interactive only: `DELETE key` or `DEL key`

33.2.6 extend

Extend a key's TTL.

Interactive only: `EXTEND key milliseconds`

33.2.7 exists

Check whether a key exists.

Interactive only: `EXISTS key`

33.2.8 Ephemeral Variants

Every persistent key-value command has an ephemeral counterpart. Ephemeral commands operate on non-replicated storage. In the interactive shell, prefix the command with `E`:

Persistent	Ephemeral
`SET`	`ESET`
`GET`	`EGET`
`DELETE` / `DEL`	`EDELETE` / `EDEL`
`EXTEND`	`EEXTEND`
`EXISTS`	`EEXISTS`
`GET BY BUCKET`	`EGET BY BUCKET`
`SCAN BY PREFIX`	`ESCAN BY PREFIX`

33.3 Lock Commands

33.3.1 lock

Acquire a distributed lock.

CLI flag: `--lock <name> [--expires <ms>]`

Interactive: `lock <name> [milliseconds]`

The interactive form acquires the lock and holds it in the shell session. The lock is released when the session ends or when `unlock` is called.

33.3.2 unlock

Release a held lock.

CLI flag: `--unlock <name> --owner <owner-id>`

Interactive: `unlock <name>`

In the interactive shell, the owner is tracked automatically from the `lock` call.

33.3.3 extend-lock

Extend a held lock's lease.

CLI flag: `--extend-lock <name> --owner <owner-id> [--expires <ms>]`

Interactive: `extend-lock <name> [milliseconds]`

33.3.4 get-lock

Display information about a held lock.

Interactive only: `get-lock <name>`

33.3.5 Ephemeral Lock Variants

Persistent	Ephemeral
`lock`	`elock`
`unlock`	`eunlock`
`extend-lock`	`eextend-lock`
`get-lock`	`eget-lock`

33.4 Sequence Commands

33.4.1 create-sequence

Create a persistent sequence.

CLI flag: `--create-sequence <name> [--initial-value <n>] [--increment <n>] [--max-value <n>]`

Interactive: `create-sequence <name> [initial-value] [increment] [max-value]`

Option	Default	Description
`--initial-value`	`0`	Starting value
`--increment`	`1`	Step size per allocation
`--max-value`	(none)	Optional upper bound

33.4.2 get-sequence

Display the current state of a sequence.

CLI flag: `--get-sequence <name>`

Interactive: `get-sequence <name>`

33.4.3 next-sequence

Reserve the next value from a sequence.

CLI flag: `--next-sequence <name> [--idempotency-key <key>]`

Interactive: `next-sequence <name> [idempotency-key]`

33.4.4 reserve-sequence

Reserve a range of values from a sequence.

CLI flag: `--reserve-sequence <name> [--count <n>] [--idempotency-key <key>]`

Interactive: `reserve-sequence <name> [count] [idempotency-key]`

Option	Default	Description
`--count`	`1`	Number of values to reserve

33.4.5 update-sequence

Update a sequence's parameters at runtime. The update starts a new incarnation. The server waits out one block lease window before confirming.

CLI flag: `--update-sequence <name> [--current-value <n>] [--increment <n>] [--max-value <n>] [--block-size <n>]`

Interactive: `update-sequence <name> [current-value] [increment] [max-value] [block-size]`

Option	Default	Description
`--current-value`	(unchanged)	New high-water mark; the next issued value is this plus the increment
`--increment`	(unchanged)	New step between values
`--max-value`	(unchanged)	New maximum ceiling
`--block-size`	(unchanged)	Per-sequence block size override; overrides the server-wide setting

33.4.6 delete-sequence

Delete a sequence.

CLI flag: `--delete-sequence <name>`

Interactive: `delete-sequence <name>`

33.5 Script Commands

33.5.1 run

Load and execute a script file.

Interactive only: `run <path-to-file.4gl>`

The script runs as a single transaction. The interactive shell also accepts inline script syntax (SET, GET, LET, IF, BEGIN, and so on) directly at the prompt.

33.6 Cluster Commands

33.6.1 cluster-members

Display the live cluster membership roster.

CLI flag: `--cluster-members`

Interactive: `cluster members`

Shows each member's endpoint, node ID, role (Learner, Voter, or Leaving), and joined version.

33.6.2 cluster-leave

Decommission a node from the cluster.

CLI flag: `--cluster-leave [--node <endpoint>]`

Interactive: `cluster leave [endpoint]`

When the connection lists several endpoints, the `--node` option (or the endpoint argument in the interactive form) is required. The CLI refuses to pick a node at random.

33.6.3 cluster-placement

Display the per-partition replica placement table.

CLI flag: `--cluster-placement [--node <endpoint>]`

Shows replicas, roles, effective replication factor, and which partitions are hosted locally.

33.6.4 set-replication-factor

Set a per-partition replication factor override.

CLI flag: `--set-replication-factor <rf> --partition <id> [--node <endpoint>]`

The command is leader-only. It tries each connected endpoint until the meta-partition leader accepts (or targets one node with `--node`). A value of 0 clears the override.

33.7 Range Commands

33.7.1 ranges

Display the key-range map.

CLI flag: `--ranges [--key-space <space>] [--node <endpoint>]`

Shows the ranges per key space, the partition serving each range, and that node's routing mode. Use `--key-space` to filter by a specific key space.

33.7.2 register-key-range

Switch a key space to key-range routing and seed its whole-space descriptor.

CLI flag: `--register-key-range <key-space> [--node <endpoint>]`

The command is sent to every connected endpoint, because the routing-mode change is node-local. Use `--node` to target a single node.

33.7.3 unregister-key-range

Remove a key space's range descriptors.

CLI flag: `--unregister-key-range <key-space> [--node <endpoint>]`

Sent to every connected endpoint, like `--register-key-range`.

33.7.4 split-range

Split the range covering a key at that exact key.

CLI flag: `--split-range <key-space> --split-key <key> [--node <endpoint>]`

The covering range `[S,E)` becomes `[S,key)` and `[key,E)`. The split key lands in the upper half. The command is leader-only.

33.7.5 merge-ranges

Run the merge pass across all key spaces.

CLI flag: `--merge-ranges [--node <endpoint>]`

Folds adjacent ranges that have fallen below the configured minimum. The command is leader-only.

33.8 Backup Commands

33.8.1 backup-full

Take a full backup on the connected node.

CLI flag: `--backup-full`

Interactive: `backup full`

33.8.2 backup-incremental

Take an incremental backup from a parent.

CLI flag: `--backup-incremental --parent-backup-id <guid>`

Requires the parent backup ID (a UUID).

33.8.3 backup-coordinated

Take a coordinated full backup using the cluster-wide safe snapshot timestamp.

CLI flag: `--backup-coordinated`

Interactive: `backup coordinated`

33.8.4 list-backups

List all backups in the node's catalog.

CLI flag: `--list-backups`

Interactive: `list backups`

33.8.5 backup-chain

Resolve and validate the backup chain for a leaf backup.

CLI flag: `--backup-chain <guid>`

33.8.6 backup-gc

Run backup garbage collection.

CLI flag: `--backup-gc [--backup-gc-dry-run]`

Reclaims orphaned artifacts and enforces retention policies. Add `--backup-gc-dry-run` to preview what would be reclaimed without deleting anything.

33.8.7 restore

Restore a backup chain to a directory.

CLI flag: `--restore <leaf-backup-guid> --target-dir <path> [--target-time-ms <ms>]`

Option	Default	Description
`--target-dir`	(required)	Destination directory for the restored data
`--target-time-ms`	`0`	Point-in-time target as Unix epoch milliseconds (0 = chain maximum)

33.9 Interactive Shell Commands

These commands are available only inside the interactive shell:

Command	Description
`clear`	Clear the terminal screen
`exit` or `quit`	Exit the shell (releases all held locks and saves history)
Ctrl+C on empty prompt	Exit the shell (same as `exit`)

The interactive shell also accepts the full Kahuna script language directly at the prompt. Any statement from Appendix A (LET, SET, GET, IF, BEGIN, FOR, and so on) can be typed or pasted. Multi-line input is supported.

34 Appendix D: Error Codes and Response Types

Every Kahuna API call returns a typed status value that tells the caller what happened and what to do next. This appendix lists every status value for each subsystem. Values are grouped into three categories:

- **Success** values mean the operation completed as requested.
- **Rejection** values mean the operation did not run. The cause is known. The caller can fix the request and retry.
- **Indeterminate** values mean the outcome is unknown. The caller must re-read the current state before it decides what to do.

Numeric codes appear in parentheses where the enum assigns an explicit integer.

34.1 Key-Value Responses

The `KeyValueResponseType` enum describes the outcome of every persistent and ephemeral key-value operation: SET, GET, DELETE, EXTEND, and EXISTS. Transaction-related operations (PREPARE, COMMIT, ROLLBACK) also use this enum.

34.1.1 Success Values

Code	Name	Meaning
0	`Set`	The value was written.
1	`NotSet`	A conditional write (NX, XX, CMP, or CMPREV) did not match. The key was not changed.
2	`Extended`	The key's TTL was extended.
3	`Get`	The value was read.
4	`Deleted`	The key was deleted.
5	`Locked`	The key was locked by the transaction.
6	`Unlocked`	The key was unlocked.
7	`Prepared`	The transaction was prepared (two-phase commit).
8	`Committed`	The transaction was committed.
9	`RolledBack`	The transaction was rolled back.
10	`Exists`	The key exists (returned by EXISTS).

34.1.2 Informational Values

Code	Name	Meaning
11	`WaitingForReplication`	The write was accepted locally but has not replicated to a quorum yet. The caller can wait or proceed.
107	`SafeTimestamp`	The response carries a safe snapshot timestamp (used by coordinated backup).

34.1.3 Rejection Values

Code	Name	Meaning
99	`Errored`	An internal error occurred. The response carries a reason string.
100	`InvalidInput`	The request is malformed (missing key, invalid TTL, or bad revision).
101	`MustRetry`	A transient conflict occurred. The caller should retry the full operation.
102	`Aborted`	The transaction was aborted by the system (deadlock, timeout, or admission refusal).
103	`DoesNotExist`	The key does not exist. Returned by GET and DELETE when the key is absent.
104	`AlreadyLocked`	The key is locked by another transaction. The caller must wait or abort.
105	`PrefixLockUnsupportedOnRangedSpace`	Prefix locks are not supported on key spaces that use key-range routing.
106	`RangeLocks`	The operation conflicts with a range lock held by another transaction.
108	`AdmissionRefused`	The transaction admission controller rejected the request. The queue is full or the priority is too low.

34.2 Lock Responses

The `LockResponseType` enum describes the outcome of distributed lock operations: LOCK, UNLOCK, EXTEND-LOCK, and GET-LOCK.

34.2.1 Success Values

Code	Name	Meaning
0	`Locked`	The lock was acquired. The response carries the owner ID and the fencing token.
1	`Busy`	The lock is held by another owner. The caller did not acquire it.
2	`Extended`	The lock's lease was extended.
3	`Unlocked`	The lock was released.
4	`Got`	The lock state was read (returned by GET-LOCK).

34.2.2 Informational Values

Code	Name	Meaning
10	`WaitingForReplication`	The lock operation was accepted locally but has not replicated to a quorum yet.

34.2.3 Rejection Values

Code	Name	Meaning
99	`Errored`	An internal error occurred.
100	`InvalidInput`	The request is malformed (missing lock name or invalid TTL).
101	`MustRetry`	A transient conflict occurred. Retry the operation.
102	`LockDoesNotExist`	The named lock does not exist. Returned by UNLOCK and EXTEND-LOCK when no lock was ever acquired.
103	`InvalidOwner`	The caller's owner ID does not match the current holder. The lock belongs to another owner.
104	`Aborted`	The operation was aborted by the system.

34.3 Sequence Responses

The `SequenceResponseType` enum describes the outcome of sequence operations: CREATE-SEQUENCE, GET-SEQUENCE, NEXT-SEQUENCE, RESERVE-SEQUENCE, and DELETE-SEQUENCE.

34.3.1 Success Values

Code	Name	Meaning
0	`Success`	The operation completed. For NEXT and RESERVE, the response carries the allocated value or range.

34.3.2 Rejection Values

Code	Name	Meaning
1	`NotFound`	The sequence does not exist.
2	`AlreadyExists`	A sequence with this name exists already (returned by CREATE).
3	`InvalidInput`	The request is malformed (invalid increment, negative count, or empty name).
4	`MaxValueExceeded`	The next allocation would exceed the sequence's configured maximum.
5	`MustRetry`	A transient conflict occurred. Retry the operation.
6	`Aborted`	The operation was aborted by the system.
99	`Error`	An internal error occurred.

34.4 Transaction Status

The `KahunaTransactionStatus` enum describes the lifecycle state of a transaction. The status appears in transaction responses and in the internal transaction record.

Code	Name	Meaning
0	`Pending`	The transaction is open. It can accept reads and writes.
1	`Finalizing`	The transaction is in the commit or rollback path. No new operations are accepted.
2	`Committed`	The transaction committed. All writes are durable and visible.
3	`Rolledback`	The transaction was rolled back. All writes were discarded.
4	`Aborted`	The system aborted the transaction (deadlock, timeout, or leadership loss).

34.5 Backup Outcomes

The `KahunaBackupOutcome` enum describes the result of a backup or restore operation. Chapter 27 covers the backup subsystem in detail.

34.5.1 Success Values

Code	Name	Meaning
0	`Ok`	The backup or restore completed.

34.5.2 Rejection Values

Code	Name	Meaning
1	`NotConfigured`	No backup directory is configured on this node. Set `--backup-directory`.
2	`ParentMissing`	The parent backup for an incremental does not exist in the catalog.
3	`NeedsFull`	An incremental backup cannot proceed because no full backup exists yet. Take a full backup first.
4	`CorruptChain`	The backup chain failed HMAC verification. One or more manifests were tampered with or corrupted.
5	`CorruptArtifact`	A backup artifact (SST file or WAL segment) failed integrity verification.
6	`TargetConflict`	The restore target directory is not empty.
7	`TargetOutsideCoverage`	The point-in-time target falls outside the chain's WAL coverage window.
8	`Cancelled`	The operation was cancelled by the caller.
9	`IoError`	A filesystem read or write failed. The reason string carries the OS error.
10	`RetryableLeadershipLoss`	Leadership moved during the backup. Retry on the new leader.
11	`ExactCheckpointUnavailable`	The requested checkpoint is not available in the WAL.
12	`UnsupportedFormat`	The manifest format version is newer than this node can read. Upgrade the node.
13	`TopologyChanged`	The cluster topology changed during a coordinated backup. Retry.
14	`NotBackupCoordinator`	This node is not the backup coordinator. Send the request to the coordinator.
15	`InsecureRoot`	The backup root directory has insecure permissions.
16	`RestrictedCoverage`	The backup covers only a subset of partitions. The reason string names the missing ones.

34.6 Cluster Leave Outcomes

The `GrpcLeaveClusterOutcome` enum describes the result of a node decommission request. Chapter 24 covers the membership protocol.

34.6.1 Success Values

Code	Name	Meaning
0	`Committed`	The node was removed from the cluster. Replica drain completed.

34.6.2 Rejection Values

Code	Name	Meaning
1	`NotAMember`	The target node is not a member of the cluster.
2	`RefusedInsufficientVoters`	Removal would drop the voter count below the minimum needed for quorum.
3	`NotInitialized`	The cluster has not completed initialization.
4	`NoLeader`	No leader is available to process the request.
5	`Timeout`	The operation timed out before the node could be removed.
6	`RefusedDrainInProgress`	A drain is already in progress on this node. Wait for it to finish.
7	`DrainTimedOut`	Replica drain started but did not complete within the allowed time.

34.7 Split Range Status

The split range response uses a string `status` field (not a numeric enum). Each status value is either determinate or indeterminate. A determinate status means the range map is in a known state. An indeterminate status means the map may have changed, and the caller must re-read `GET /v1/ranges` before it takes the next action.

34.7.1 Determinate (Success)

Status	Meaning
`Succeeded`	The split completed. The range `[S,E)` became `[S,key)` and `[key,E)`. The response carries the new partition ID and the new routing generation.

34.7.2 Determinate (Rejection)

Status	Meaning
`NotLeader`	This node does not lead the partition that owns the range map. Retry against the leader. The `leaderHint` field names the leader when gossip knows it.
`NoRange`	No descriptor covers the key. The key space is unregistered or unseeded.
`InvalidSplitKey`	The key is outside the covering range, or equal to the range start. Either case would produce an empty half.
`BelowMinRangeSize`	One of the halves holds no keys. The split was refused by the minimum-size policy.
`PartitionCreationFailed`	The destination partition could not be created. No descriptor changed. A retry allocates a fresh partition ID.
`InvalidInput`	The request is malformed.
`KeyRangeDisabled`	The cluster has no data partition to split onto.

34.7.3 Indeterminate

Status	Meaning
`TransferFailed`	Data transfer to the new partition failed mid-split. Re-read the map.
`QuiesceFailed`	The source partition could not quiesce writes. Re-read the map.
`CutoverFailed`	The routing cutover did not commit. Re-read the map.
`ConcurrentSplit`	Another split is in progress on the same range. Re-read the map.
`Indeterminate`	Leadership or transport was lost mid-split. Re-read the map.

34.8 Merge Ranges Status

The merge response uses a string `status` field. A merge pass scans every key space and folds adjacent ranges that fall below the configured minimum size.

Status	Determinate	Meaning
`Completed`	Yes	The pass ran to completion. The `merges` field tells how many pairs it folded. A value of `0` means nothing was eligible.
`NotLeader`	Yes	This node does not lead the partition that owns the range map. Retry against the leader.
`Indeterminate`	No	The pass failed partway. Merges that already committed stay committed. Re-read the map.

34.9 Register Key Range Status

The register response uses a string `status` field. Registration has two parts: a node-local routing-mode flip and a replicated seed descriptor.

Status	Meaning
`Seeded`	This call committed the whole-space descriptor. The key space is now key-range routed on this node.
`AlreadySeeded`	A descriptor already existed. This call still registered the routing mode on this node.
`Indeterminate`	The mode was flipped, but no descriptor is visible yet. The seed may have committed on the leader and not replicated within the wait. Re-read the map.
`InvalidInput`	Empty key space, or a `/meta` schema-log space.
`KeyRangeDisabled`	The cluster has no data partition to seed onto. Key-range routing cannot be enabled.

The `routingMode` field in the response shows how this node routes the space after the call: `"KeyRange"` when registered, `"Hash"` when refused.

34.10 Remove Key Range Status

The remove response uses a string `status` field. Removal clears the descriptors for a key space.

Status	Meaning
`Removed`	The space carries no descriptors on this node. Removal is idempotent: this status appears whether this call removed them or they were already gone.
`Indeterminate`	The removal was accepted, but descriptors are still visible. The commit may not have applied yet. Re-read the map.
`QuiesceWindowOpen`	A split is mid-cutover on a range in this space. The quiesce window is short. Retry.
`InvalidInput`	Empty key space, or a `/meta` schema-log space.
`KeyRangeDisabled`	The cluster has no data partition. No space was ever key-range routed.

The `routingMode` field shows the routing mode after the call. The `descriptorCount` field shows how many descriptors remain visible on this node. A successful removal shows `0`.

34.11 Common Patterns

Several patterns appear across all subsystems. The following table summarizes how to handle them.

Pattern	Subsystems	Action
`MustRetry`	Key-value, locks, sequences	Retry the full operation from the start. The conflict was transient.
`Aborted`	Key-value, locks, sequences, transactions	The system cancelled the operation. Start a new transaction or operation.
`InvalidInput`	All	Fix the request. Do not retry with the same parameters.
`NotLeader`	Split, merge, cluster leave	Send the request to the leader. Use the `leaderHint` field when it is present.
`Indeterminate`	Split, merge, register, remove	Re-read the current state with `GET /v1/ranges` before the next action. Do not assume the operation failed.
`WaitingForReplication`	Key-value, locks	The write was accepted but not yet durable on a quorum. Wait or proceed, depending on the application's consistency requirements.
`Errored` / `Error`	Key-value, locks, sequences	An internal error occurred. The response carries a reason string. Log it and contact the operator.

35 Appendix E: Comparison with etcd, ZooKeeper, Consul, and Redis

This appendix compares Kahuna with four systems that operators often evaluate for similar workloads: etcd, ZooKeeper, Consul, and Redis. The comparison is factual, not promotional. Each system was built for a different primary use case, and the right choice depends on the workload.

35.1 Feature Matrix

Feature	Kahuna	etcd	ZooKeeper	Consul	Redis
Key-value store	Yes	Yes	Yes (znodes)	Yes	Yes
Distributed locks	Yes (with fencing tokens)	Yes (via leases)	Yes (ephemeral znodes)	Yes (sessions)	Yes (Redlock)
Distributed sequencer	Yes (block-based)	No	No	No	No (manual with INCR)
Multi-key transactions	Yes (2PC, ACID)	Yes (mini-transactions)	Yes (multi-op)	Yes (single-key CAS)	Yes (MULTI/Lua)
Script language	Yes (Kahuna Script)	No	No	No	Yes (Lua)
MVCC / revision history	Yes (per-key version chain)	Yes (global revision)	Yes (per-znode version)	No	No
Snapshot reads (as-of)	Yes (HLC-based)	Yes (revision-based)	No	No	No
Range scans	Yes (key-range routing)	Yes (native)	Yes (children)	Yes (prefix)	Yes (SCAN)
Automatic sharding	Yes (hash + key-range)	No (single Raft group)	No (single leader)	No (single leader per DC)	Yes (Redis Cluster)
Watch / notifications	No	Yes (watch)	Yes (watcher)	Yes (blocking queries)	Yes (Pub/Sub, keyspace)
TTL / ephemeral data	Yes	Yes (leases)	Yes (ephemeral znodes)	Yes (sessions)	Yes (EXPIRE)
Backup / PITR	Yes (full, incremental, coordinated)	Yes (snapshot)	Yes (snapshot)	Yes (snapshot)	Yes (RDB/AOF)

35.2 Architecture Comparison

35.2.1 Consensus and Replication

Kahuna uses per-partition Raft groups. Each partition is an independent Raft group with its own leader, log, and election cycle. The system can run many partitions across many nodes. Leadership is distributed: different partitions can have leaders on different nodes. This architecture scales write throughput horizontally because writes to different partitions do not contend for the same Raft log.

etcd uses a single Raft group for the entire data store. All writes go through one leader. The leader serializes all proposals into one log. This design is simple and provides strong consistency, but write throughput is bounded by the single leader's capacity. etcd is designed for small metadata workloads (configuration, service discovery), not high-throughput data storage.

ZooKeeper uses a single-leader protocol (ZAB, a Raft-like atomic broadcast). All writes go through the leader. Reads can be served by followers, but follower reads may be stale unless the client issues a `sync` call first. Like etcd, ZooKeeper is designed for small coordination workloads.

Consul uses a single Raft group per datacenter. Cross-datacenter replication uses a gossip-based WAN federation, not Raft. Writes within a datacenter go through the local leader. Consul is primarily a service mesh and service discovery tool; its key-value store is a secondary feature.

Redis does not use consensus for its primary replication. Redis uses asynchronous leader-follower replication. In this mode, a write acknowledged by the leader may be lost if the leader crashes before the follower receives it. Redis Sentinel provides automated failover but does not guarantee that the promoted follower has all writes. Redis Cluster adds hash-based sharding across multiple leaders, but each shard still uses asynchronous replication by default. The `WAIT` command can force synchronous replication to a specified number of replicas, but this is opt-in per write.

35.2.2 Data Model

Kahuna stores arbitrary byte values keyed by strings. Keys are organized into key spaces (the portion before the last `/` separator). Each key maintains a version chain with revision numbers and HLC timestamps. Deleted keys leave tombstones that are eventually pruned.

etcd stores arbitrary byte values keyed by byte strings. etcd maintains a global revision counter that increments on every write. Watches observe changes by revision. The entire key space is one flat namespace with byte-order range queries.

ZooKeeper stores data in a hierarchical namespace of znodes (similar to a filesystem). Each znode can hold up to 1 MiB of data. Znodes have versions, ACLs, and optional ephemeral or sequential properties. The hierarchical model encourages a tree-structured naming convention.

Consul stores values keyed by strings in a flat key space with `/` as a conventional separator. Values are limited to 512 KiB. Consul does not maintain revision history. It provides blocking queries (long-poll) for change notification.

Redis stores values in a flat key space with multiple data structure types: strings, hashes, lists, sets, sorted sets, streams, and more. Redis does not maintain revision history. Keys have optional TTL but no version chain.

35.2.3 Consistency Model

Kahuna provides linearizable reads and writes through Raft read-index confirmation. Transactions provide snapshot isolation with serializable conflict detection. The system uses HLC timestamps for MVCC ordering.

etcd provides linearizable reads and writes. Mini-transactions (If/Then/Else) provide atomic conditional updates on multiple keys, but they execute in a single Raft round and cannot span multiple round-trips.

ZooKeeper provides linearizable writes. Reads from the leader are linearizable. Reads from followers are sequentially consistent (they may lag behind the leader). A `sync` call forces a follower to catch up before the next read.

Redis provides eventual consistency in its default asynchronous replication mode. A write acknowledged by the leader is not guaranteed to survive a failover. Redis does not provide linearizable reads across replicas. Redis transactions (MULTI/EXEC) are serialized on a single node but do not span multiple nodes in Redis Cluster.

Consul provides linearizable reads when the `consistent` query mode is used. The default mode (`stale`) allows follower reads that may lag. Consul's key-value operations are single-key; there is no multi-key transaction primitive.

35.3 Distributed Locks

Kahuna provides first-class distributed locks with fencing tokens. Each lock acquisition returns a monotonically increasing fencing token. The fencing token allows downstream services to reject stale operations from a lock holder whose lease expired. Locks support configurable TTL, automatic expiry, and explicit unlock.

etcd provides distributed locks through its lease mechanism. A client creates a lease with a TTL and associates a key with that lease. The key is deleted when the lease expires or is revoked. etcd does not provide built-in fencing tokens; the application must implement fencing using the key's revision or create-revision.

ZooKeeper provides distributed locks through ephemeral sequential znodes. A client creates an ephemeral sequential znode under a lock path and watches the znode with the next-lower sequence number. The lock is released when the ephemeral znode is deleted (either explicitly or when the session expires). ZooKeeper does not provide fencing tokens.

Consul provides distributed locks through sessions. A client creates a session with a TTL and acquires a lock by writing a key with the session attached. The lock is released when the session is invalidated or expires. Consul does not provide fencing tokens.

Redis provides distributed locks through the Redlock algorithm. Redlock acquires locks on a majority of independent Redis instances and uses clock-based TTL for expiry. The Redlock algorithm has been the subject of academic debate about its safety properties under clock skew and network partitions. Redis does not provide fencing tokens as part of the lock primitive.

35.4 Transactions

Kahuna provides two transaction models. Script transactions execute a sequence of operations atomically on the server. Interactive transactions allow the client to read, compute, and write across multiple round-trips, with a final two-phase commit. Both models support pessimistic and optimistic locking. Transactions can span multiple partitions. Conflict detection uses read-set validation and write-intent comparison.

etcd provides mini-transactions (Txn) that evaluate a set of conditions and execute one of two operation sets atomically. Mini-transactions execute in a single Raft round-trip. They cannot span multiple round-trips or perform client-side computation between reads and writes.

ZooKeeper provides multi-op transactions that execute a batch of operations atomically. Like etcd, these execute in a single round-trip. ZooKeeper does not support interactive transactions.

Consul does not provide multi-key transactions. Its key-value operations are single-key with CAS (check-and-set) semantics.

Redis provides MULTI/EXEC transactions that queue commands and execute them atomically on a single node. Redis transactions do not support conditional logic between commands (the WATCH command provides optimistic locking but not read-modify-write in a single transaction). In Redis Cluster, transactions are limited to keys that hash to the same slot.

35.5 Sharding and Scalability

Kahuna supports automatic sharding through two modes: hash-based routing (the default, which distributes keys evenly across a fixed partition pool) and key-range routing (which assigns contiguous key intervals to partitions and supports automatic splitting and merging). Write throughput scales with the number of partitions and nodes.

etcd does not shard data. All data lives in a single Raft group. Scalability is vertical: a larger machine handles more data and more throughput, up to the limits of a single Raft leader.

ZooKeeper does not shard data. All data lives on every node. Reads scale with the number of followers (when using stale reads), but writes do not scale beyond the single leader.

Consul does not shard its key-value store within a datacenter. Cross-datacenter federation provides geographic distribution but not throughput scaling.

Redis Cluster shards data across multiple leaders using hash slots (16,384 slots). Each leader handles a subset of the key space. Resharding is manual or semi-automatic. Redis Cluster does not support multi-key transactions across different hash slots.

35.6 Use-Case Fit

Use Case	Best Fit
Small configuration store (< 1 GiB)	etcd, ZooKeeper, Consul
Service discovery and health checking	Consul
Distributed coordination with fencing	Kahuna
High-throughput key-value with transactions	Kahuna, Redis
Caching with rich data structures	Redis
Leader election for microservices	Kahuna, etcd, ZooKeeper
Distributed sequence generation	Kahuna
MVCC reads at a past point in time	Kahuna, etcd
Multi-key ACID transactions	Kahuna
Watch-based change notification	etcd, ZooKeeper
Service mesh integration	Consul

35.7 Summary

Kahuna occupies a different point in the design space than etcd, ZooKeeper, and Consul. Those three systems are optimized for small metadata workloads with a single Raft group. They provide strong consistency with simple operations. Kahuna is built for workloads that need multiple primitives (key-value, locks, sequences, transactions) under one system, with automatic sharding for throughput scaling.

Redis occupies yet another point. It provides the richest data structure model and the highest single-node throughput, but its default replication model trades consistency for performance. Applications that need strong consistency with Redis must use the `WAIT` command and accept the latency cost, or accept the risk of data loss on failover.

The choice depends on the workload's requirements: data size, throughput, consistency guarantees, transaction scope, and operational complexity. No single system is the best fit for every use case.

36 Appendix F: Glossary

Actor. A concurrent processing unit with a private mailbox. Each actor processes one message at a time. Kahuna uses the Nixie actor framework. Key-value, lock, and sequencer partitions are each served by an actor.

Admission control. A mechanism that limits the number of concurrent transactions a node accepts. When the admission queue is full or the request priority is too low, the controller rejects the transaction with `AdmissionRefused`. Admission control prevents overload from cascading into unbounded intent accumulation.

Apply fingerprint. A rolling digest of the log entries a partition applied. Nodes exchange fingerprints over inter-node gRPC. A divergence at the same log index surfaces as a diagnostic metric. Range splits check the source partition's fingerprint before they proceed.

Anchor partition. The partition that holds the canonical transaction record for a distributed transaction. The anchor is determined by hashing the transaction ID. All participants look up the anchor to learn whether a transaction committed or aborted.

Background writer. The `BackgroundWriterActor` that batches committed Raft entries and flushes them to the storage backend asynchronously. The background writer decouples the Raft commit path from disk latency.

Backup chain. A sequence of backups that starts with a full backup and continues with zero or more incremental backups. Each incremental links to its parent by backup ID. Restoring from any point in the chain requires every ancestor back to the root full backup.

CAS (compare-and-swap). An atomic operation that writes a new value only if the current value matches an expected value. Kahuna supports CAS through the `CMP` flag on SET and through the `CMPREV` flag that compares the revision number.

Check-quorum. A Raft safety mechanism where the leader steps down if it does not receive heartbeat responses from a majority of voters within a configurable interval. Check-quorum prevents a network-partitioned leader from serving stale reads or accepting writes that cannot replicate.

Commit. The act of making a write durable and visible. In Raft, a log entry is committed when a majority of nodes have appended it to their logs. In a transaction, a commit is the decision to apply all writes.

Completion receipt. A record that proves a transaction's result has been applied to a specific key on a specific partition. Completion receipts prevent duplicate application during recovery.

Consensus. Agreement among a group of nodes on a single value or sequence of values, despite failures. Kahuna uses the Raft consensus algorithm through the Kommander library.

Coordinated omission. A measurement error in benchmarks where slow responses reduce the request rate, which hides tail latency. Open-loop benchmarks (such as `kahuna-bench --rate N`) avoid this error by measuring latency from the intended start time, not the actual start time.

Cutover. The atomic step in a range split or merge that swaps the old range descriptors for the new ones in a single Raft-committed mutation. After cutover, routing uses the new descriptors.

Deadband. A tolerance band that prevents oscillation in a balancing system. The leader balancer's count deadband prevents leadership from bouncing between nodes when their leader counts differ by a small amount.

Deferred settlement. The practice of separating the transaction decision (commit or abort) from the resolution of individual write intents. After the decision is committed, intents are settled asynchronously. This reduces the latency of the commit path.

Descriptor. See Range descriptor.

Durability. The guarantee that a committed write survives node restarts and crashes. Kahuna achieves durability through Raft replication (the write exists on a majority of nodes) and persistence to the storage backend.

Durability floor. The highest WAL index whose state has been flushed to the storage backend for a given partition. Kommander uses this floor to decide when WAL entries can be compacted.

Ephemeral. Data that is not replicated through Raft and does not survive node restarts. Ephemeral key-value entries and ephemeral locks are stored only in the local node's memory.

Fencing token. A monotonically increasing integer returned with each lock acquisition. Downstream services use the fencing token to reject operations from a lock holder whose lease expired. If a stale holder sends a request with token 5 but the current holder has token 6, the downstream service rejects the stale request.

Flush. The act of writing buffered data from memory to the storage backend. The background writer accumulates entries and flushes them in batches.

Generation. A version number on a range descriptor. The generation increments on every split, merge, or modification to the range. Clients carry the generation with each request. A stale generation causes a routing error that forces the client to refresh its range map.

Generation fence. The mechanism that rejects requests routed to a partition with a stale generation number. The fence prevents writes from landing on a range that no longer owns the key.

Hop budget. A forward-count limit on inter-node request forwarding. Each forward increments a hop counter. When the counter reaches `MaxForwardHops` (2), the receiver returns `MustRetry` instead of forwarding again. The hop budget prevents infinite routing loops during range map convergence.

HLC (hybrid logical clock). A clock that combines a physical timestamp (wall-clock time) with a logical counter. HLCs provide a total order across events on different nodes without requiring perfectly synchronized clocks. Kahuna encodes HLCs as a 64-bit value: 44 bits for the physical timestamp (milliseconds), 16 bits for the logical counter, and 4 bits for flags.

Incarnation. A version boundary for a sequence. Each `UpdateSequence` call starts a new incarnation. Values across incarnations are unique to the caller, not to the system.

Idempotent. An operation that produces the same result whether it is applied once or multiple times. Kahuna's sequence reservations are idempotent: retrying a reservation with the same owner returns the same value.

Intent. See Write intent.

Key space. The portion of a key before the last `/` separator. Keys in the same key space share routing decisions. A key space can be registered for key-range routing or left on the default hash-based routing.

Kommander. The Raft consensus library that Kahuna uses. Kommander manages leader election, log replication, and state machine application for each partition.

Leader. The Raft node that handles all writes for a partition. The leader replicates log entries to followers and responds to clients after a majority acknowledges each entry.

Learner. A Raft node that receives replication traffic but does not count toward quorum. New nodes join the cluster as learners and are promoted to voters after they catch up.

Lease. A time-bounded grant. Distributed locks use leases: the lock is held for a specified duration and expires automatically if the holder does not renew it.

Lock grant term fence. A safety check that records the Raft term under which each lock was issued. At commit time, the conflict probe verifies that the partition still leads under the same term. A term change means the locks were dropped, and the commit is refused.

Linearizability. A consistency guarantee that every operation appears to take effect at a single instant between its invocation and its completion. The result is equivalent to a single serial execution. Kahuna provides linearizable reads and writes through Raft read-index confirmation.

Log entry. A single record in a Raft partition's write-ahead log. Each entry has an index, a term, and a payload. The payload can be a key-value write, a lock operation, a transaction record, a range map change, or another replicated operation.

Meta partition. Partition zero. The meta partition stores the range map (range descriptors), the snapshot floor registry, and the cluster membership roster. It is a Raft group like any other partition, but it holds system metadata rather than user data.

MVCC (multi-version concurrency control). A concurrency control method that keeps multiple versions of each key-value entry. Readers see a consistent snapshot without blocking writers. Kahuna's MVCC uses HLC timestamps to order versions.

Nemesis. In Jepsen testing, a fault injector. A nemesis introduces failures such as network partitions, process kills, or clock skew during a test run.

Nixie. The actor framework that Kahuna uses. Nixie provides actor lifecycle management, mailbox processing, and consistent-hash routing.

One-phase commit. An optimization for transactions where all writes land on the same partition. The prepare, validate, and commit steps collapse into a single Raft round-trip. The transaction skips the full two-phase commit protocol.

Pipelined batching. A write aggregation mode where multiple batches per partition await Raft concurrently. FIFO ordering is preserved. Controlled by `--kv-write-max-in-flight-batches`.

Placement group. The portion of a key space's name before the first `|` separator. All key spaces that share the same placement group hash to the same partition. If the key space has no `|`, the entire key space is its own placement group.

Partition. A subset of the data managed by an independent Raft group. Each partition has its own leader, log, and set of replicas. Partitions are the unit of scaling: more partitions distribute load across more nodes.

PITR (point-in-time recovery). The ability to restore data to any specific moment within the backup retention window. PITR replays WAL segments from a full backup checkpoint up to the target HLC timestamp.

Placement. The assignment of partition replicas to nodes. The placement rebalancer distributes replicas to maintain the target replication factor, spread replicas across zones, and balance replica counts.

Prepared intent. See Write intent.

Pre-decision fence confirmation. A coordinator-driven check that queries replicas for their staged-base fence verdicts before the CAS decide step. If any replica reports a stale base, the coordinator aborts the transaction before committing. Complements the follower-driven stale-base veto.

Presumed-abort. A recovery rule for distributed transactions. If no committed decision record exists for a transaction, the transaction is presumed aborted. This rule allows recovery to clean up orphaned intents without contacting the coordinator.

Proposal. A write request submitted to the Raft leader for replication. The leader appends the proposal to its log and replicates it to followers. The proposal is committed when a majority acknowledges it.

Quiesce. A temporary pause in routing to a range during a split or merge. The quiesce step prevents new writes from arriving while the cutover executes.

Quorum. The minimum number of nodes that must agree for a Raft decision to commit. For a group of N nodes, the quorum is `floor(N/2) + 1`. A three-node group has a quorum of 2. A five-node group has a quorum of 3.

Raft. A consensus algorithm that elects a leader and replicates a log of commands to a group of nodes. Raft guarantees that all nodes apply the same commands in the same order, even if some nodes fail.

Range descriptor. A record that maps a contiguous interval of keys (defined by a start key and an end key) to a partition. Range descriptors live on the meta partition and are replicated through Raft.

Range map. The collection of all range descriptors for all key spaces. The range map is the source of truth for key-range routing.

Rate limiter (distributed). A `System.Threading.RateLimiting` implementation whose budget lives in Kahuna instead of in one process. Every replica that names the same key spends one shared budget. Four types are available: fixed window, sliding window, token bucket, and concurrency.

Read-index. A Raft mechanism for linearizable reads without writing to the log. The leader confirms that it still holds leadership by exchanging heartbeats with a majority, then serves the read at the committed index. Kahuna coalesces multiple concurrent read-index requests into a single heartbeat round.

Replication factor. The number of voter replicas for each partition. A replication factor of 3 means three nodes hold copies of each partition. A replication factor of 0 means full replication: every node holds every partition.

Routing mode. The client-side strategy for selecting which server receives an operation. Four modes are available: `RoundRobin` (rotate over configured endpoints), `Learned` (reuse the endpoint a previous response reported), `Metadata` (learned routing plus hash-based partition resolution from cluster metadata), and `Auto` (default, selects `Learned` for multi-endpoint clients and `RoundRobin` for single-endpoint clients).

Revision. A version number on a key-value entry. The revision increments on every write to that key. Clients use revisions for conditional writes (`CMPREV`) and for reading historical values (`GET key @ revision`).

Serializability. An isolation level where concurrent transactions produce the same result as some serial execution. Kahuna's interactive transactions provide snapshot isolation with serializable conflict detection.

Slow node avoidance. A routing optimization that classifies followers by their WAL commit-wait latency and steers replication traffic away from slow nodes. The classifier uses hysteresis (separate promote and demote thresholds) to prevent oscillation. A slow follower still receives replication traffic but is deprioritized for quorum acknowledgment.

Snapshot floor. The minimum HLC timestamp below which MVCC revision pruning is blocked. Active snapshot holds pin the floor. The snapshot floor prevents the pruning system from deleting revisions that an active snapshot still needs.

Snapshot isolation. An isolation level where each transaction reads from a consistent snapshot taken at the transaction's start time. Writers do not block readers. Write-write conflicts are detected at commit time.

Stale-base veto. A replica-side defense where a follower rejects a commit whose base timestamp is older than the follower's last committed write for that key. The veto prevents a delayed coordinator from overwriting a newer value. The leader counts upheld vetoes as a conflict and returns `MustRetry` to the caller.

Stale-leader fencing. A safety mechanism that prevents an outdated leader from applying proposals after a new election. When a partition actor receives a proposal, it checks the current Raft term against its own term. If the term has advanced, the actor rejects the proposal. On leadership loss, the `LockActor` and `KeyValueActor` clear their in-memory state so that a new leader starts from a clean baseline.

Split. The act of dividing one range into two. A split creates a new partition for the right half of the key range and atomically updates the range descriptors.

State transfer. The process of copying all data for a partition from one node to another. State transfer is used when a follower's WAL has been compacted past the entries it needs, so normal log replication cannot bring it up to date.

SWIM (Scalable Weakly-consistent Infection-style Membership). A failure detection protocol that uses direct and indirect probes to detect failed nodes. Kahuna uses SWIM to feed liveness information to the cluster membership system.

System partition. See Meta partition.

Term. A Raft concept that represents an election epoch. Each election increments the term. A node that discovers a higher term updates its own term and steps down if it was the leader. Terms provide a logical clock for leader identity.

Tombstone. A marker left when a key is deleted. The tombstone records the deletion revision and timestamp. Tombstones are eventually pruned by the revision cleanup system.

Transaction record. The canonical commit or abort decision for a distributed transaction. The transaction record lives on the anchor partition. All participants consult the record to learn the transaction's outcome.

Two-phase commit (2PC). A protocol for atomic commitment across multiple partitions. Phase one (prepare) asks each participant to promise it can commit. Phase two (decide) records the commit or abort decision. Kahuna uses 2PC for interactive and script transactions that span multiple partitions.

Unflushed overlay. A read-through layer that bridges the gap between Raft-committed entries and entries flushed to the storage backend. The overlay stores committed entries in memory so that reads can see them immediately, even before the background writer flushes them.

Voter. A Raft node that counts toward quorum. Voters can vote in elections and can become leaders.

WAL (write-ahead log). A durable, append-only log that records every committed Raft entry. On restart, Kahuna replays the WAL to reconstruct in-memory state. The WAL is compacted after entries are flushed to the storage backend.

Write intent. A provisional write created during the prepare phase of a two-phase commit. The intent is visible to the transaction that created it but blocked for other transactions. When the transaction commits, the intent is materialized into a regular key-value entry. When the transaction aborts, the intent is discarded.

Yielding transaction. A transaction that uses the `TransactionConflictPolicy.Yield` conflict policy. When a yielding transaction encounters a lock held by a non-yielding transaction, it aborts instead of waiting. This policy is designed for background or maintenance work that must not block user-facing operations. Yielding transactions cannot acquire prefix or range locks.

Zone. A locality label assigned to a node through the `--raft-zone` flag. The placement planner spreads replicas across distinct zones to tolerate zone-level failures (such as a rack or availability zone going offline).